

AI + Scientific computing

May 22, 2025

[Tailin Wu](#), Westlake University

Website: ai4s.lab.westlake.edu.cn/course

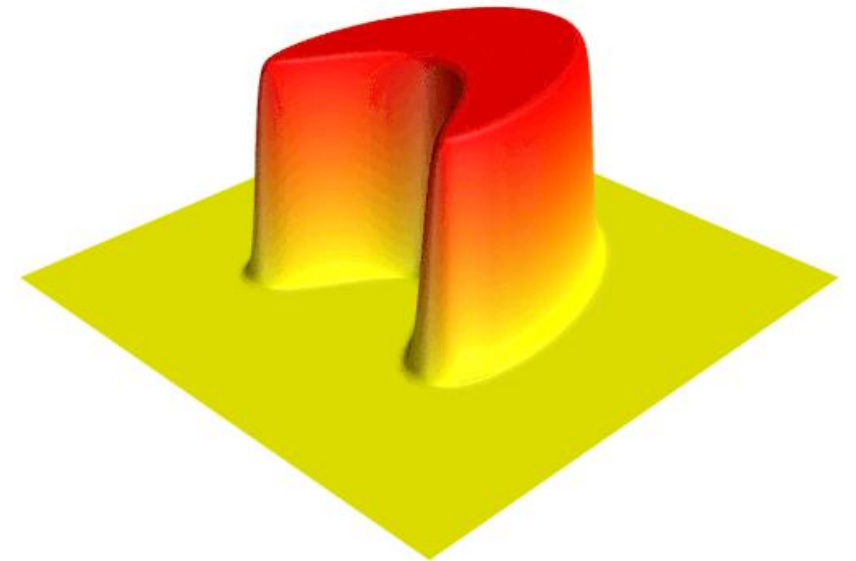
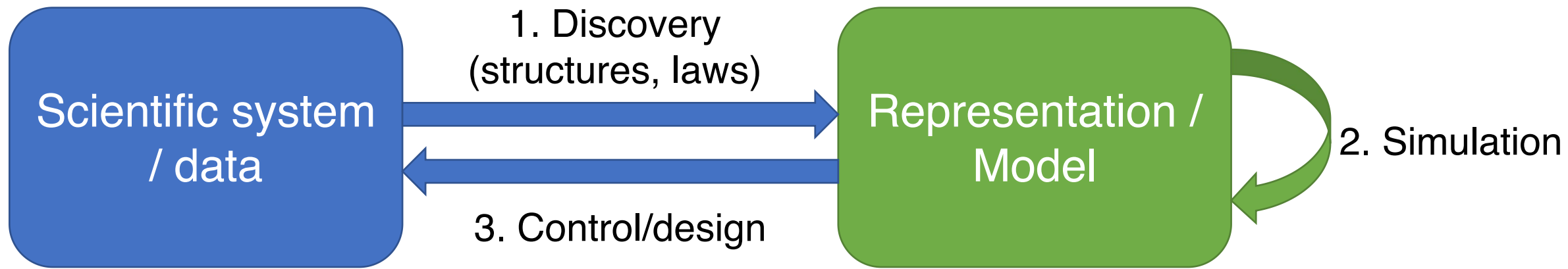
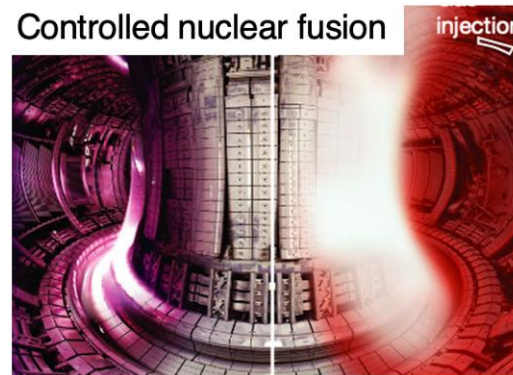
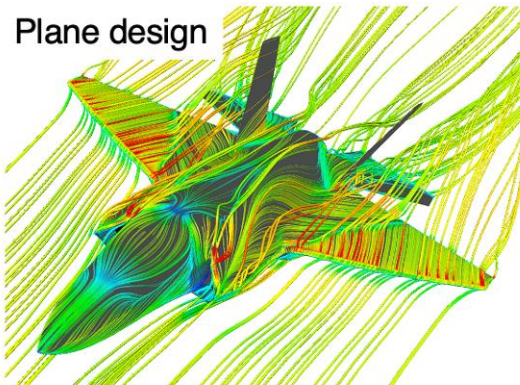


Image from: Wikipedia



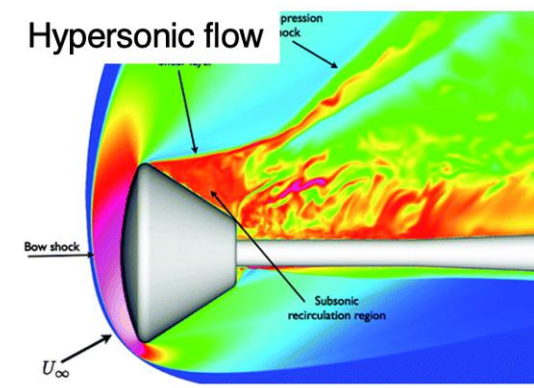
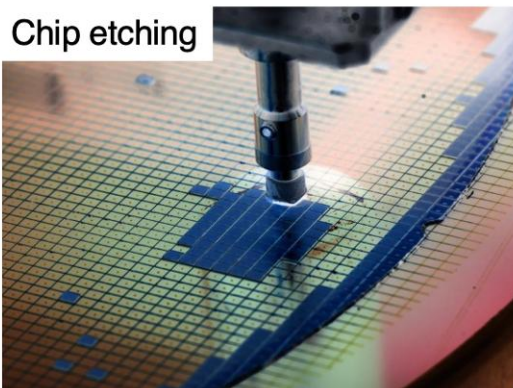
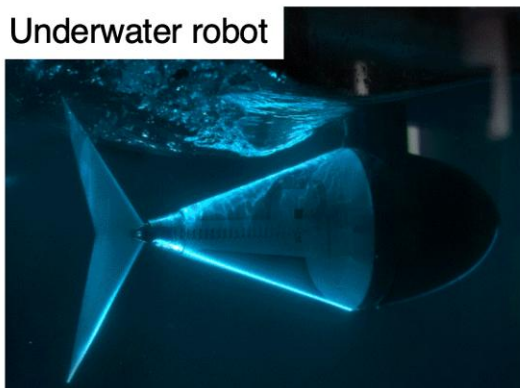
These three tasks are fundamental in **science** and **engineering**

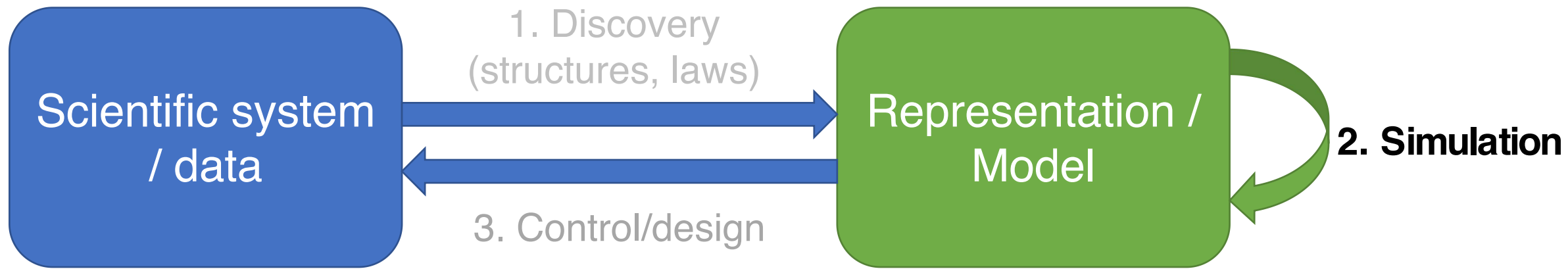


@Princeton

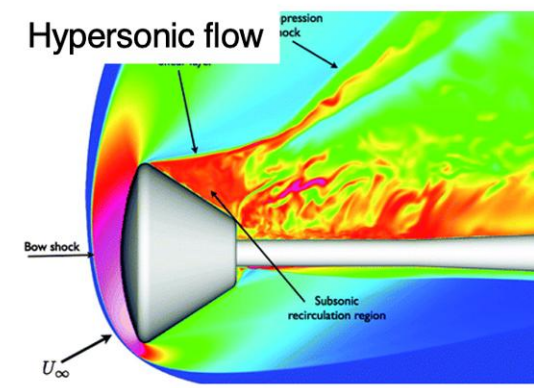
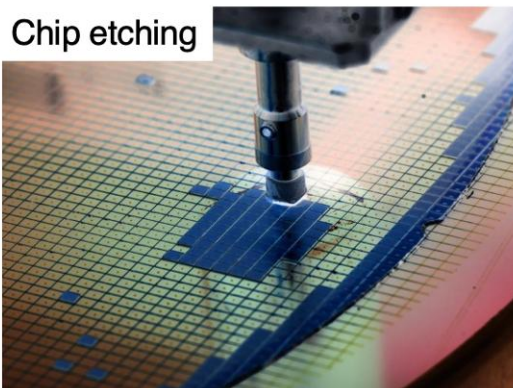
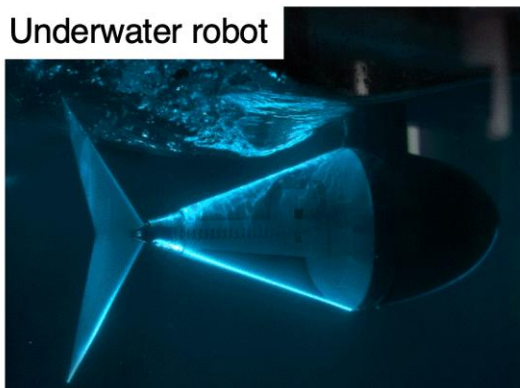
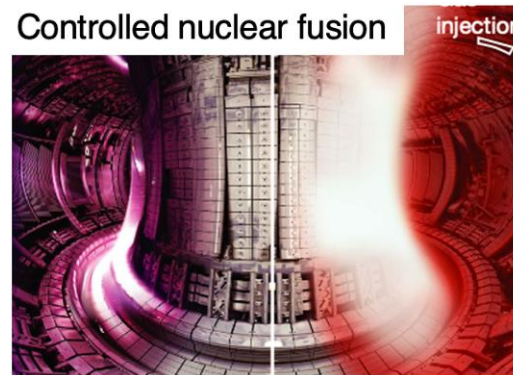
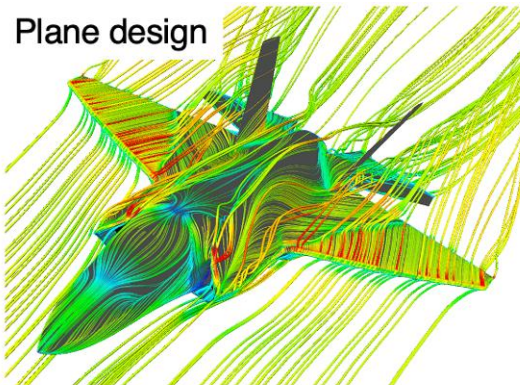


@New Scientist





These three tasks are fundamental in **science** and **engineering**

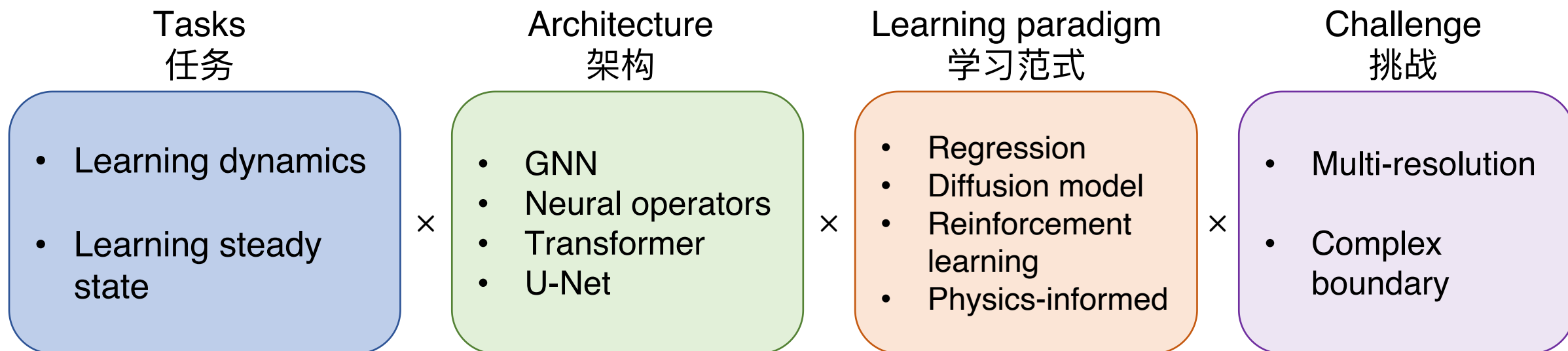


AI for scientific simulation

Scientific simulation: simulate the **dynamics** or **steady state** of the system, given initial state, boundary condition and parameters of the system

AI for scientific simulation: develop machine learning (ML) methods for scientific simulation, improving its **speed** and/or **accuracy**

AI for scientific simulation



以上各个部分为**正交**关系，即任何任务可选择合适的架构和学习范式。它们都有可能面临多分辨率和复杂边界条件等挑战

AI for scientific simulation

两类任务：学习系统动力学和稳态

两个重要架构：图神经网络和神经算子

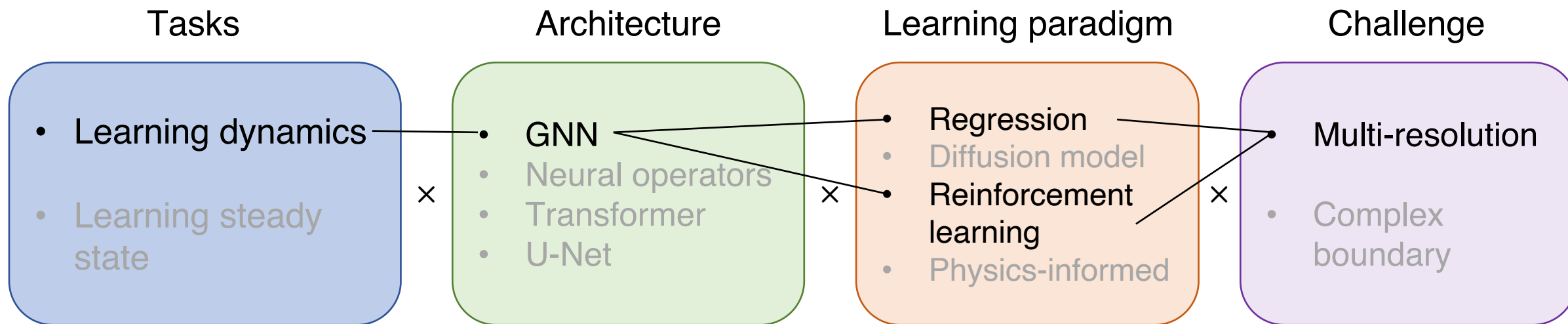
两个挑战和解决方式：多分辨率和复杂边界条件

AI for scientific simulation

两类任务：学习系统动力学和稳态

两个重要架构：图神经网络和神经算子

两个挑战和解决方式：多分辨率和复杂边界条件

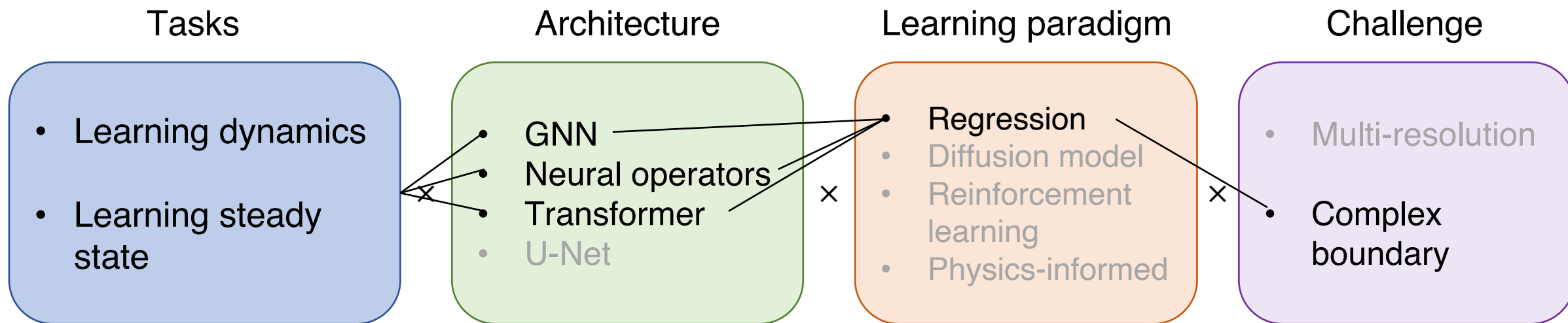


AI for scientific simulation

两类任务：学习系统动力学和稳态

两个重要架构：图神经网络和神经算子

两个挑战和解决方式：多分辨率和复杂边界条件

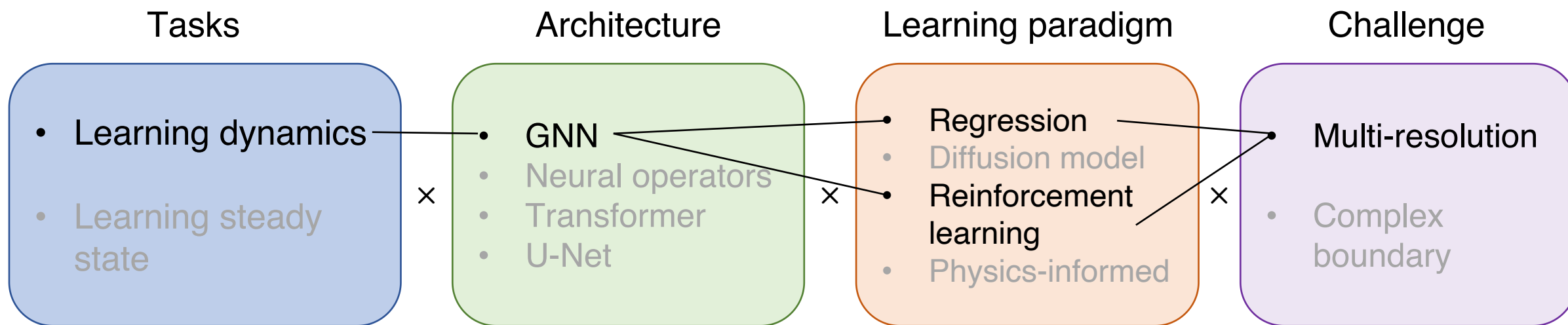


AI for scientific simulation

两类任务：学习系统动力学和稳态

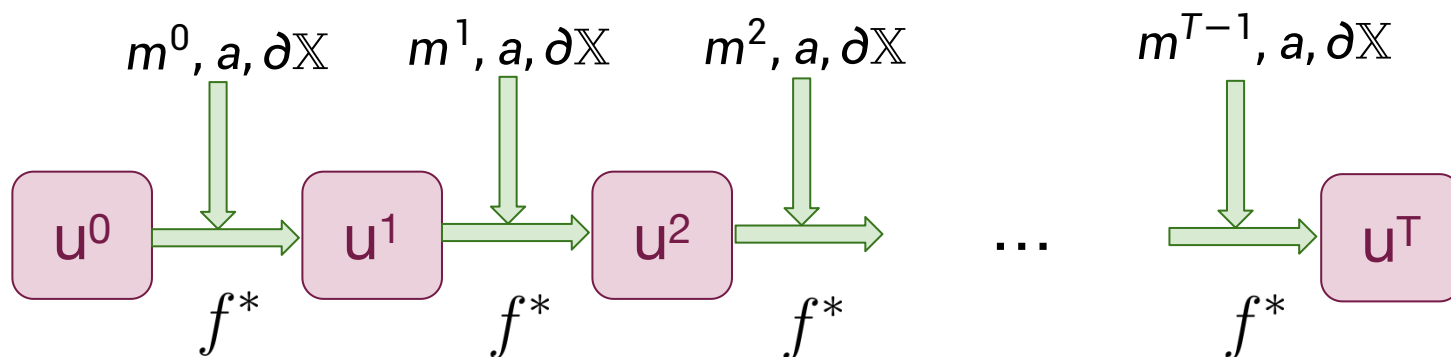
两个重要架构：图神经网络和神经算子

两个挑战和解决方式：多分辨率和复杂边界条件



Task setup for evolving dynamics

Goal: 根据状态 u^t 预测未来状态 $u^{t+1}, u^{t+2}, \dots$:



u^t : original **state** (状态) of the system. Can be an infinite-dimensional function $u(t, x)$ as solution to a PDE, or a graph (e.g., mesh, particle-based systems, molecules)

f^* : Evolution (系统演化). By a classical solver (求解器) or in the real world (真实观测)

m^t : **external control** (外界控制)

a : **static parameters** (静态参数) of the system that does not change with time (e.g. parameters of PDE, spatially varying diffusion coefficient)

$\partial\mathbb{X}$: **boundary condition** (边界条件) of the system

PDE: partial differential equation

Spectrum of methods for simulating dynamics

First principle

Data driven



Classical solver

Accurate;
Interpretable;
Error guarantee

Slow;
Assumption may
be incorrect

Pure deep learning

Fast ($10 - 10^4\times$);
Can directly learn
from data

Challenge in
long-term acc.

Classical solvers and limitations

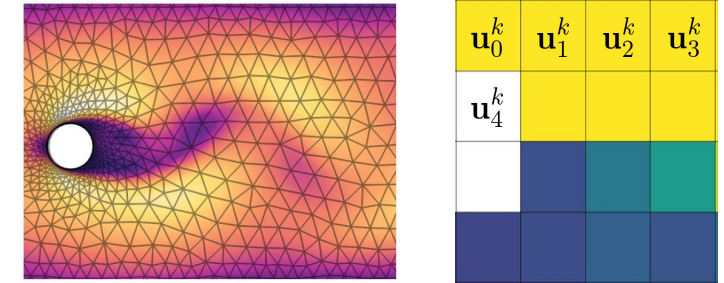
Classical solvers:

Based on Partial Differential Equations (PDEs)

$$\frac{\partial \mathbf{u}}{\partial t} = F\left(x, \mathbf{u}, \frac{\partial \mathbf{u}}{\partial \mathbf{x}}, \frac{\partial^2 \mathbf{u}}{\partial \mathbf{x}^2}, \dots\right)$$

$\mathbf{u}$: state
 $\mathbf{x}$: spatial coordinate
 t : time

Discretize the PDE, then use finite difference, finite element, finite volume, *etc.* to evolve the system.



mesh

grid

$\mathbf{u}(t, \mathbf{x}) \longrightarrow \mathbf{u}_i^k$

discrete time index k
discrete cell id i

Pros and challenges:

- **Pros:** (1) Based on first principles and interpretable, (2) accurate, (3) have error guarantee.
- **Challenges: Slow and computational expensive**, due to
 - (1) Small time interval to ensure numerical stability, or use implicit method.
 - (2) For multi-resolution systems, typically need to resolve to the lowest resolution

Classical solvers: Finite difference

E.g. Finite difference method, discretizes the PDE into equations how the values at each cell change with time:

$$\begin{aligned}\frac{\partial \mathbf{u}(t, \mathbf{x})}{\partial t} &\longrightarrow \frac{\mathbf{u}_i^{k+1} - \mathbf{u}_i^k}{\Delta t} \\ \frac{\partial \mathbf{u}(t, \mathbf{x})}{\partial \mathbf{x}} &\longrightarrow \frac{\mathbf{u}_{i+1}^k - \mathbf{u}_{i-1}^k}{2\Delta x} \quad \frac{\partial^2 \mathbf{u}(t, x)}{\partial \mathbf{x}^2} \longrightarrow \frac{\mathbf{u}_{i+1}^k + \mathbf{u}_{i-1}^k - 2\mathbf{u}_i^k}{\Delta x^2} \quad (1D \text{ regular grid})\end{aligned}$$

$$\partial_t \mathbf{u} = F(\mathbf{x}, \mathbf{a}, \mathbf{u}, \partial_{\mathbf{x}} \mathbf{u}, \partial_{\mathbf{x}\mathbf{x}} \mathbf{u}, \dots) \longrightarrow$$

$$\frac{\mathbf{u}_i^{k+1} - \mathbf{u}_i^k}{\Delta t} = F(\mathbf{x}_i, \mathbf{a}, \mathbf{u}_i^k, \frac{\mathbf{u}_{i+1}^k - \mathbf{u}_{i-1}^k}{2\Delta x}, \frac{\mathbf{u}_{i+1}^k + \mathbf{u}_{i-1}^k - 2\mathbf{u}_i^k}{\Delta x^2}) \quad i = 1, 2, \dots, N$$

Each cell will have one equation, to predict $U^{k+1} = \{\mathbf{u}_i^{k+1}\}_{i=1}^N$ from $U^k = \{\mathbf{u}_i^k\}_{i=1}^N$

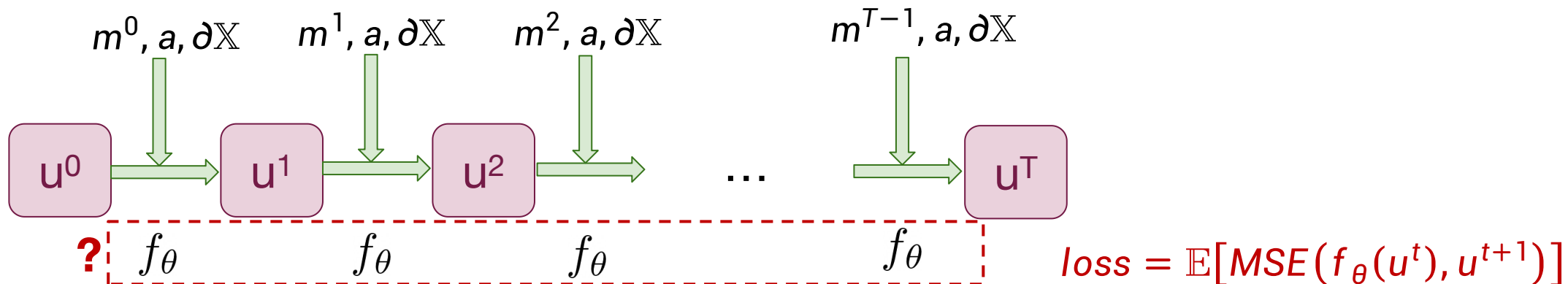
Deep learning-based surrogate models

Recently, deep learning based surrogate modeling has emerged as attractive alternative to replace or complement classical solvers. They:

- Offer **speedup (>10-1000 fold)** via:
 - Larger spatial resolution
 - Larger time intervals
 - Use explicit forward
 - Better representations

Task setup for learning dynamics

Goal: learn the mapping f_θ from u^t to u^{t+1} :



u^t : original **state** (状态) of the system. Can be an infinite-dimensional function $u(t, x)$ as solution to a PDE, or a graph (e.g., mesh, particle-based systems, molecules)

f_θ : **neural surrogate models** (神经网络代理模型)

m^t : **external control** (外界控制)

a : **static parameters** (静态参数) of the system that does not change with time (e.g. parameters of PDE, spatially varying diffusion coefficient)

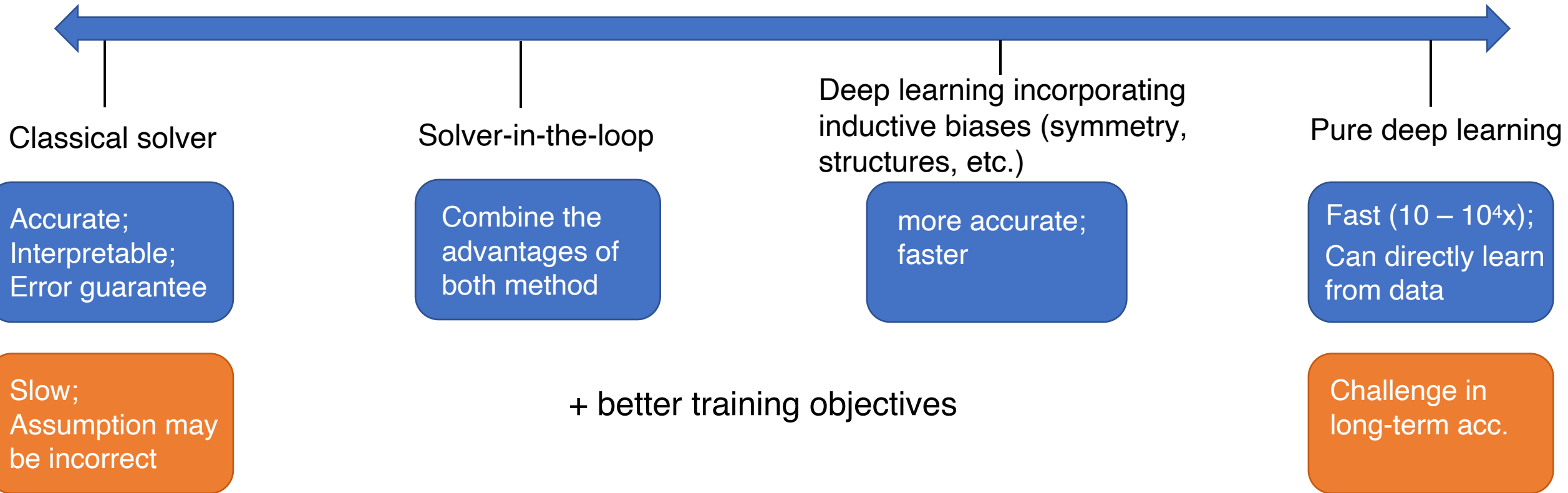
$\partial\mathbb{X}$: **boundary condition** (边界条件) of the system

PDE: partial differential equation

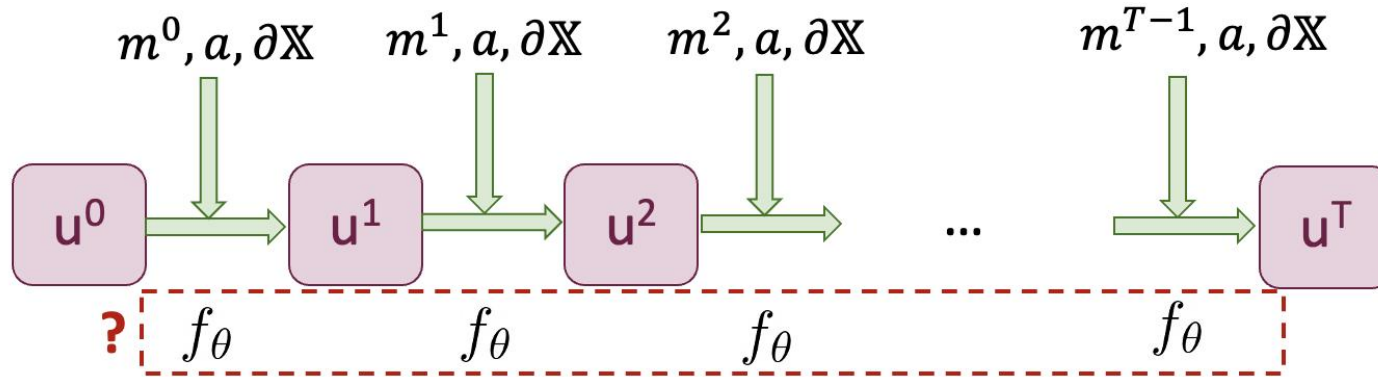
Spectrum of methods for simulating dynamics

First principle

Data driven



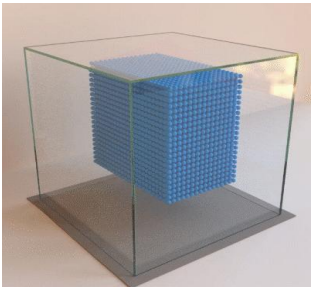
Case study: GNN (图神经网络) -based simulation



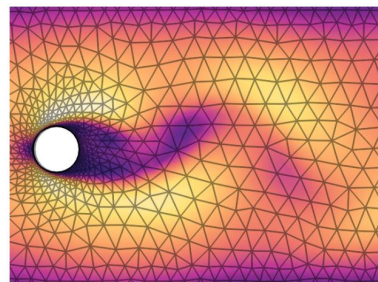
u^t : **state** of the system. Represented as a **graph** (e.g., mesh, particles, molecules)

f_θ : Graph Neural Network (GNN)

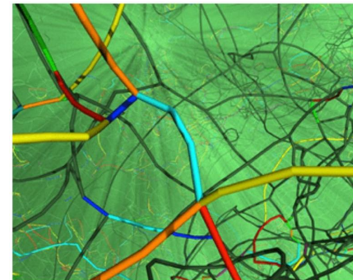
Such graph-structured data is universal across disciplines:



Fluid dynamics,
computer graphics



Mesh-based
simulation for PDEs

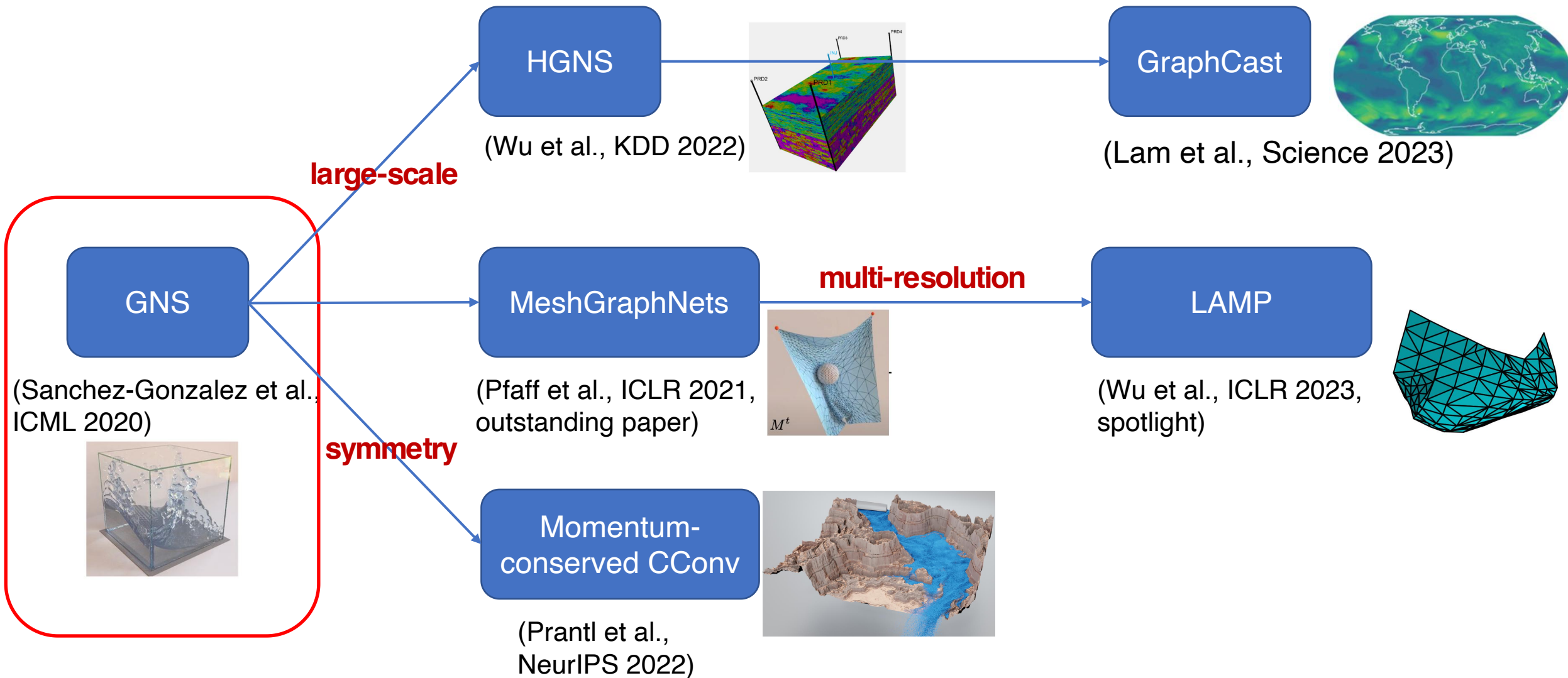


Dislocation in
materials



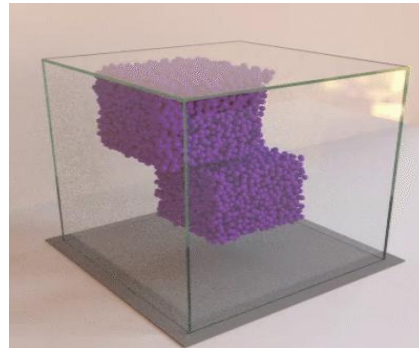
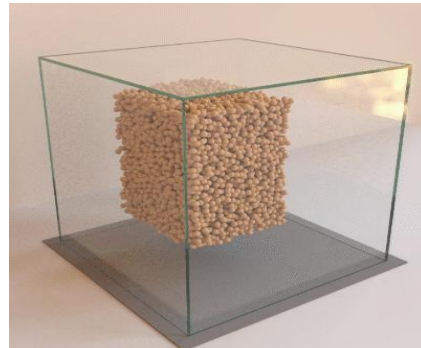
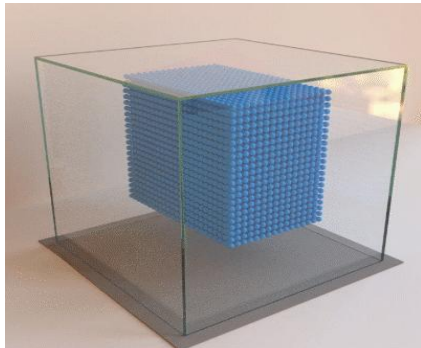
Proteins and small
molecules

Case study: GNN-based simulation

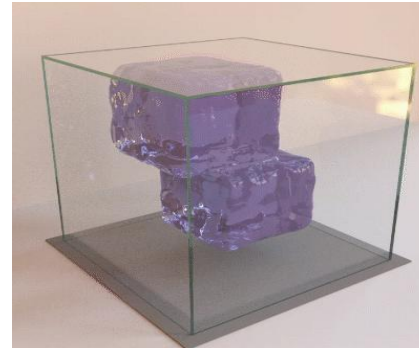
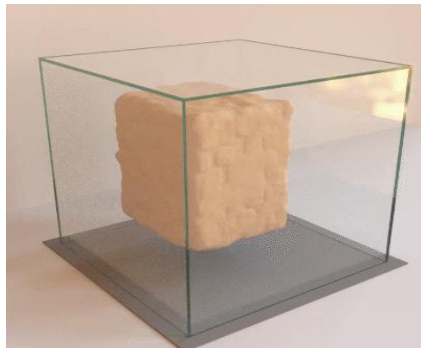
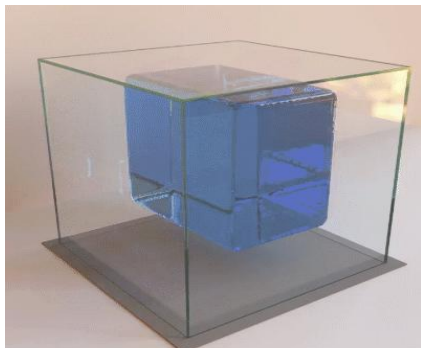


Case study: Graph Network Simulator (GNS)

Graph Network Simulator (GNS) [1] introduced a GNN-based simulator that learns to simulate particle-based systems



GNN Model predicts particle positions and velocity



Predicted simulation after rendering

[1] Sanchez-Gonzalez, Alvaro, et al. "Learning to simulate complex physics with graph networks." *International conference on machine learning*. PMLR, 2020.

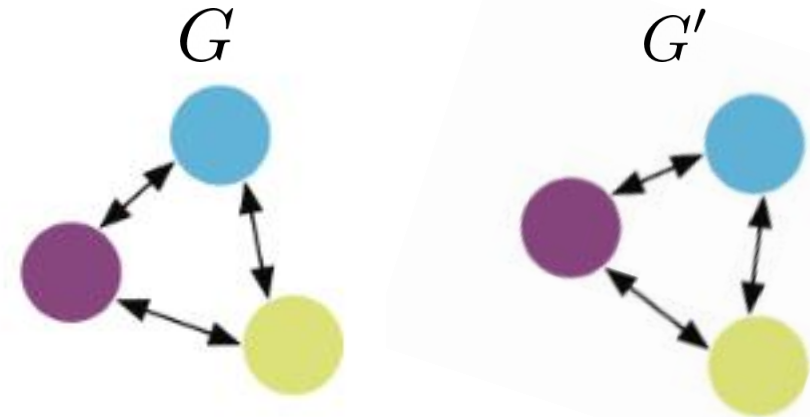
Preliminary: graph neural networks (GNNs)

A GNN f_θ takes **graph-structured data** $G=(V, E)$ as input, and typically maps to another $G'=(V', E')$:

$$G' = f_\theta(G)$$

n-body system:

1. Compute **forces** on each **edge**
2. **Accumulate** forces on each **node**
3. Update the **node feature** (position, velocity) based on the **accumulated force** and **previous node feature**



node feature: position, velocity

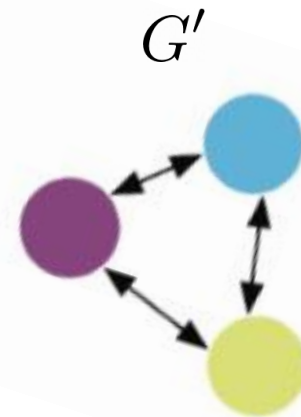
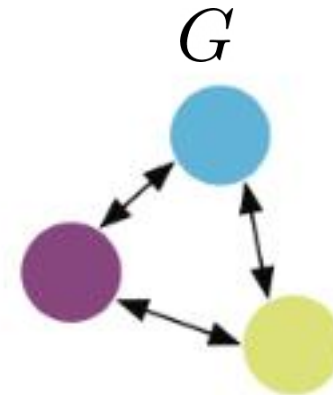
Preliminary: graph neural networks (GNNs)

A GNN f_θ takes **graph-structured data** $G=(V, E)$ as input, and typically maps to another $G'=(V', E')$:

$$G' = f_\theta(G)$$

Generic GNN:

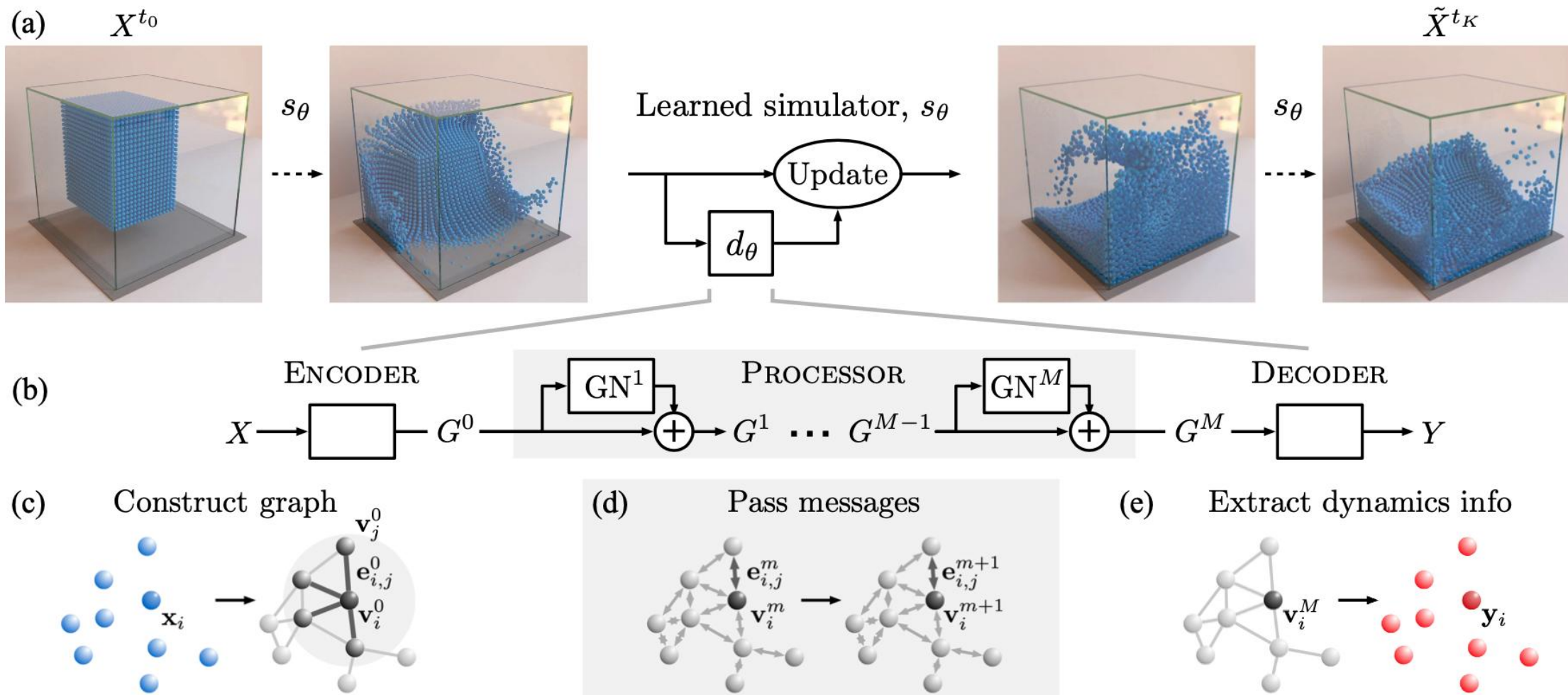
1. Compute learnable **messages** on each **edge**
2. **Accumulate messages** on each **node**
3. Perform a learnable update on the **node feature** using the **accumulated messages** and **previous node feature**



The GNN learns by minimizing the prediction loss w.r.t. the parameter θ

$$loss = \mathbb{E}[MSE(f_\theta(u^t), u^{t+1})]$$

Case study: Graph Network Simulator (GNS)



Case study: Graph Network Simulator (GNS)

Training: trained with 1-step prediction, minimizing MSE.

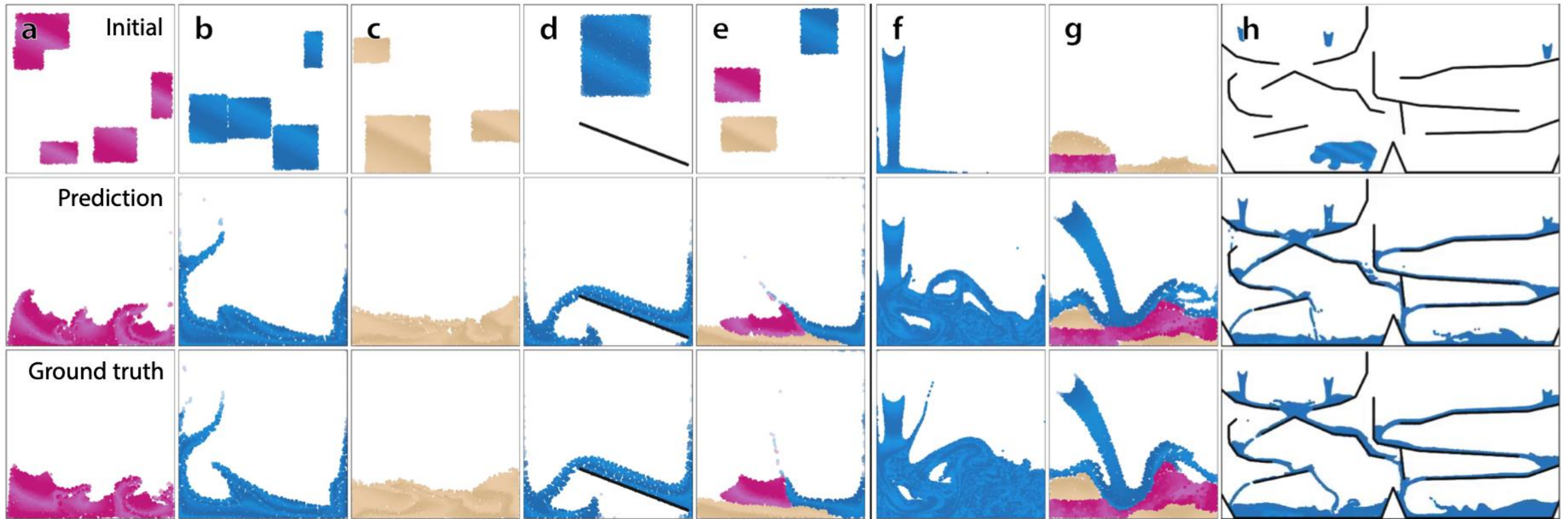
在代理模型的输入加入高斯噪音，使得训练的代理模型更加鲁棒：

$$L = \mathbb{E}[(f_{\theta}(\mathbf{u}_t + \boldsymbol{\sigma} \cdot \boldsymbol{\epsilon}) - \mathbf{u}_{t+1})^2] \quad \boldsymbol{\epsilon} \sim \mathbf{N}(\mathbf{0}, \mathbf{I}), \boldsymbol{\sigma}: \text{amplitude of noise on each feature}$$

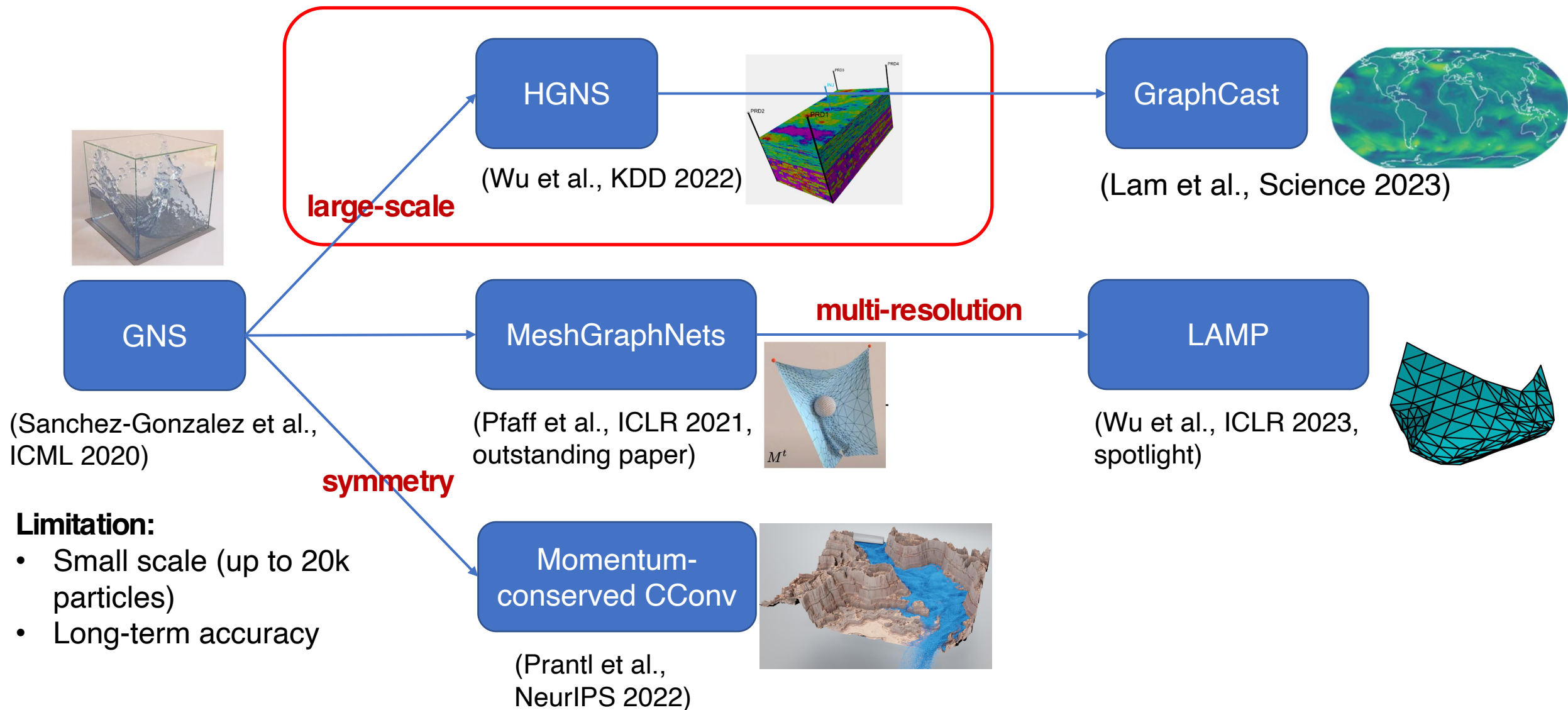
Inference: autoregressively rollout for hundreds of steps

Case study: Graph Network Simulator (GNS)

Result:



Case study: GNN-based simulation



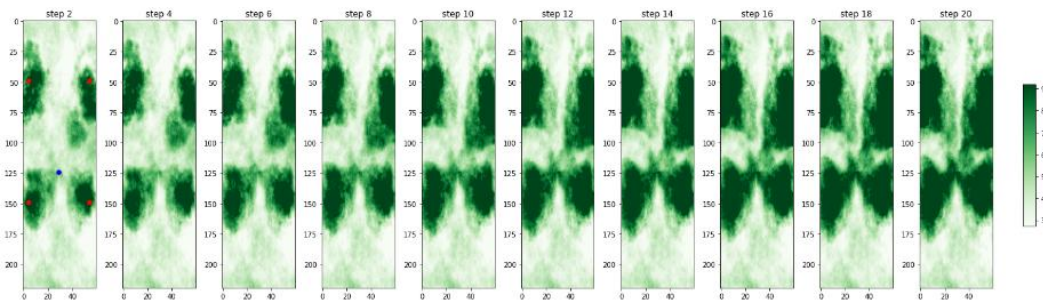
Case study: Hybrid Graph Network Simulator (HGNS)

Task: Subsurface fluid simulation (critical in energy, carbon capture, etc.)

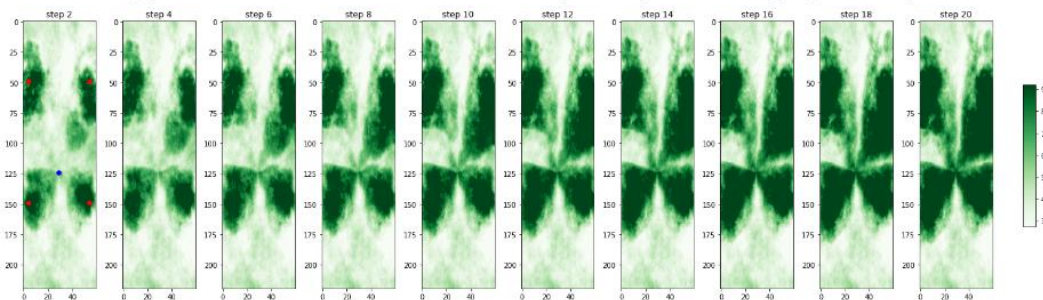
Main contribution: Introduced HGNS [1] for fluid simulation, which use

- multi-step prediction during training to improve long-term prediction accuracy
- Sector-based training and inference

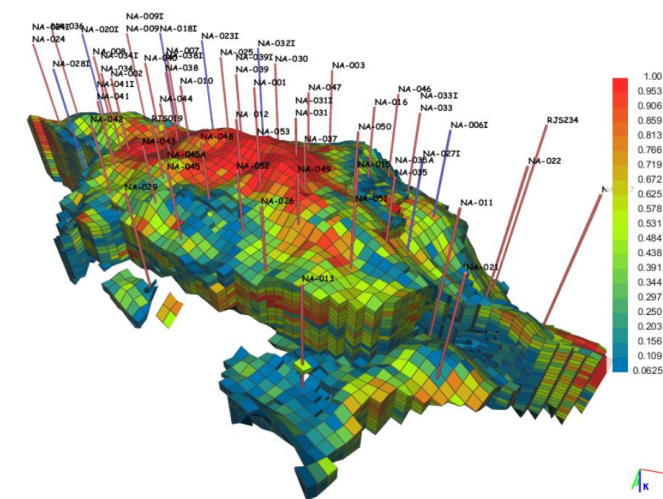
Results: Up to 18x faster than classical solver. Apply to **10 million** cells per step. Deployed in industry



(a) HGNS rollout of water volume (barrel) for 20 steps (months)

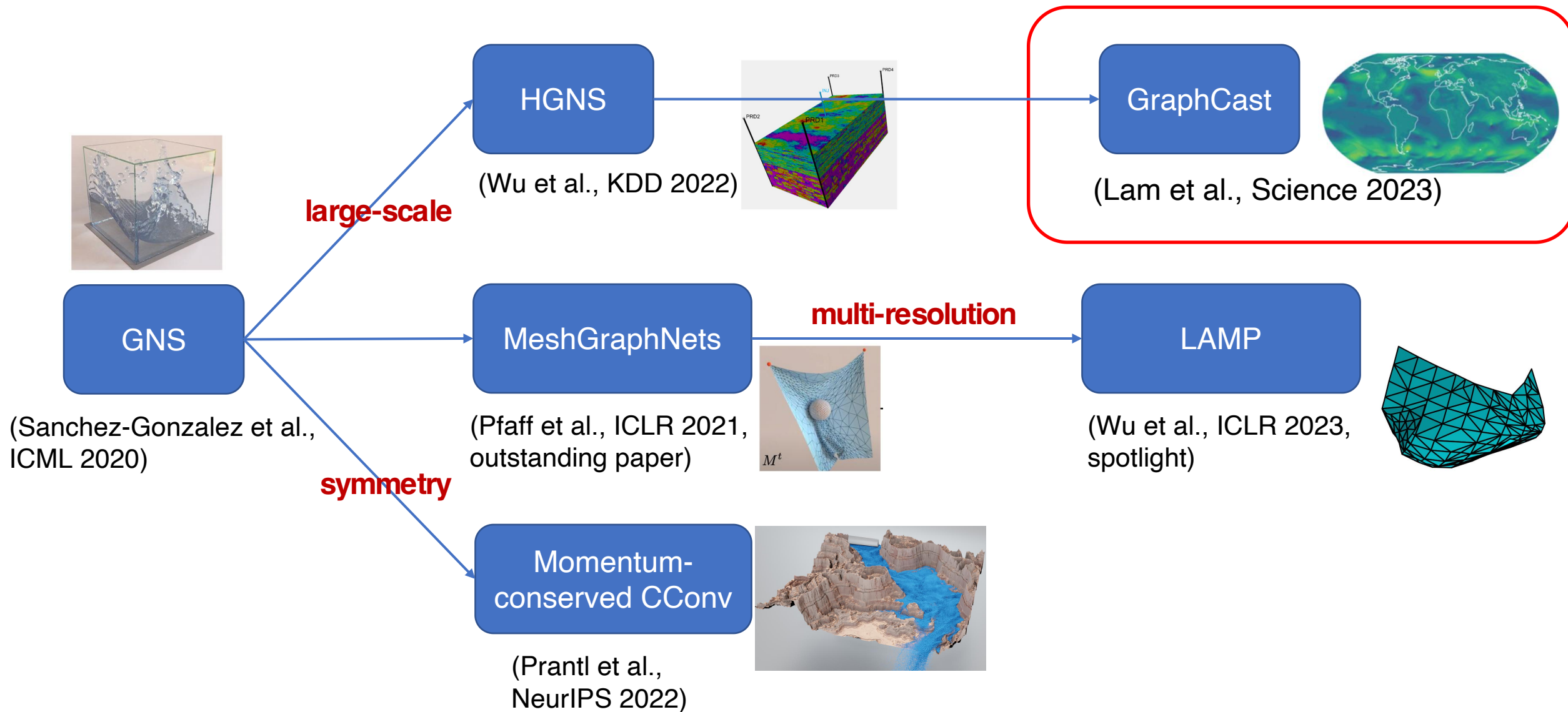


(b) Ground-truth of water volume (barrel) for 20 steps



Subsurface (consisting of cells, wells, fractures, etc.)

Case study: GNN-based simulation



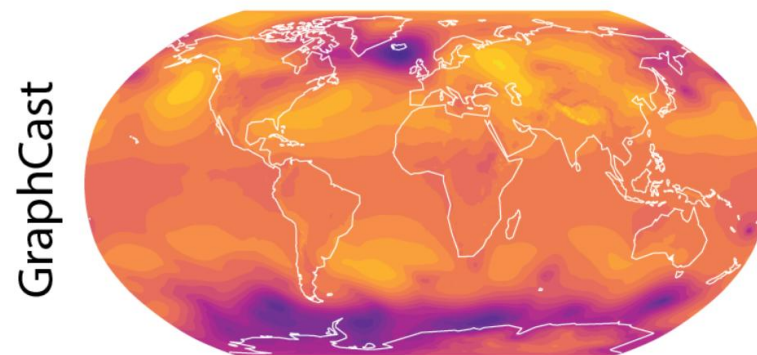
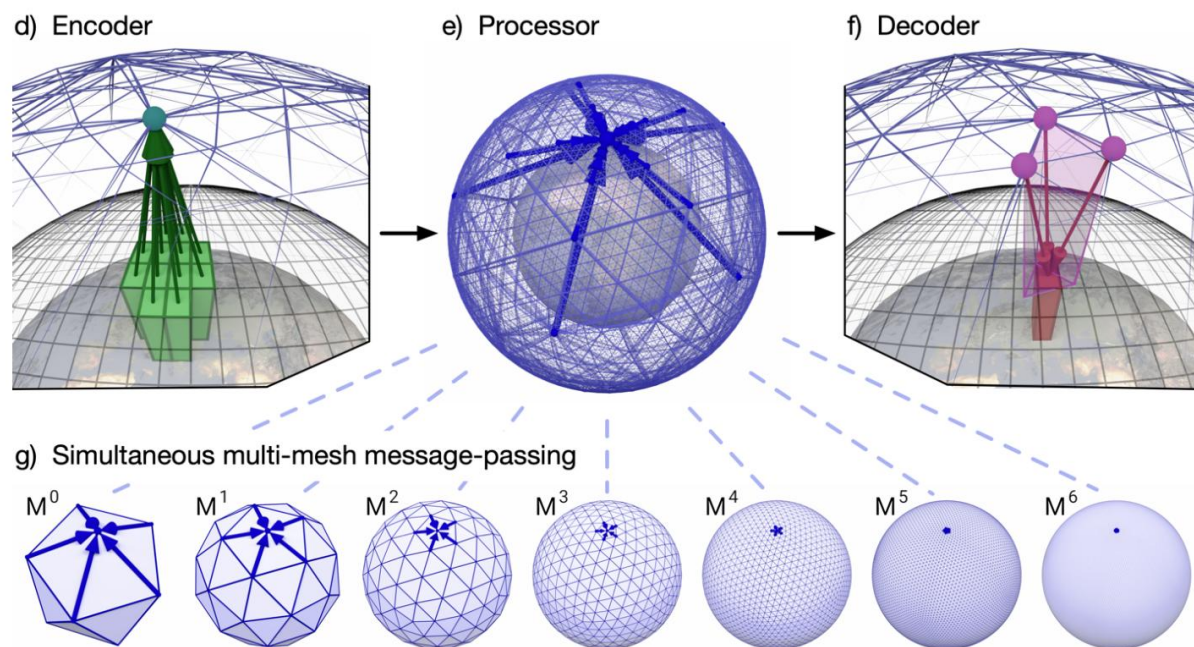
Case study: GraphCast

Task: Weather forecasting (mid-range, 10-day)

Main contribution: Introduced GraphCast [1]:

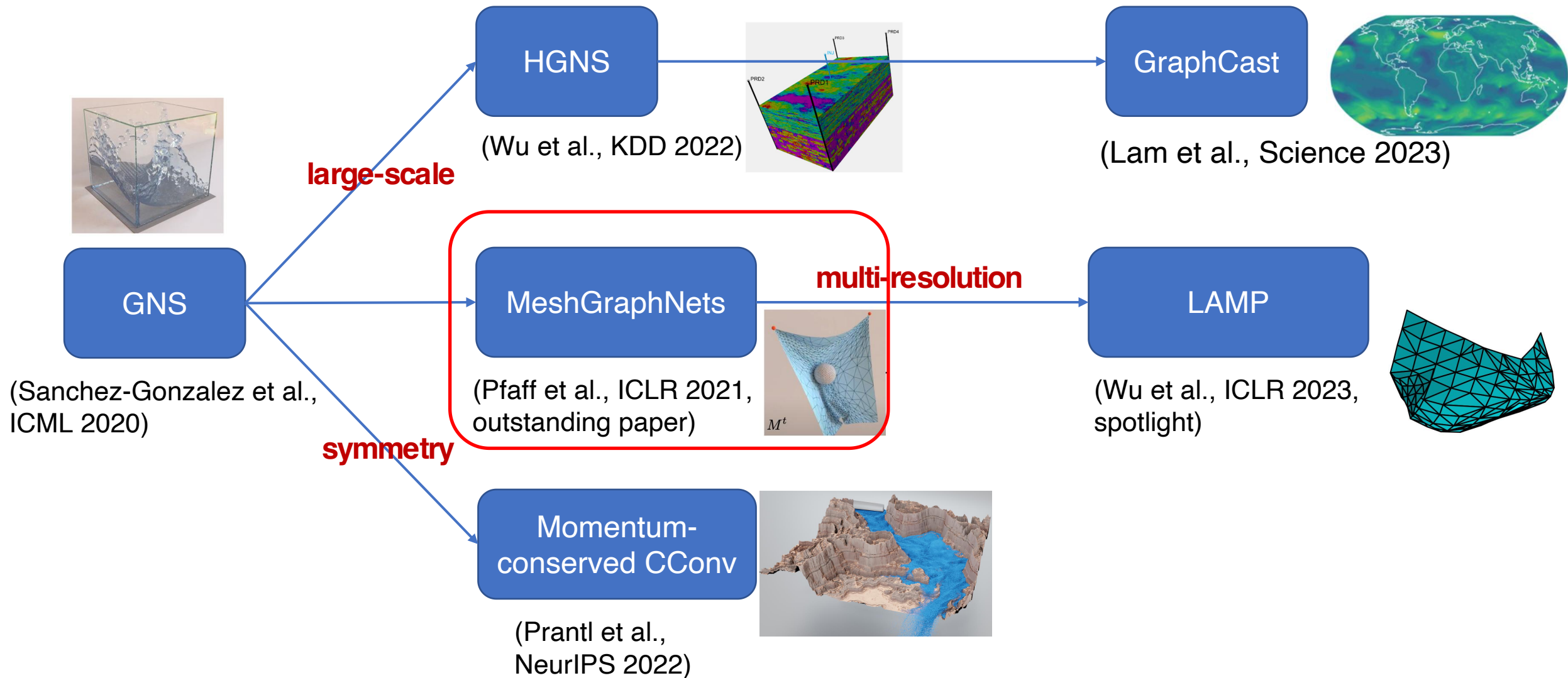
- Multi-scale GNN (多尺度图神经网络)
- Annealed multi-step learning objective

Results: outperforms state-of-the-art weather forecasting method (HRES) in 10-day prediction acc.



Prediction by GraphCast

Case study: GNN-based simulation



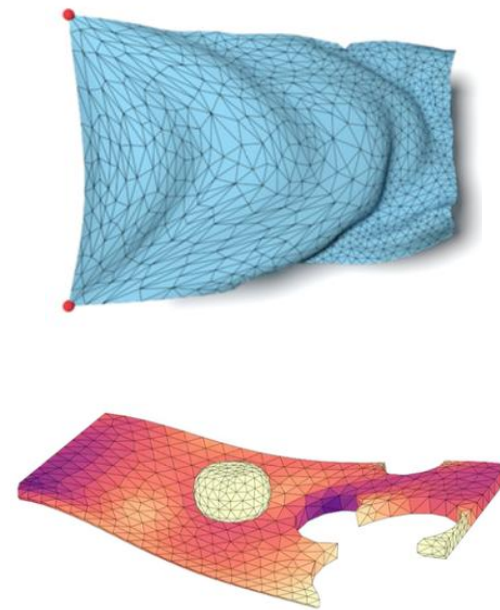
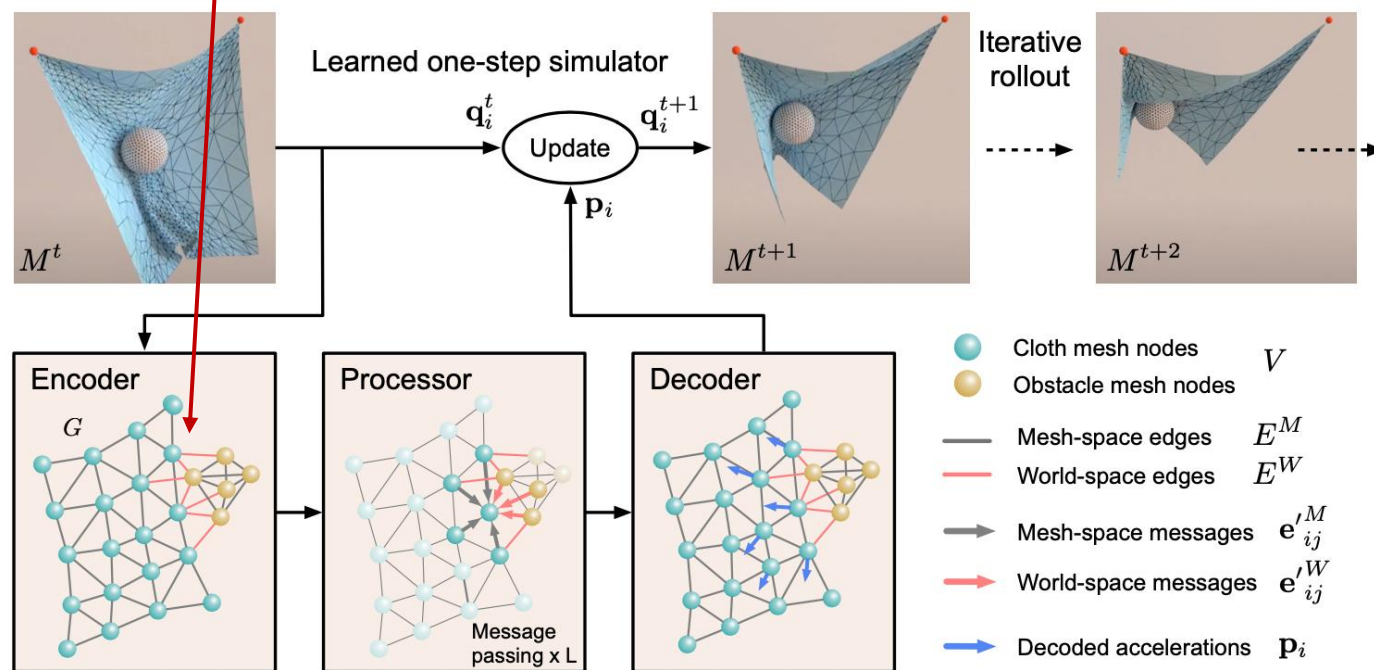
Case study: MeshGraphNets

Task: Mesh-based simulation

Main contribution: Introduced MeshGraphNets [1]:

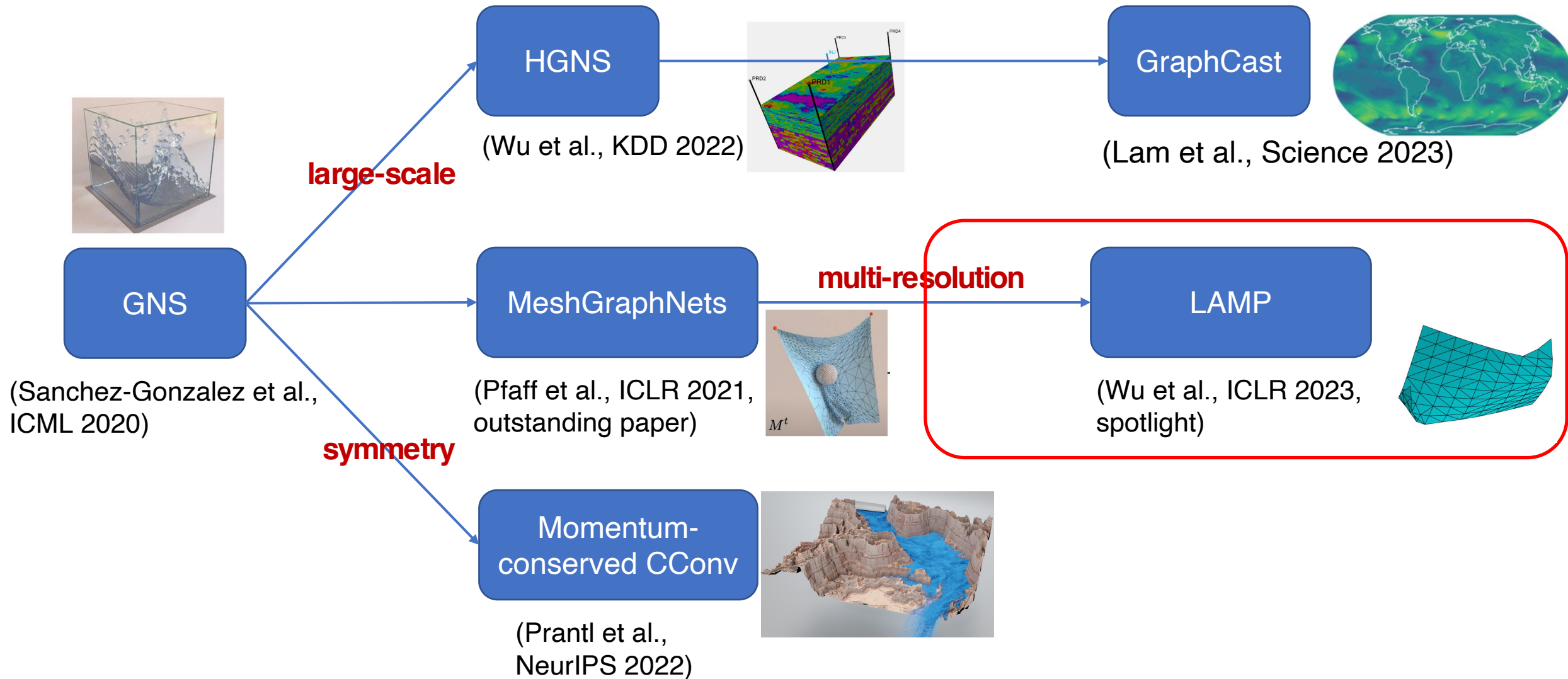
- **Two types of edges** (mesh-space and world-space edges)
- Supervised remeshing

Results: accurate prediction on many different systems.



Example predictions

Case study: GNN-based simulation



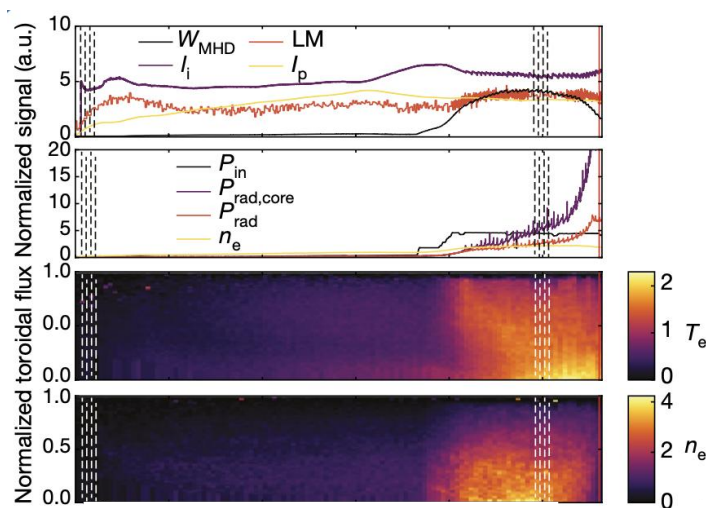
Simulating multi-resolution dynamics: significance

How to simulate a **multi-resolution** (多分辨率) dynamical system in an **accurate** and **efficient** way. (less computation)

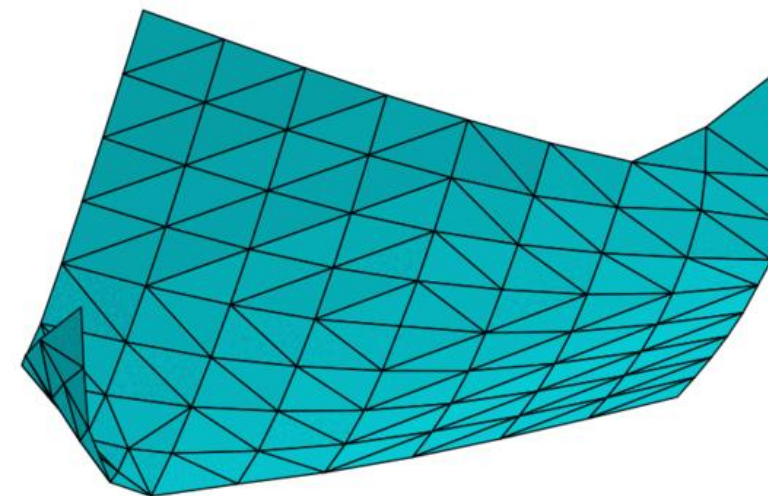
Multi-resolution systems are prevalent across different disciplines, where a small subset of the system is highly dynamic, and requires delicate simulation



Weather prediction



Disruptive instabilities in controlled fusion plasmas



Simulating cloth

Limitation of prior methods for multi-resolution challenges

However, current methods are **insufficient** to address the **multi-resolution** challenges

- Today's deep learning-based surrogate models mostly optimize the prediction accuracy, **without optimizing** the computational cost
- Classical solvers use heuristics for remeshing, which is suboptimal

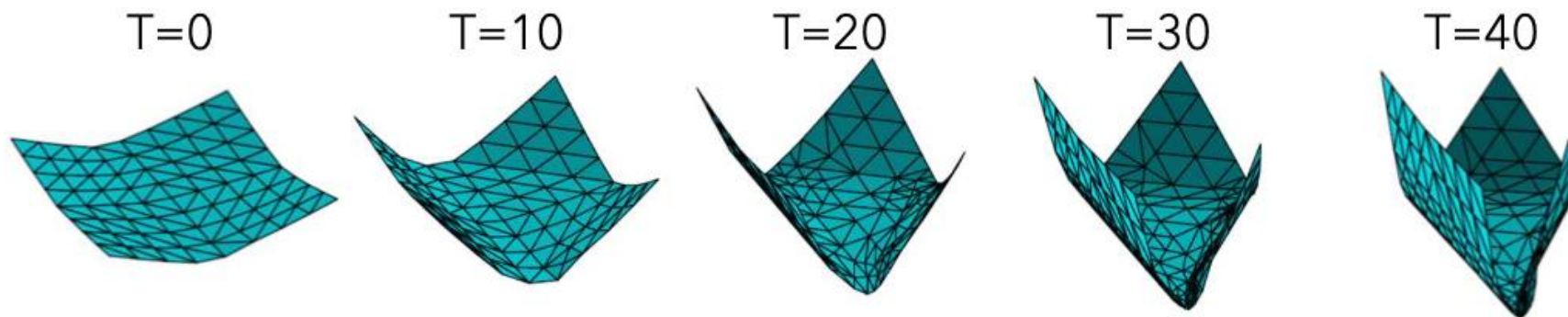
My contribution

Wu, Tailin, et al. "Learning Controllable Adaptive Simulation for Multi-resolution Physics." ICLR 2023 **notable top-25%**

We introduced the **first** deep learning-based surrogate model that jointly learns the **evolution** and optimize **computational cost**.

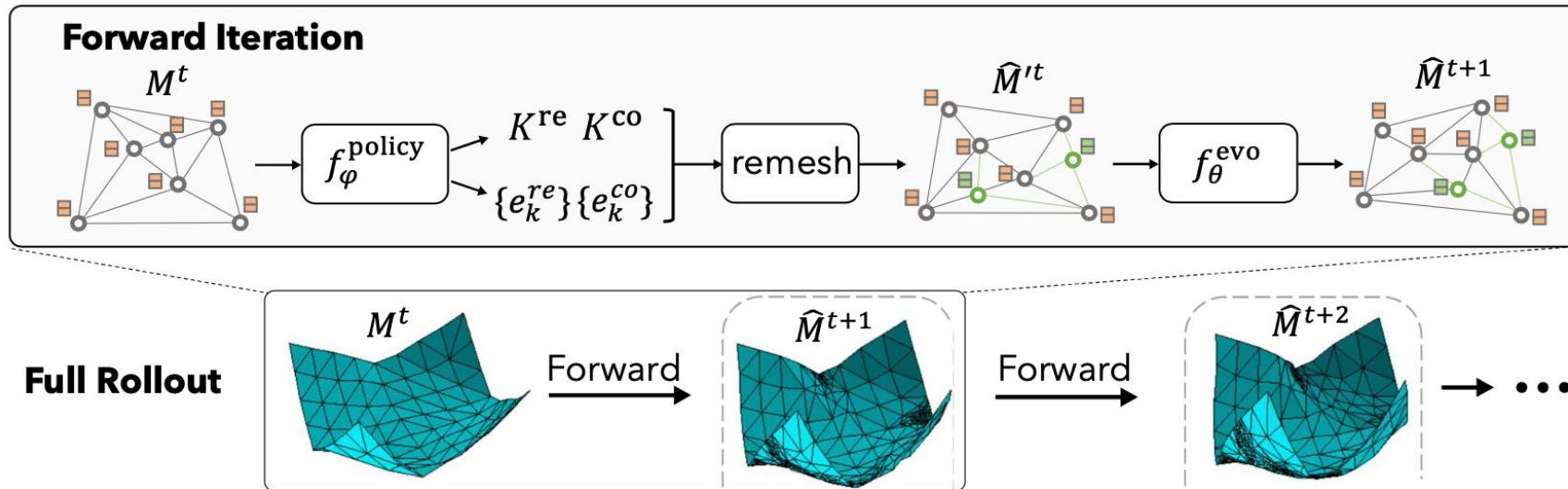
$$L = (1 - \beta) \cdot \text{Error} + \beta \cdot \text{Computation}$$

Key component: GNN-based RL agent, which learns to coarsen or refine the mesh, to achieve a controllable **tradeoff** between **prediction error** and **computational cost**.

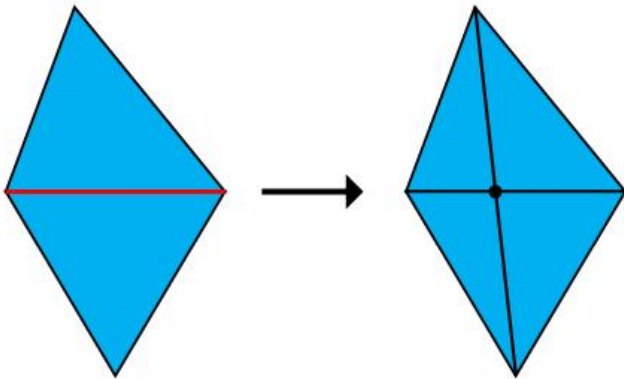


Method

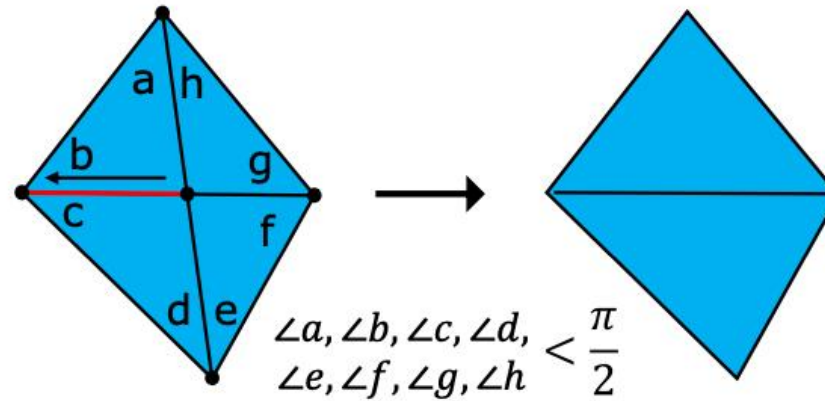
f_{θ}^{evo} : GNN-based evolution model, evolving the system while keeping the mesh topology
 $f_{\varphi}^{\text{policy}}$: GNN-based policy, which refines/coarsens the mesh based on current state and β



Action space



(1) **Refining** an edge

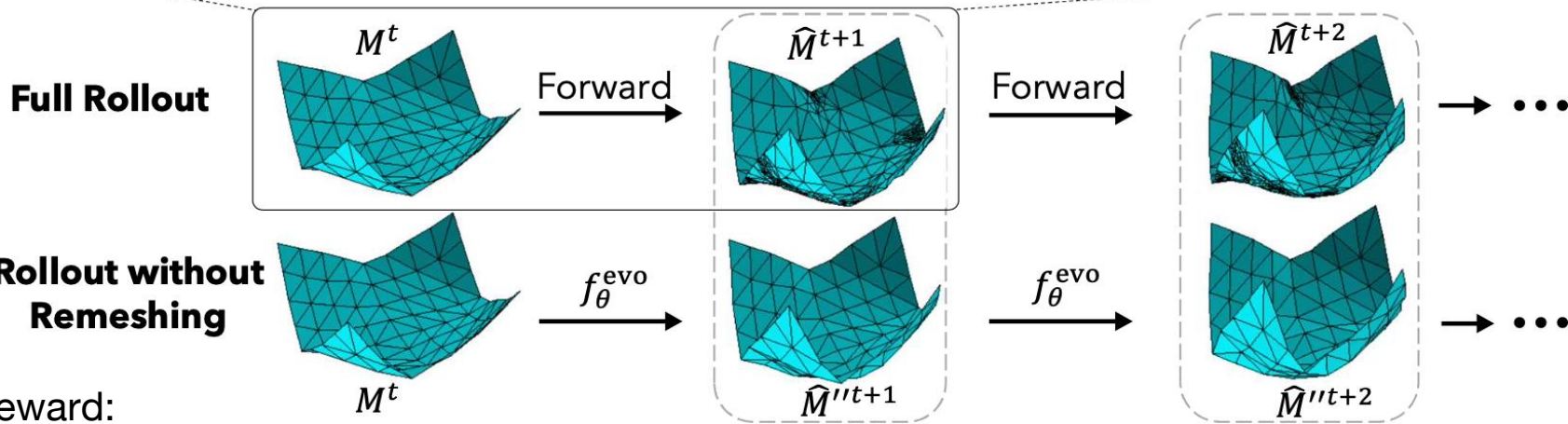
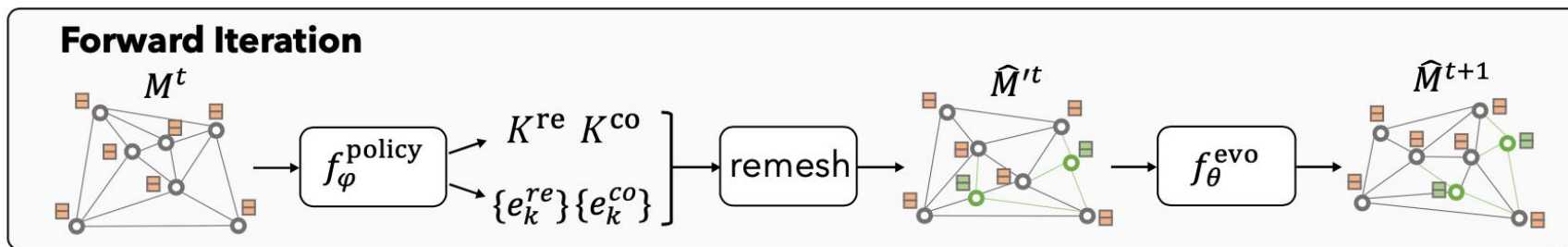


(2) **Coarsening** an edge

Such action is performed on **all** the cells simultaneously

Method

f_{θ}^{evo} : GNN-based evolution model, evolving the system while keeping the mesh topology
 $f_{\varphi}^{\text{policy}}$: GNN-based policy, which refines/coarsens the mesh based on current state and β



奖励 Reward:

$$r^t = (1 - \beta) \cdot \Delta\text{Error} + \beta \cdot \Delta\text{Computation}$$

Reward is based on the **improvement** of both **error** and **computational cost**.

- **Error** is the multi-step prediction error
- **Computational cost** is measured by number of vertices in the mesh

β is also an input to the policy

Experiment 1: Burgers' equation

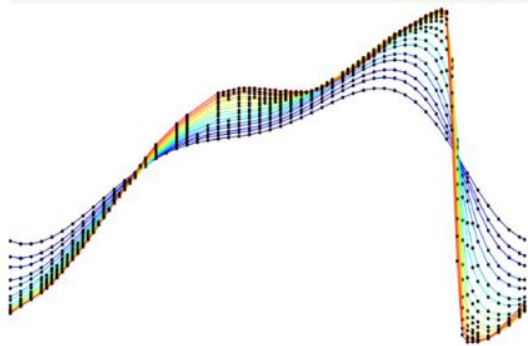
$$L = (1 - \beta) \cdot \text{Error} + \beta \cdot \text{Computation}$$

Example rollout:

Ours , $\beta = 0.1$

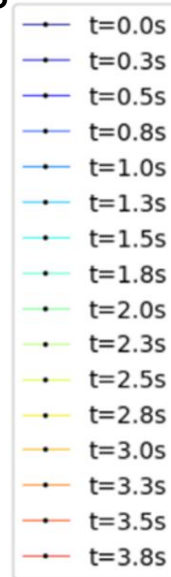
Added cells

(+34)



(-3)

removed cells



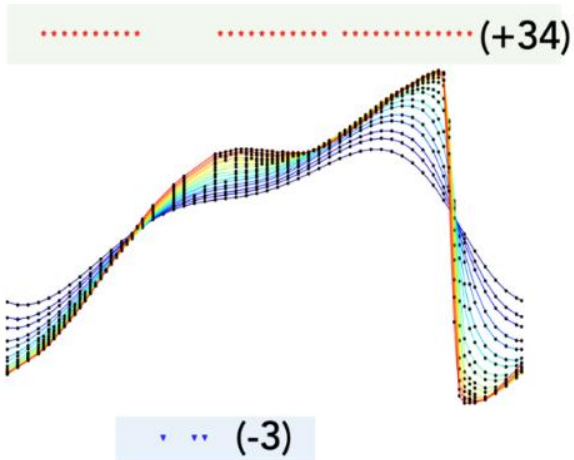
- Refines more near the shock front, and coarsens more in static regions
- With larger β that focuses more on reducing computation, it refines less and coarsens more

Experiment 1: Burgers' equation

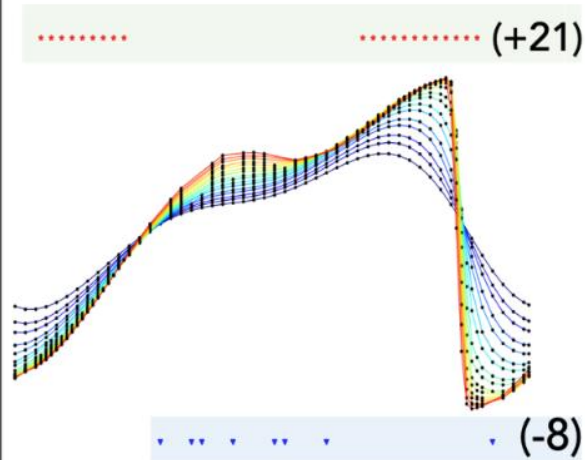
$$L = (1 - \beta) \cdot \text{Error} + \beta \cdot \text{Computation}$$

Example rollout:

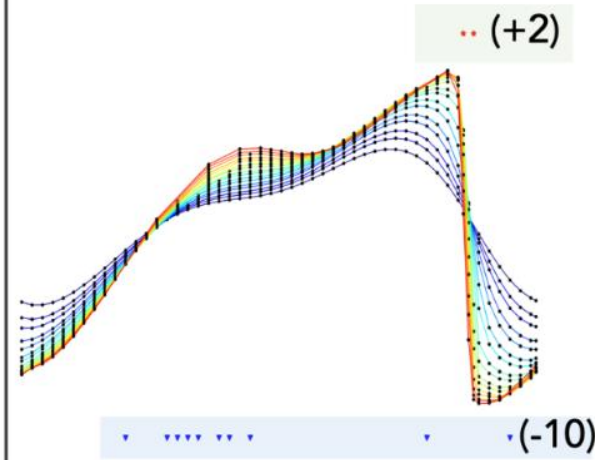
Ours , $\beta = 0.1$



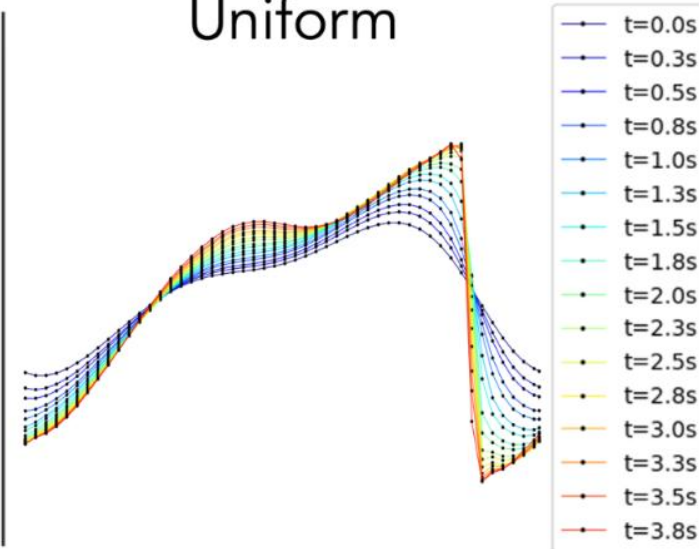
Ours , $\beta = 0.2$



Ours , $\beta = 0.7$



Uniform

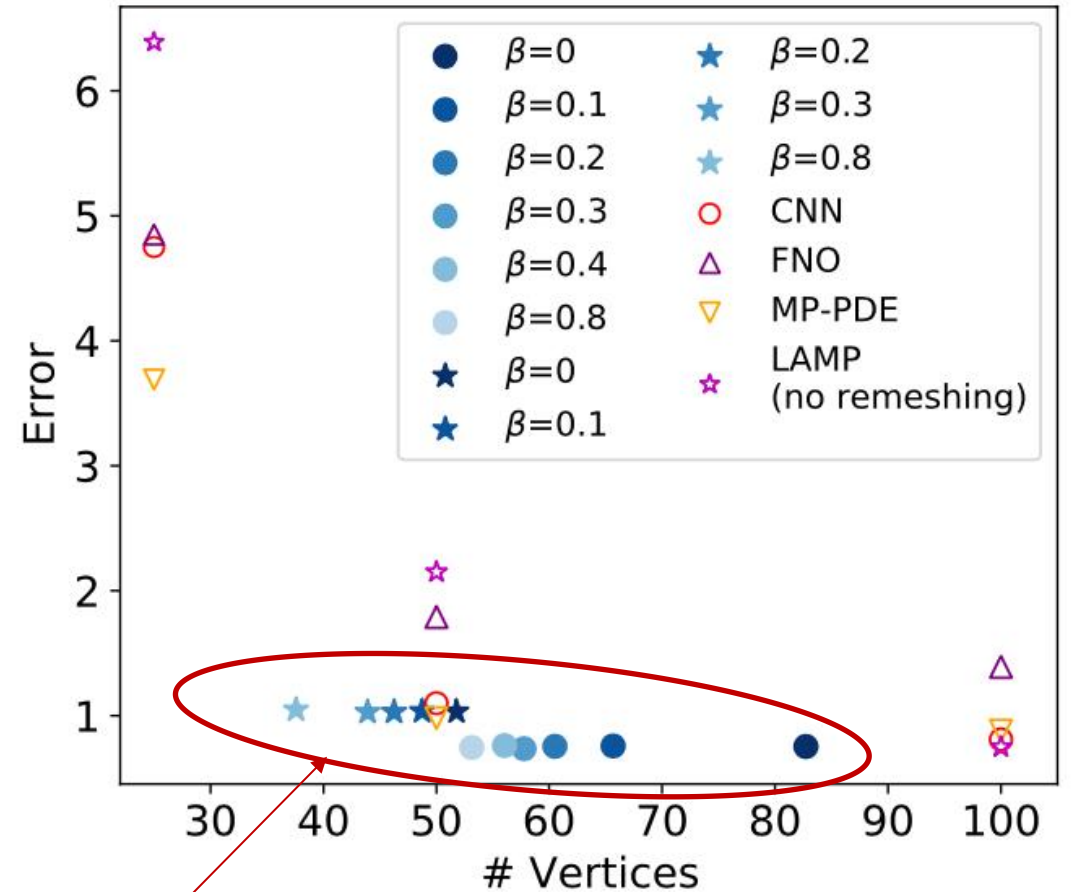


- Refines more near the shock front, and coarsens more in static regions
- With larger β that focuses more on reducing computation, it refines less and coarsens more

Experiment 1

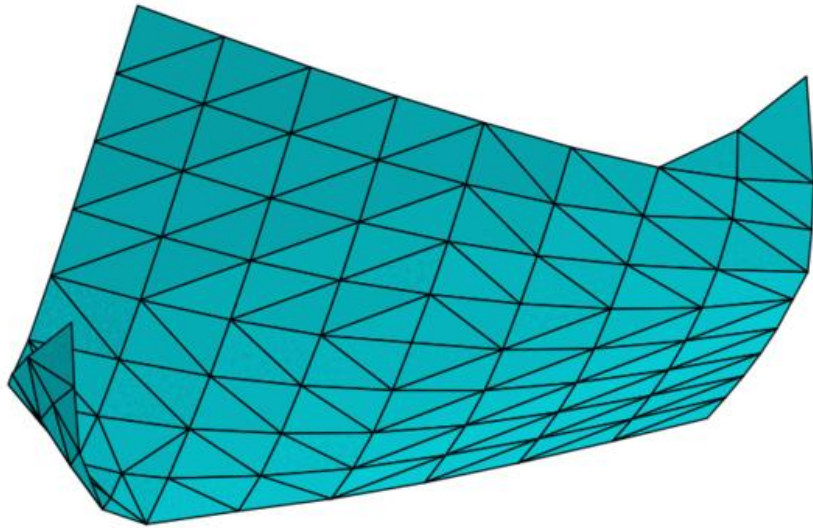
Fig. 3 shows the average error and # nodes over full test trajectories. We see that

With increasing β , LAMP improves the **Pareto frontier** over other models



Our model

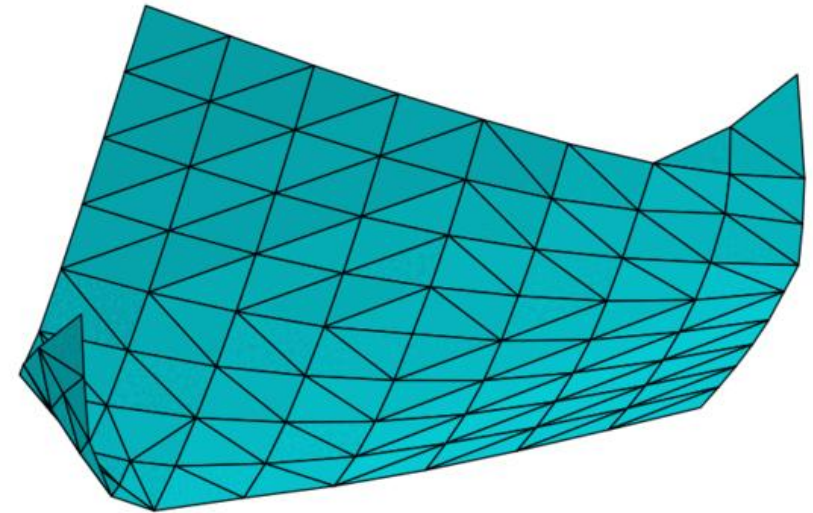
Experiment 2: mesh-based simulation visualization



**MeshGraphNets + GT
remeshing**

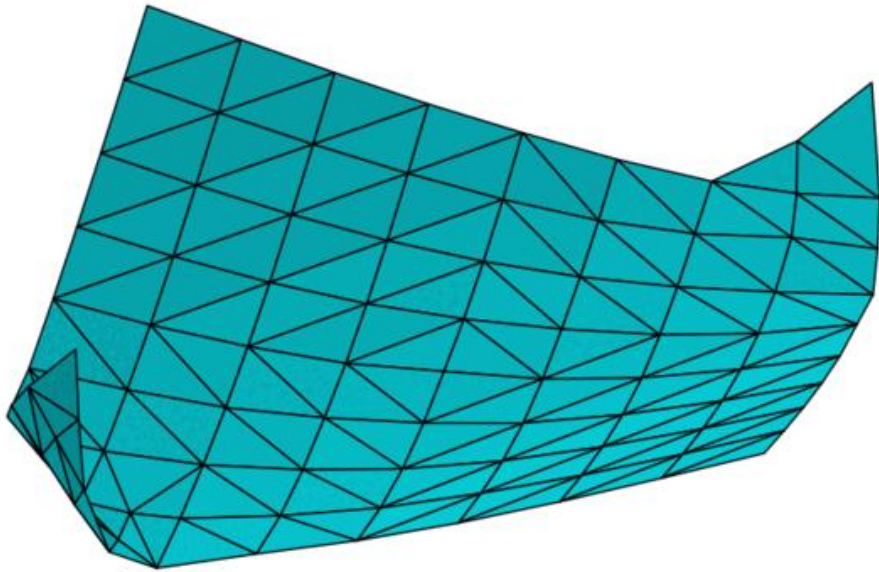
MSE: 5.91e-

4



ground-truth (fine-grained)

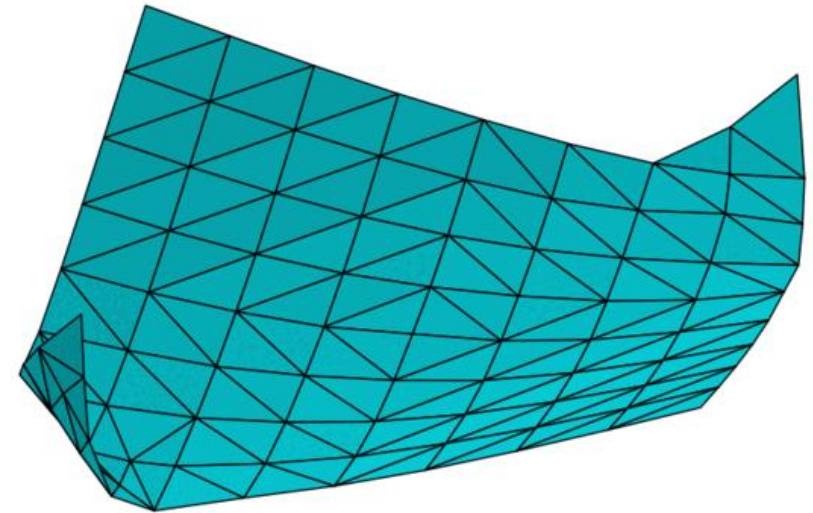
Experiment 2: mesh-based simulation visualization



**LAMP + heuristic
remeshing**

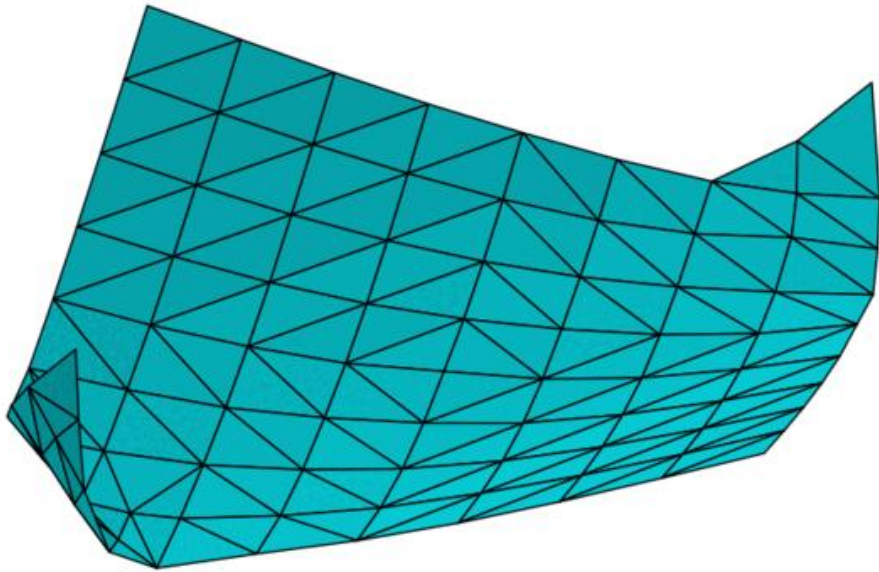
MSE: $6.38e-$

4



ground-truth (fine-grained)

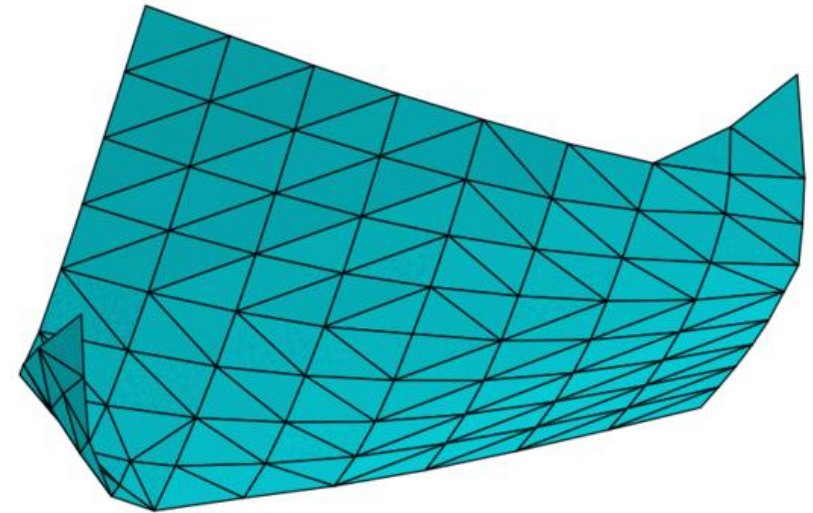
Experiment 2: mesh-based simulation visualization



**LAMP + no
remeshing**

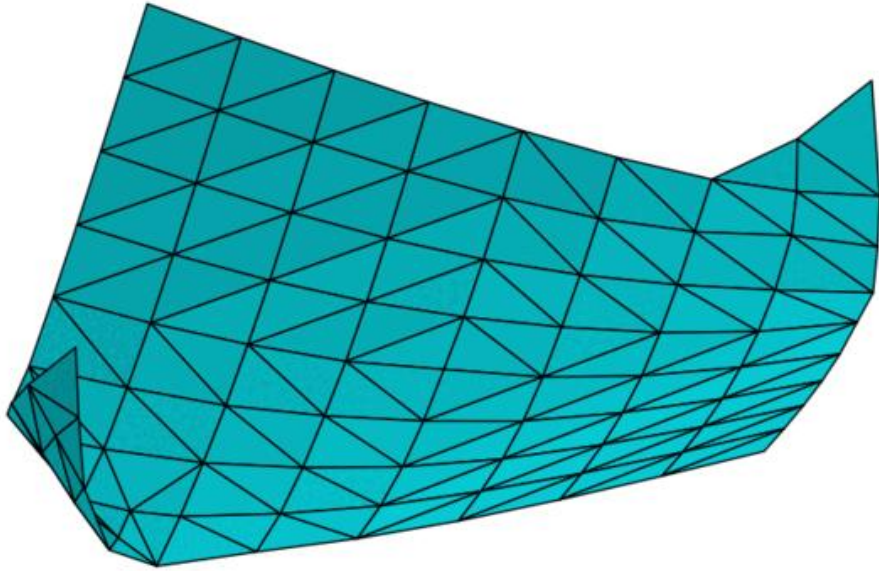
MSE: $6.13e-$

4



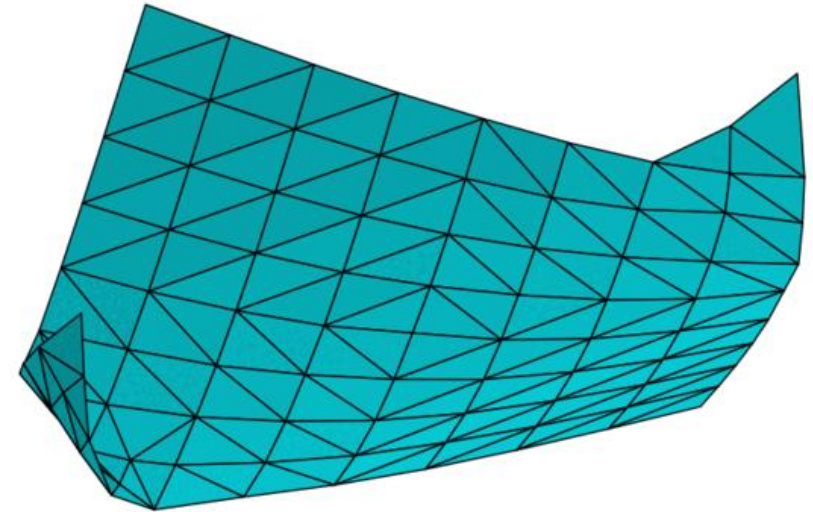
ground-truth (fine-grained)

Experiment 2: mesh-based simulation visualization



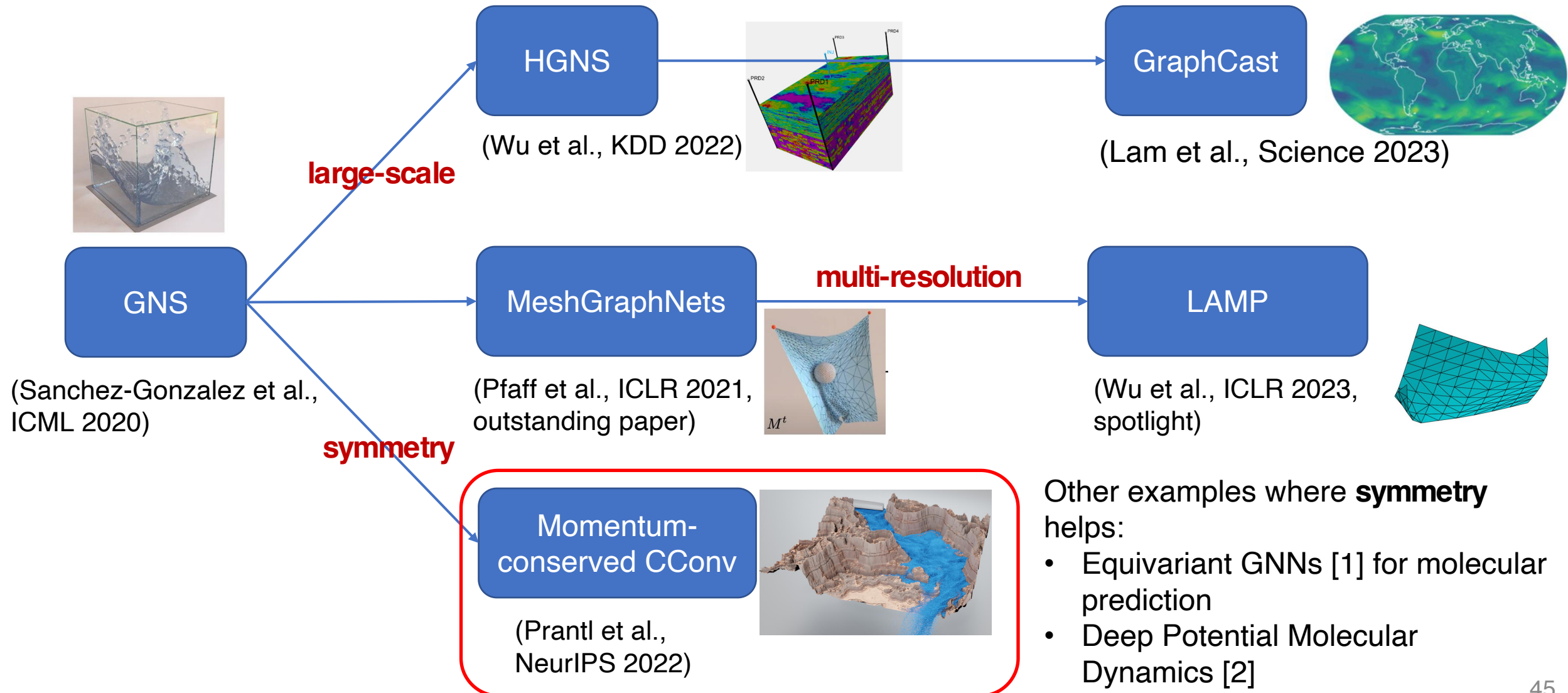
LAMP

MSE: 5.80e-
4



ground-truth (fine-grained)

Case study: GNN-based simulation



[1] Satorras, Victor Garcia, Emiel Hoogeboom, and Max Welling. "E (n) equivariant graph neural networks." *ICML*, 2021.

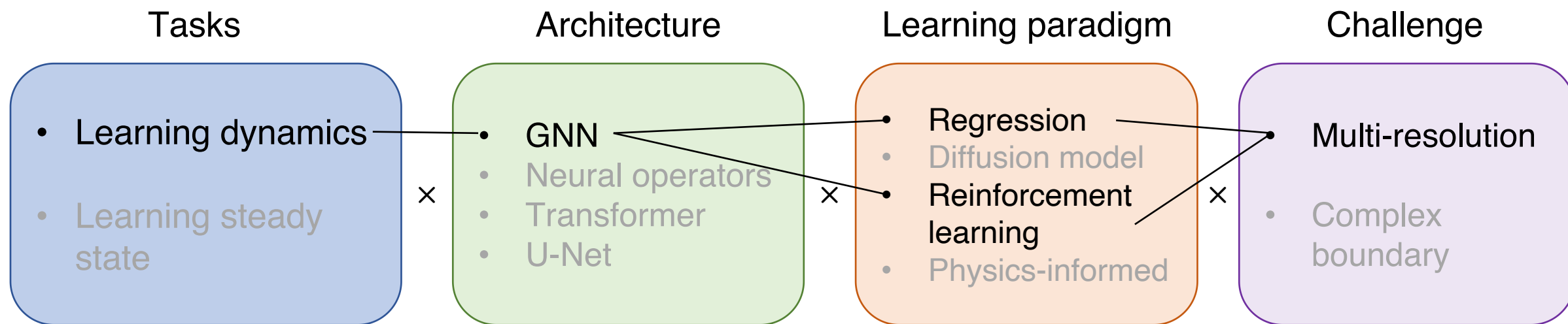
[2] Zhang, Linfeng, et al. "Deep potential molecular dynamics: a scalable model with the accuracy of quantum mechanics." *Physical review letters* 120.14 (2018): 143001.

今天报告（一）：总结

两类任务：学习系统动力学和稳态

两个重要架构：图神经网络和神经算子

两个挑战和解决方式：多分辨率和复杂边界条件

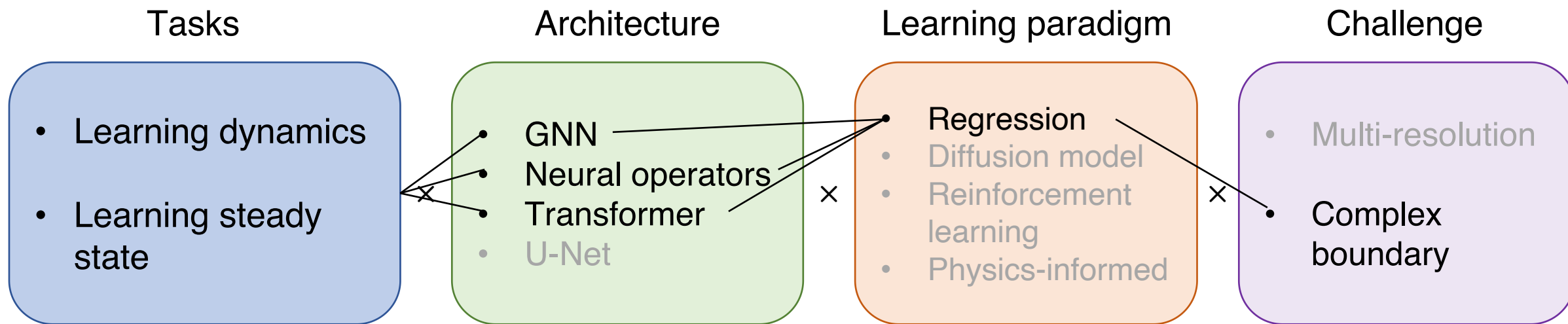


今天报告（二）

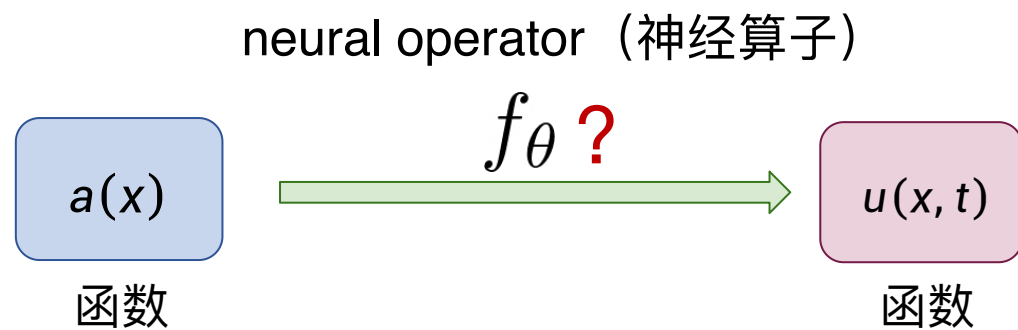
两类任务：学习系统动力学和稳态

两个重要架构：图神经网络和神经算子

两个挑战和解决方式：多分辨率和复杂边界条件

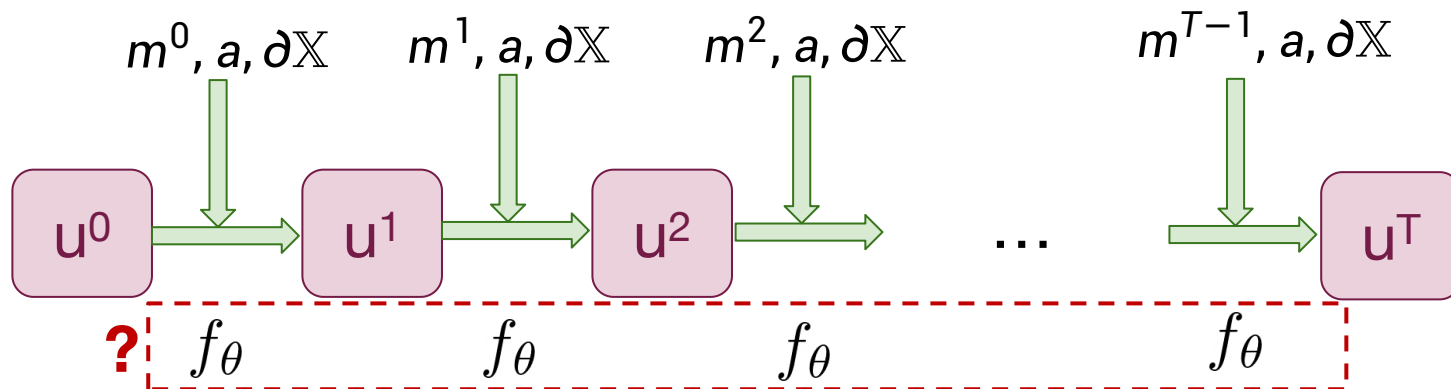


神经算子neural operator: 函数到函数的映射



Task setup 1: learning **dynamics**

Goal: learn the mapping f_θ from u^t to u^{t+1} :



u^t : original **state** (状态) of the system. Can be an **infinite-dimensional function** $u(t, x)$ as solution to a PDE, or a graph (e.g., mesh, particle-based systems, molecules)

f_θ : **neural operators** (神经算子)

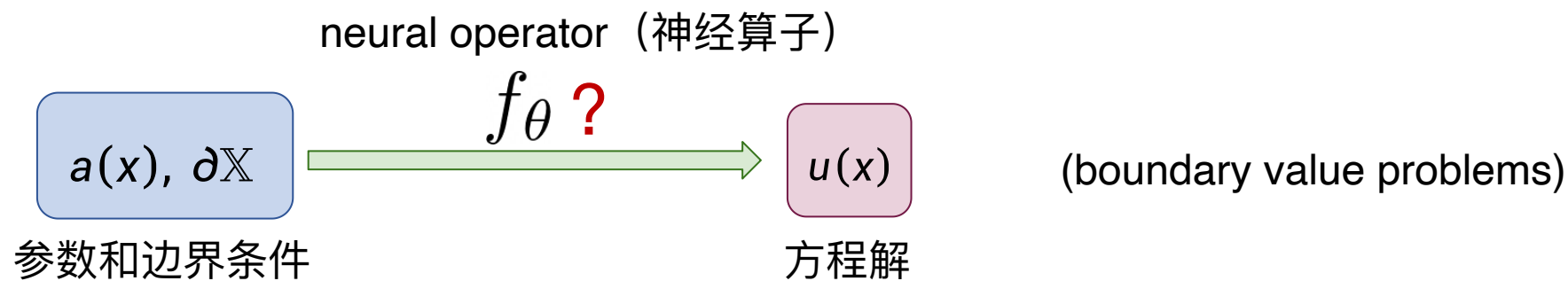
m^t : **external control** (外界控制)

a : **static parameters** (静态参数) of the system that does not change with time (e.g. parameters of PDE, spatially varying diffusion coefficient)

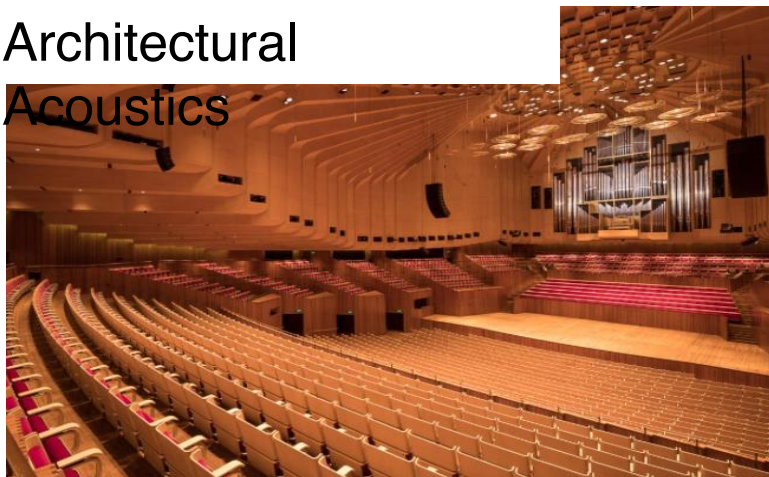
$\partial\mathbb{X}$: **boundary condition** (边界条件) of the system

PDE: partial differential equation

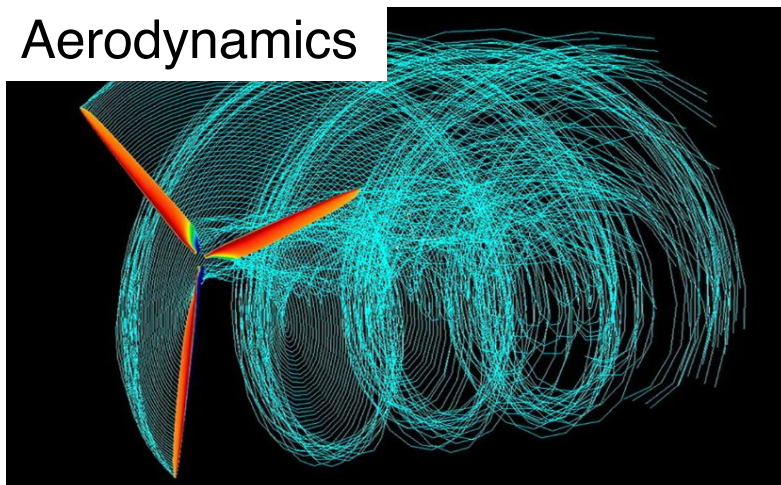
Task setup 2: learning **steady state** 学习稳态解



Architectural
Acoustics



Aerodynamics



Fracture mechanics



Elliptic PDEs (椭圆型偏微分方程)

There are three types of second-order PDEs, elliptic, parabolic (e.g., N-S equation), and hyperbolic (e.g., wave equation)

Elliptic PDEs are important across different scientific fields. Examples:

Poisson's equation 泊松方程:

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x), \quad x \in \mathbb{X}$$

$$\mathbb{B}[u(x)] = g(x), \quad x \in \partial\mathbb{X}$$

Important in materials, plasma physics, elasticity, hydrology.

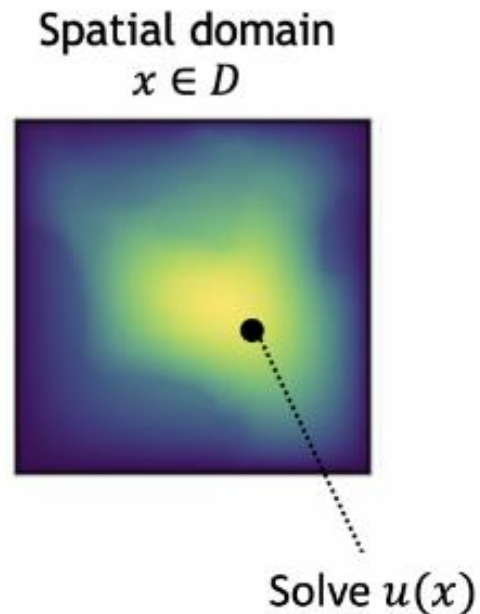
Grad-Shafranov equation:

$$-\mu_0 r^2 \frac{d}{d\Psi} - \frac{1}{2} \frac{dF^2}{d\Psi} = \frac{\partial^2 \Psi}{\partial r^2} - \frac{1}{r} \frac{\partial \Psi}{\partial z^2} + \frac{\partial^2 \Psi}{\partial z^2}$$

Important in *controlled nuclear fusion* (可控核聚变)

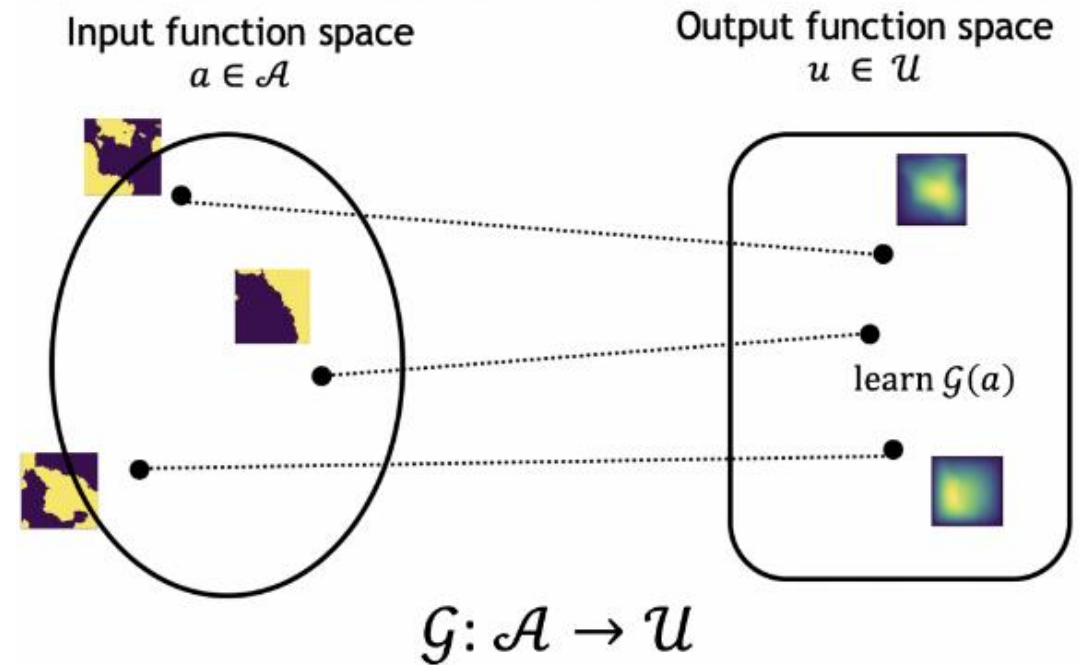
Solving vs. learning PDE

Solving for a PDE instance Numerical solvers and PINNs focus on solving one specific instance



Learn solution operator $\mathcal{G}$

Neural operators learn the solution operator for a family of equations



Zongyi Li *et al.* (2021)

Neural operators

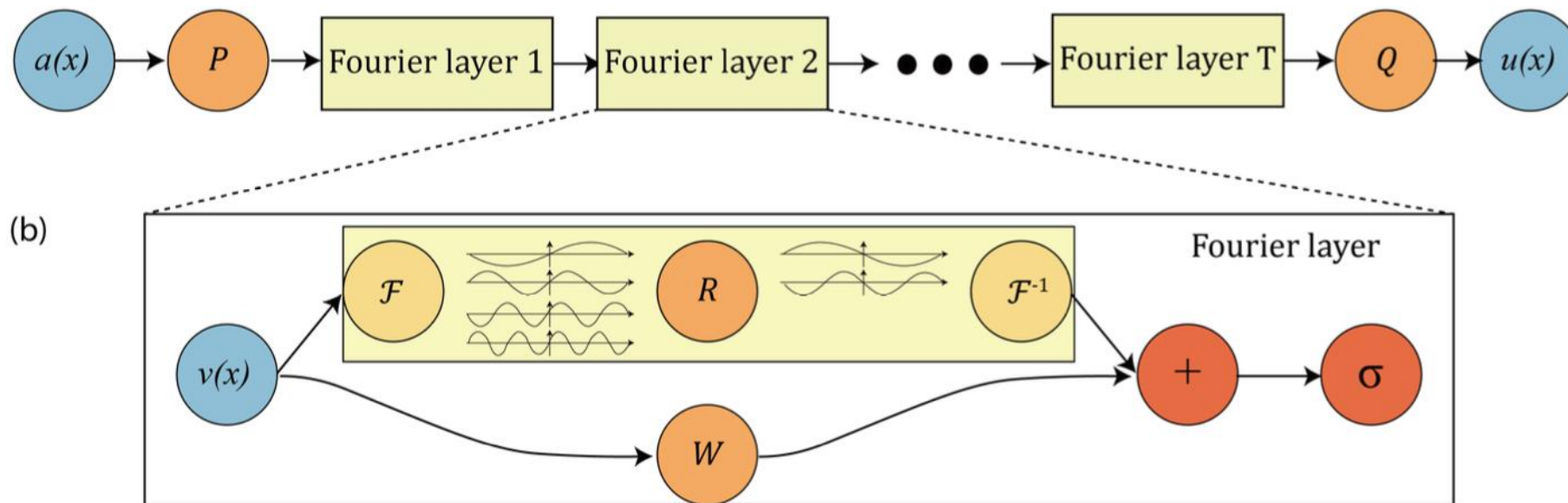
- Fourier Neural Operator (FNO)
 - Based on Fourier transformation
- Boundary-embedded Neural Operator (BENO)
 - Based on Green's function

Neural operators

- Fourier Neural Operator (FNO)
 - Based on Fourier transformation
- Boundary-embedded Neural Operator (BENO)
 - Based on Green's function

Li, Zongyi, et al. "Fourier neural operator for parametric partial differential equations." ICLR 2021, *arXiv preprint arXiv:2010.08895* (2020).

Fourier Neural Operators (FNO)



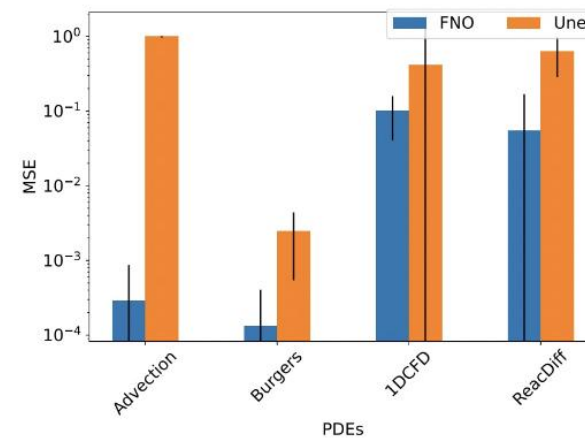
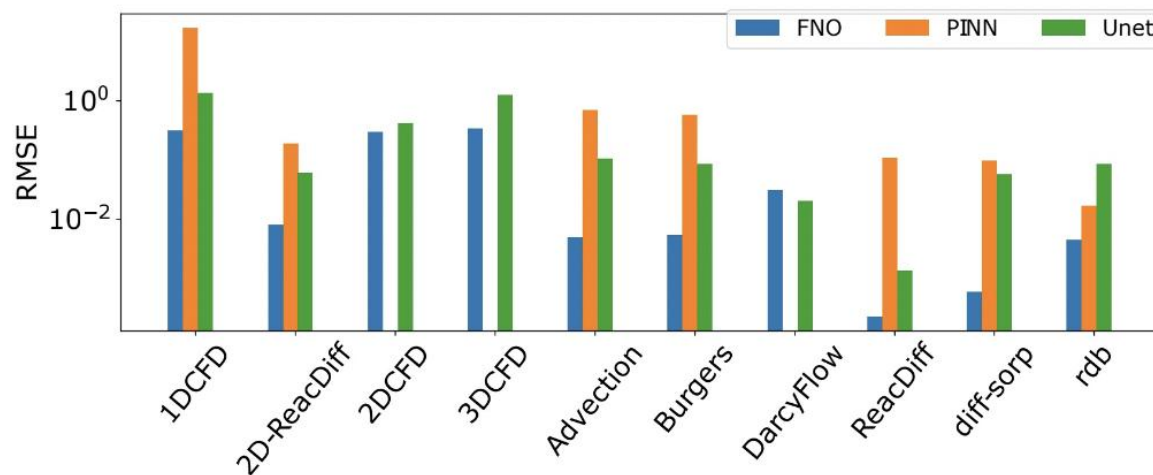
类比有限维的一层神经网络:

$$v_{t+1}(x) := \underset{\text{非线性}}{\sigma} \left(\underset{\text{局域变换}}{W v_t(x)} + \underset{\text{全局积分}}{(\mathcal{K}(a; \phi) v_t)(x)} \right)$$

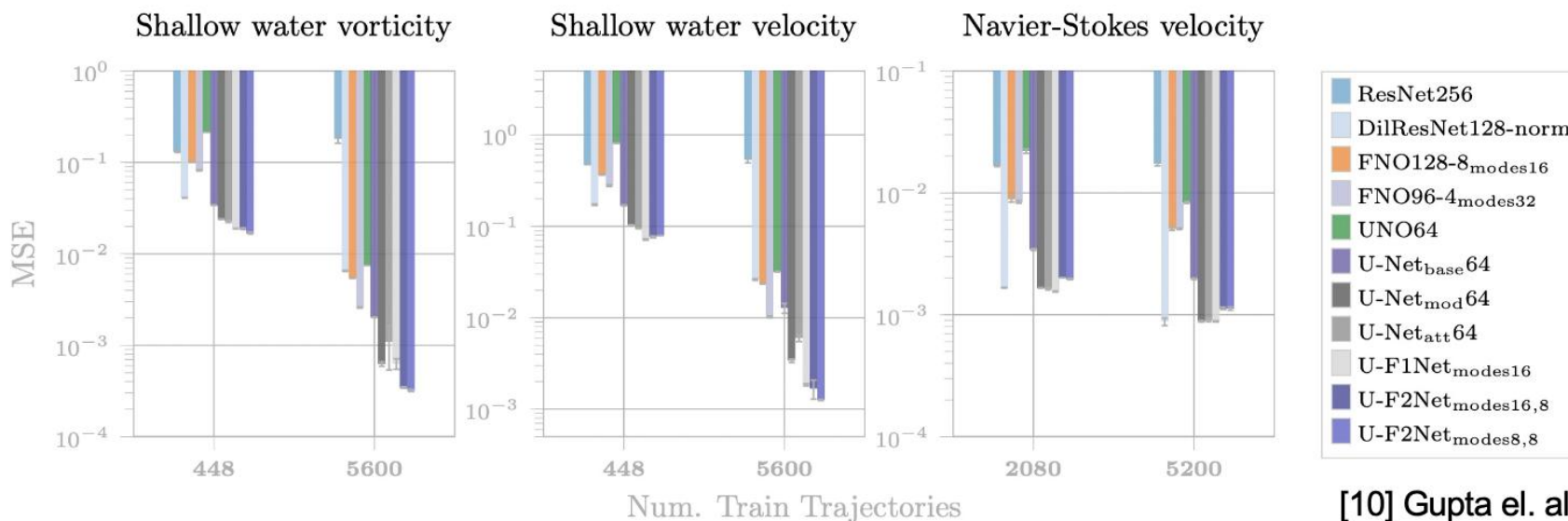
$$v_{L+1} = \sigma(b + W v_L), \quad v_L \in R^d, W \in R^{d' \times d}$$

$$(\mathcal{K}(a; \phi) v_t)(x) := \int_D \kappa(x - y) v_t(y) dy = \mathfrak{F}^{-1}(R_\phi \cdot \mathfrak{F} v_t)(x) \quad (\text{卷积在傅里叶空间中为相乘})$$

FNO compared to other neural networks

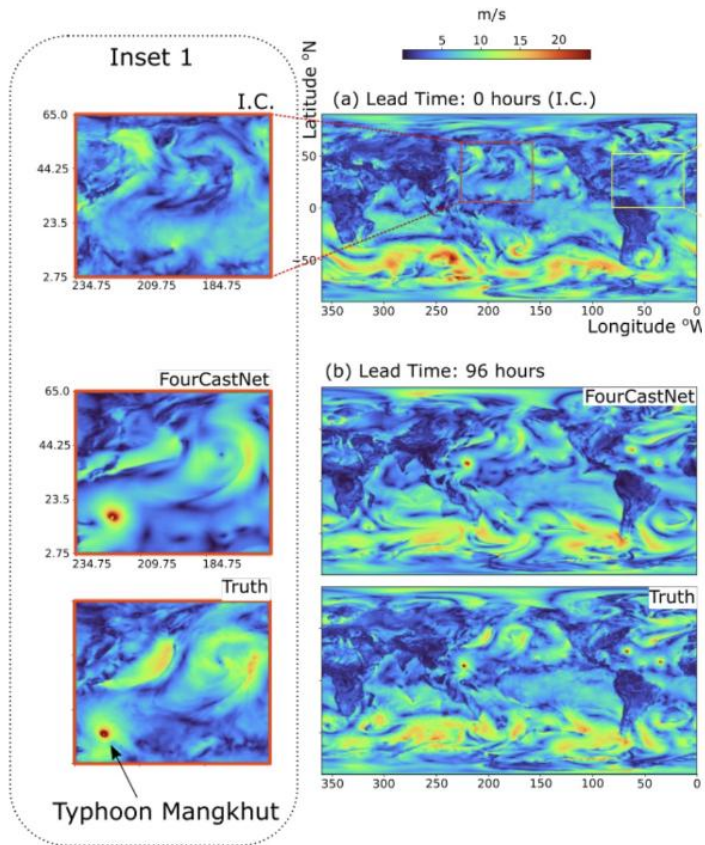


[9] Takamoto et. al. PDEBench Fig.2

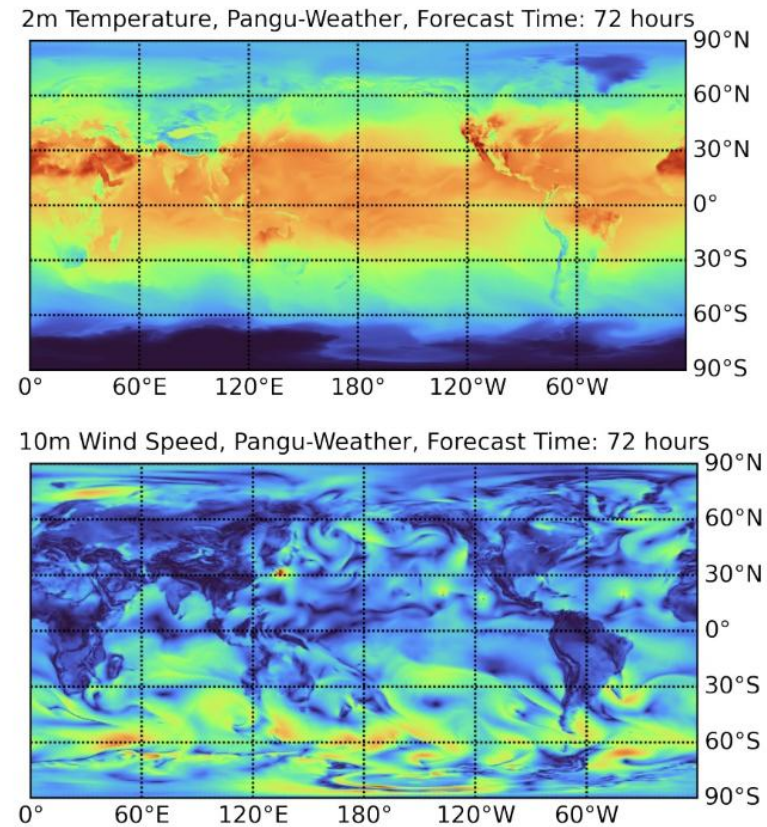


[10] Gupta et. al. PDEarena Fig.4

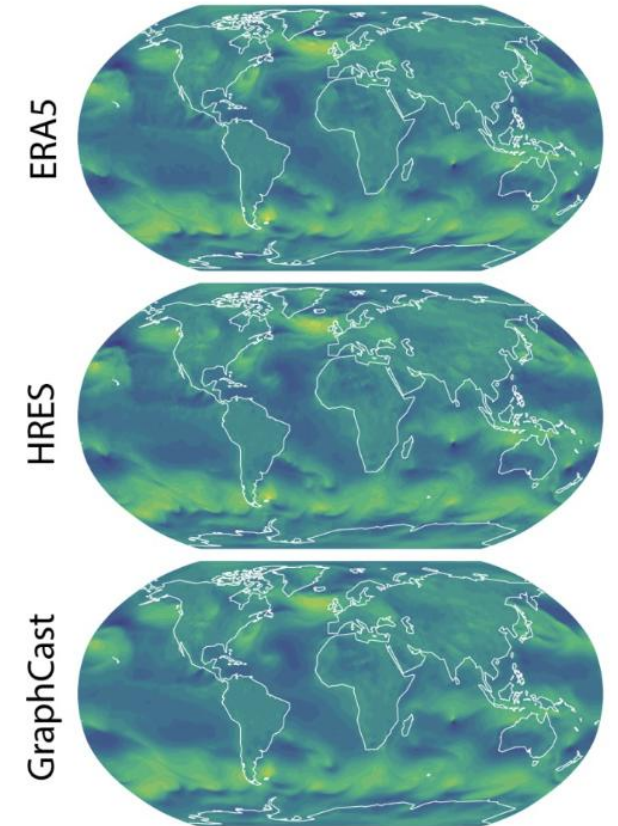
Weather forecast



Fourcastnet [1]
Jaideep Pathak et. al. Feb 2022



Pangu-weather [11]
Kaifeng Bi et. al. Nov 2022



GraphCast [12]
Remi Lam et. al. Dec 2022

FNO: advantages and disadvantages

Advantages:

- Typically very good accuracy, near state-of-the-art
- Can do super-resolution (超分辨率)

Disadvantages:

- Learns a global mapping, requires large amount of training samples
- Requires regular grid

Neural operators

- Fourier Neural Operator (FNO)
 - Based on Fourier transformation
- Boundary-embedded Neural Operator (BENO)
 - Based on Green's function

Wang, Haixin, et al. "BENO: Boundary-embedded Neural Operators for Elliptic PDEs." *NeurIPS 2023 AI for Science Workshop*. 2023.
ICLR: <https://openreview.net/forum?id=ZZTkLDRmkg>

Elliptic PDEs

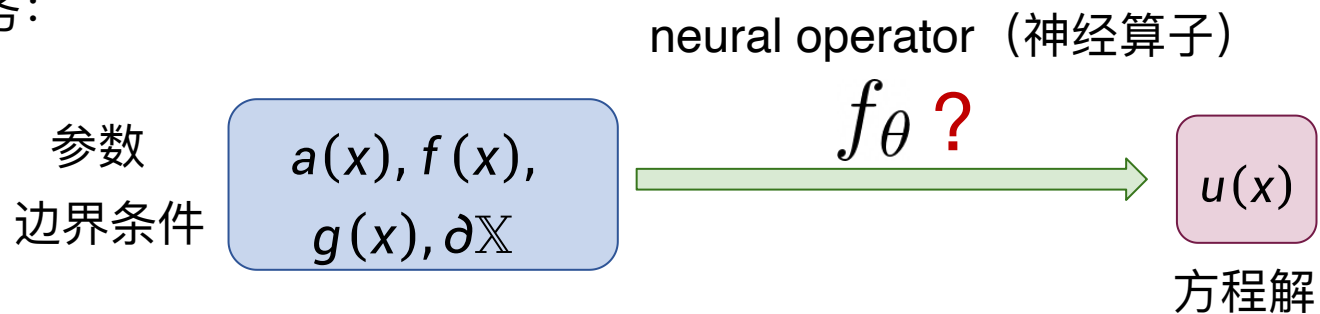
Poisson's equation 泊松方程:

$$-\nabla \cdot (a(x) \nabla u(x)) = f(x), \quad x \in \mathbb{X}$$

$$\mathbb{B}[u(x)] = g(x), \quad x \in \partial\mathbb{X}$$

Important in materials, plasma physics, elasticity, hydrology.

任务:



Complex Boundary Conditions

非均匀边界值

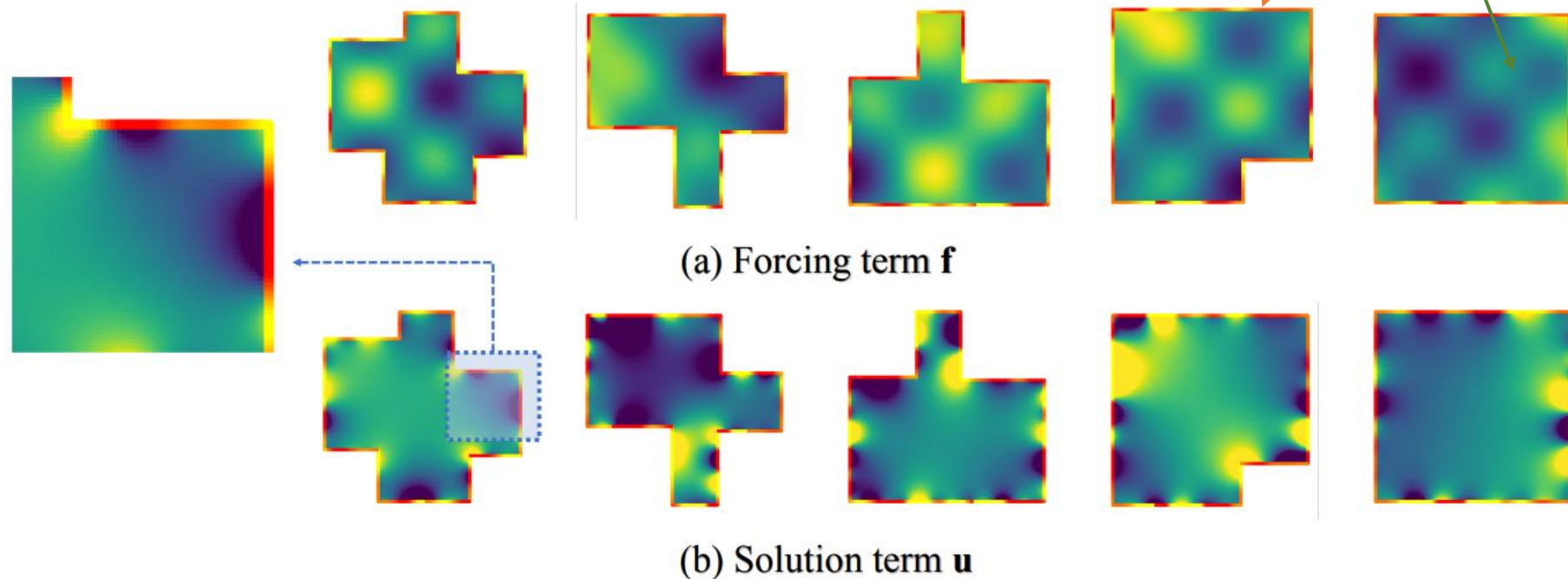
$$\begin{aligned} -\nabla \cdot (a(x)\nabla u(x)) &= f(x), & x \in \mathbb{X} \\ \mathbb{B}[u(x)] &= g(x), & x \in \partial\mathbb{X} \end{aligned}$$

边界值

内部源

Inhomogeneous Value

Challenges



Complex Geometry

复杂边界形状

Haixin Wang *et al.* (2023)

Comparison

Methods		GKN ^[1]	FNO ^[2]	GNN-PDE ^[3]	MP-PDE ^[4]	BENO(ours) ^[5]
1	PDE-agnostic prediction on new initial condition	✓	✓	✓	✓	✓
2	Train/Test space grid independence	✓	✗	✓	✗	✓
3	Evaluation at unobserved spatial locations	✓	✓	✗	✗	✓
4	Free-form spatial domain for boundary shape (多样边界形状)	✗	✗	✓	✗	✓
5	Inhomogeneous boundary condition value (非均匀边界值)	✗	✗	✗	✗	✓

[1] Neural operator: Graph kernel network for partial differential equations. (2020)

[2] Fourier neural operator for parametric partial differential equations. (2020)

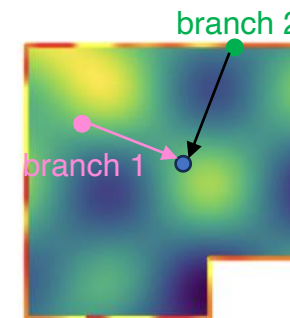
[3] Learning the solution operator of boundary value problems using graph neural networks.(2022)

[4] Message passing neural pde solvers.(2022)

[5] Wang, Haixin, et al. "BENO: Boundary-embedded Neural Operators for Elliptic PDEs." *NeurIPS 2023 AI for Science Workshop*. 2023.

Our method: motivation

- Inspiration: Green's function method (格林函数)



$$\begin{cases} \nabla^2 G = \delta(x - x_0)\delta(y - y_0) \\ G|_{\partial\Omega} = 0 \end{cases} \quad (1)$$

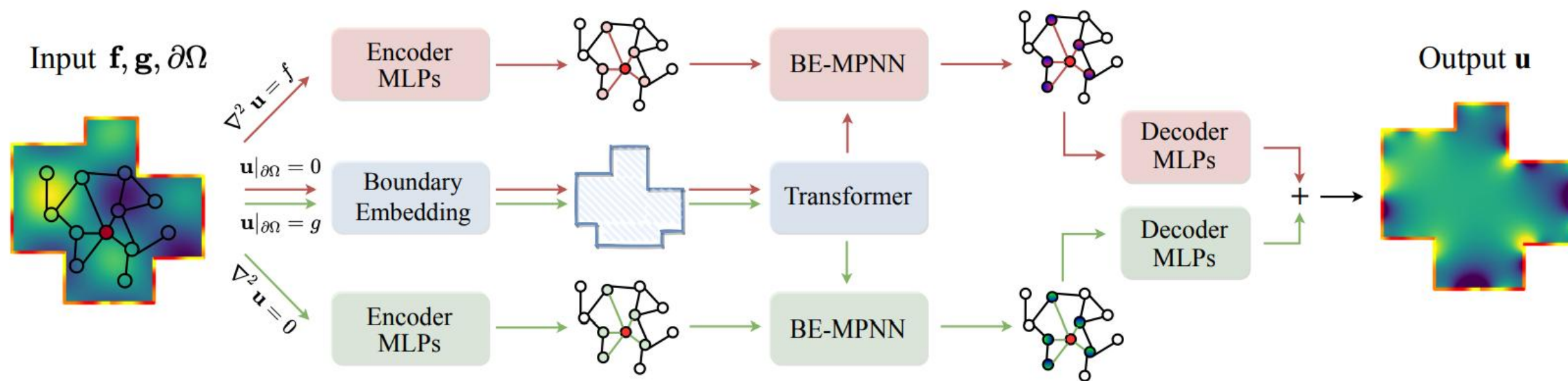
$$u(x, y) = \underbrace{\iint_{\Omega} G(x, y, x_0, y_0) f(x_0, y_0) d\sigma_0}_{\text{内部源的影响}} - \underbrace{\int_{\partial\Omega} g(x_0, y_0) \frac{\partial G(x, y, x_0, y_0)}{\partial n_0} dl_0}_{\text{边界的影响}} \quad (2)$$

$$\begin{cases} \nabla^2 u(x, y) = f(x, y) \\ u(x, y) = g(x, y) \end{cases} \Rightarrow \underbrace{\begin{cases} \nabla^2 u(x, y) = f(x, y) \\ u(x, y) = 0 \end{cases}}_{\text{Branch 1}} + \underbrace{\begin{cases} \nabla^2 u(x, y) = 0 \\ u(x, y) = g(x, y) \end{cases}}_{\text{Branch 2}} \quad (3)$$

- How to embed boundary?

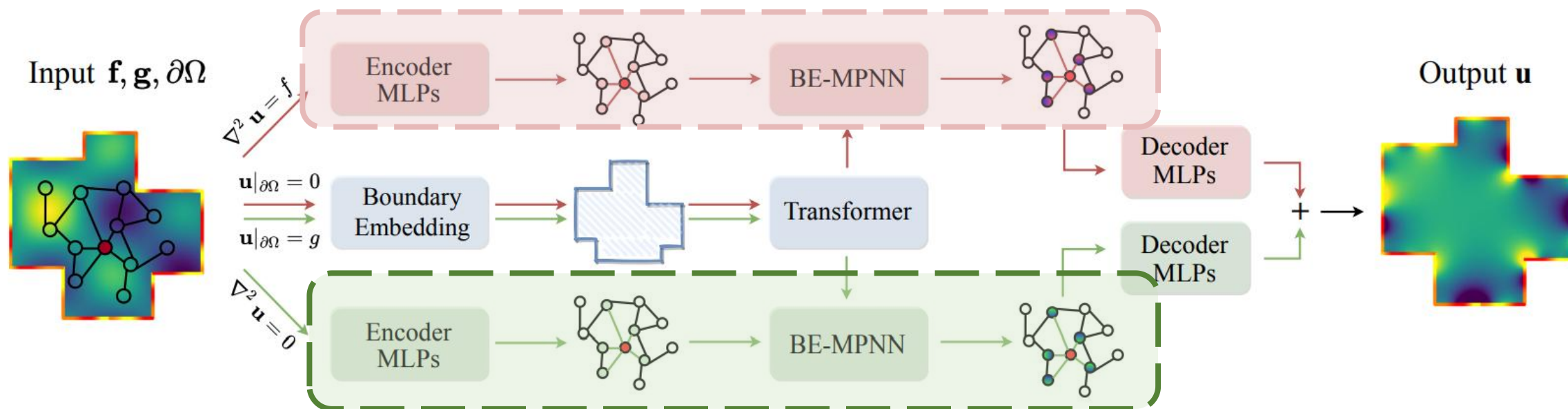
We propose to embed the boundary to represent its global information with Transformer (Vaswani et al., 2017) .

Our method: overall architecture

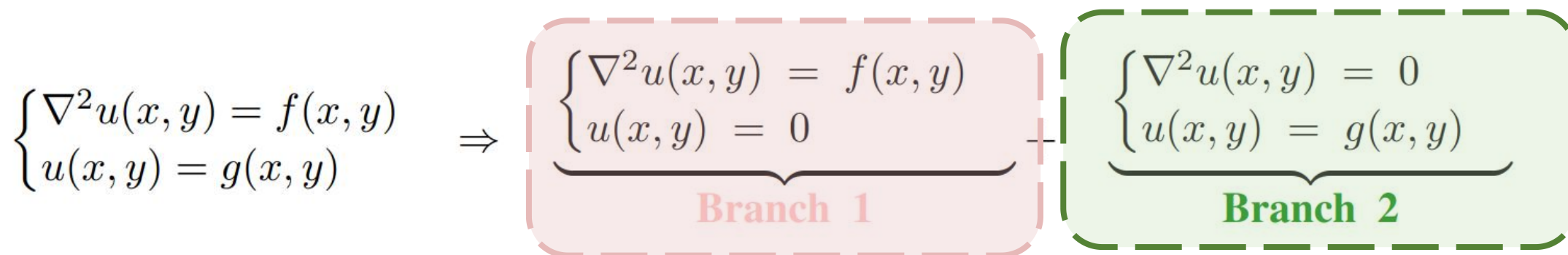


$$\begin{cases} \nabla^2 u(x, y) = f(x, y) \\ u(x, y) = g(x, y) \end{cases} \Rightarrow \underbrace{\begin{cases} \nabla^2 u(x, y) = f(x, y) \\ u(x, y) = 0 \end{cases}}_{\text{Branch 1}} + \underbrace{\begin{cases} \nabla^2 u(x, y) = 0 \\ u(x, y) = g(x, y) \end{cases}}_{\text{Branch 2}}$$

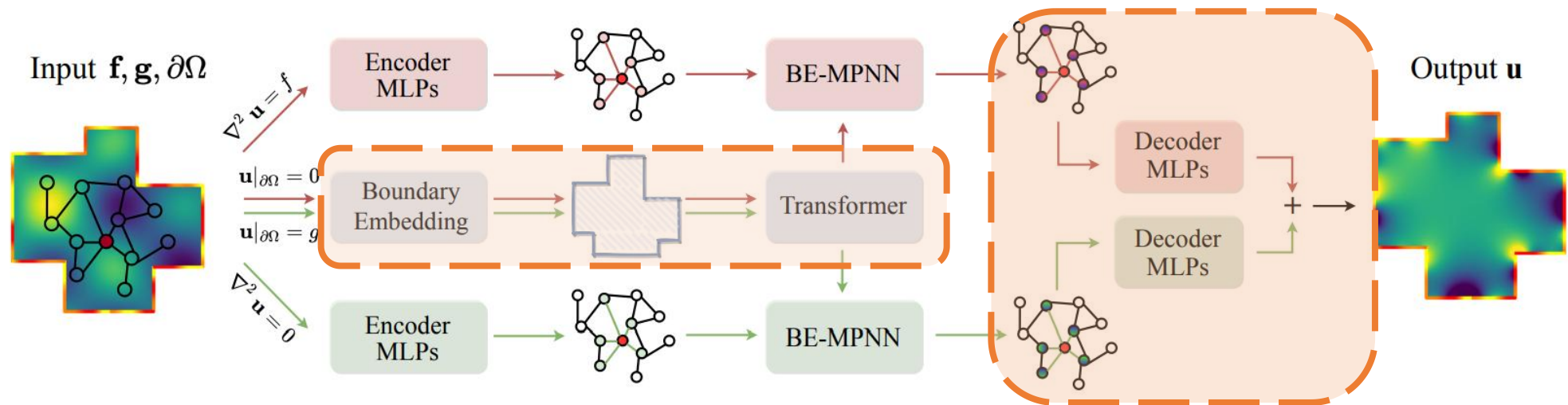
Our method: overall architecture



两个多层图神经网络，分别建模内部源和边界对解的映射



Embedding boundary with transformer



Transformer将复杂边界形状和边界值映射到一个潜在表示，并影响图神经网络每一层的消息传递（message passing）

输入特征

- 边界上每个点 (x, y) 坐标
- 该点上的值 $g(x, y)$
- 该点与上下左右边界的距离

输出特征

- 潜在表示(Latent embedding)

Experiment 1: Poisson Equation with Dirichlet BCs

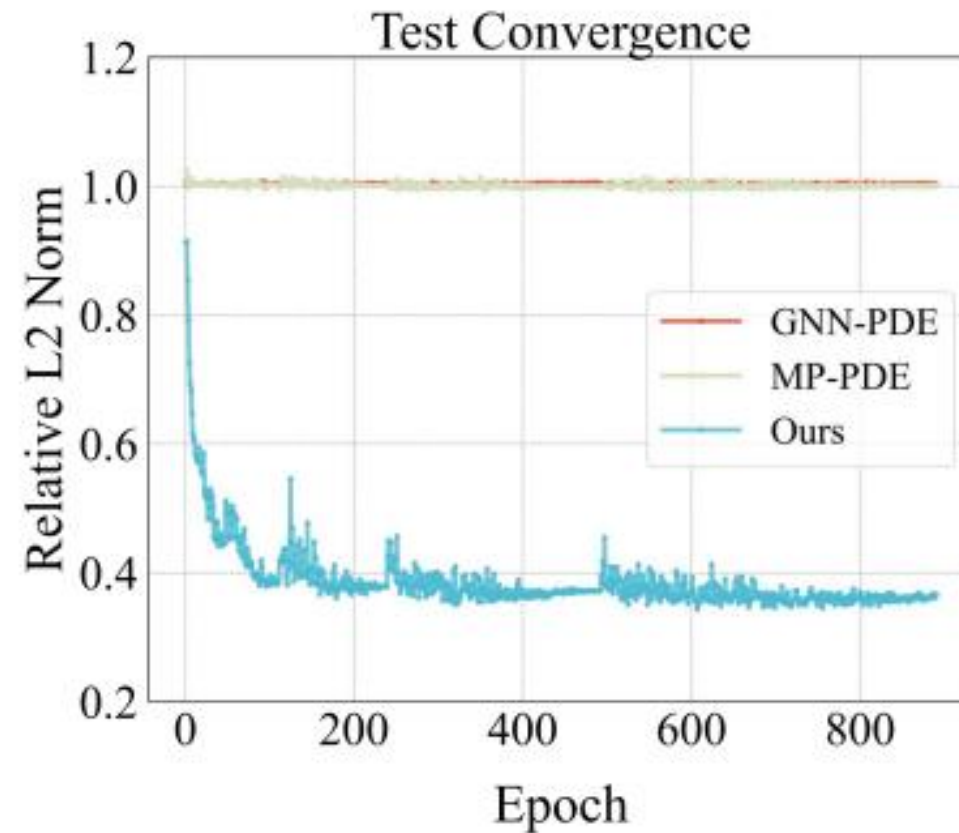
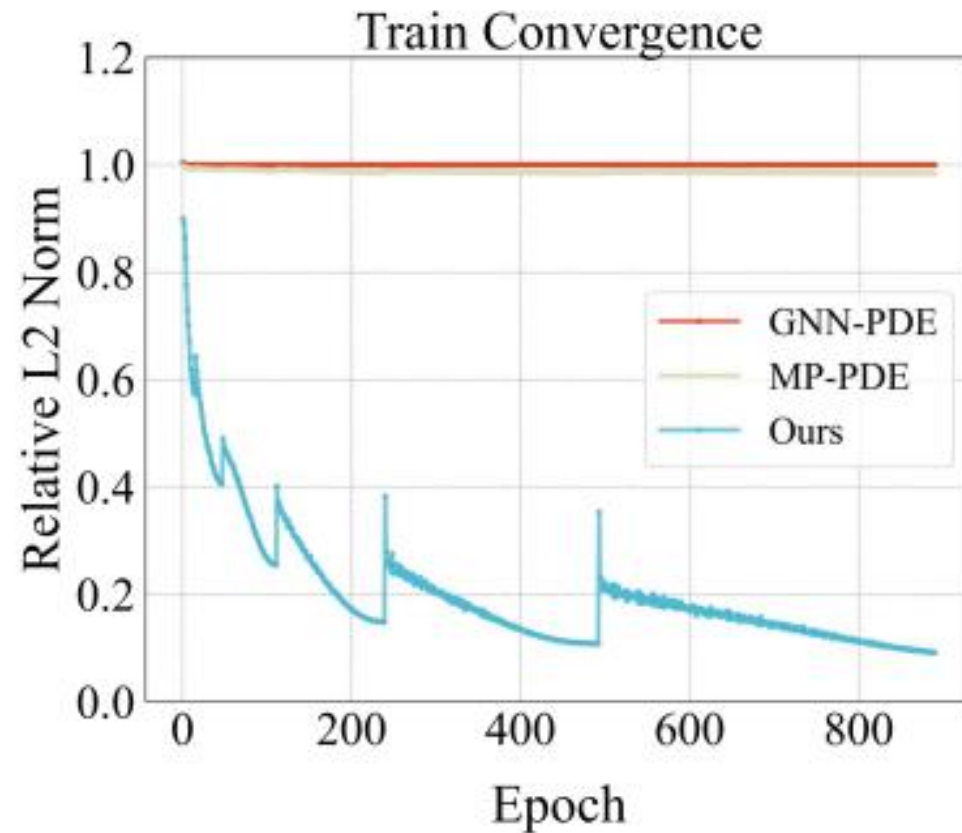
$$\left\{ \begin{array}{ll} \Delta u(x, y) = f(x, y) & \forall (x, y) \in \Omega \\ u(x, y) = g(x, y) & \forall (x, y) \in \partial\Omega \end{array} \right.$$

Train on 4-Corners dataset

Test set	4-Corners		3-Corners		2-Corners		1-Corner		No-Corner	
Metric	L2	MAE	L2	MAE	L2	MAE	L2	MAE	L2	MAE
GKN	1.1146± 0.3936	3.6497± 1.1874	1.0692± 0.2034	3.7059± 0.9543	1.0673± 0.1393	3.6822± 0.9819	1.1063± 0.1905	3.4898± 0.9469	1.0728± 0.2074	3.9551± 0.9791
FNO	1.0947± 0.3265	2.2707± 0.3361	1.0742± 0.3418	2.1657± 0.3976	1.0672± 0.3736	2.2617± 0.2449	1.0921± 0.2935	2.3922± 0.3526	1.0762± 0.4420	2.2281± 0.4192
GNN-PDE	1.0026± 0.0093	3.1410± 0.8751	1.0009± 0.0101	3.2812± 0.8839	1.0015± 0.0099	3.3557± 0.8521	1.0002± 0.0153	3.1421± 0.8685	1.0011± 0.0152	3.7561± 1.0274
MP-PDE	1.0007± 0.0677	3.1018± 0.8431	1.0003± 0.0841	3.2464± 0.8049	0.9919± 0.0699	3.2765± 0.8632	0.9829± 0.07199	3.0163± 0.8272	0.9882± 0.0683	3.6522± 0.8961
BENO (ours)	0.3523± 0.1245	0.9650± 0.3131	0.4308± 0.1994	1.2206± 0.4978	0.4910± 0.1888	1.4388± 0.5227	0.5416± 0.2133	1.4529± 0.4626	0.5542± 0.1952	1.7481± 0.5394

Experiment 1: Poisson Equation with Dirichlet BCs

Learning curve:

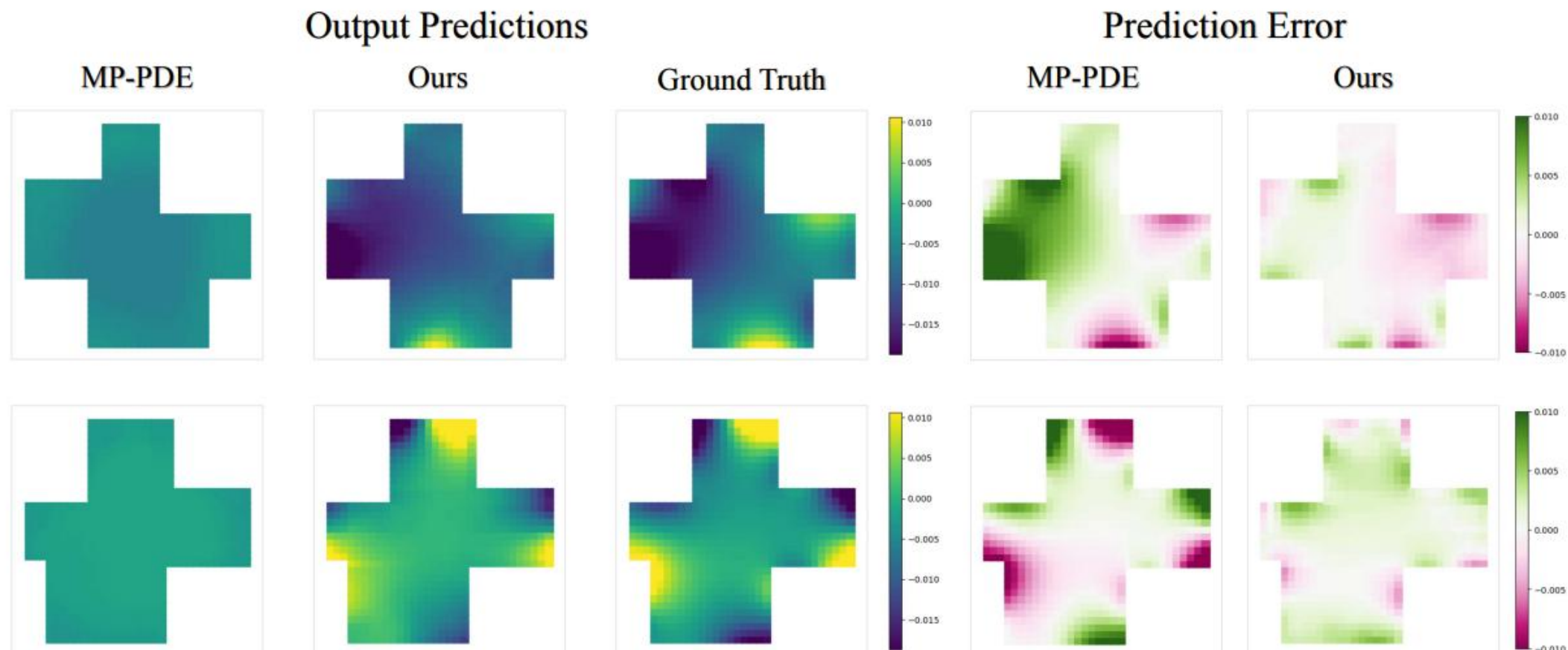


Experiment 2: Poisson Equation with hybrid BCs

$$\left\{ \begin{array}{ll} \Delta u(x, y) = f(x, y) & \forall (x, y) \in \Omega \\ u(x, y) = g(x, y) & \forall (x, y) \in \partial\Omega_{Dirichlet} \\ \frac{\partial u(x, y)}{\partial n} = 0 & \forall (x, y) \in \partial\Omega_{Neumann} \end{array} \right.$$

Train on 4-Corners dataset											
Test set	4-Corners		3-Corners		2-Corners		1-Corner		No-Corner		
Metric	L2	MAE	L2	MAE	L2	MAE	L2	MAE	L2	MAE	
GKN	1.0118±0.1031	3.7105±1.0699	1.0046±0.1284	3.6013±1.2937	1.0301±0.1417	3.3880±1.0127	1.0025±0.1326	3.3675±0.9112	0.9827±0.1270	3.5691±1.2194	
FNO	1.0547±0.1643	4.0316±0.8953	1.0587±0.1761	4.0219±0.8210	1.0519±0.1822	4.0308±0.8369	1.0533±0.1782	4.0276±0.8554	1.0549±0.1842	4.0417±0.8063	
GNN-PDE	1.0105±0.0898	2.3685±0.6933	0.9907±0.1054	2.5474±0.8863	1.0132±0.1208	2.7348±0.8461	0.9821±0.1225	2.9824±0.8106	0.9711±0.1071	3.4930±1.1110	
MP-PDE	1.0070±0.0813	2.3595±0.6941	0.9895±0.0973	2.5480±0.8955	1.0134±0.1120	2.7345±0.8393	0.9782±0.1240	2.9679±0.7958	0.9670±0.1164	3.4807±1.1143	
BENO (ours)	0.3568±0.0988	0.8311±0.2864	0.4201±0.1170	1.0814±0.3938	0.5020±0.1648	1.3918±0.5454	0.5074±0.1422	1.5676±0.4815	0.5221±0.1474	1.8649±0.5472	

Visualization



BENO: Paper and Code

Paper:

BENO: Boundary-embedded Neural Operators for Elliptic PDEs

Our paper link:

<https://openreview.net/forum?id=xZqEdxTqd2>

Our Code link:

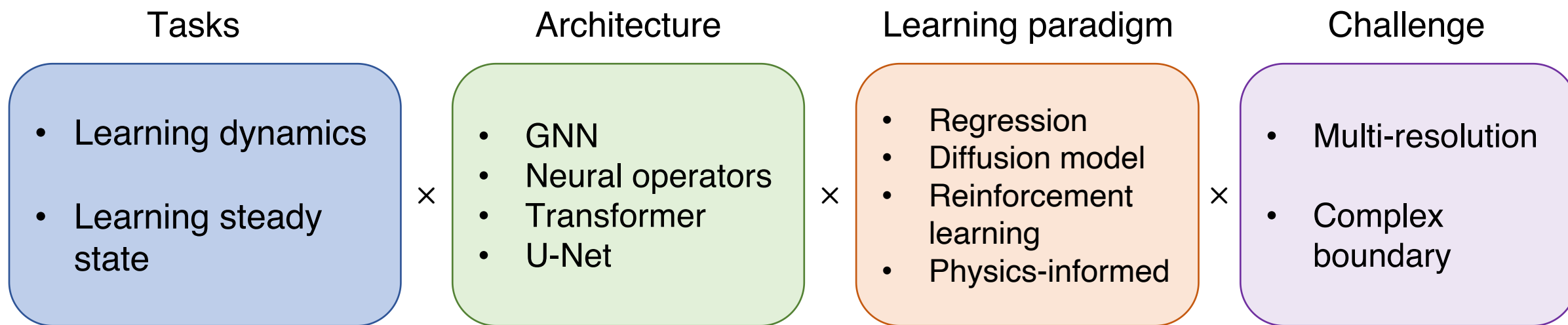
<https://anonymous.4open.science/r/BENO-3D53>

报告总结

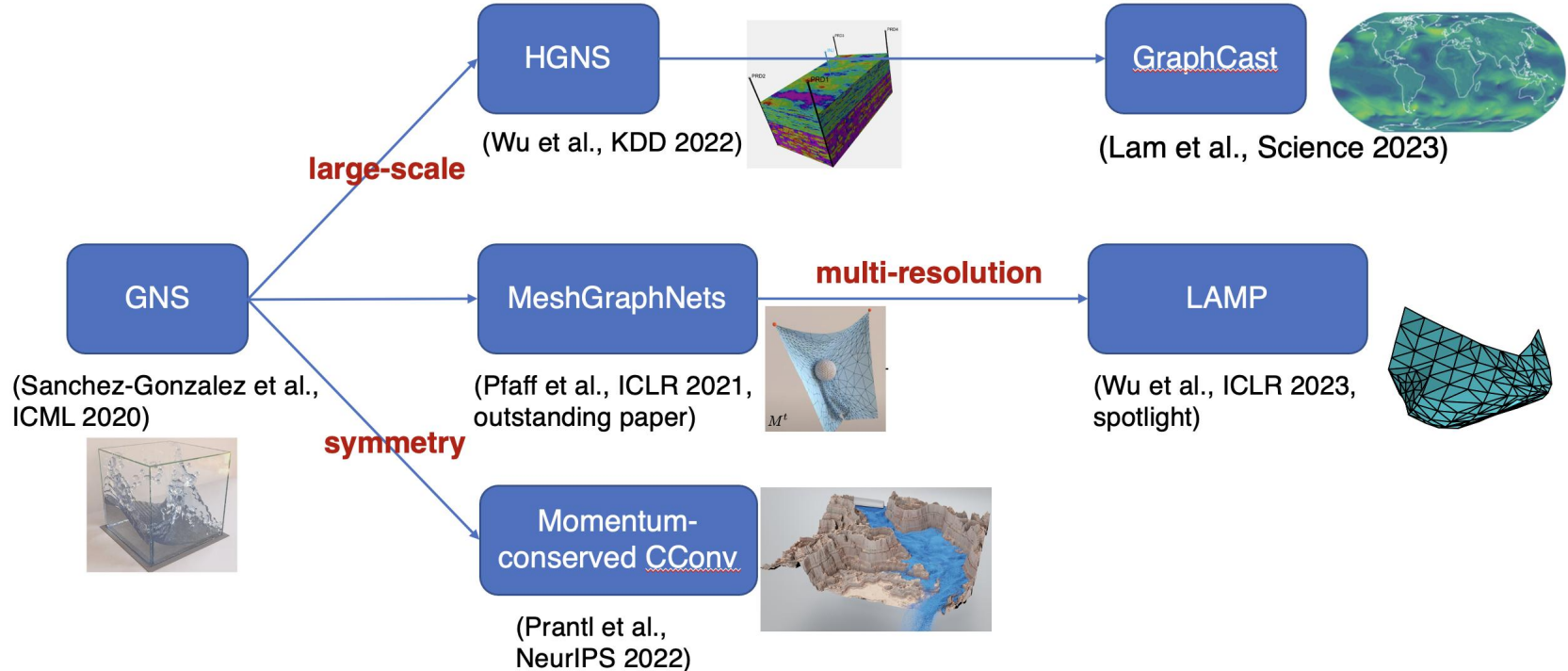
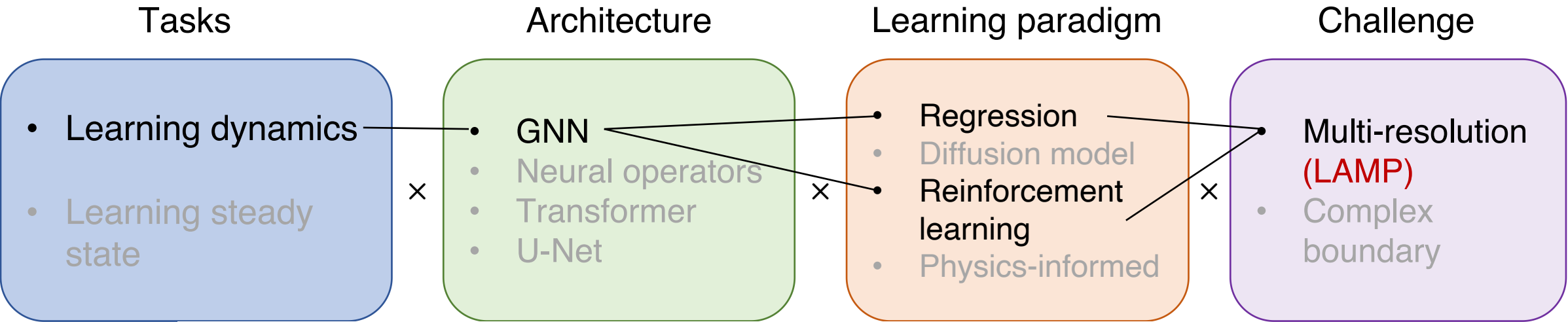
两类任务：学习系统动力学和稳态

两个重要架构：图神经网络和神经算子

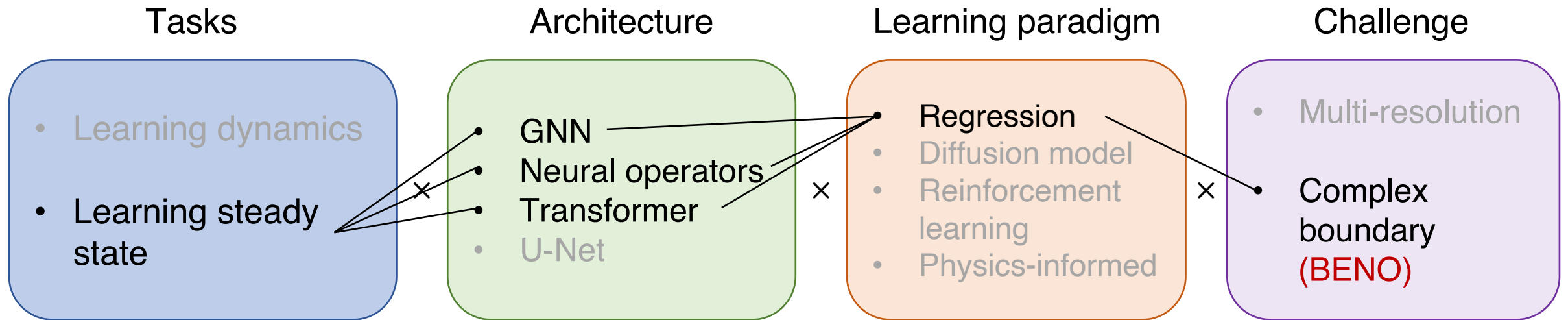
两个挑战和解决方式：多分辨率和复杂边界条件



报告总结 (一)



报告总结 (二)



- Fourier Neural Operator (FNO)
 - Based on Fourier transformation
- Boundary-embedded Neural Operator (BENO)
 - For solving **complex boundary shape** and **inhomogeneous boundary values**

开放问题

AI for scientific simulation:

- Multi-scale
- Improving trustworthiness:
 - Uncertainty quantification 不确定性估计 [1]
 - Error guarantees, 误差保证:
 - e.g., with conformal prediction [2]
- Better incorporation of data and physics equation
 - Existing works:
 - Solver-in-the-loop [3]
 - PEDS [4]
 - Physics-informed DeepONet [5]
 - Physics-informed diffusion model [6]

[1] Wu, Tailin *et al.*, Uncertainty Quantification for Forward and Inverse Problems of PDEs via Latent Global Evolution, AAAI 2024, <https://github.com/AI4Science-WestlakeU/le-pde-ug>

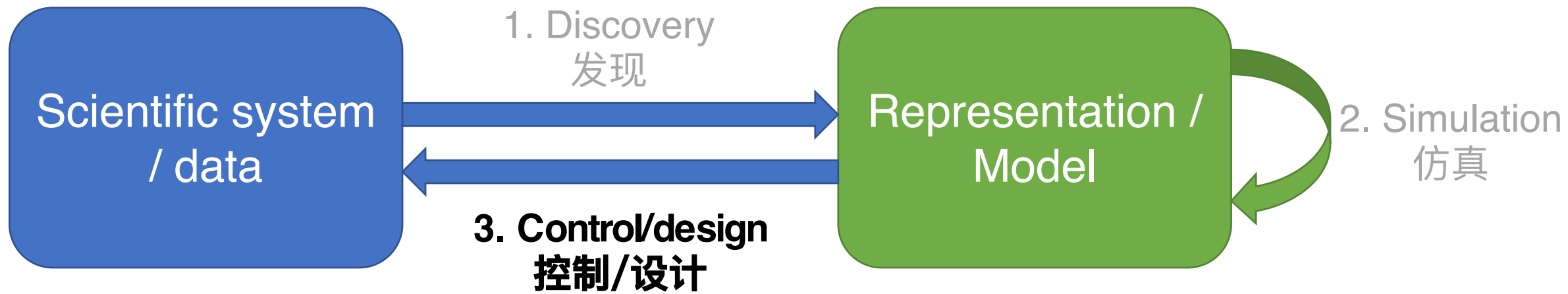
[2] Stankeviciute, Kamile, Ahmed M Alaa, and Mihaela van der Schaar. "Conformal time-series forecasting." *Advances in neural information processing systems* 34 (2021): 6216-6228.

[3] Um, Kiwon, et al. "Solver-in-the-loop: Learning from differentiable physics to interact with iterative pde-solvers." *NeurIPS* 2020, 6111-6122.

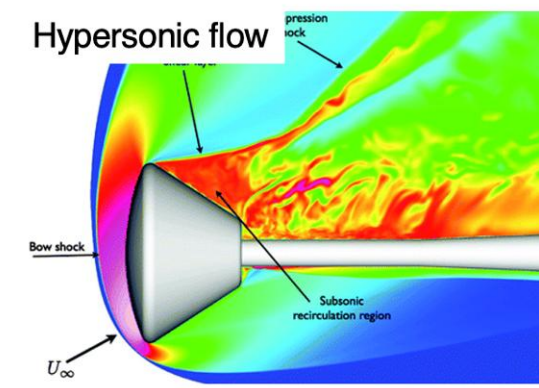
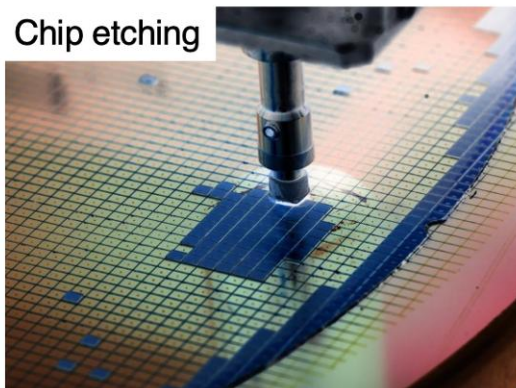
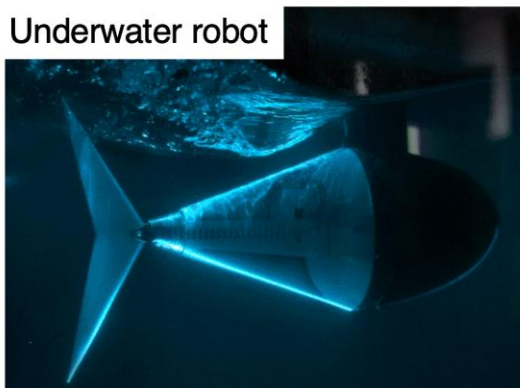
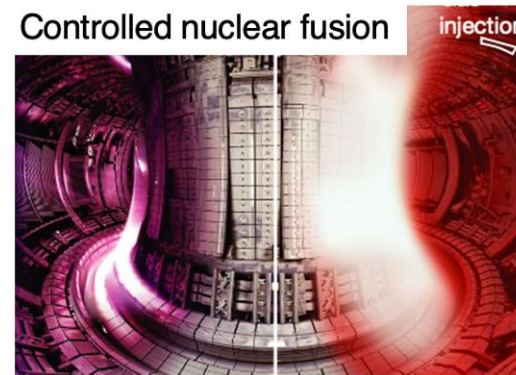
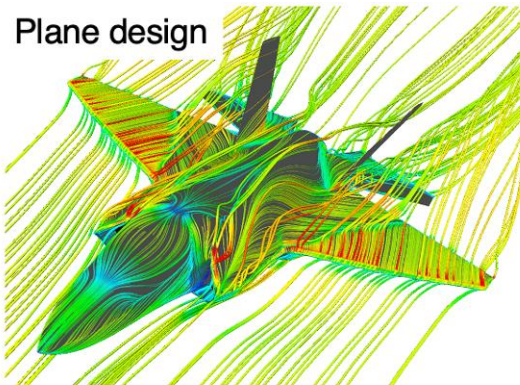
[4] Pestourie, Raphaël, et al. "Physics-enhanced deep surrogates for partial differential equations." *Nature Machine Intelligence* (2023): 1-8.

[5] Wang, Sifan, Hanwen Wang, and Paris Perdikaris. "Learning the solution operator of parametric partial differential equations with physics-informed DeepONets." *Science advances* 7.40 (2021): eabi8605.

[6] Shu, Dule, Zijie Li, and Amir Barati Farimani. "A physics-informed diffusion model for high-fidelity flow field reconstruction." *Journal of Computational Physics* 478 (2023): 111972.

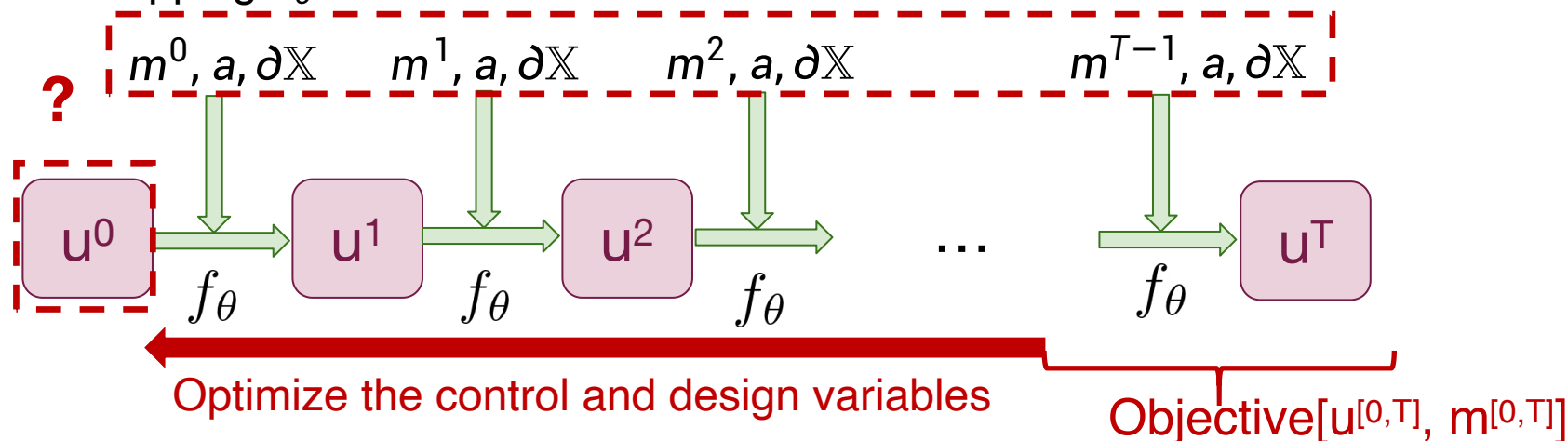


These three tasks are fundamental in **science** and **engineering**



Task setup for learning control/design

Goal: learn the mapping f_θ from u^t to u^{t+1} :



u^t : original **state** of the system. Can be an infinite-dimensional function $u(t, x)$ as solution to a PDE, or a graph (e.g., mesh, particle-based systems, molecules)

f_θ : neural surrogate models

m^t : external **control** (外界控制)

a : **static parameters** (静态参数) of the system that does not change with time (e.g. parameters of PDE, spatially varying diffusion coefficient)

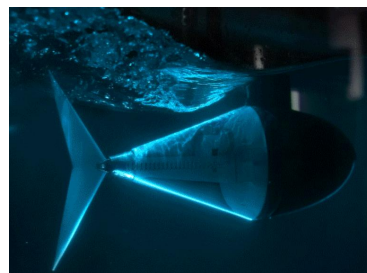
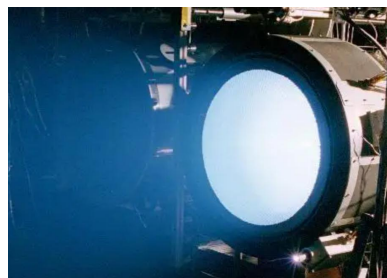
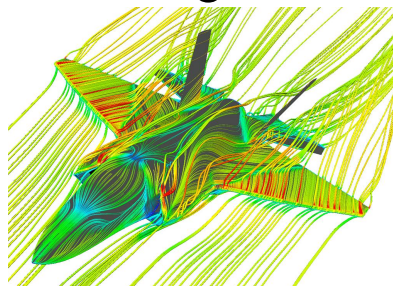
$\partial \mathbb{X}$: **boundary condition** (边界条件) of the system

} control (控制)

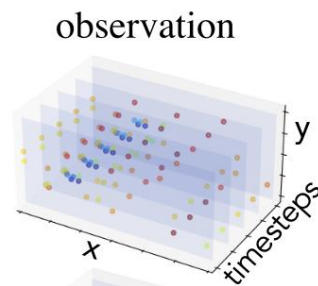
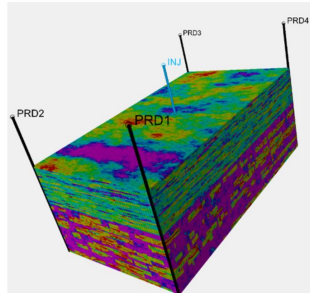
} inverse design (反向设计)

任务类型

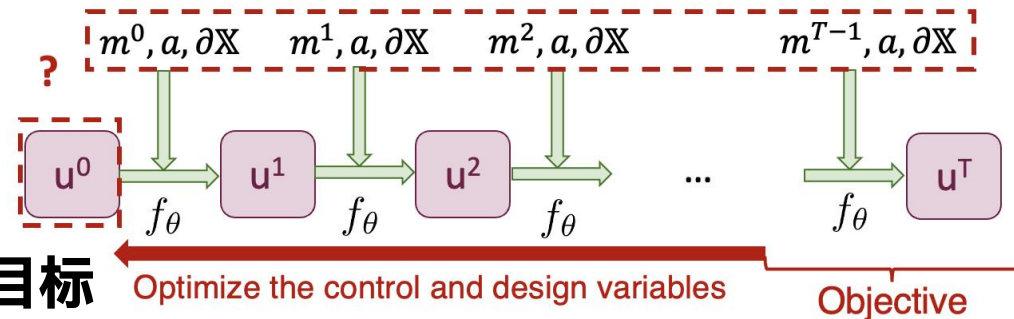
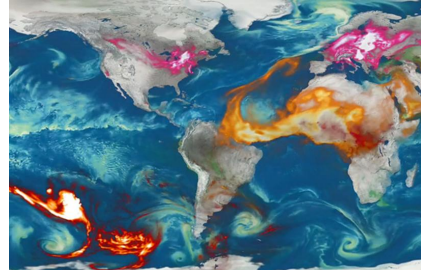
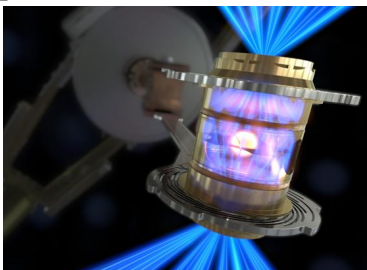
- **反向设计**: 边界条件 $\partial\mathbb{X}$, 初条件 u^0 , 参数 a 以**优化设计目标**
plane design, rocket shape, underwater robot shape



- **反问题**: 推测初条件 u^0 , 参数 a 使得**预测与观测相符**

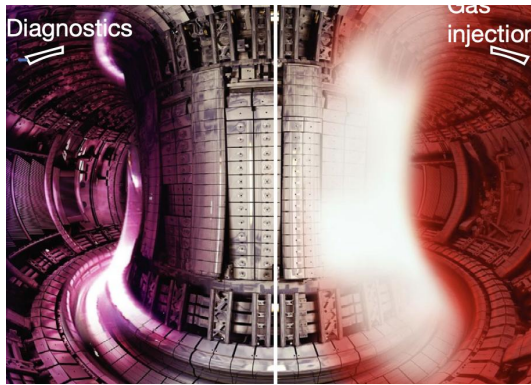


- **控制**: 优化控制序列 m^t 以**优化控制目标**: control pulses for controlled nuclear fusion

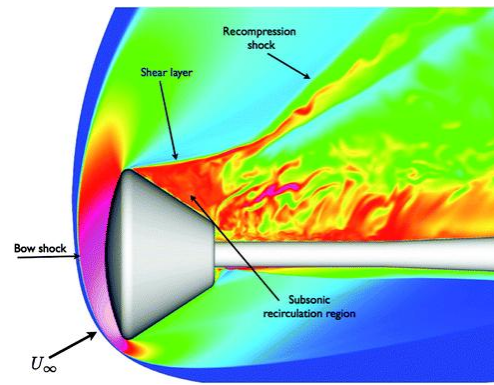


AI for control and inverse design: significance

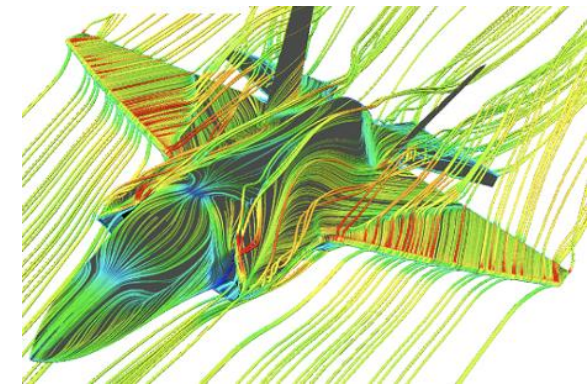
- Control and Inverse design is **prevalent** in science and engineering:



Controlled nuclear fusion



Mechanical engineering



Plane design

- Helps to explore continuous, **high-dimensional** design space, potential to find control/designs not imagined by humans

AI for control and inverse design: difficulty

- **Complex control/design space:**
 - High Computational cost
 - Complex composition relations
- **Complex dynamics:**
 - How to characterize interaction between optimization of shape with physical process
- **Generalization:**
 - How to generalize to more complex compositional scenarios

Limitation of prior methods

For **control**: PID is the most prevalent

- Hard to deal with nonlinear systems with high-dimensional, coupled controls
- Tuning it requires with expertise

For **design**: e.g., cross-entropy method:

- High accuracy but low efficiency
- Need rich expert knowledge
- Hard to deal with high-dimensional design space

How can AI help scientific design?

What is the most **important insight** from 30 years of deep learning?



Method 1: go from A to B in a **straight line**
(learn a **direct mapping** from A to B)

Hard to learn due to the complexity
of A, B and their difference!

How can AI help scientific design?

Insight: Model a hard transformation by composing many simple, easy transformations.



Method 2: go from A to B in **small, easier steps**
(**compose step-by-step** simple mappings to map A to B)

Much easier!

How can AI help scientific design/control?

Insight: to construct a complex mapping from A to B, it is much easier to compose simple mappings



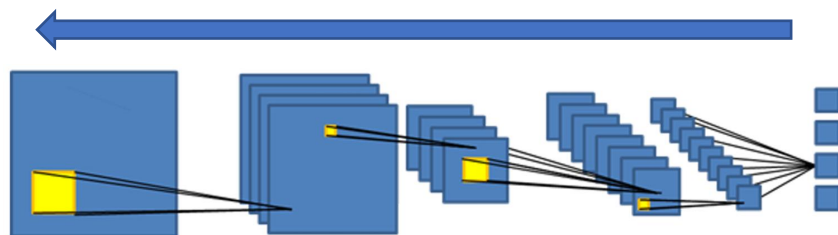
AI for scientific design:
1. backpropagation

gradient w.r.t.
parameter

$$\frac{\partial L}{\partial \theta}$$

Backpropagation 反向传播

loss



$$L = \text{MSE} (f_{\theta}(x), y)$$

How can AI help scientific design/control?

Insight: to construct a complex mapping from A to B, it is much easier to compose simple mappings



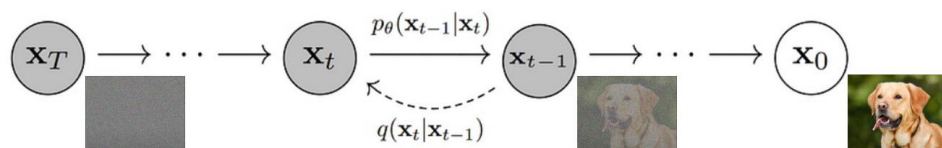
AI for scientific design:

1. backpropagation
2. diffusion models

Gaussian distribution



diffusion model



data distribution



How can AI help scientific design/control?

Insight: to construct a complex mapping from A to B, it is much easier to compose simple mappings



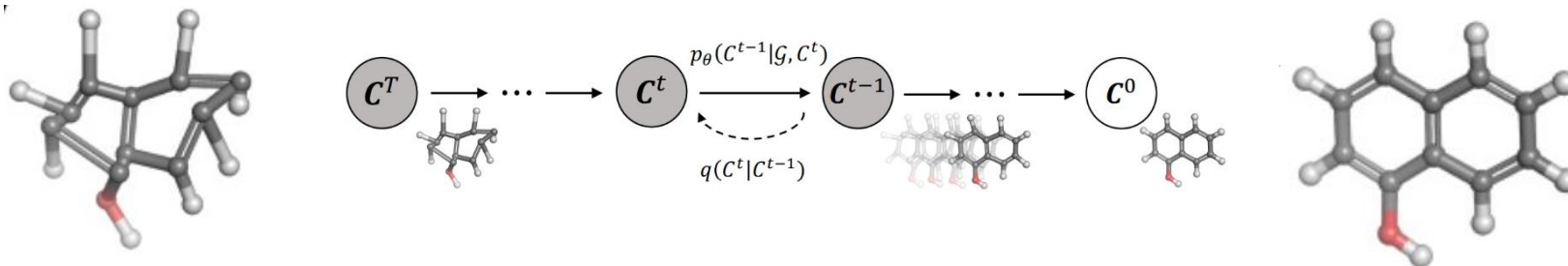
AI for scientific design:

1. backpropagation
2. diffusion models

Gaussian distribution

diffusion model

data distribution



How can AI help scientific design/control?

Insight: to construct a complex mapping from A to B, it is much easier to compose simple mappings



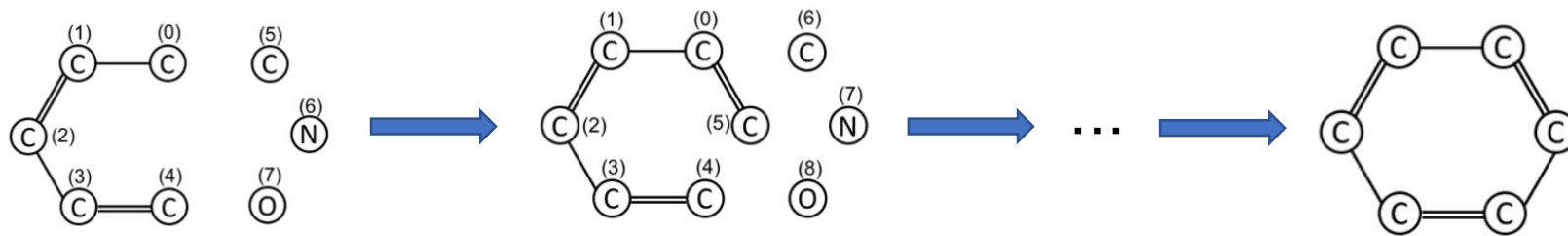
AI for scientific design:

1. backpropagation
2. diffusion models
3. reinforcement learning

starting configuration

reinforcement learning

target configuration



How can AI help scientific design/control?

1. **Surrogate model + backpropagation** for design [1][2] and control [3]
2. **Reinforcement learning** (e.g., RL for controlling plasma fusion [4], GCPN [5], GFlowNet [6], RL for designing quantum experiments [7])
3. **Diffusion models** for design [8][9] and control [10]
4. Direct mapping [11]
5. Bayesian methods / sample-based methods [12]
6. ...

[1] Allen, Kelsey R., et al. "Physical design using differentiable learned simulators." *NeurIPS 2022*

[2] Zhao, Qingqing, David B. Lindell, and Gordon Wetzstein. "Learning to solve pde-constrained inverse problems with graph networks." *ICML 2022*

[3] Hwang, Rakhoon, et al. "Solving pde-constrained control problems using operator learning." *AAAI 2022*.

[4] Degraeve, Jonas, et al. "Magnetic control of tokamak plasmas through deep reinforcement learning." *Nature* 602.7897 (2022): 414-419.

[5] You, Jiaxuan, et al. "Graph convolutional policy network for goal-directed molecular graph generation." *NeurIPS 2018*

[6] Jain, Moksh, et al. "Biological sequence design with GflowNets." *ICML 2022*

[7] Nautrup, Hendrik Poulsen, et al. "Optimizing quantum error correction codes with reinforcement learning." *Quantum* 3 (2019): 215.

[8] Wu, Tailin, et al. "Compositional Generative Inverse Design." *NeurIPS 2023 AI for Science Workshop. 2023*.

[9] Xu, Minkai, et al. "Geodiff: A geometric diffusion model for molecular conformation generation." *ICLR 2022*

[10] Holzschuh, Benjamin, Simona Vegetti, and Nils Thuerey. "Solving Inverse Physics Problems with Score Matching." *NeurIPS 2023*

[11] Guo, Ruchi, Shuhao Cao, and Long Chen. "Transformer meets boundary value inverse problems." *NeurIPS 2022*

[12] Rubinstein, R. Y. and Kroese, D. P. "The cross-entropy method: A unified approach to monte carlo simulation, randomized optimization and machine learning". *Information Science & Statistics*, Springer Verlag, NY, 2004.

AI for scientific design/control

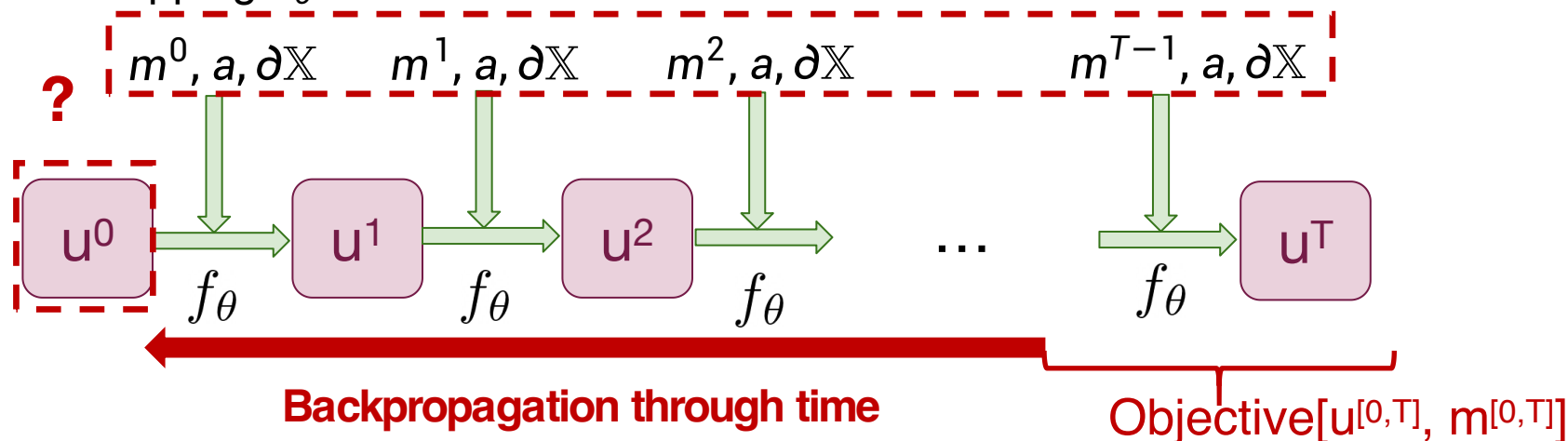
1. **Surrogate model + backpropagation** for design and control
2. **Reinforcement learning** for control
3. **Diffusion models** for design and control

AI for scientific design/control

1. **Surrogate model + backpropagation** for design and control
2. **Reinforcement learning** for control
3. **Diffusion models** for design and control

Task setup for learning control/design

Goal: learn the mapping f_θ from u^t to u^{t+1} :



u^t : original **state** of the system. Can be an infinite-dimensional function $u(t, x)$ as solution to a PDE, or a graph (e.g., mesh, particle-based systems, molecules)

f_θ : neural surrogate models

m^t : external **control** (外界控制)

a : **static parameters** (静态参数) of the system that does not change with time (e.g. parameters of PDE, spatially varying diffusion coefficient)

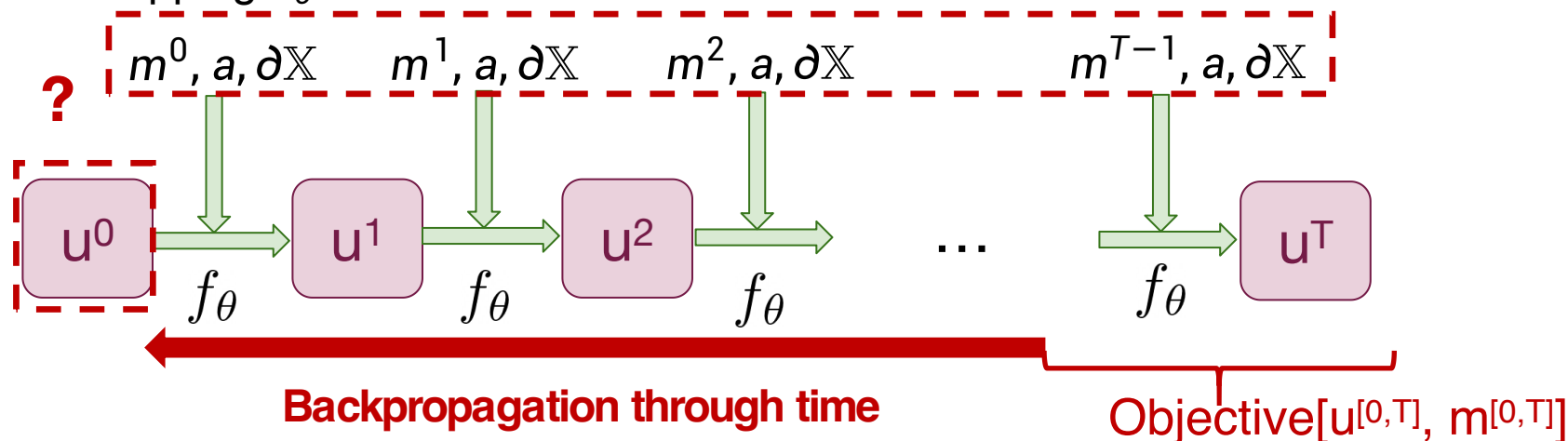
$\partial\mathbb{X}$: **boundary condition** (边界条件) of the system

} control (控制)

} inverse design (反向设计)

Task setup for learning control/design

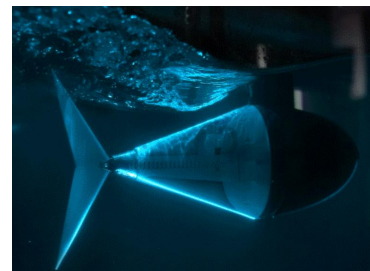
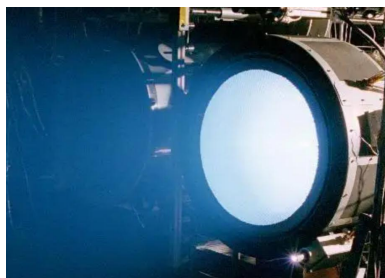
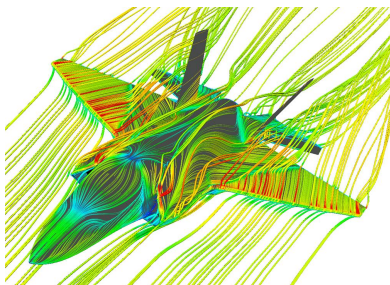
Goal: learn the mapping f_θ from u^t to u^{t+1} :



1. 反向设计: Optimize $J[u^T(\partial\mathbb{X}, a)]$ w.r.t. boundary $\partial\mathbb{X}$ [1] or system parameter a :

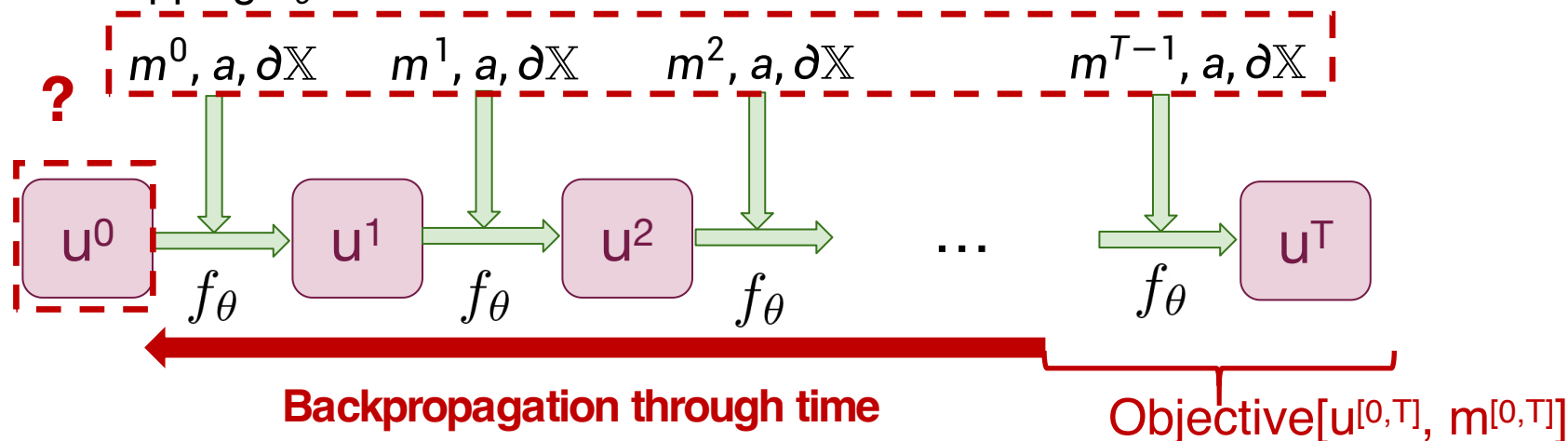
目标函数 J : drag (for plane and under water robots), specific impulse (for rocket engine)

边界 $\partial\mathbb{X}$: plane shape, rocket engine shape, robot shape

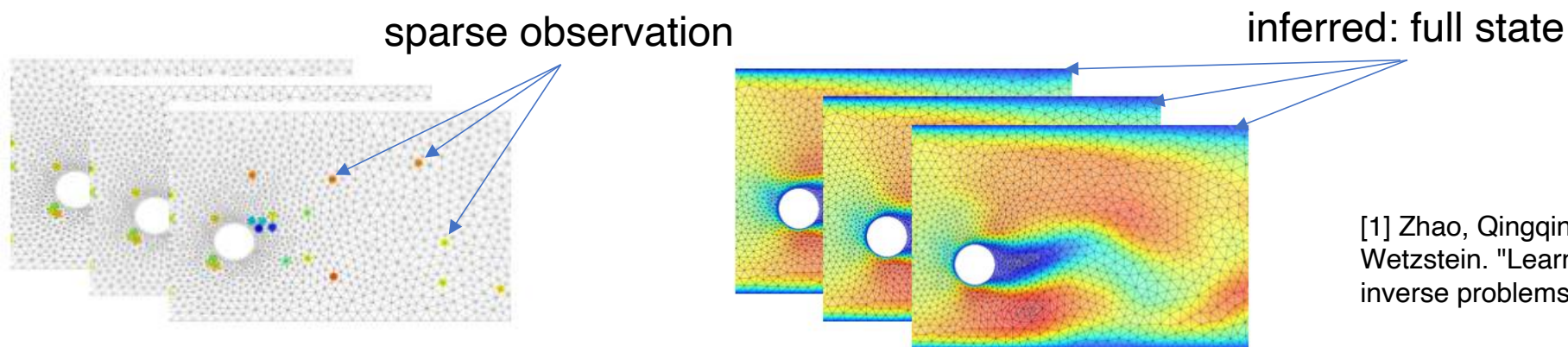


Task setup for learning control/design

Goal: learn the mapping f_θ from u^t to u^{t+1} :



2. 反问题或状态估计: optimize $MSE[\hat{u}^t(x), u^t(x)]$ w.r.t. initial state u^0 or parameter a [1], given sparse observation of $u^t(x)$ at certain locations $x \in \Omega$



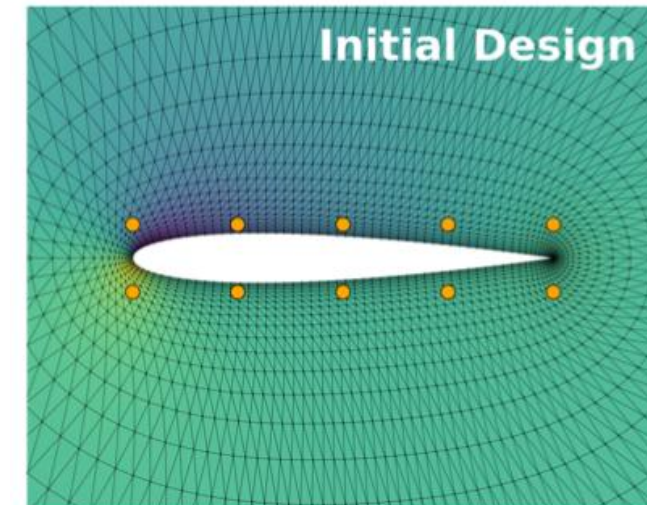
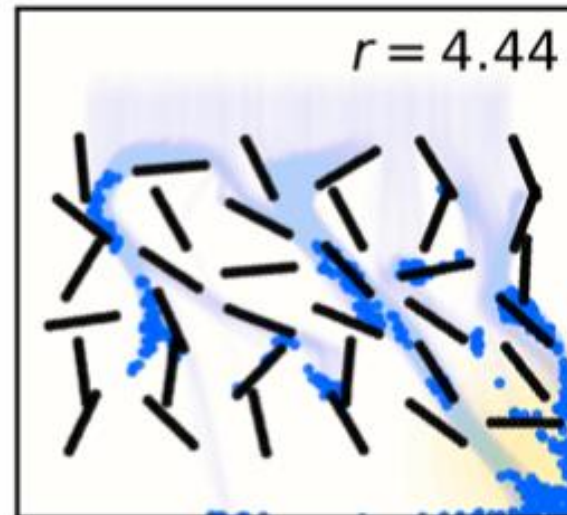
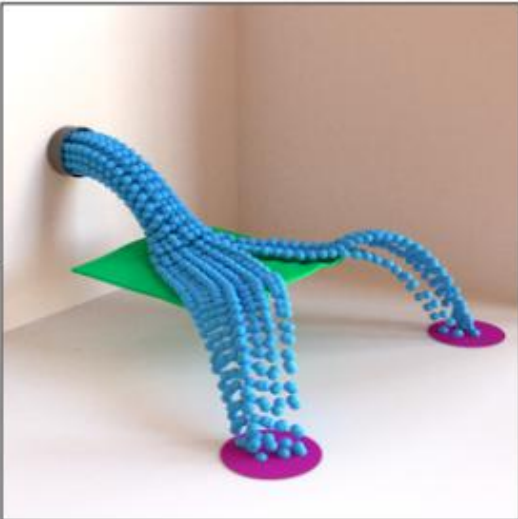
[1] Zhao, Qingqing, David B. Lindell, and Gordon Wetzstein. "Learning to solve pde-constrained inverse problems with graph networks." *ICML 2022*

Case study: physical design with differentiable learned simulators

Task: design boundary for physical systems to optimize certain objective

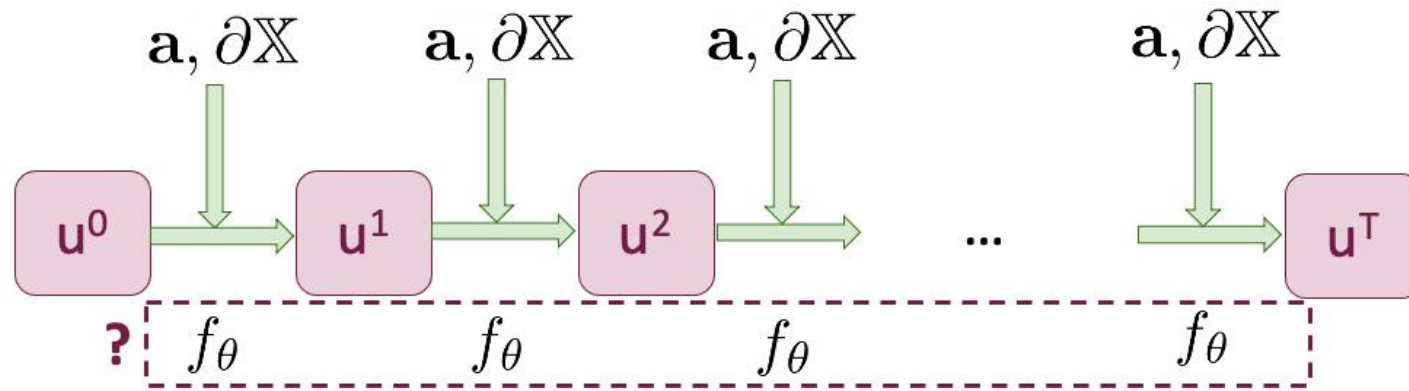
Main contribution:

- Backpropagation through the entire simulation rollout

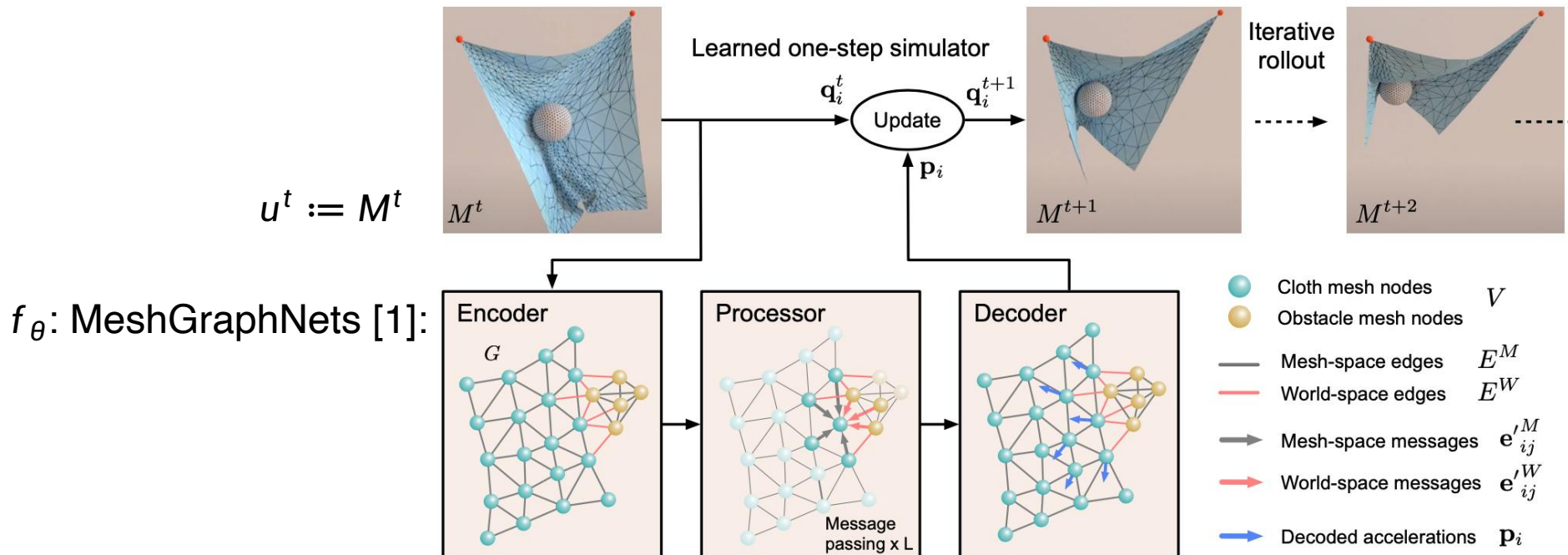


minimize drag for airfoil

Case study: physical design with differentiable learned simulators



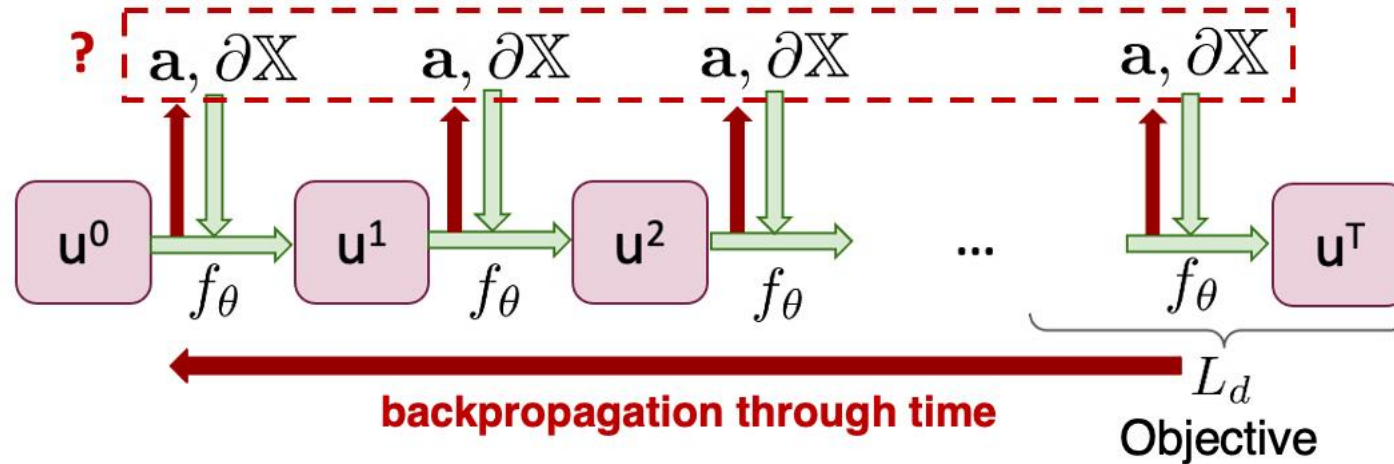
Training: given tuples of $\{(u^t, a, \partial\mathbb{X}, u^{t+1})\}$, learn $f_\theta: (u^t, a, \partial\mathbb{X}) \rightarrow u^{t+1}$



[1] Pfaff, Tobias, et al. "Learning mesh-based simulation with graph networks." ICLR 2021

Case study: physical design with differentiable learned simulators

Method:



Inference:

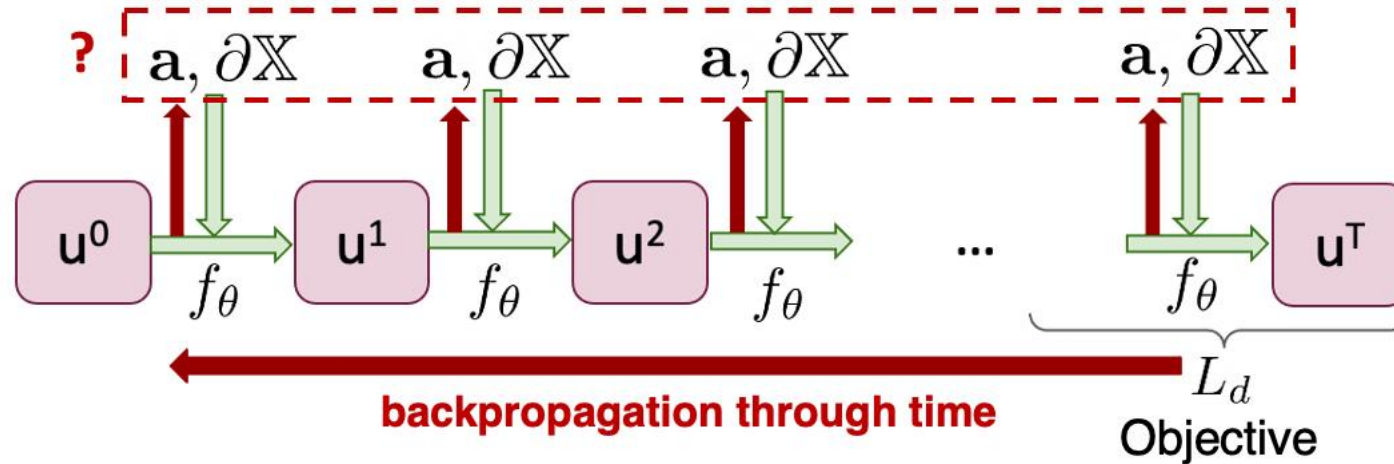
Objective for inverse design: maximize $L_d[\partial \mathbb{X}]$

$\partial \mathbb{X}$: boundary

To compute $\frac{\partial L_d[\partial \mathbb{X}]}{\partial (\partial \mathbb{X})}$, do backpropagation through time through the entire simulation

Case study: physical design with differentiable learned simulators

Method:



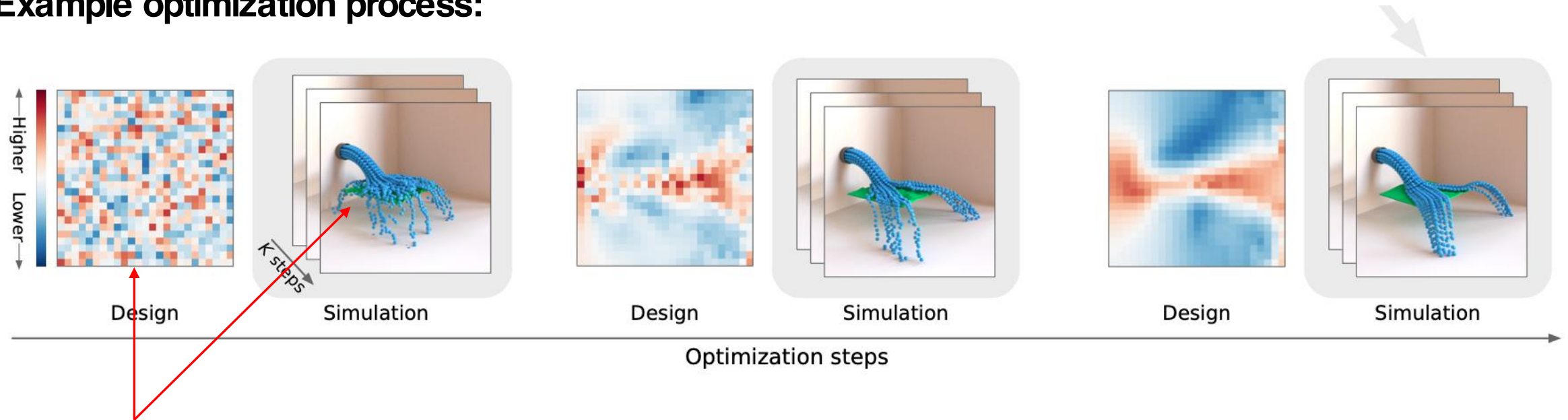
Inference:

Also employs two techniques:

1. **Ensembles:** train multiple forward models each with different data split. At inference, average the gradient
2. **Gradient checkpointing:** recompute the intermediate activations for each step as needed when doing the backpropagation through time

Case study: physical design with differentiable learned simulators

Example optimization process:

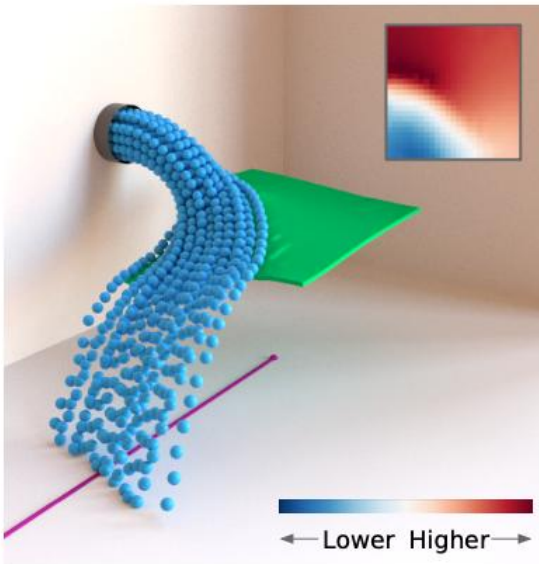


height of the green pad ($\partial \mathbb{X}$)

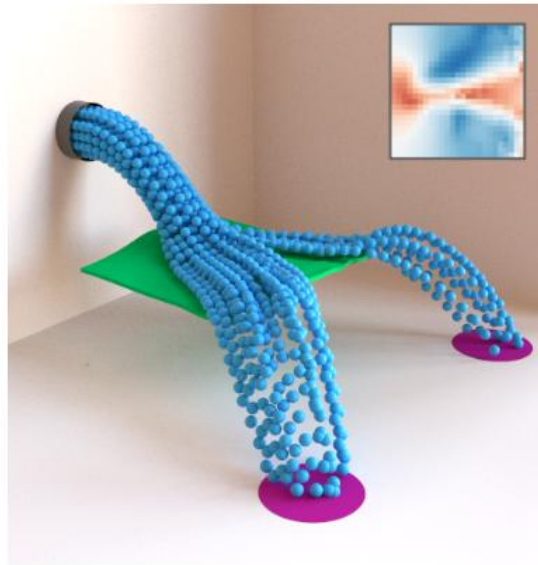
Case study: physical design with differentiable learned simulators

Results 1: 3D Watercourse

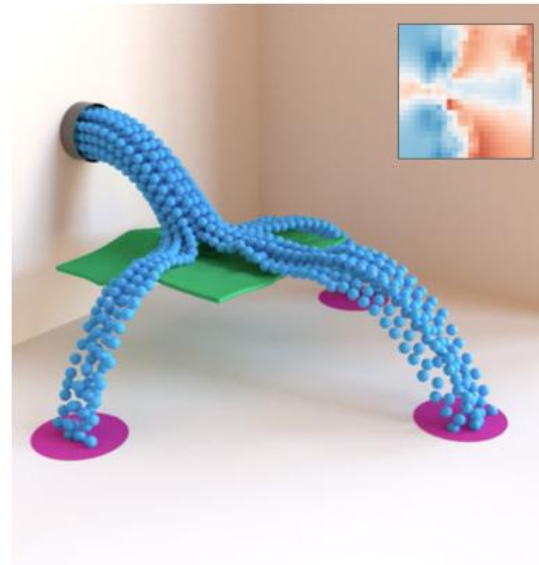
(a) Direction (GD-M)
 $r = 1.19 [1.184, 1.201]$



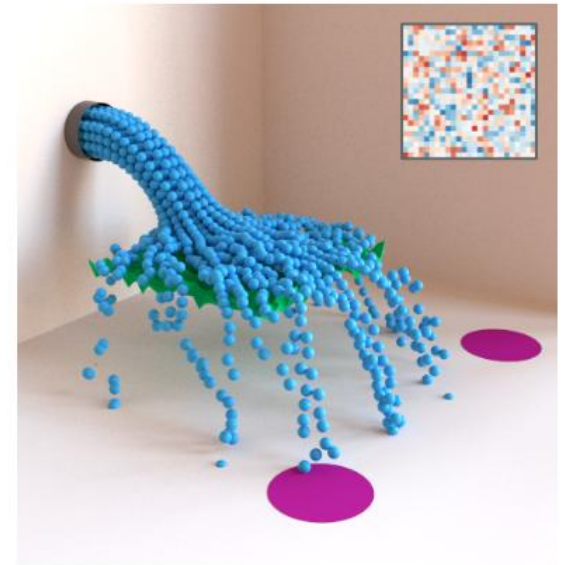
(b) 2 Pools (GD-M)
 $r = 0.90 [0.897, 0.909]$



(c) 3 Pools (GD-M)
 $r = 0.87 [0.844, 0.887]$



(d) 2 Pools (CEM-M)
 $r = 0.01 [0.004, 0.019]$

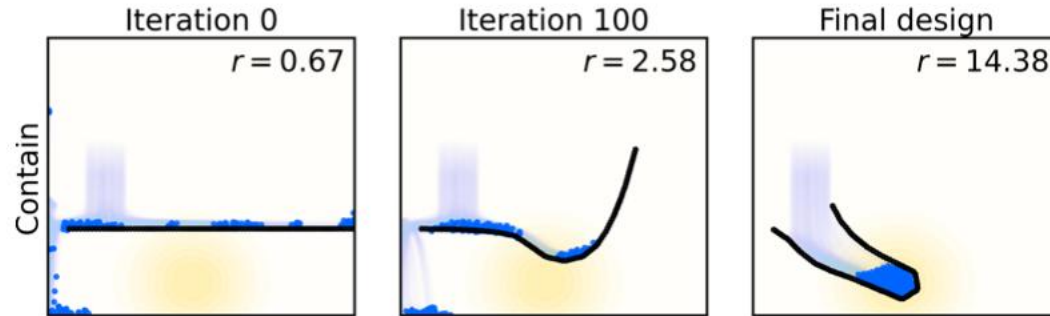


Proposed method (GD-M)

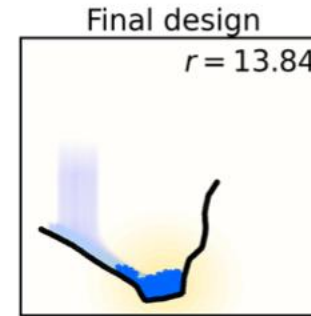
CEM: cross-entropy method [1],
a sample-based method

Case study: physical design with differentiable learned simulators

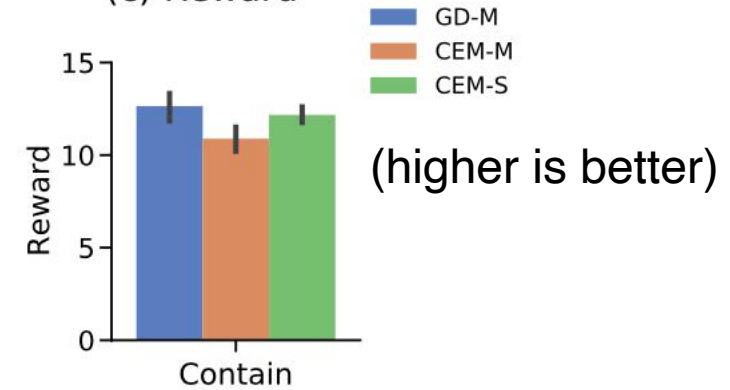
Results 2: 2D Fluid Tools (a) GD-M design evolution



(b) CEM-M

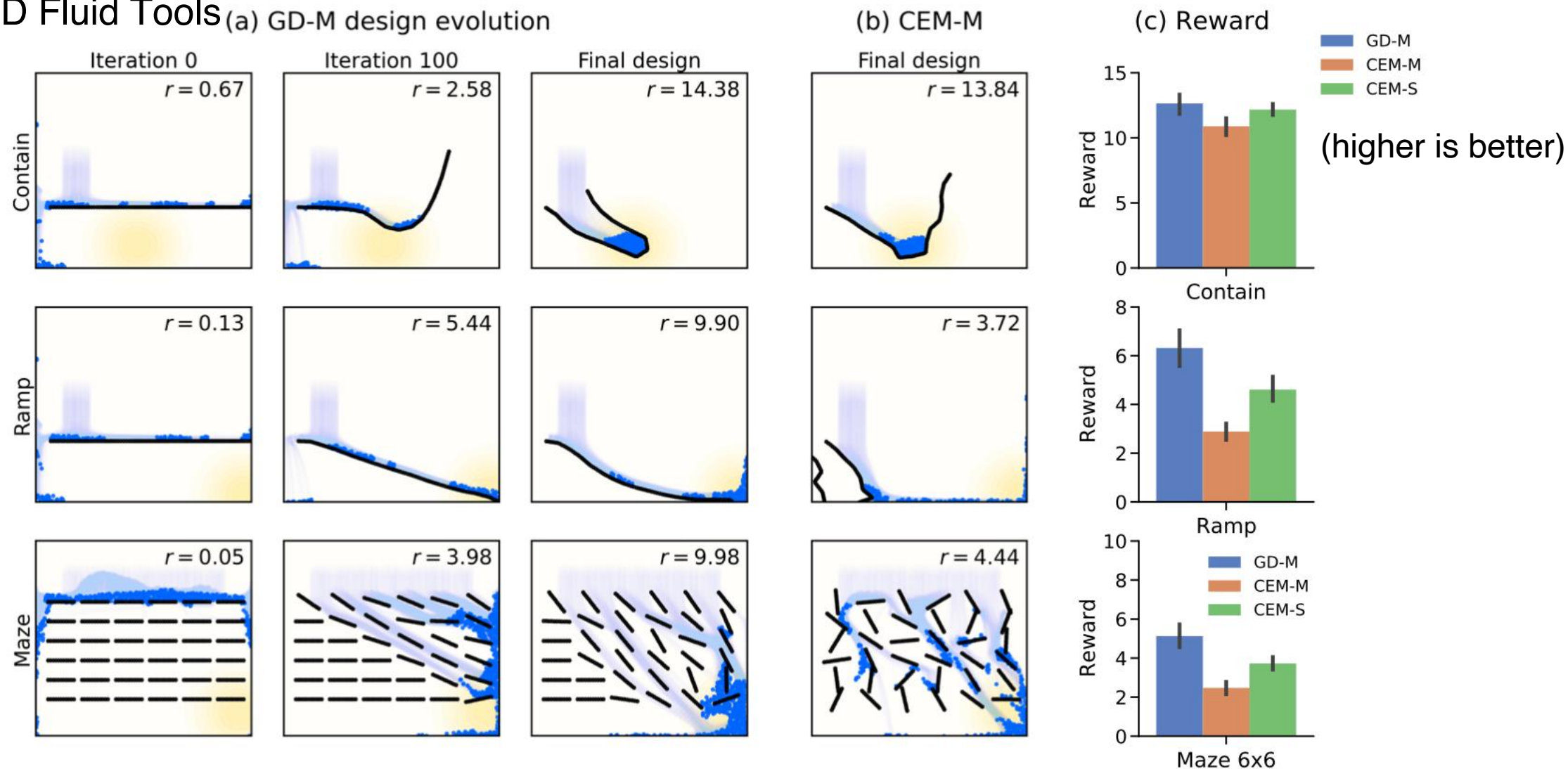


(c) Reward



Case study: physical design with differentiable learned simulators

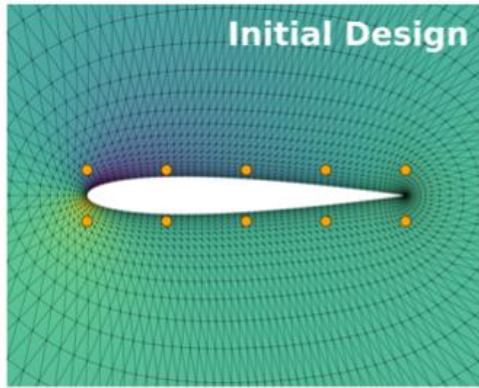
Results 2: 2D Fluid Tools



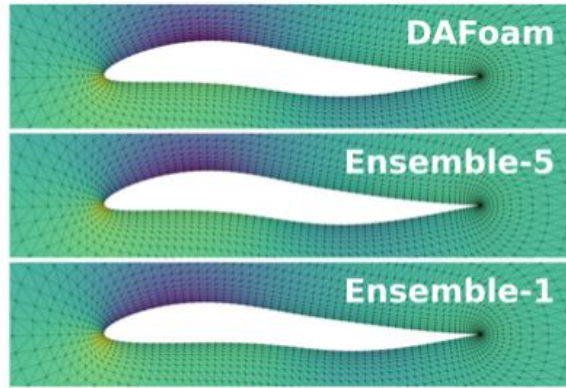
Case study: physical design with differentiable learned simulators

Results 3: airfoil

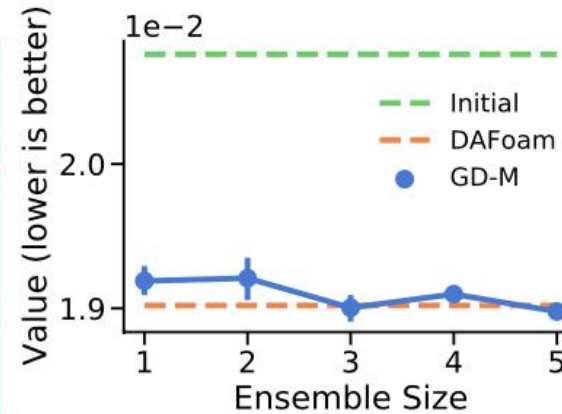
(a) Initial design



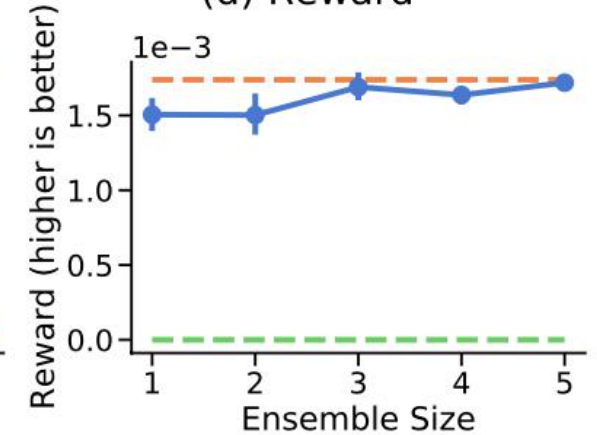
(b) Optimized designs



(c) Drag



(d) Reward



Case study: physical design with differentiable learned simulators

Significance: First work that combines GNN-based surrogate model with backpropagation through time for inverse design

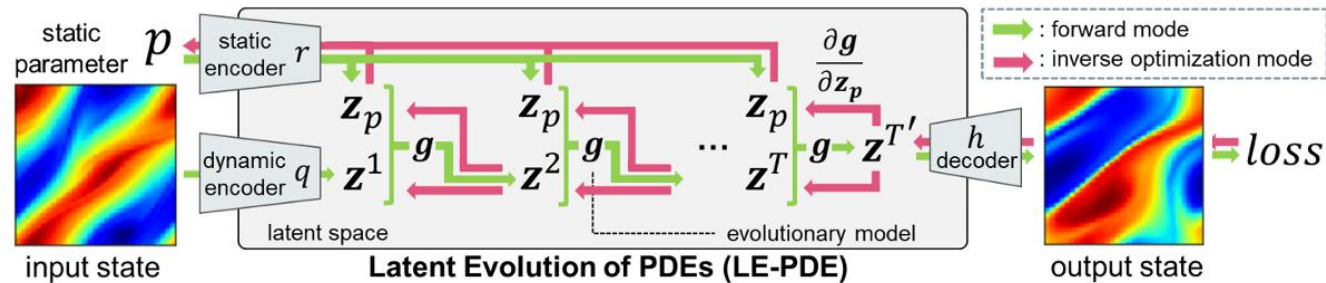
Limitation: computationally expensive, since

- The surrogate model f_θ is learning a mapping in the **input space**, which can have high dimension
- Simulation length is long (up to 300 steps)

Improvements: inverse design/problem in latent space

Inverse design by backpropagation through time in latent space

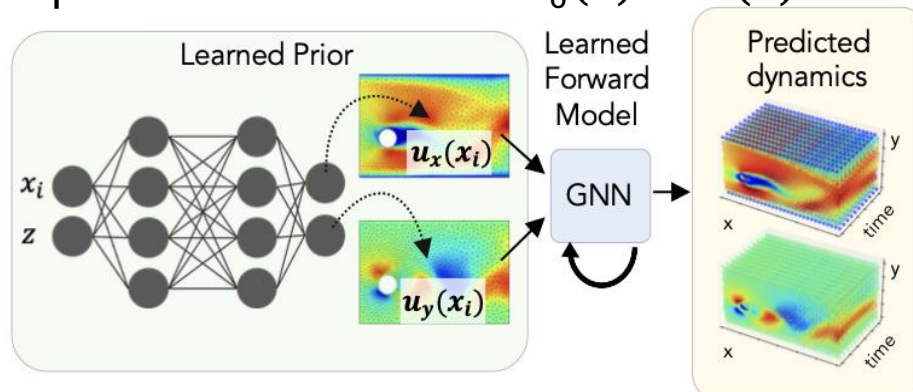
Wu *et al.* proposed LE-PDE[1] that learns forward model and performs inverse design in **latent space**, significantly improve speed while maintaining competitive accuracy (compared with FNO, MP-PDE, etc.)



[1] Wu, Tailin, Takashi Maruyama, and Jure Leskovec. "Learning to accelerate partial differential equations via latent global evolution." *NeurIPS* 2022

Solving inverse problem by optimization in latent space

Zhao et al. proposed solving inverse problem in latent space, which maps a latent vector z to $u_0(x)$ or $a(x)$.



[2] Zhao, Qingqing, David B. Lindell, and Gordon Wetzstein. "Learning to solve pde-constrained inverse problems with graph networks." *ICML* 2022.

AI for scientific design/control

1. **Surrogate model + backpropagation** for design and control
2. **Reinforcement learning** for control
3. **Diffusion models** for design and control

Task and challenge (lecture 6)

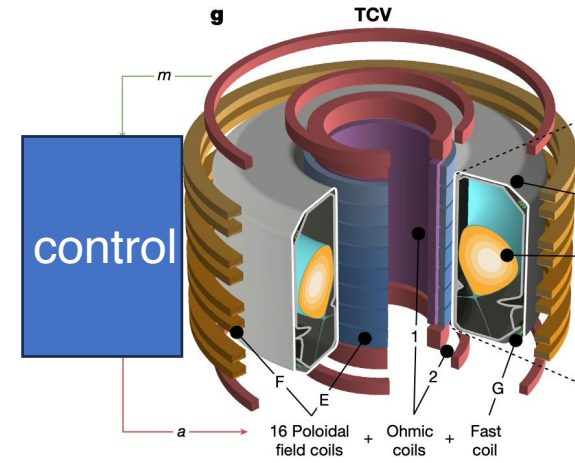
Task: To shape and maintain a high-temperature plasma within the tokamak vessel.

Challenge: This requires high-dimensional, high-frequency, closed-loop control using magnetic actuator coils, further complicated by the diverse requirements across a wide range of plasma configurations

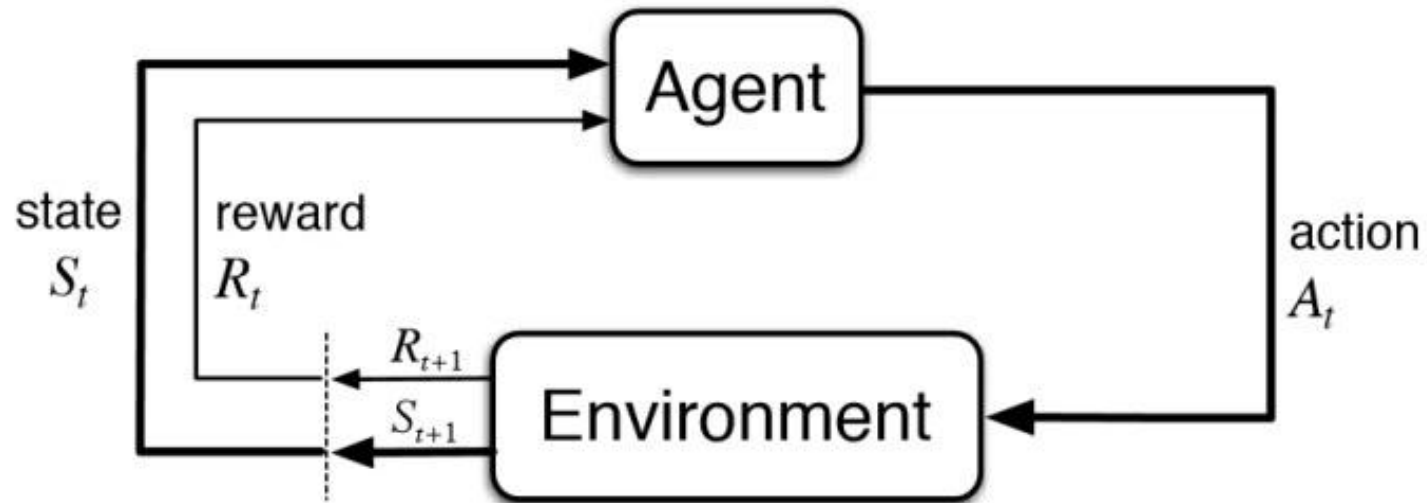
Each time step t have observation
and needs to output a control signal:

观测 m : input observation, R^{92}

控制 a : output control, R^{19}



Reinforcement learning (RL)



Environment: the tokamak

Goal: maximize the long-term expected reward w.r.t. to the policy $\pi(A_t|S_t)$

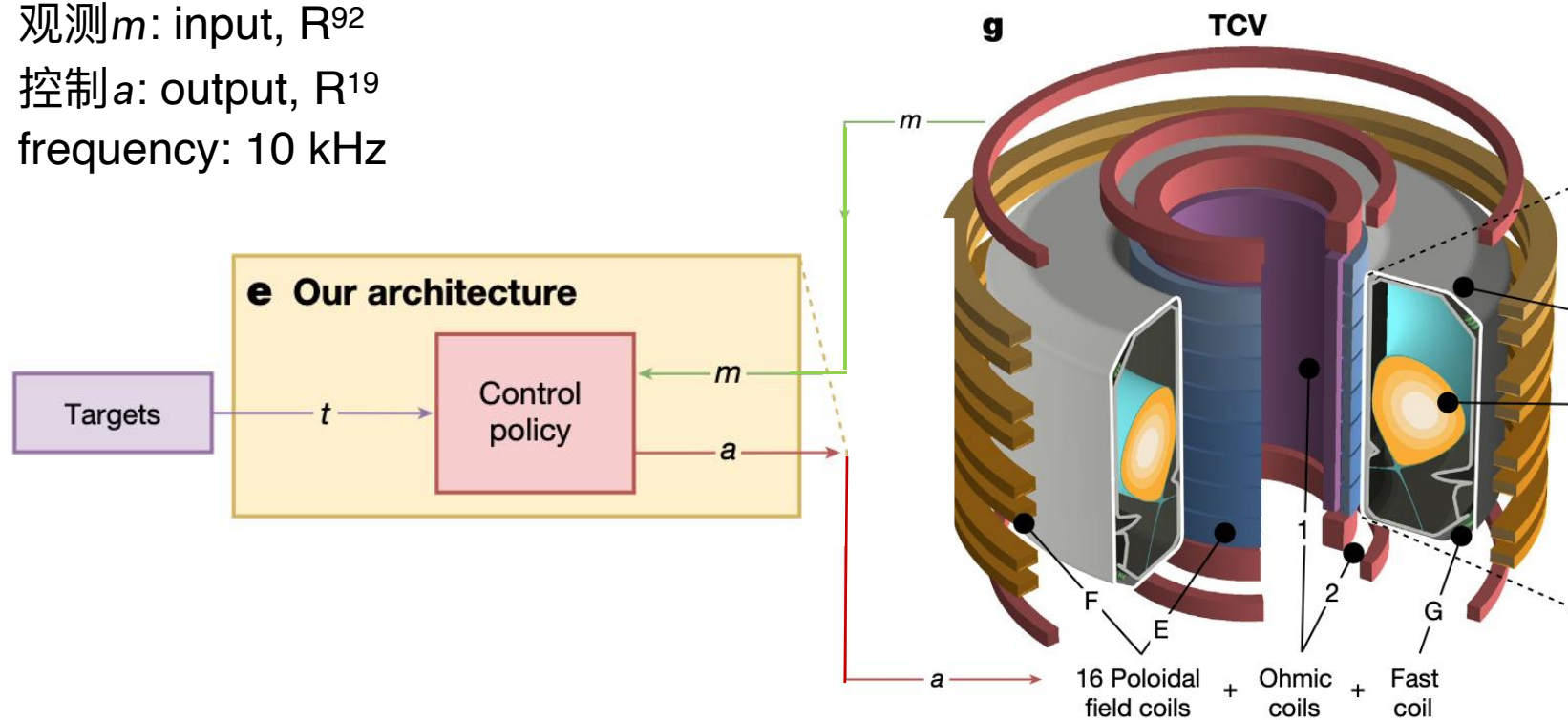
$$\max_{\pi(A_t|S_t)} \mathbb{E}_t[R_t]$$

Reinforcement learning (RL)

观测 m : input, $\mathbb{R}^{92}$

控制 a : output, $\mathbb{R}^{19}$

frequency: 10 kHz



AI for scientific design/control

1. **Surrogate model + backpropagation** for design and control
2. **Reinforcement learning** for control
3. **Diffusion models** for design and control

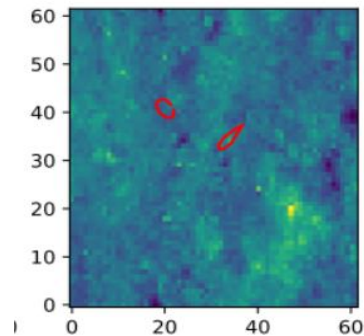
Limitation of prior methods

1. Surrogate model + backpropagation

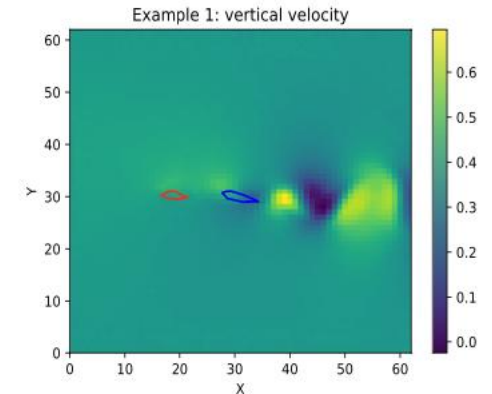
Limitations:

(1) Easy to fall into adversarial modes

Designed boundary
shape and fluid velocity



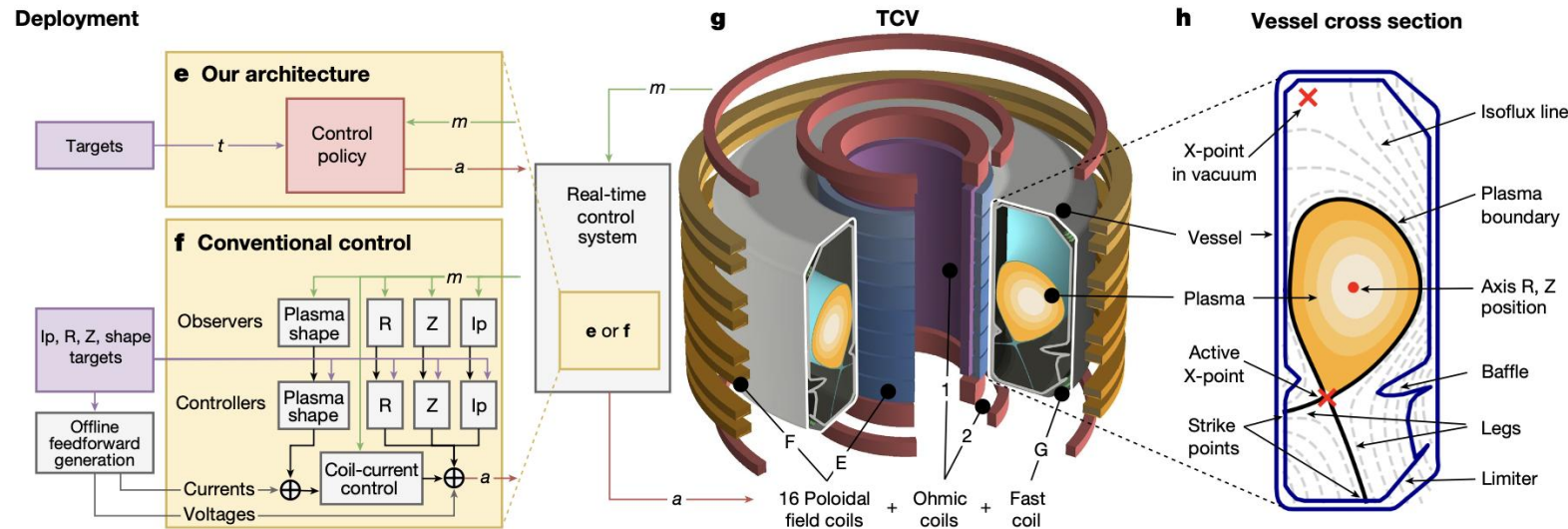
Reasonable boundary
shape and fluid velocity



(2) Hard to design more complex parameters

Limitation of prior methods

Reinforcement learning for **control** [1][2]



Limitations:

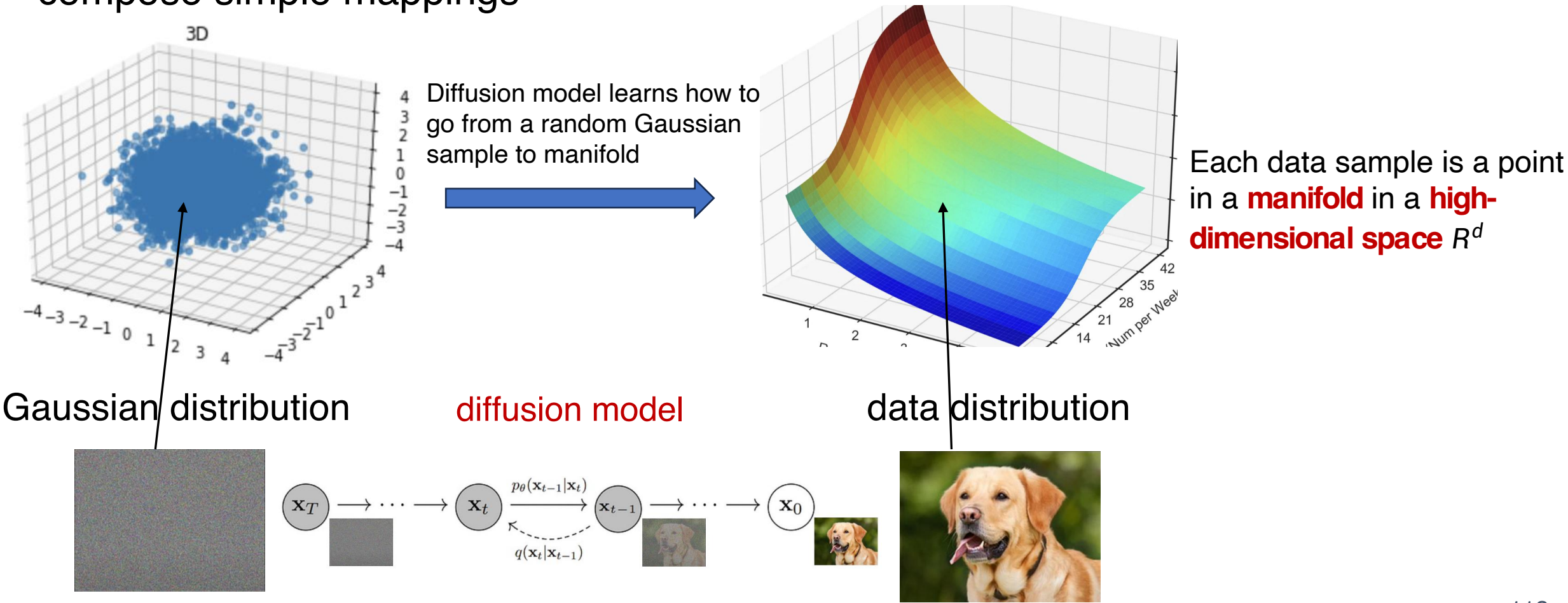
- (1) **Low sample efficiency:** requires large number of interactions
- (2) Difficulty for **high-dimensional** control (action space)
- (3) **Generalization:** not flexible to generalize to unseen objectives

[1] Degraeve, Jonas, et al. "Magnetic control of tokamak plasmas through deep reinforcement learning." *Nature* 602.7897 (2022): 414-419.

[2] Haarnoja, Tuomas, et al. "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor." International conference on machine learning. PMLR, 2018.

Diffusion models: Denoising perspective

Insight: to construct a complex mapping from A to B, it is much easier to compose simple mappings



Diffusion models: Denoising perspective, training and inference

x_0 : training data

x_t : training data with t steps of added noise

Training: ϵ_θ : denoising network to be learned

Inference (sampling):

Algorithm 1 Training

```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_\theta \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$   
6: until converged
```

input: data with t steps of added noise

predict: the noise ϵ added

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

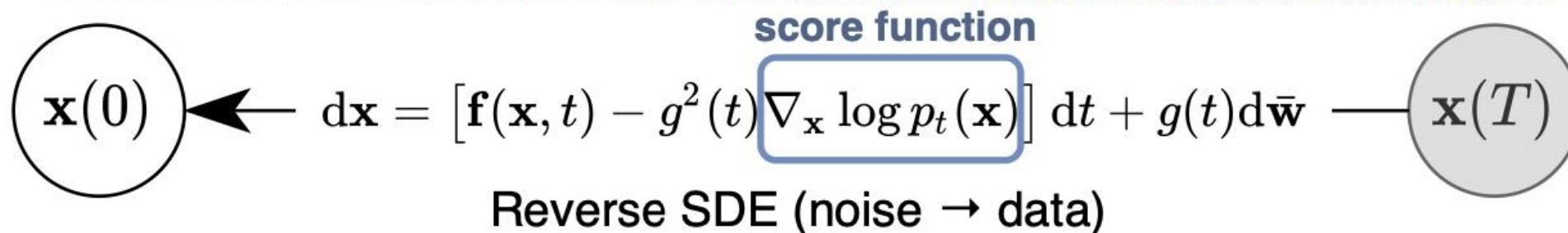
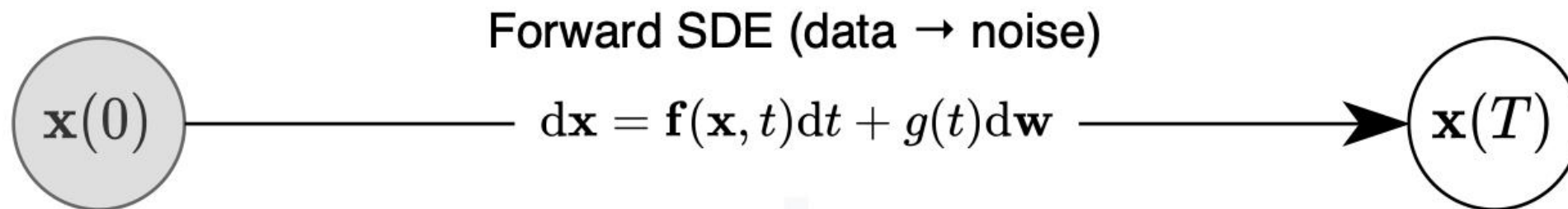
↑
denoise step-by-step

Limitation: Does not consider optimizing the samples with design objectives.
Can only sample examples similar to the training distribution.

[1] Ho, Jonathan, Ajay Jain, and Pieter Abbeel. "Denoising diffusion probabilistic models." *Advances in neural information processing systems* 33 (2020): 6840-6851.

Diffusion models: SDE perspective

SDE: stochastic differential equation



[1] Song, Yang, et al. "Score-based generative modeling through stochastic differential equations." *ICLR 2021*

Diffusion model: Energy perspective

x : data samples (e.g., image, trajectory)

Diffusion model essentially learns a “**energy**”-based model E_θ to model the **probability distribution**

$$p_\theta(x) \propto e^{-E_\theta(x)}$$

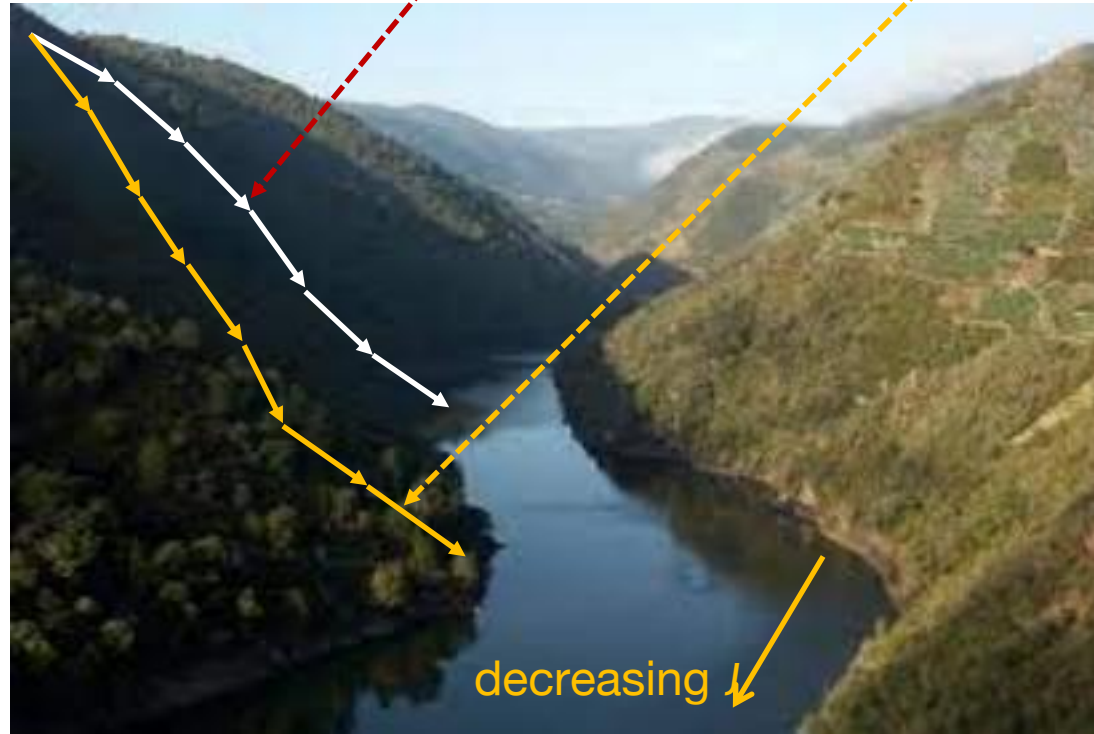
The denoising function $\epsilon_\theta(x_t)$ is essentially the gradient of the energy-based model

$$\epsilon_\theta(x) = \nabla_x E_\theta(x)$$

$$\hat{\gamma} = \arg \min_{\gamma, U_{[0,T]}} [E_\theta(U_{[0,T]}, \gamma) + \lambda \cdot \mathcal{J}(U_{[0,T]}, \gamma)]$$

Training: only learn E_θ

Inference: have an additional objective J



$U_{[0,T]}$: state sequence

γ : boundary condition

Composition by Addition of energy-based models

$$E(x) = E_{\text{smiling}}(x) + E_{\text{male}}(x) \quad \longleftrightarrow \quad p(x) = p_{\text{smiling}}(x)p_{\text{male}}(x)$$

Smiling



Male



Smiling
AND
Male

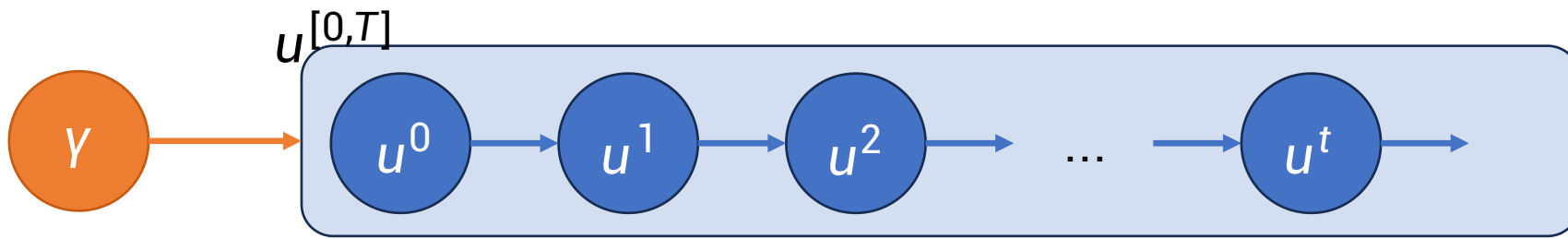


Prior autoregressive approach

γ : parameters

u^t : state at physical time t

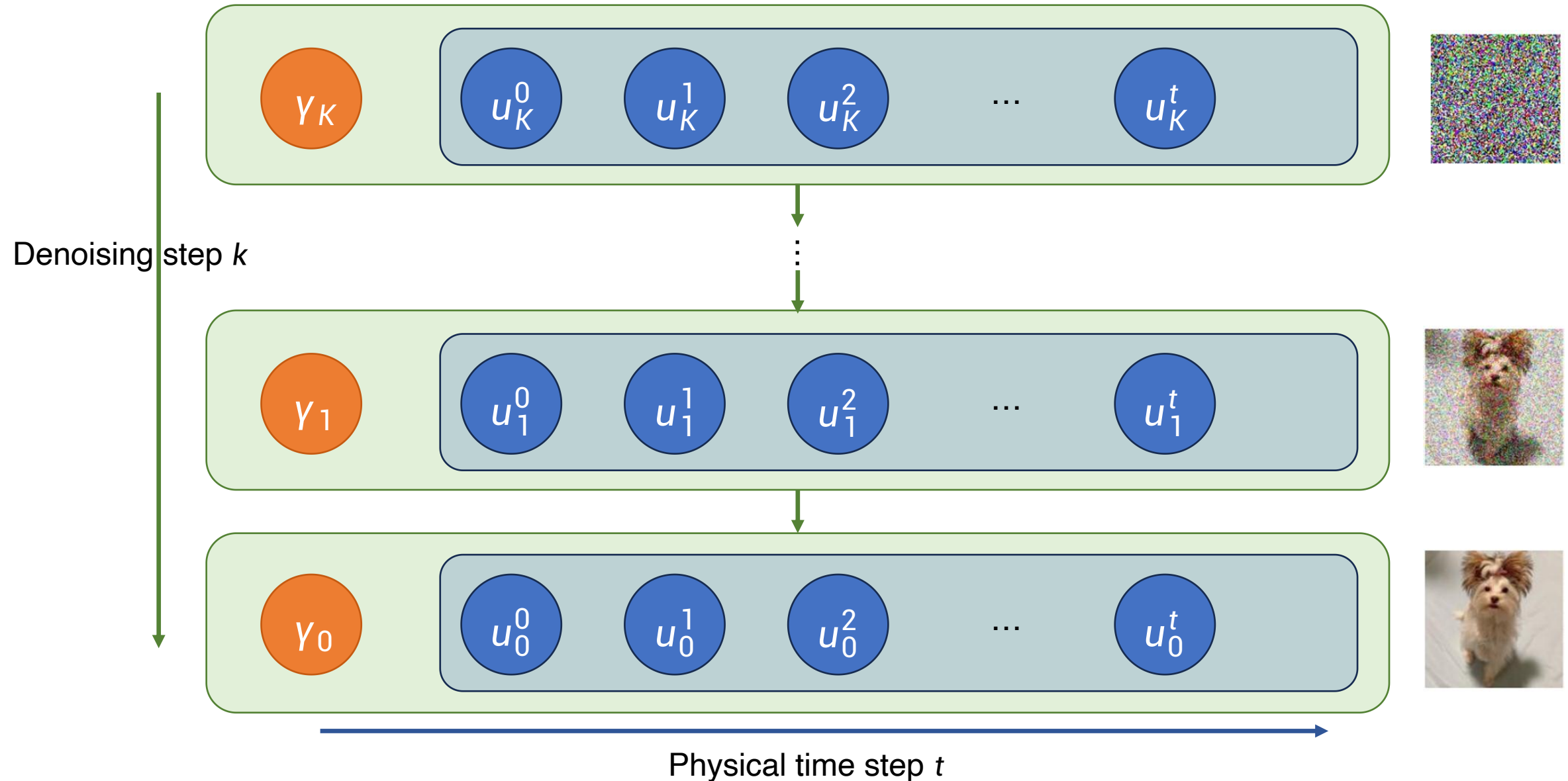
Arrow means conditional distribution



$$p(u^{[0,T]}|\gamma) = p(u^0) \prod_{t=1}^T p_{\theta}(u^t|u^{t-1}, \gamma)$$

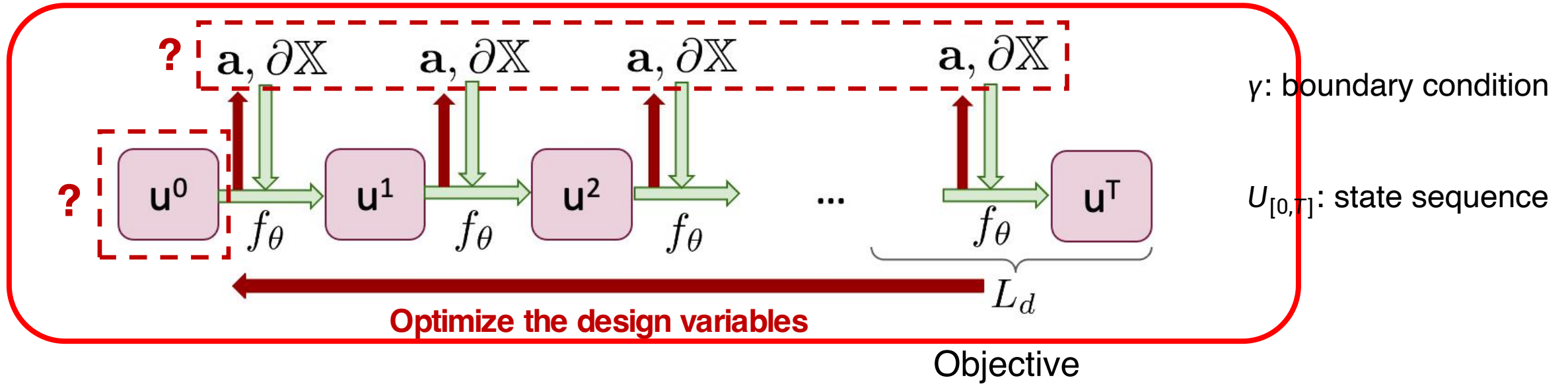
My contribution: CinDM

γ_k : parameters γ at denoising step k ,
 u_k^t : state at physical time t , at denoising step k



Our contribution

[1] Wu, Tailin, et al. "[Compositional Generative Inverse Design](#)." ICLR 2024 **spotlight**

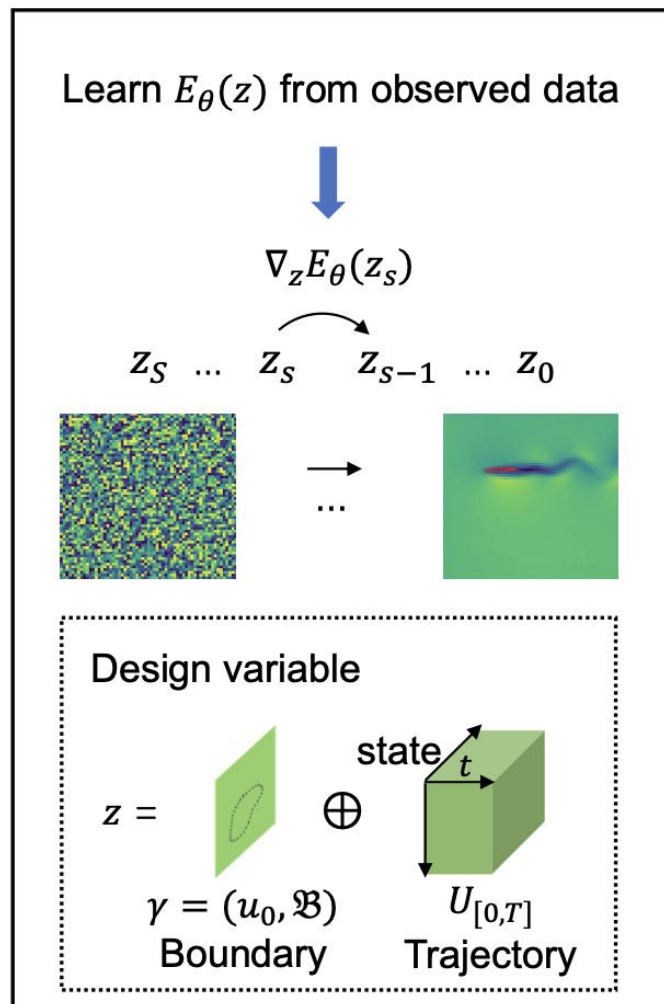


- Treat all the variables as a **single variable** $(U_{[0,T]}, \gamma)$ and learn to generate **simultaneously**, by learning $p(U_{[0,T]}, \gamma) \propto e^{-E_\theta(U_{[0,T]}, \gamma)}$
- **Composition** by adding the learned energy function E_θ on subsets of variables

$$\hat{\gamma} = \operatorname{argmin}_{\gamma, U_{[0,T]}} \left(\sum_{k=1}^M E_\theta(U_{[0,T]}, \gamma_k) + \lambda \cdot J(U_{[0,T]}, \gamma) \right)$$

Our method: architecture

Train

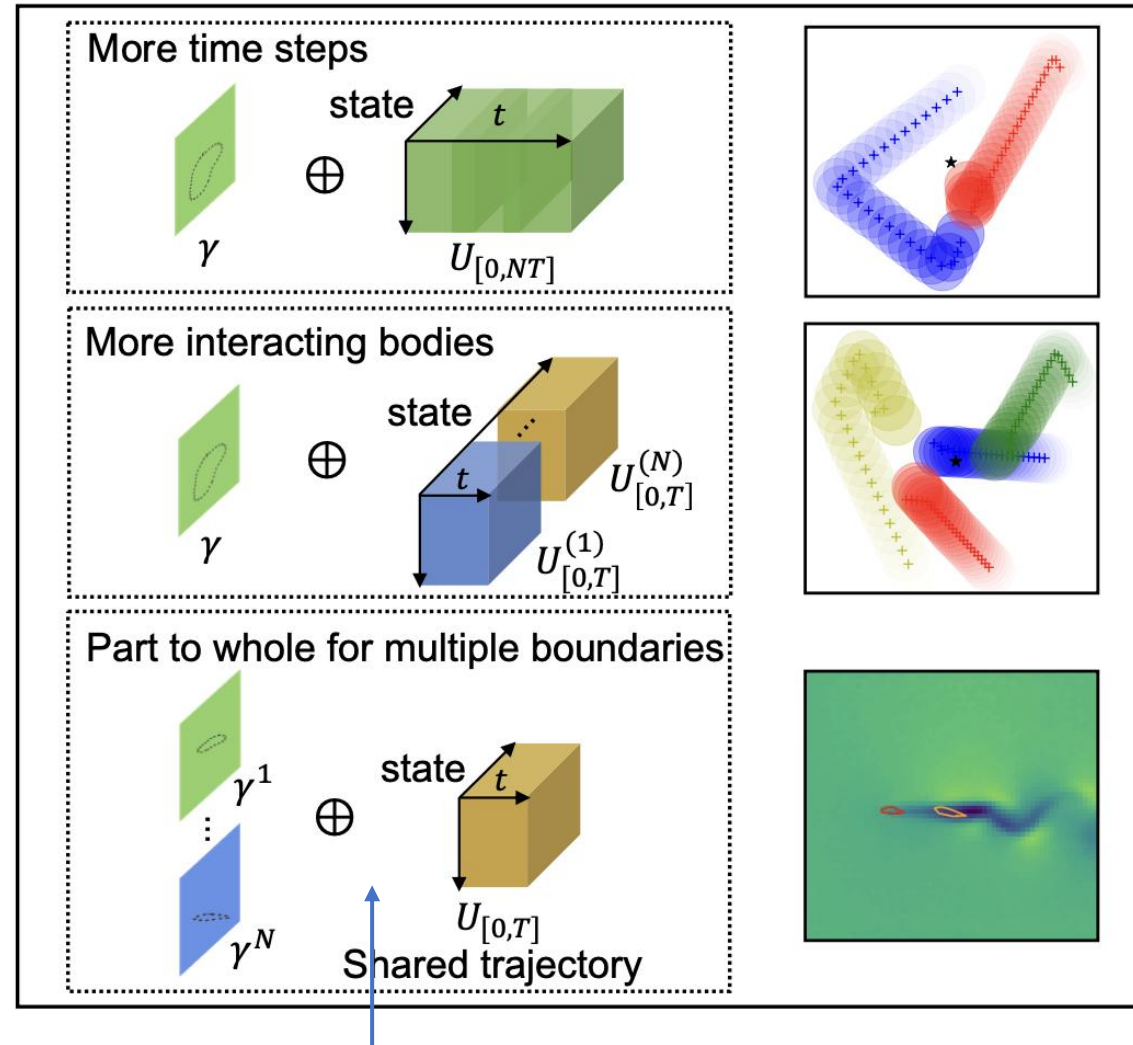


Design objective

$$\left. \begin{array}{l} \lambda \mathcal{J}(z) \\ + \\ E_\theta(z) \end{array} \right\} \rightarrow$$

Energy function

Compositional Design



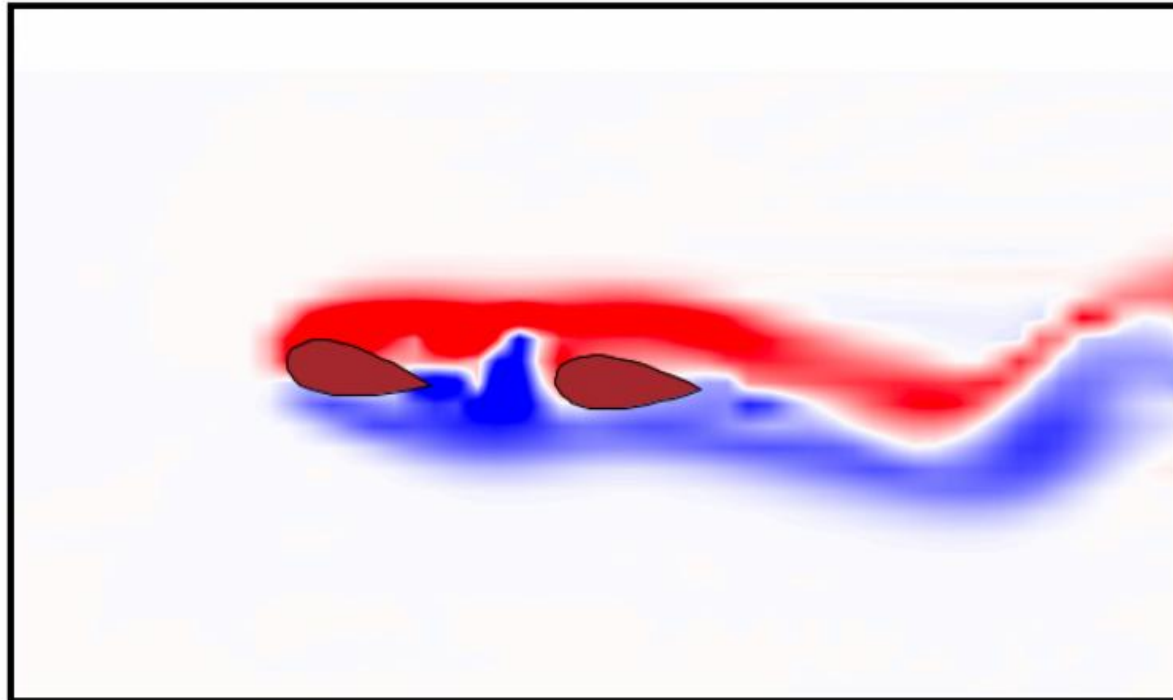
$$E_\theta(U_{[0,T]}, \gamma_1) + \dots + E_\theta(U_{[0,T]}, \gamma_N) + \mathcal{J}(U_{[0,T]}, \cup \{\gamma_i\})$$

Experiment 2: airfoil design, part-to-whole composition

Training: Consider a **single airfoil** interacting with the air flow Learning $E_{\theta}(U_{[0,T]}, Y)$

Inference: Consider **multiple airfoils**, maximize lift-to-drag ratio (升阻比) : $(J = \frac{lift}{drag})$

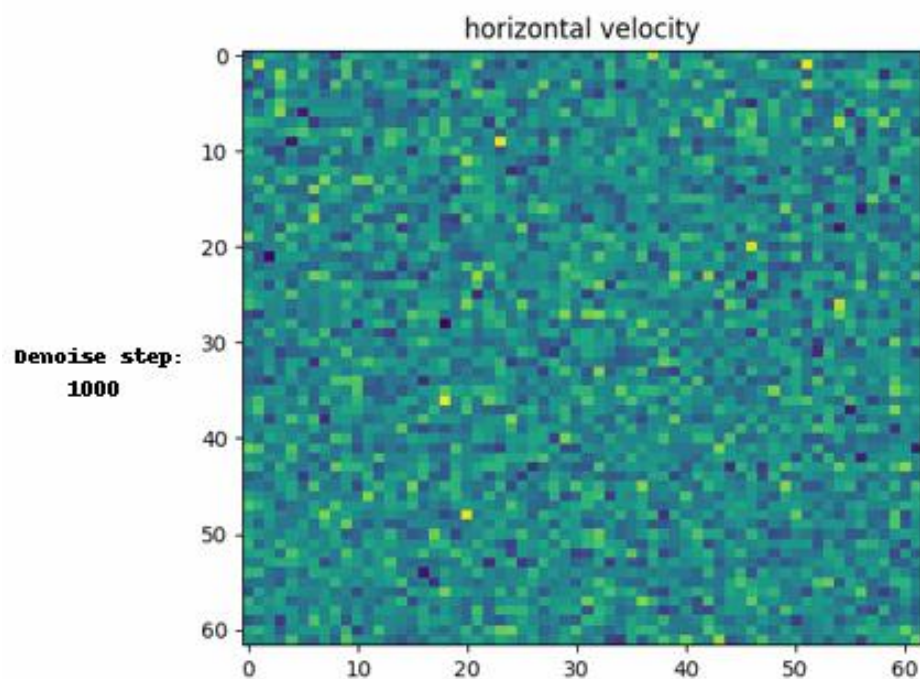
Sampling from $E = E_{\theta}(U_{[0,T]}, Y_1) + \dots + E_{\theta}(U_{[0,T]}, Y_N) + J(U_{[0,T]}, \cup \{Y_i\})$



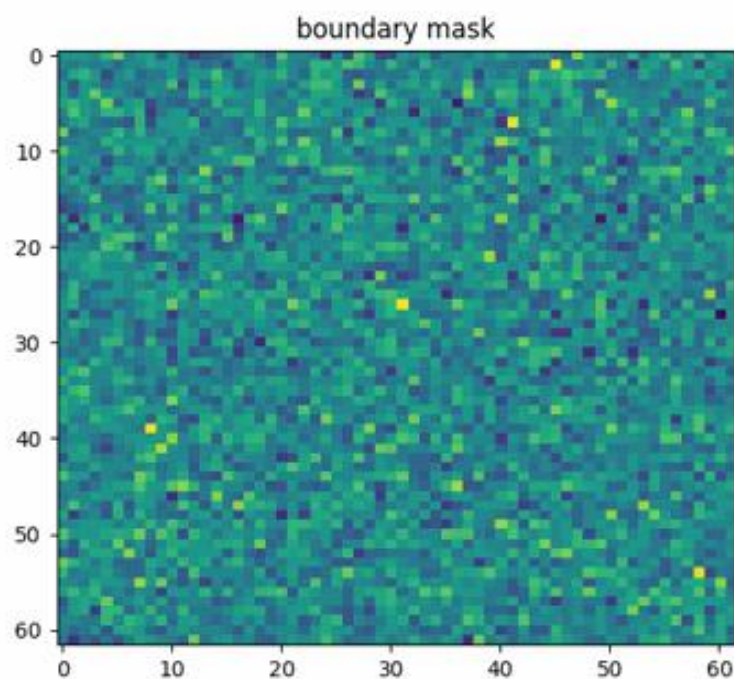
Example of Lily-Pad simulation

Experiment 2: airfoil design, part-to-whole composition

推理生成过程： 从高斯分布到设计出来的速度场轨迹和机翼边界设计

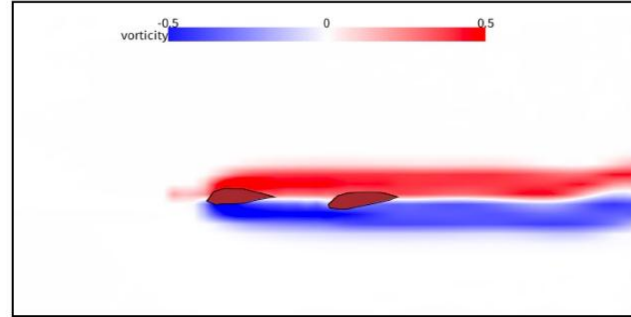


流体速度场

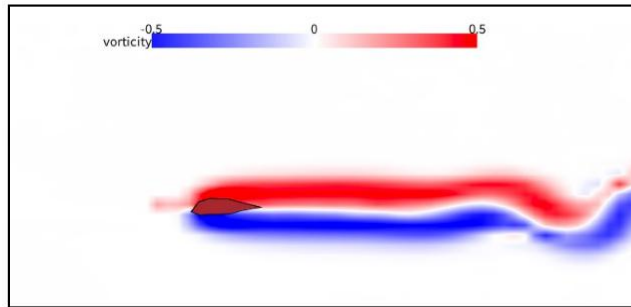


机翼边界设计

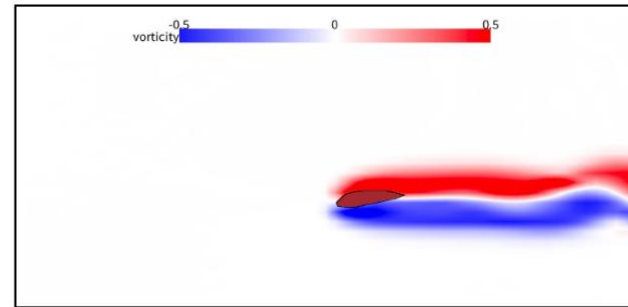
Experiment 2: airfoil design, part-to-whole composition



(a) Formation flying of airfoils A and B



(b) Single flying of airfoil A



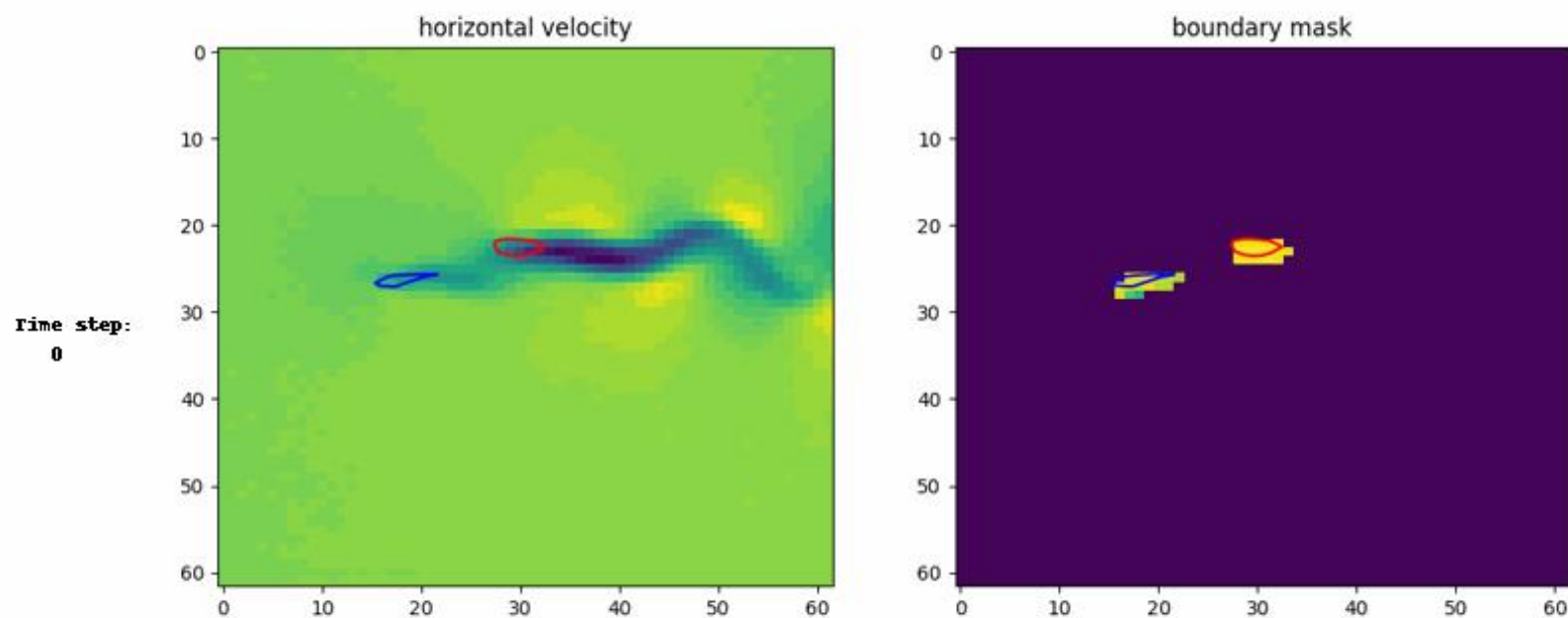
(c) Single flying of airfoil B

Our model discovers **formation flying**

- Reducing the drag by 53.6%
- increasing the lift-to-drag ratio by 66.1%

Experiment 2: airfoil design, part-to-whole composition

生成的速度场轨迹和机翼边界设计：

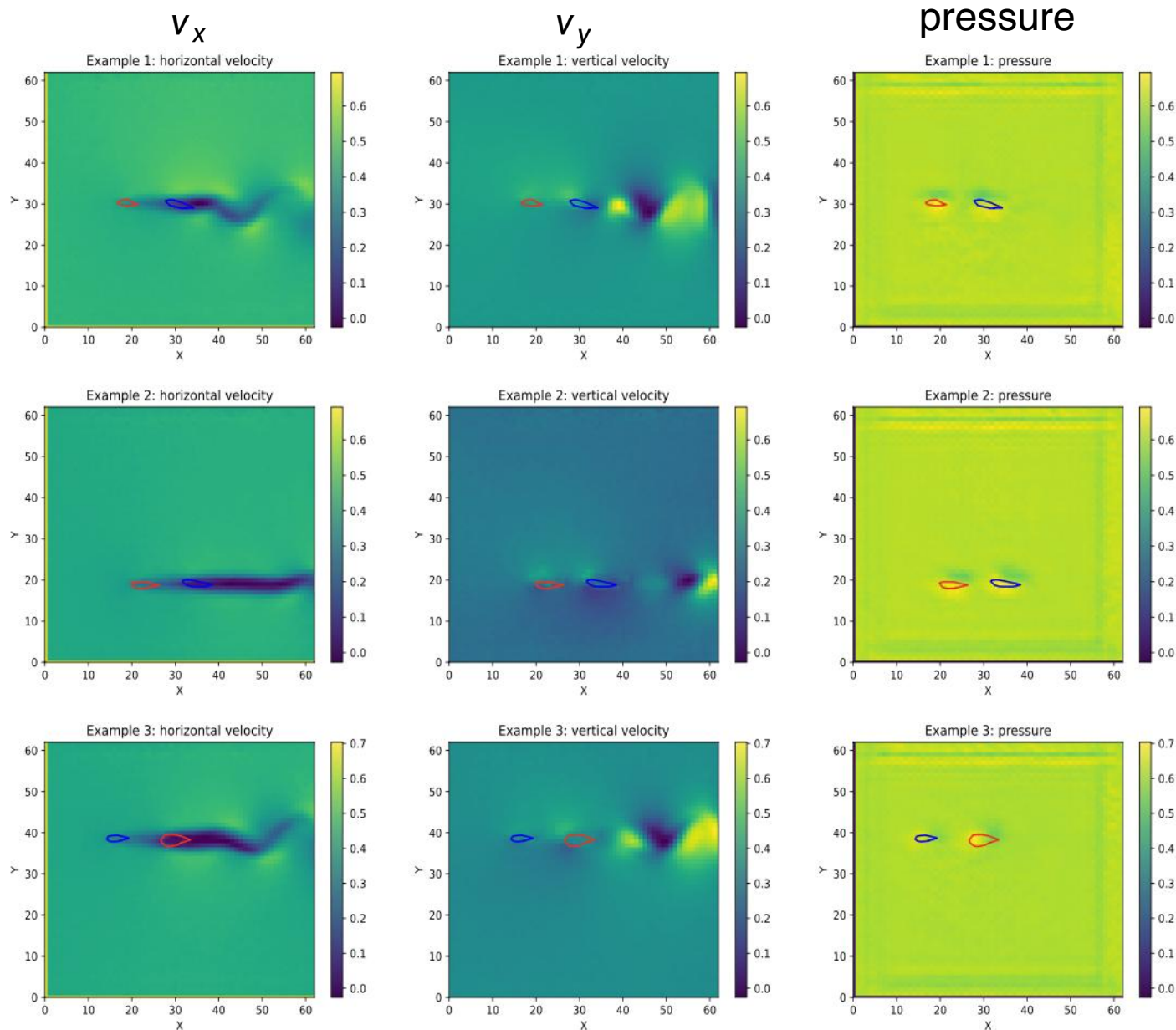


Experiment 2: airfoil design, part-to-whole composition

Method	1 airfoil		2 airfoils	
	design obj ↓	lift-to-drag ratio ↑	design obj ↓	lift-to-drag ratio ↑
CEM, FNO	0.0932	1.4005	0.3890	1.0914
CEM, LE-PDE	0.0794	1.4340	0.1691	1.0568
Backprop, FNO	0.0281	1.3300	0.1837	0.9722
Backprop, LE-PDE	0.1072	1.3203	0.0891	0.9866
CinDM (ours)	0.0797	2.1770	0.1986	1.4216

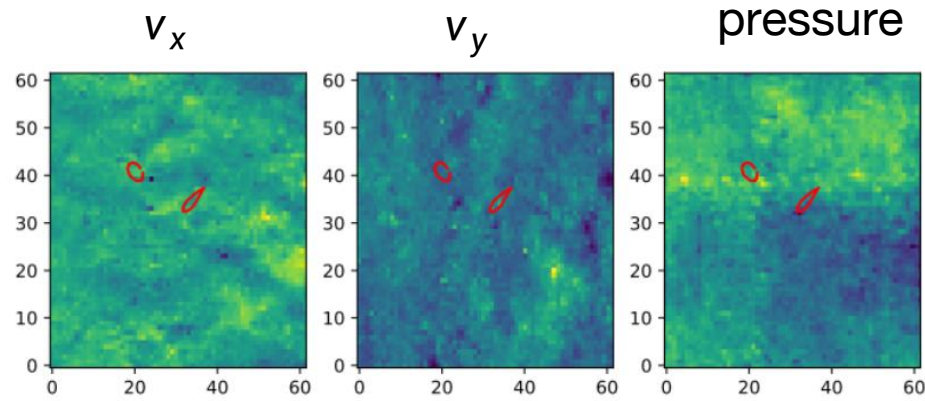
Experiment 2: airfoil design, part-to-whole composition

CinDM
generated:

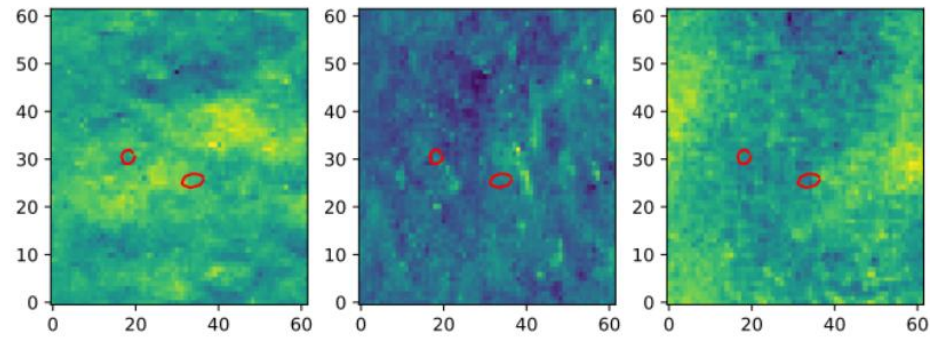


Experiment 2: inverse design by **baseline neural models**

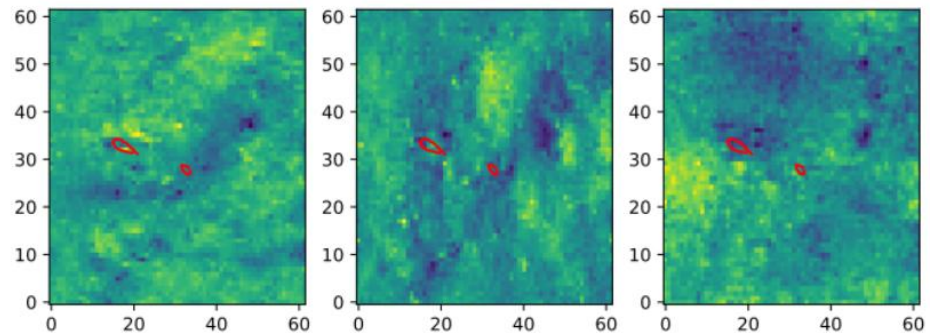
Adversarial modes by
baseline neural model
(FNO with CEM):



(a)



(b)



AI for scientific design/control

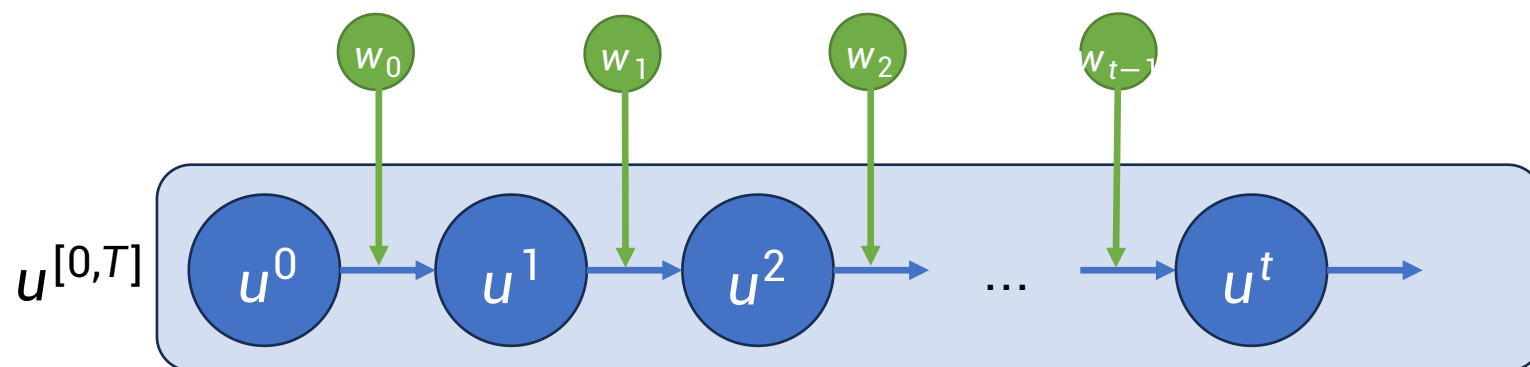
1. **Surrogate model + backpropagation** for design and control
2. **Reinforcement learning** for control
3. **Diffusion models** for design and **control**



w^t : external control at time t

u^t : state at physical time t

Arrow means conditional distribution



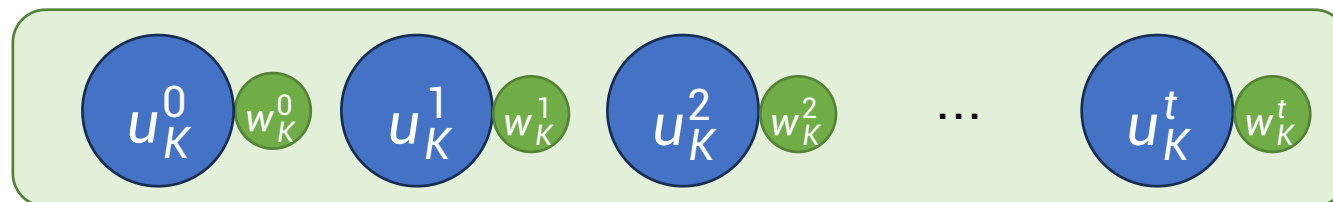
$$p(u^{[0,T]} | w^{[0,T-1]}) = p(u^0) \prod_{t=1}^T p_{\theta}(u^t | u^{t-1}, w^{t-1})$$



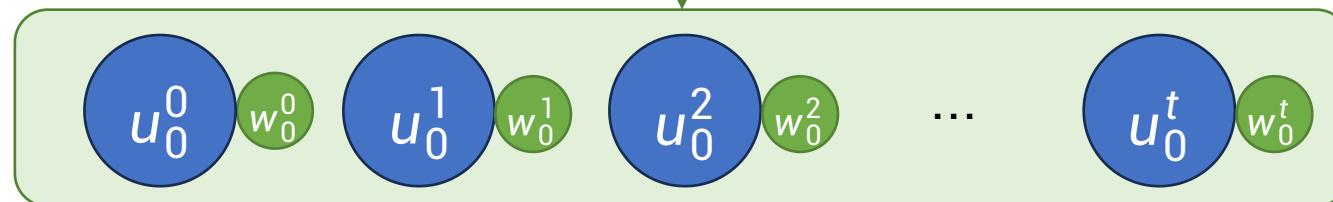
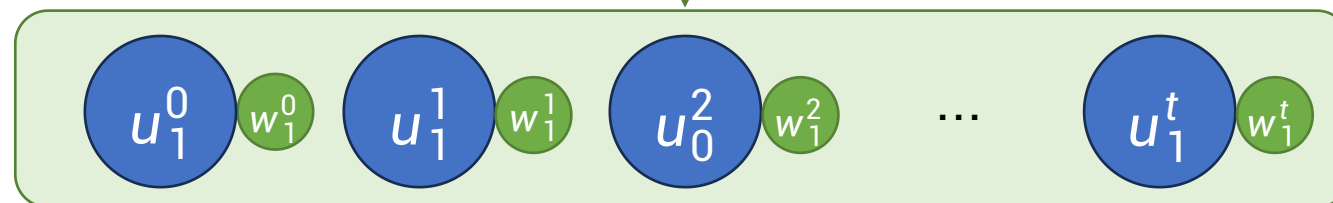


贡献1：将仿真和控制融为同一任务

去噪时间步 k



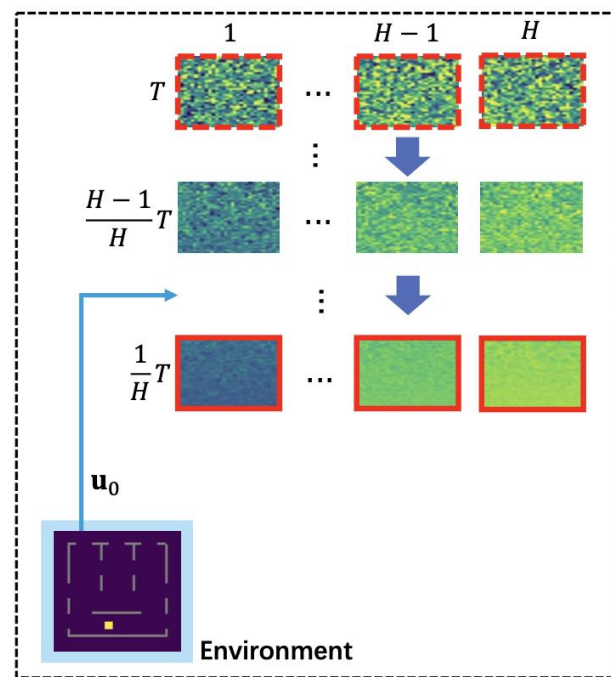
w_k^t : control at physical time t , at denoising step k ,
 u_k^t : state at physical time t , at denoising step k



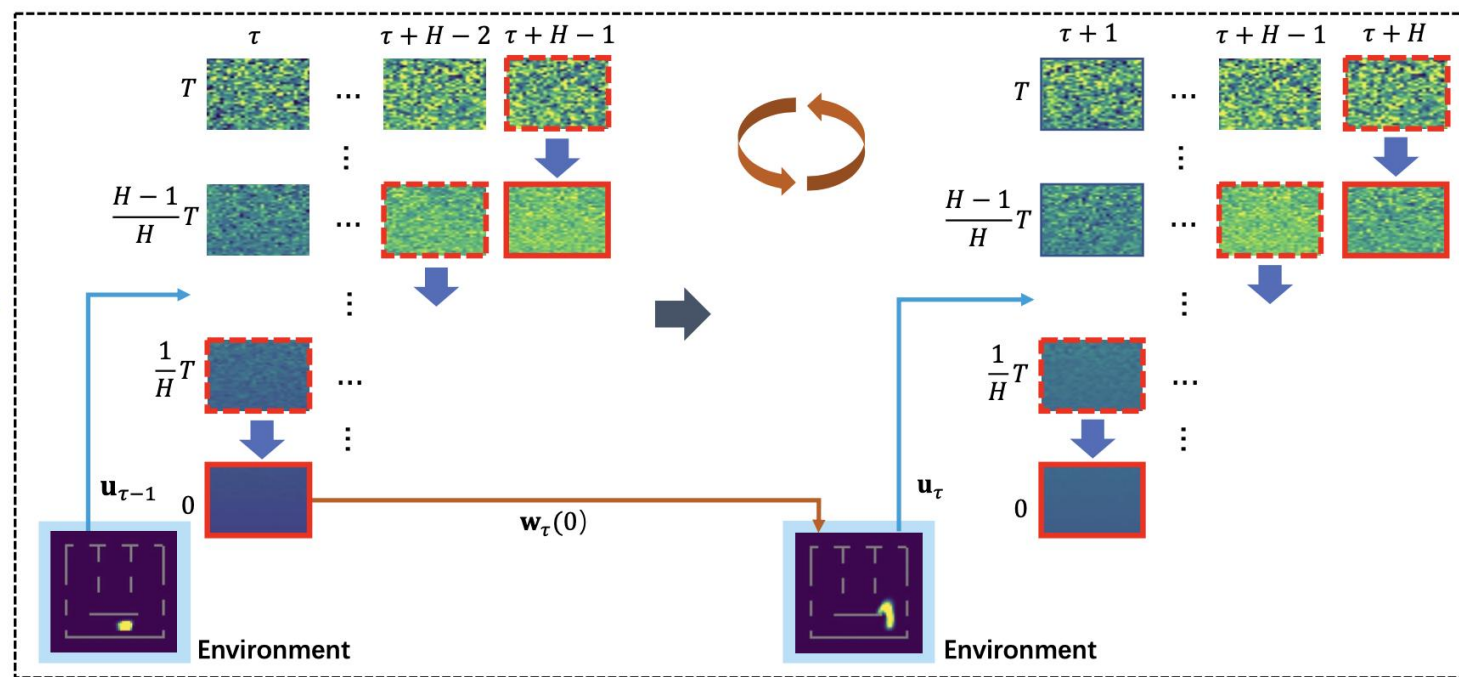
物理时间步 t

不同的物理时间步有不同的噪声（越靠近当下去噪步数越多），同时将环境的新状态实时反馈到未来控制序列和状态轨迹的生成过程中，实现高效闭环控制

Initialization

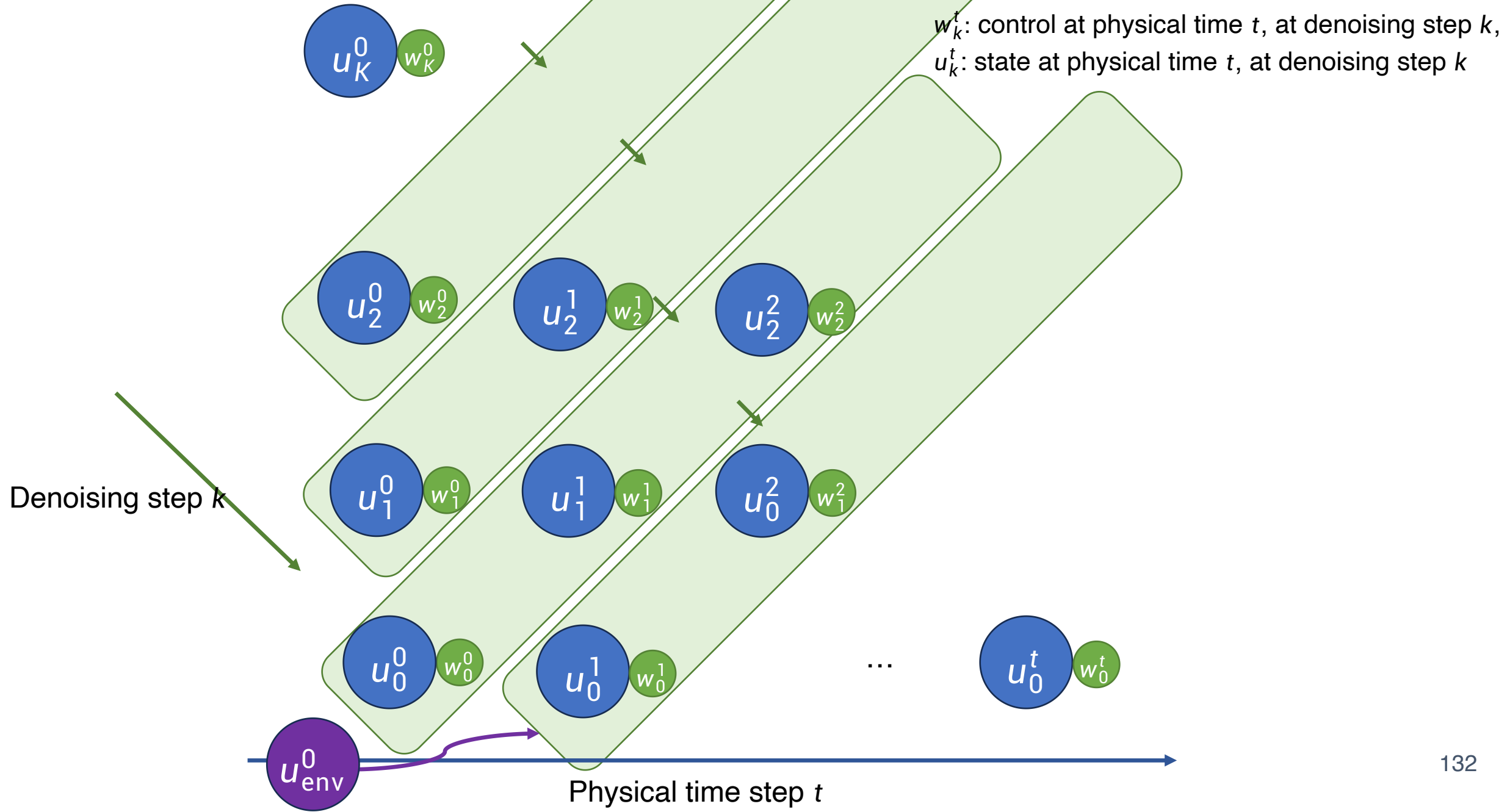


Closed-loop Control ($\tau \geq 1$)



$= z_\tau(t) = [u_\tau(t), w_\tau(t)]$ Sampling process Start sampling End sampling Control signal System State

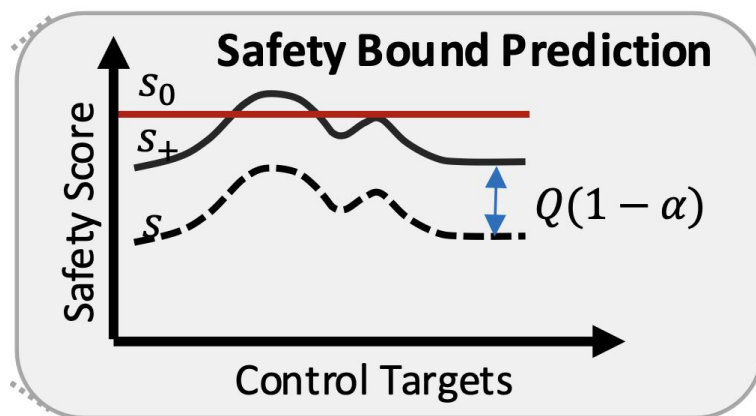
Our contribution:



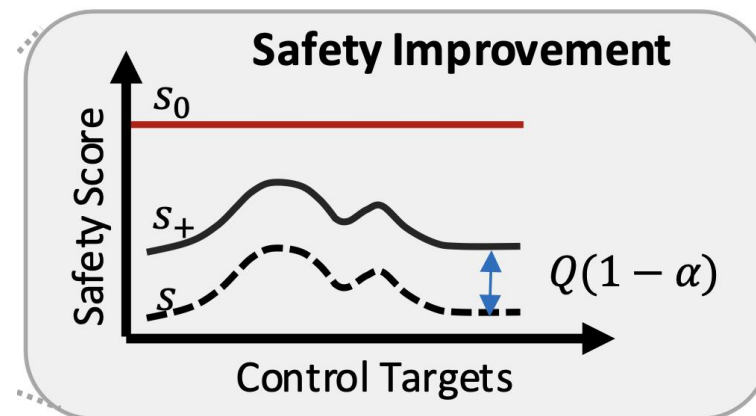
将共形预测融入扩散生成控制中，实现可证明的安全控制

$$\mathbb{P}(s(\mathbf{u}) \in CI_{\theta}(1 - \alpha, D_{cal})) \geq 1 - \alpha.$$

系统状态以大于等于 $1 - \alpha$ 的概率位于安全区间内



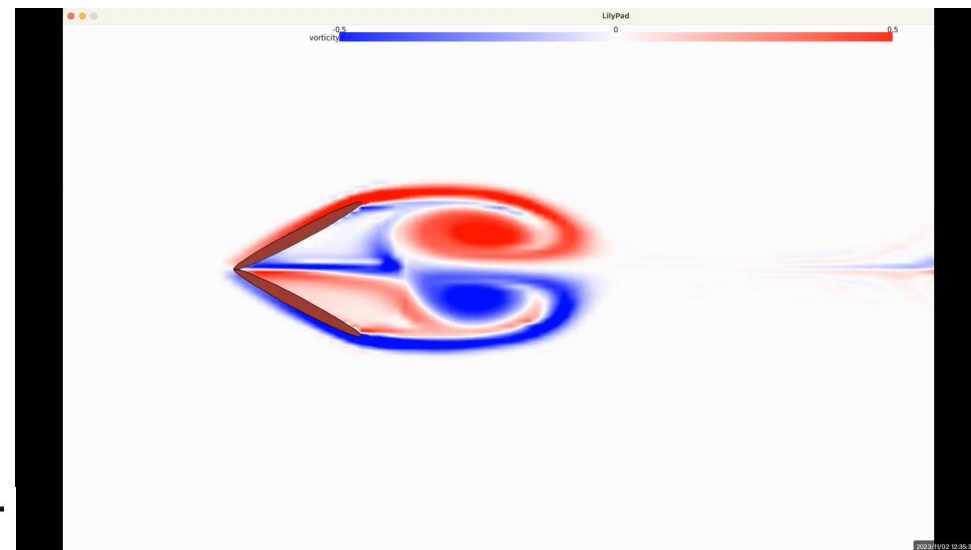
得到预测值和共形置信区间



通过微调优化安全上界

The fluid dynamic is governed by Navier-Stokes Equation:

$$\begin{cases} \frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} - \nu \nabla^2 \mathbf{v} + \nabla p = 0 \\ \nabla \cdot \mathbf{v} = 0 \\ \mathbf{v}(0, \mathbf{x}) = \mathbf{v}_0(\mathbf{x}) \end{cases}$$

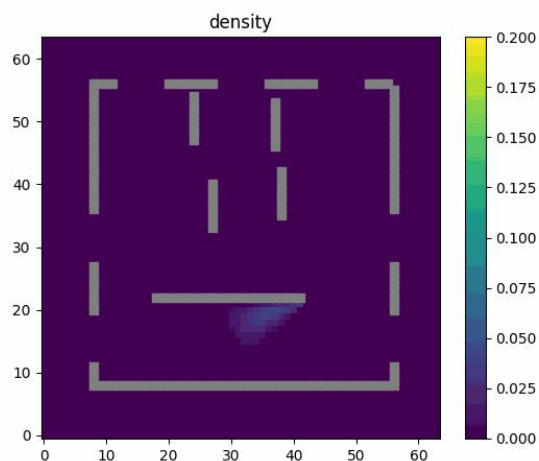
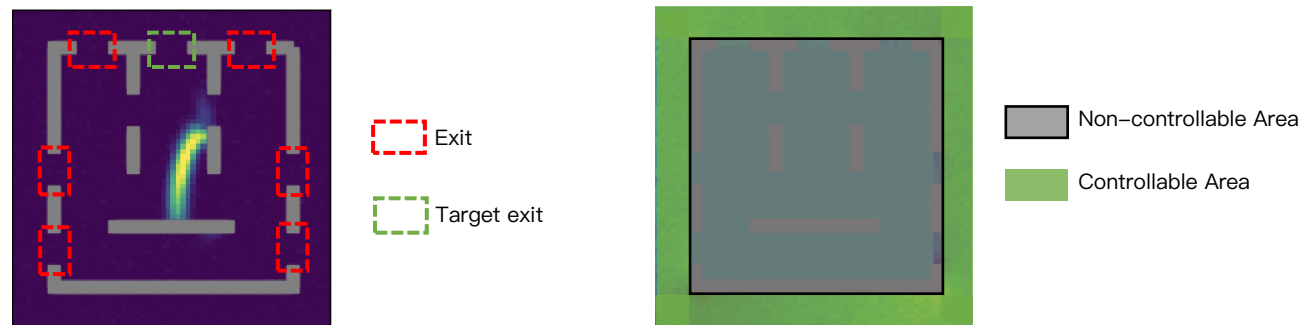


	Full observation			Partial observation		
	$\bar{v} \uparrow$	$R(\mathbf{w}) \downarrow$	$\mathcal{J} \downarrow$	$\bar{v} \uparrow$	$R(\mathbf{w}) \downarrow$	$\mathcal{J} \downarrow$
MPC	25.72	0.0112	109.17	-150.51	0.1791	329.59
SL	-76.94	0.1286	205.57	-102.98	0.1188	221.79
SAC (surrogate-solver)	-166.96	0.0069	18.14	-153.09	0.0057	158.82
SAC (offline)	-158.66	0.0069	165.58	-206.21	0.0058	211.96
BC	30.48	0.0629	32.44	20.08	0.0556	35.48
BPPO	<u>107.67</u>	0.0867	<u>-20.93</u>	<u>54.83</u>	0.0518	<u>-3.02</u>
DiffPhyCon-lite (ours)	95.04	0.0746	-20.47	2.92	0.0779	74.97
DiffPhyCon (ours)	279.87	0.2058	-74.11	150.21	0.1269	-23.32

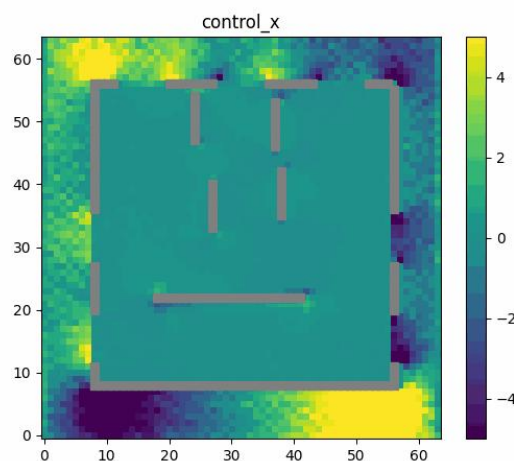
模型效果超过深度强化学习



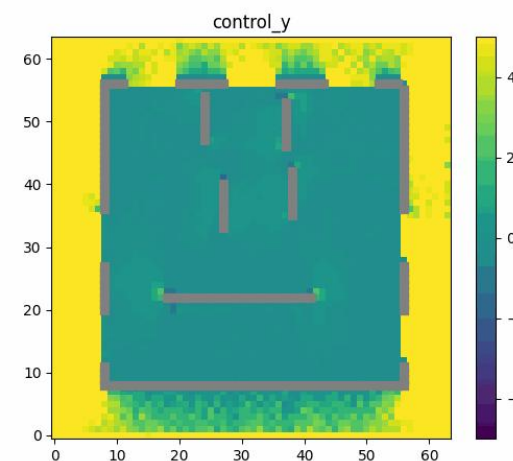
Method	$\mathcal{J} \downarrow$
BC	0.3085
BPPO	0.3066
SAC (surrogate-solver)	0.3212
SAC(offline)	0.6503
DiffPhyCon-lite (ours)	0.2324
DiffPhyCon (ours)	0.2254



Smoke in a fluid

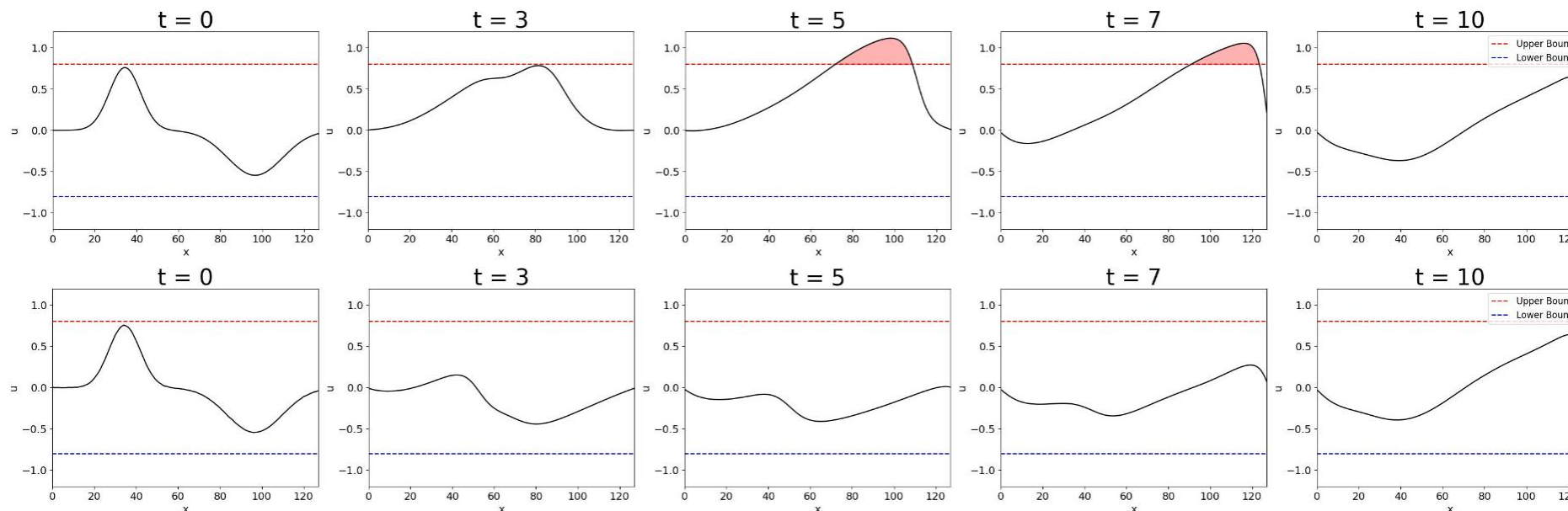


External control





实验3: Burgers方程安全控制



Methods	$\mathcal{J} \downarrow$	$\mathcal{R}_{\text{sample}} \downarrow$	$\mathcal{R}_{\text{time}} \downarrow$	$\mathcal{R}_{\text{point}} \downarrow$
BC	0.0001	38%	13%	1.2%
BC-Safe	0.0002	14%	3%	0.2%
PID	0.0968	0%	0%	0.0%
SL-Lag	0.0115	0%	0%	0.0%
MPC-Lag	0.0092	0%	0%	0.0%
CDT	0.0012	16%	3%	0.2%
TREBI	0.0074	0%	0%	0.0%
SafeDiffCon	0.0011	0%	0%	0.0%

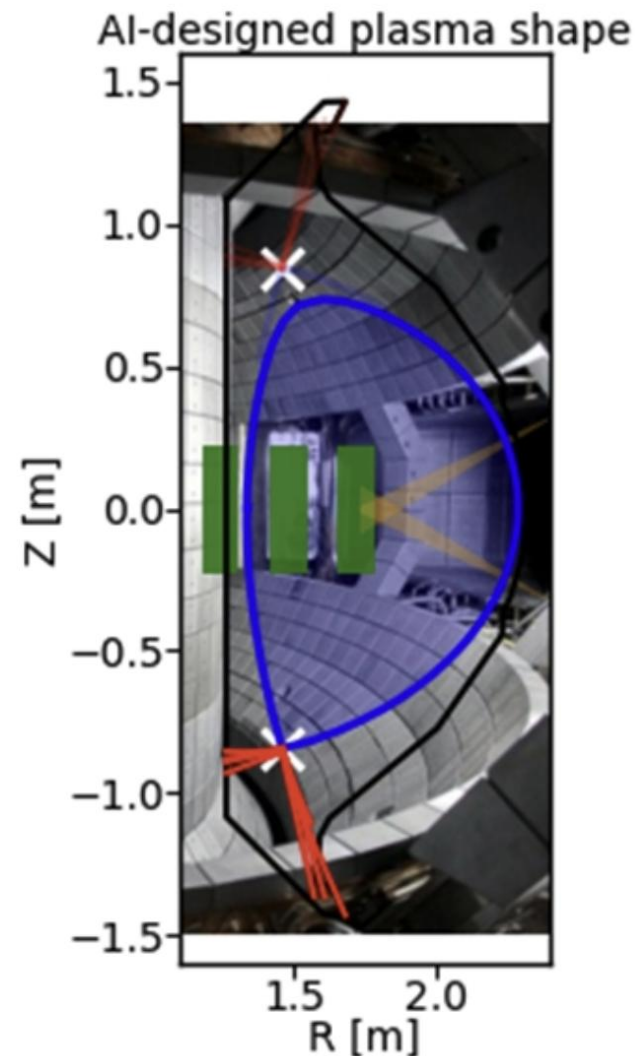


KSTAR代理模型环境：

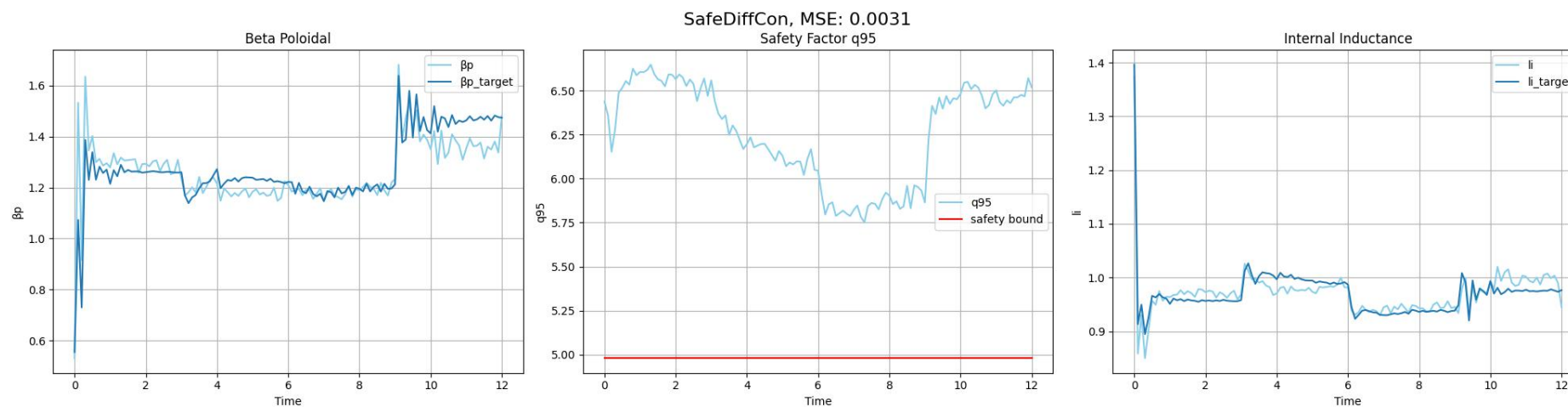
$$\mathcal{J} := \int_{\Omega \times [0, T]} |\beta_p(t, x) - \beta_p^*(x)|^2 dx dt,$$

$$s := - \inf_{(t, x) \in [0, T] \times \Omega} \{q_{95}(t, x)\}.$$

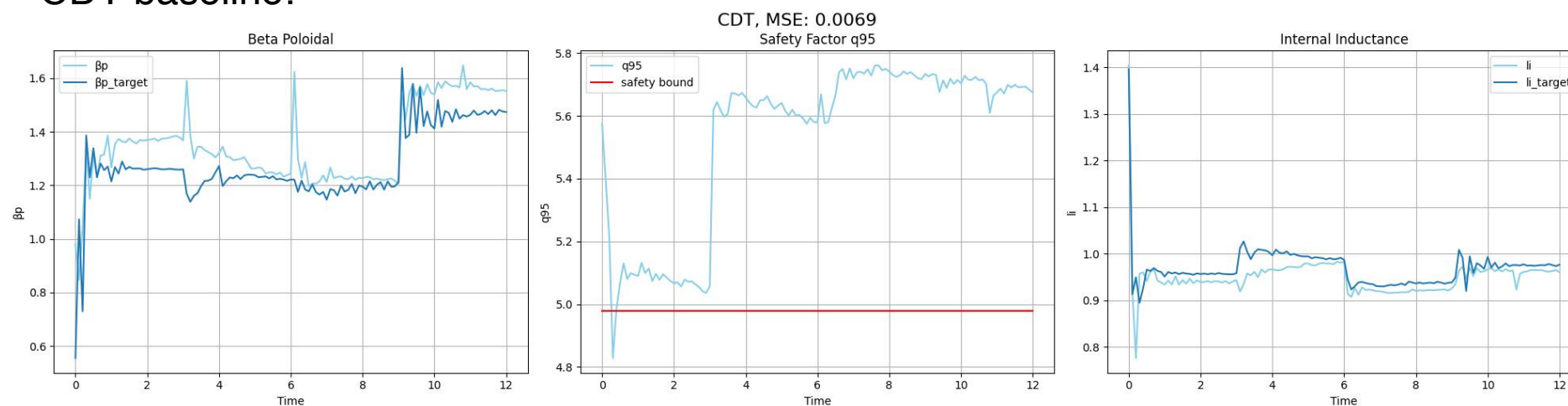
Methods	$\mathcal{J} \downarrow$	$\mathcal{R}_{\text{sample}} \downarrow$	$\mathcal{R}_{\text{time}} \downarrow$
BC	0.0610	42%	1.34%
BC-Safe	0.0811	4%	0.03%
SL-Lag	0.8812	0%	0.00%
MPC-Lag	0.8659	0%	0.00%
CDT	0.0071	8%	0.54%
TREBI	0.0261	0%	0.00%
SafeDiffCon	0.0094	0%	0.00%



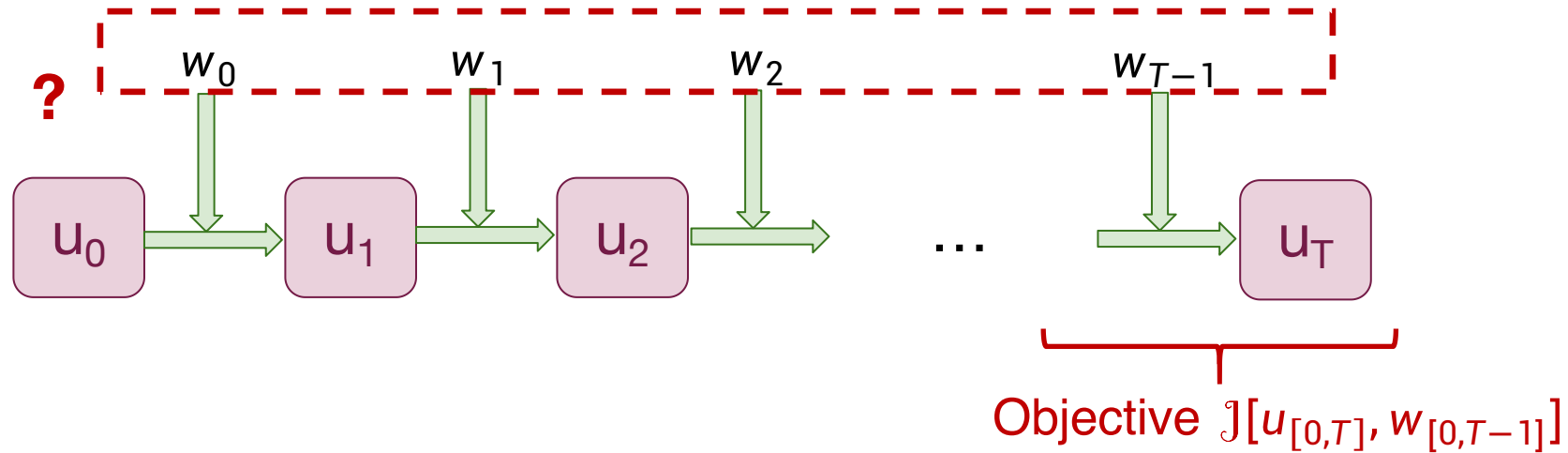
Our SafeDiffCon method:



CDT baseline:



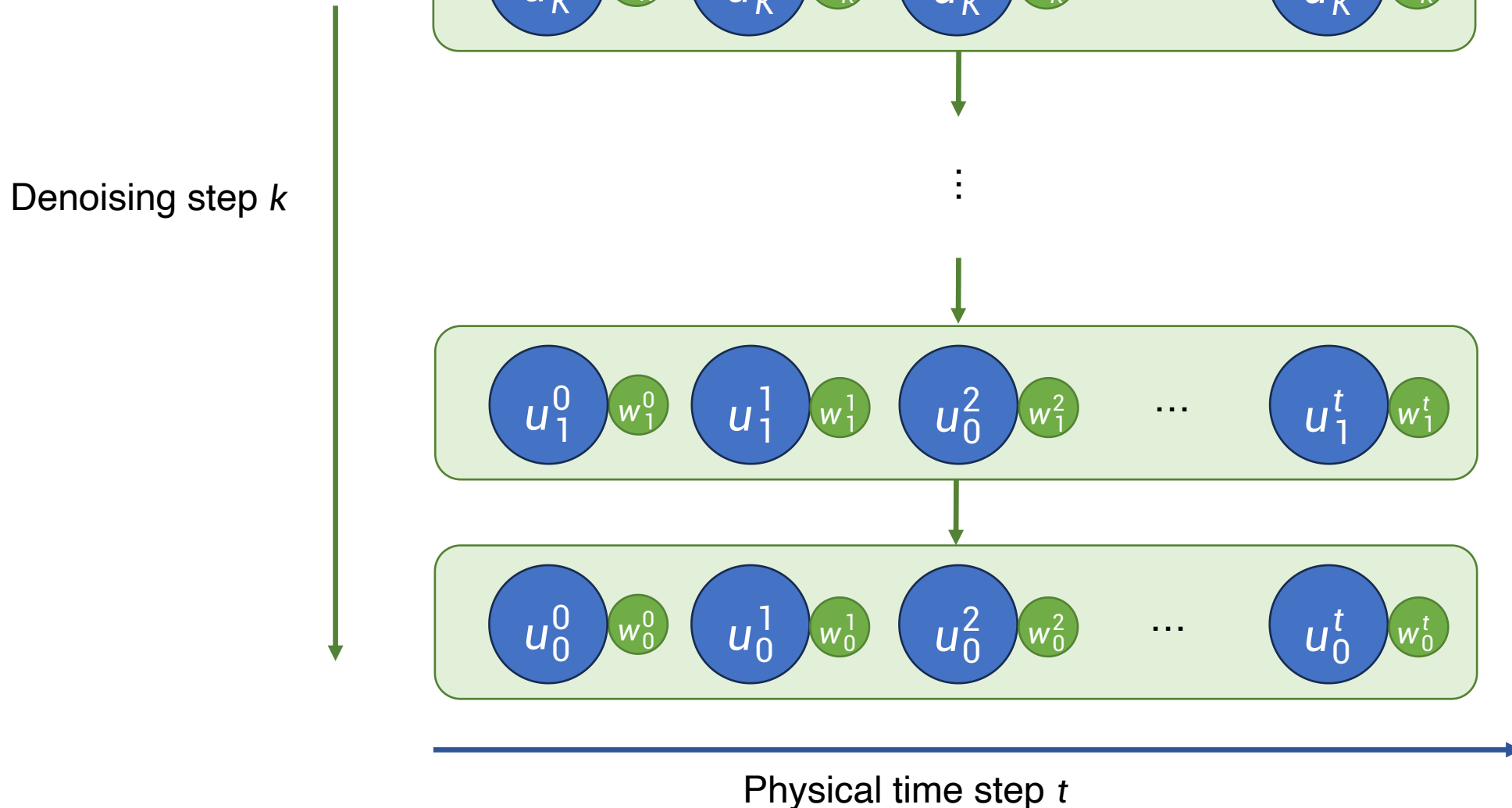
Task setup for learning control (控制)



Offline learning task: Given many example trajectories $\{(u_t, w_t, u_{t+1})\}$, $t = 0, 1, \dots, T$, generate a control sequence $(w_0, w_1, \dots, w_{T-1})$ to minimize the objective $J(u_{[0,T]}, w_{[0,T-1]})$

Our contribution: DiffPhyCon

w_k^t : control at physical time t , at denoising step k ,
 u_k^t : state at physical time t , at denoising step k



Important challenge

w : control sequence

u : state trajectory

Here, similarly, we use a diffusion model to learn the joint distribution $p(u, w)$.

Challenge: The training distribution of $p(u, w)$ may deviate significantly from the optimal control (u^*, w^*) that minimizes the objective $J(u, w)$

Our solution [1]

w : control sequence

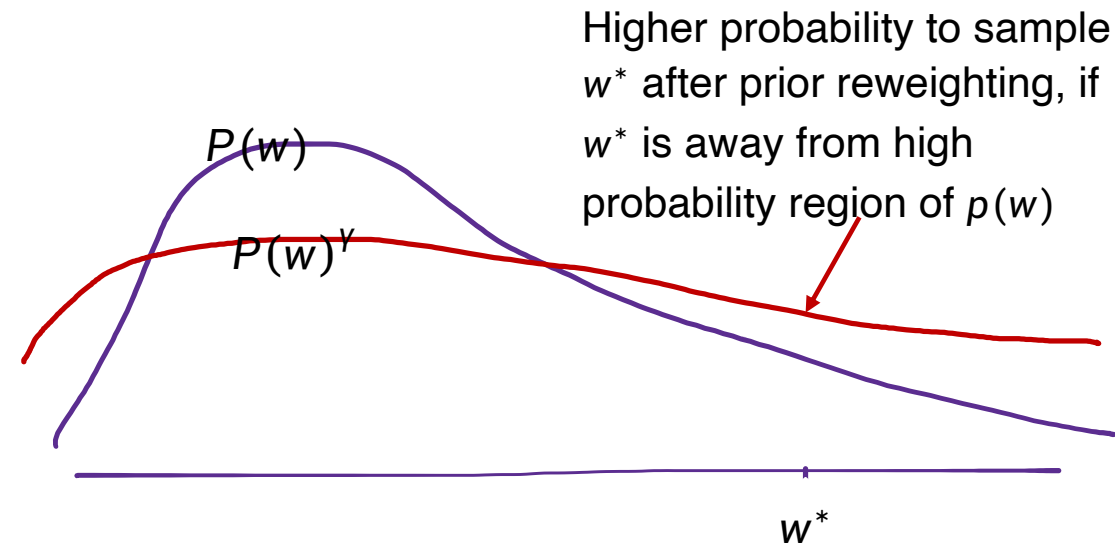
u : state trajectory

Prior reweighting:

Note that for training distribution, $p(u, w) = p(u|w)p(w)$, we instead learn

$$p_\gamma(u, w) \propto p(u|w)p^\gamma(w)$$

where $\gamma \in \mathbb{R}^+$ tunes the strength of the prior.



[1] L. Wei, ... **T. Wu**. 2024, DiffPhyCon: A Generative Approach to Control Complex Physical Systems. NeurIPS 2024

Our solution [1]

w : control sequence

u : state trajectory

Prior reweighting:

Learn **two diffusion models**,

- One for the joint distribution of $p(u, w)$
- The other for the prior control sequence $p(w)$

$$p_\gamma(u, w) \propto p(u|w)p^\gamma(w) = p(u, w)p^{\gamma-1}(w)$$



$$\epsilon(\cdot) = -\nabla \log p(\cdot)$$

$$\epsilon_{\theta, \gamma}(u, w) = \epsilon_\theta(u, w) + (\gamma - 1)\epsilon_\theta(w)$$

Theoretical analysis of prior reweighting

Definitions. Consider a pair (u, w) sampled using the prior reweighting technique with hyperparameter γ : $(u, w) \sim p_\gamma(u, w) = p(u|w)p^\gamma(w)/C_\gamma$. Denote J^* as the global minimum of the control objective J . Define $Q(\epsilon)$ to be the " ϵ -optimal" solution set of (u, w) such that $J(u, w) - J^* \leq \epsilon$, whose complement set is $Q(\epsilon)^c$, and denote $\mathbb{I}_{Q(\epsilon)}(u, w)$ as its indicator function, i.e. $\mathbb{I}_{Q(\epsilon)}(u, w) = 1$ if $(u, w) \in Q(\epsilon)$; otherwise 0. Define Y to be the random variable of "whether use J as a guidance for sampling", namely, $p(Y|u, w) = \begin{cases} e^{-J(u, w)}/Z, & Y = 1 \\ 1 - e^{-J(u, w)}/Z, & Y = 0 \end{cases}$

Consider $E(\gamma) = \mathbb{E}_{(u, w) \sim p_\gamma(u, w)}[\mathbb{I}_{Q(\epsilon)}(u, w)|Y = 1]$, which indicates the expectation of getting an ϵ -optimal solution by using the prior reweighting technique with γ under the guidance of J . Define

$$F(\gamma) = \frac{\mathbb{E}_{u, w}[\mathbb{I}_{Q(\epsilon)}(u, w)p(Y = 1|u, w) \ln(p(w))]}{\mathbb{E}_{u, w}[\mathbb{I}_{Q(\epsilon)}(u, w)p(Y = 1|u, w)]} - \frac{\mathbb{E}_{u, w}[\mathbb{I}_{Q(\epsilon)^c}(u, w)p(Y = 1|u, w) \ln(p(w))]}{\mathbb{E}_{u, w}[\mathbb{I}_{Q(\epsilon)^c}(u, w)p(Y = 1|u, w)]}$$

Theoretical analysis of prior reweighting

Theorem 1. Assume $E(\gamma)$ is a smooth function, then:

- (i) If $F(1) < 0$, there exists $\gamma_- < 1$, s.t., $E(\gamma_-) > E(1)$;
- (ii) If $F(1) > 0$, there exists $\gamma_+ > 1$, s.t., $E(\gamma_+) > E(1)$.

$Q(\epsilon)$: ϵ -optimal solution set of (u, w)
such that $J(u, w) - J^* < \epsilon$

$$p(Y|u, w) = \begin{cases} e^{-J(u, w)}/Z, & Y = 1 \\ 1 - e^{-J(u, w)}/Z, & Y = 0 \end{cases}$$

$$E(\gamma) = \mathbb{E}_{(u, w) \sim p_\gamma(u, w)} [\mathbb{I}_{Q(\epsilon)}(u, w) | Y = 1]$$

Here $F(\gamma)$ can be interpreted as some kind of difference between "entropies" in $Q(\epsilon)^c$ and $Q(\epsilon)$. When $F(1) < 0$, it means that $Q(\epsilon)^c$ has higher "entropies", implying that the training trajectories are far from optimal. As a result, we may need to flatten the distribution of training trajectories, which corresponds to using the prior reweighting technique with $\gamma < 1$.

$$F(\gamma) = \frac{\mathbb{E}_{u, w} [\mathbb{I}_{Q(\epsilon)}(u, w) p(Y = 1 | u, w) \ln(p(w))]}{\mathbb{E}_{u, w} [\mathbb{I}_{Q(\epsilon)}(u, w) p(Y = 1 | u, w)]} - \frac{\mathbb{E}_{u, w} [\mathbb{I}_{Q(\epsilon)^c}(u, w) p(Y = 1 | u, w) \ln(p(w))]}{\mathbb{E}_{u, w} [\mathbb{I}_{Q(\epsilon)^c}(u, w) p(Y = 1 | u, w)]}$$

Theoretical analysis of prior reweighting

w : control sequence
 u : state trajectory

Proof

$$\begin{aligned} E(\gamma) &= \int \mathbb{I}_{Q(\varepsilon)}(u, w) p_\gamma(u, w | Y = 1) d(u, w) \\ &= \int \mathbb{I}_{Q(\varepsilon)}(u, w) \frac{p(Y = 1 | u, w) p_\gamma(u, w)}{p(Y = 1)} d(u, w) \\ &= \frac{\mathbb{E}_{(u, w)}[\mathbb{I}_{Q(\varepsilon)}(u, w) p(Y = 1 | u, w)]}{\mathbb{E}_{(u, w)}[p(Y = 1 | u, w)]} \\ &= \frac{\mathbb{E}_{(u, w)}[\mathbb{I}_{Q(\varepsilon)}(u, w) p(Y = 1 | u, w)]}{\mathbb{E}_{u, w}[\mathbb{I}_{Q(\varepsilon)}(u, w) p(Y = 1 | u, w)] + \mathbb{E}_{u, w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w) p(Y = 1 | u, w)]} \\ &= \frac{1}{1 + \frac{\mathbb{E}_{u, w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w) p(Y = 1 | u, w)]}{\mathbb{E}_{u, w}[\mathbb{I}_{Q(\varepsilon)}(u, w) p(Y = 1 | u, w)]}} \end{aligned}$$

Define

$$\begin{aligned} G(\gamma) &= \frac{\mathbb{E}_{u, w}[\mathbb{I}_{Q(\varepsilon)}(u, w) p(Y = 1 | u, w)]}{\mathbb{E}_{u, w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w) p(Y = 1 | u, w)]} \\ &= \frac{\mathbb{E}_w[\mathbb{E}_u[\mathbb{I}_{Q(\varepsilon)}(u, w) p(Y = 1 | u, w) | w]]}{\mathbb{E}_w[\mathbb{E}_u[\mathbb{I}_{Q(\varepsilon)^c}(u, w) p(Y = 1 | u, w) | w]]} \\ &= \frac{\int \mathbb{E}_u[\mathbb{I}_{Q(\varepsilon)}(u, w) p(Y = 1 | u, w) | w] p^\gamma(w) dw}{\int \mathbb{E}_u[\mathbb{I}_{Q(\varepsilon)^c}(u, w) p(Y = 1 | u, w) | w] p^\gamma(w) dw} \end{aligned}$$

Theoretical analysis of prior reweighting

w : control sequence
 u : state trajectory

Then

$E(\gamma) = \frac{1}{1 + \frac{1}{G(\gamma)}}$. Since $G(\gamma) > 0$, $E(\gamma)$ and $G(\gamma)$ have the same monotonicity.

$$\begin{aligned} G'(\gamma) &= \frac{\int \mathbb{E}_u[\mathbb{I}_{Q(\varepsilon)}(u, w)p(Y = 1|u, w)|w]p^\gamma(w) \ln(p(w))dw \mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w)p(Y = 1|u, w)]}{(\mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w)p(Y = 1|u, w)])^2} \\ &\quad - \frac{\mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)}(u, w)p(Y = 1|u, w)] \int \mathbb{E}_u[\mathbb{I}_{Q(\varepsilon)^c}(u, w)p(Y = 1|u, w)|w]p^\gamma(w) \ln(p(w))dw}{(\mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w)p(Y = 1|u, w)])^2} \\ &= \frac{\mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)}(u, w)p(Y = 1|u, w) \ln(p(w))] \mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w)p(Y = 1|u, w)]}{(\mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w)p(Y = 1|u, w)])^2} \\ &\quad - \frac{\mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)}(u, w)p(Y = 1|u, w)] \mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w)p(Y = 1|u, w) \ln(p(w))]}{(\mathbb{E}_{u,w}[\mathbb{I}_{Q(\varepsilon)^c}(u, w)p(Y = 1|u, w)])^2} \end{aligned}$$

By definition, $F(\gamma)$ is a positive multiple of $G'(\gamma)$, which implies our conclusion:

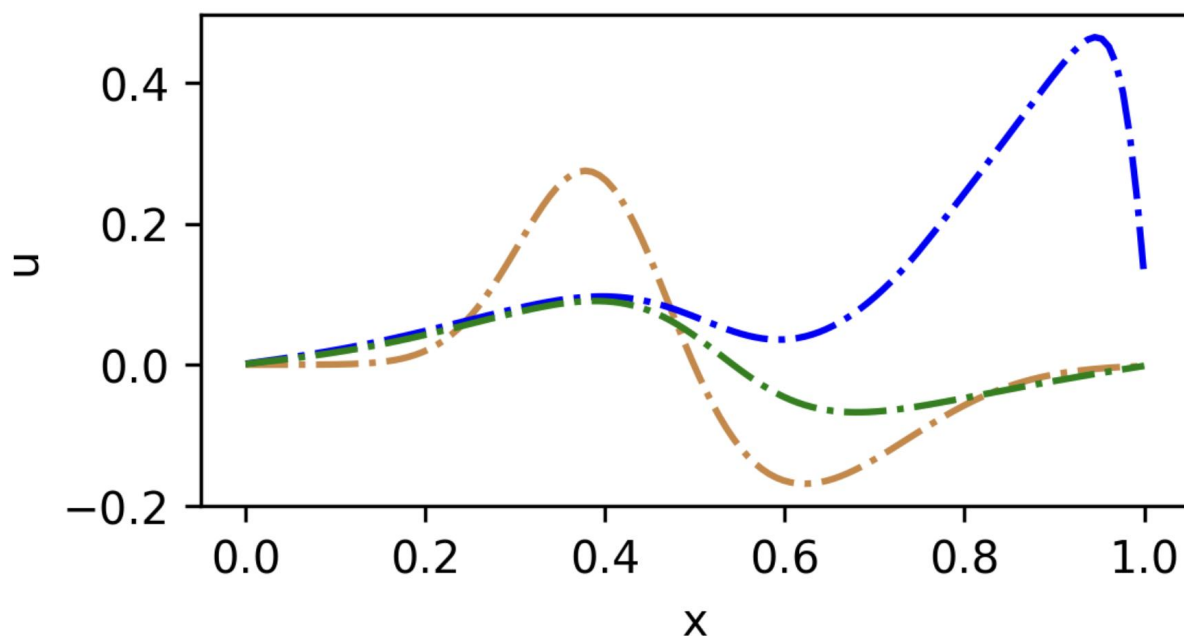
- If $F(1) < 0$, then $G'(1) < 0$ and thus $E(\gamma)$ decreases around 1. Hence, there exists $\gamma_- < 1$, s.t., $E(\gamma_-) > E(1)$, thus (i) holds;
- otherwise, for similar reason, (ii) holds.

Experiment 1: Diffusion model for control, task setting

[1] Long Wei, ..., Tailin Wu#, Generative PDE control, ICLR 2024 AI4PDE workshop [oral](#)

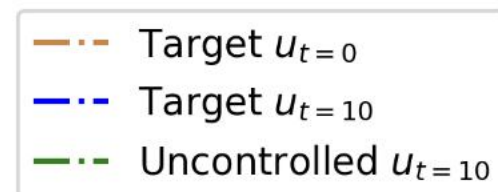
Controlling Burgers' equation:

$$\begin{cases} \frac{\partial u}{\partial t} = -u \cdot \frac{\partial u}{\partial x} + \nu \frac{\partial^2 u}{\partial x^2} + m(t, x) & \text{in } [0, T] \times \Omega \\ u(t, x) = 0 & \text{on } [0, T] \times \partial\Omega \\ u(0, x) = u^0(x) & \text{in } \{t = 0\} \times \Omega \end{cases}$$

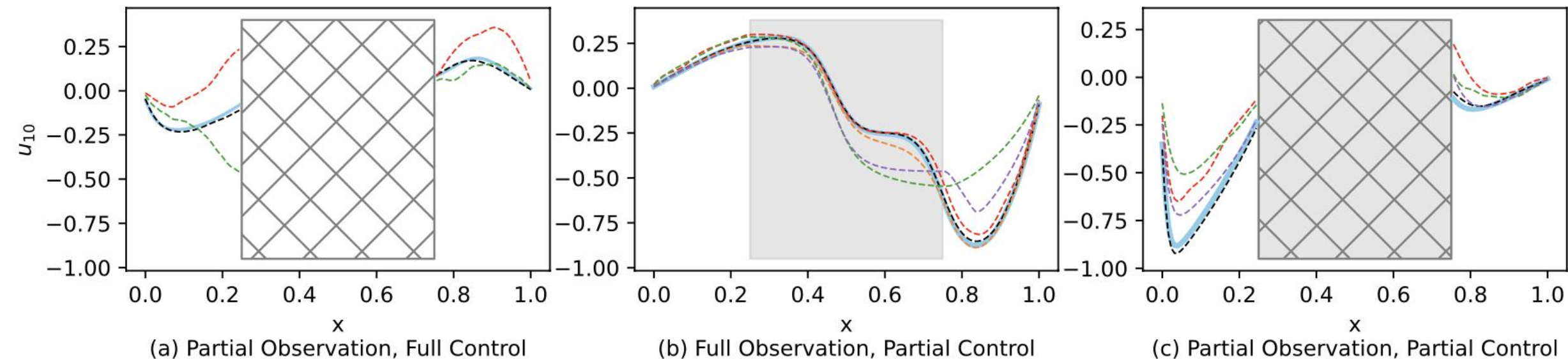


Objective: $J = \|u_T^{(control)} - u_T^{(target)}\|_2^2$

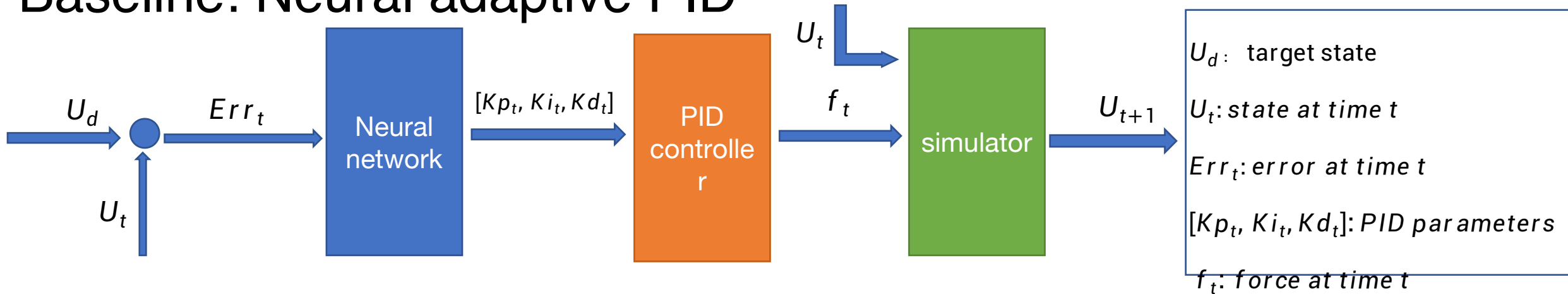
Control energy: $\sum_t \|m_t\|_2^2$



Experiment 1: Diffusion model for control, task setting



Baseline: Neural adaptive PID



Use MIMO PID controller to control U_t to U_d , we train a PID parameters neural network planner output MIMO PID parameters based on Err_t , then use PID controller to output control signal f_t

Problem setting:

- MIMO control

Challenge:

- Traditional PID controllers are limited to SISO linear time-invariant system, to address these gaps we use an NN to automatically plan PID parameters based on current errors.

[1] Slama, Sabrine, Ayachi Errachdi, and Mohamed Benrejeb. "Neural adaptive PID and neural indirect adaptive control switch controller for nonlinear MIMO systems." Mathematical Problems in Engineering 2019 (2019).

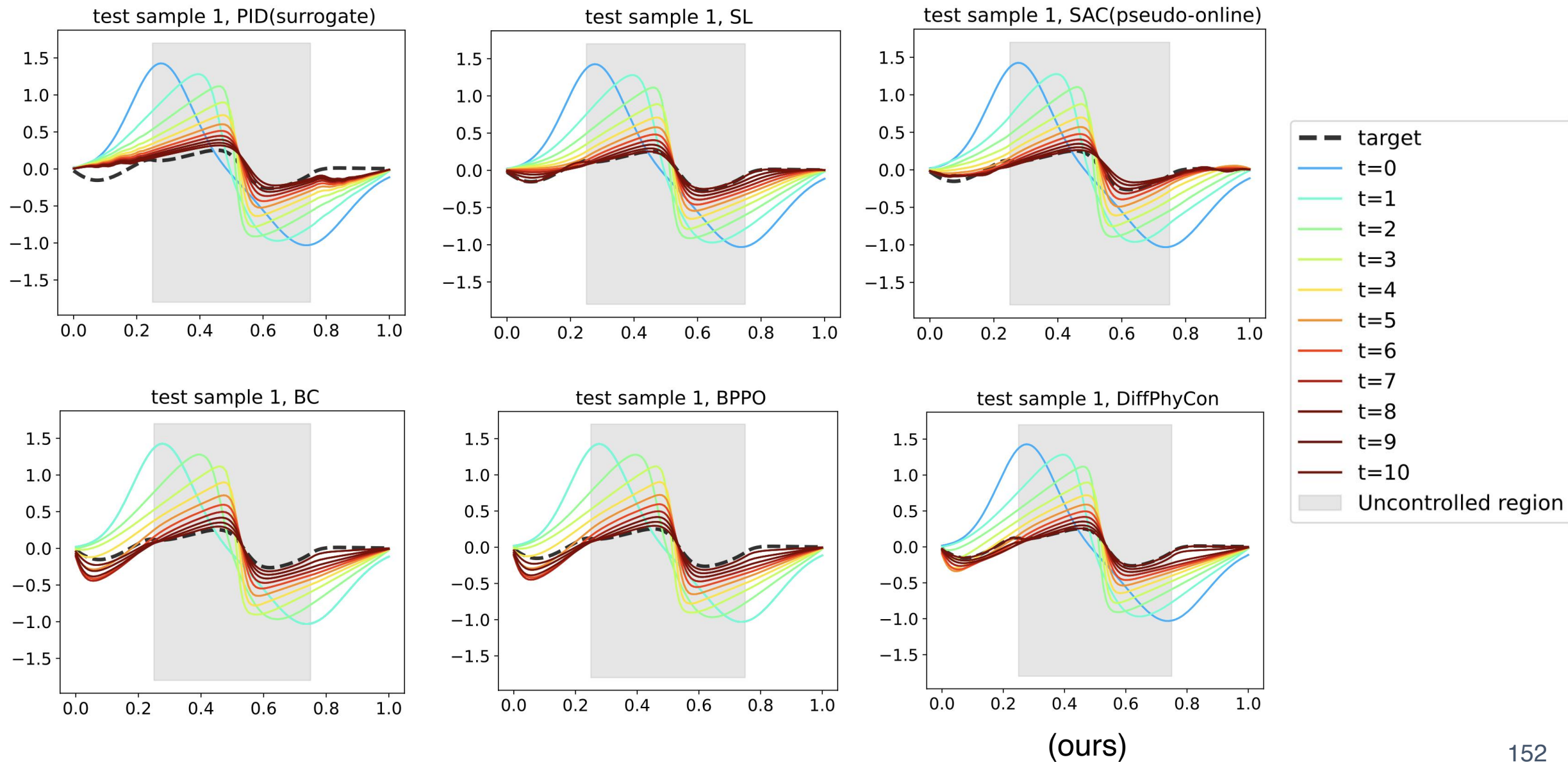
[2] Esmail, Manal Shaban, et al. "Comparison of various Control Techniques Applied to a Quadcopter." Journal of Advanced Engineering Trends 42.2 (2023): 233-244.

Experiment 1: 1D Burgers' Control

PO: partially observed
FO: fully observed
PC: partially control
FC: fully control

	PO-FC	FO-PC	PO-PC
PID (surrogate-solver)	-	0.09115	0.09631
SL	0.09752	<u>0.00078</u>	0.02328
SAC (surrogate-solver)	0.01577	0.03426	0.02149
SAC (offline)	0.03201	0.04333	0.03328
BC	0.02836	0.00856	0.00952
BPPO	0.02771	0.00852	<u>0.00891</u>
DiffPhyCon-lite (ours)	<u>0.01139</u>	0.00037	0.00494
DiffPhyCon (ours)	0.01103	0.00037	0.00494

Experiment 1: 1D Burgers' Control



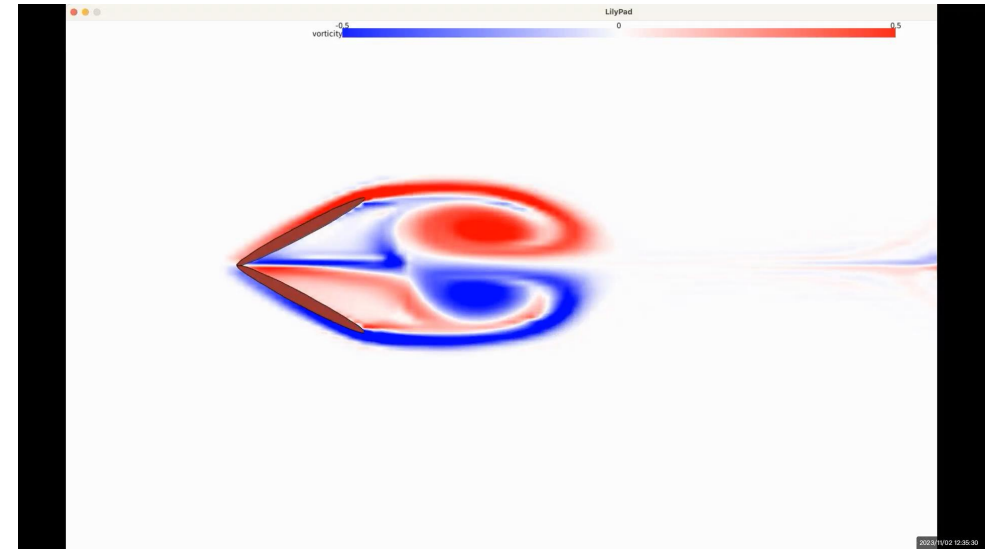
Experiment 2: Controlling a jellyfish

The fluid dynamic is governed by Navier-Stokes Equation:

$$\begin{cases} \frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} - \nu \nabla^2 \mathbf{v} + \nabla p = 0 \\ \nabla \cdot \mathbf{v} = 0 \\ \mathbf{v}(0, \mathbf{x}) = \mathbf{v}_0(\mathbf{x}) \end{cases}$$

Control objective: maximize average moving speed $\bar{v}$ of the jellyfish, under energy cost constraints $R(\mathbf{w})$ of opening angles $\mathbf{w}$ of it wings:

$$\mathcal{J} = -\bar{v} + \zeta \cdot R(\mathbf{w})$$

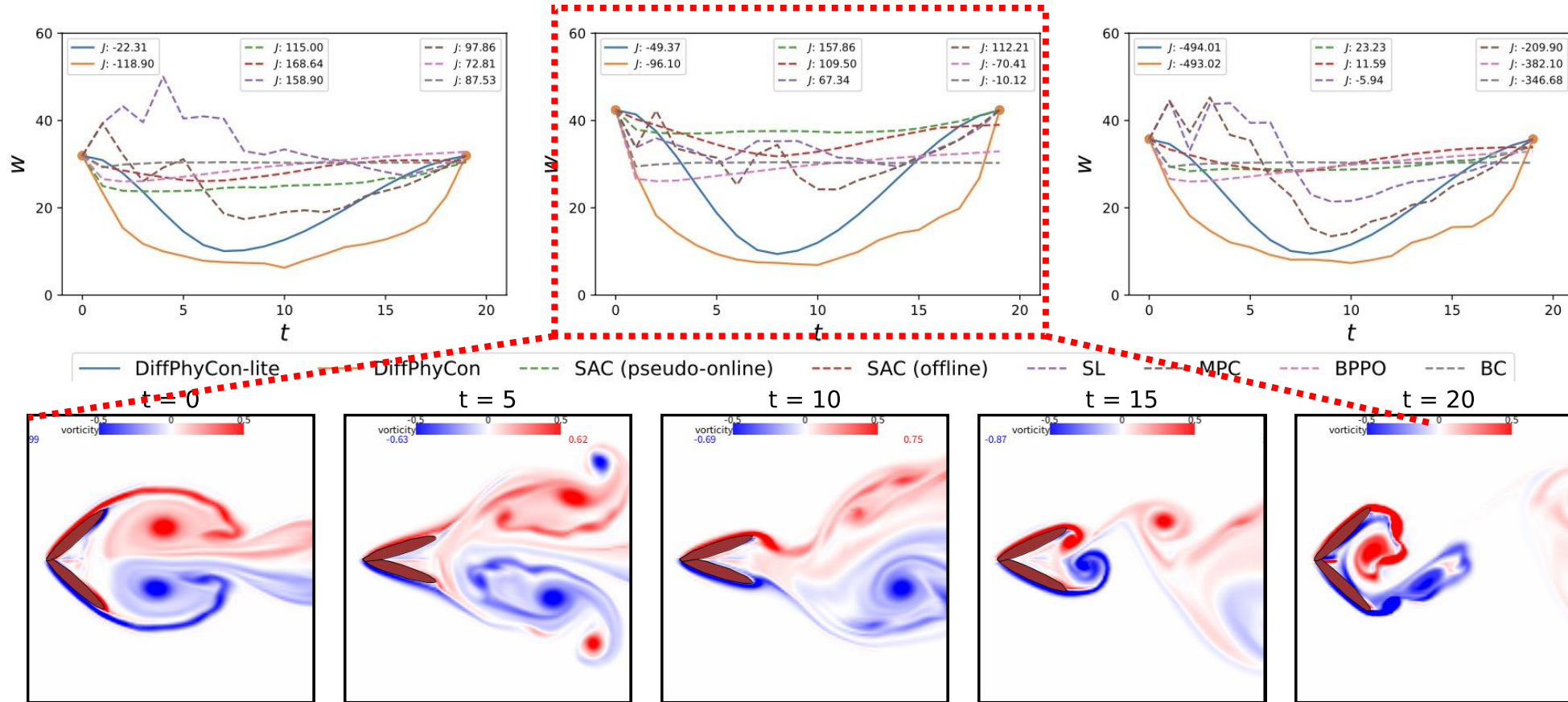


Experiment 2: Controlling a jellyfish

	Full observation			Partial observation		
	$\bar{v} \uparrow$	$R(\mathbf{w}) \downarrow$	$\mathcal{J} \downarrow$	$\bar{v} \uparrow$	$R(\mathbf{w}) \downarrow$	$\mathcal{J} \downarrow$
MPC	25.72	0.0112	109.17	-150.51	0.1791	329.59
SL	-76.94	0.1286	205.57	-102.98	0.1188	221.79
SAC (surrogate-solver)	-166.96	0.0069	18.14	-153.09	0.0057	158.82
SAC (offline)	-158.66	0.0069	165.58	-206.21	0.0058	211.96
BC	30.48	0.0629	32.44	20.08	0.0556	35.48
BPPO	<u>107.67</u>	0.0867	<u>-20.93</u>	<u>54.83</u>	0.0518	<u>-3.02</u>
DiffPhyCon-lite (ours)	95.04	0.0746	-20.47	2.92	0.0779	74.97
DiffPhyCon (ours)	279.87	0.2058	-74.11	150.21	0.1269	-23.32

Comparing with MPC, Reinforcement learning (SAC, BPPO), etc., our method (DiffPhyCon) achieves the highest moving speed and lowest control objective.

Experiment 2: Controlling a jellyfish



Comparison of generated control curves. DiffPhyCon presents a desired **fast-close-slow-open** pattern.

Experiment 2: Controlling a jellyfish

Propulsive performance and vortex dynamics of jellyfish-like propulsion with burst-and-coast strategy

Cite as: Phys. Fluids **35**, 091904 (2023); doi: 10.1063/5.0160878

Submitted: 6 June 2023 · Accepted: 12 August 2023 ·

Published Online: 6 September 2023



Linlin Kang (康林林),¹  An-Kang Gao (高安康),^{2,a)}  Fei Han (韩非),¹  Weicheng Cui (崔维成),^{1,b)} 
and Xi-Yun Lu (陆夕云)^{a)} 

AFFILIATIONS

¹Key Laboratory of Coastal Environment and Resources of Zhejiang Province, School of Engineering, Westlake University, Hangzhou, Zhejiang 310030, China

²Department of Modern Mechanics, University of Science and Technology of China, Hefei 230026, Anhui, China

^{a)}Author to whom correspondence should be addressed: ankanggao@ustc.edu.cn

^{b)}Electronic mail: cuiweicheng@westlake.edu.cn

ABSTRACT

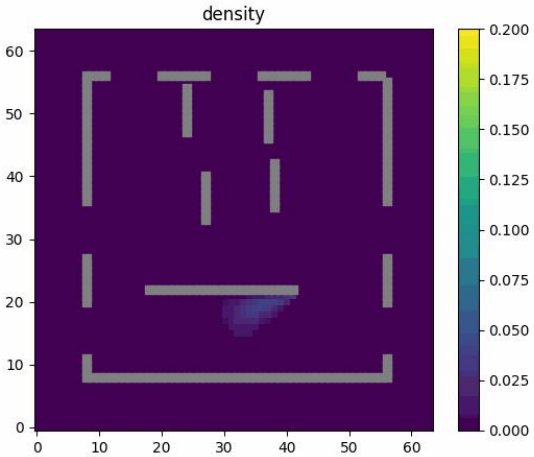
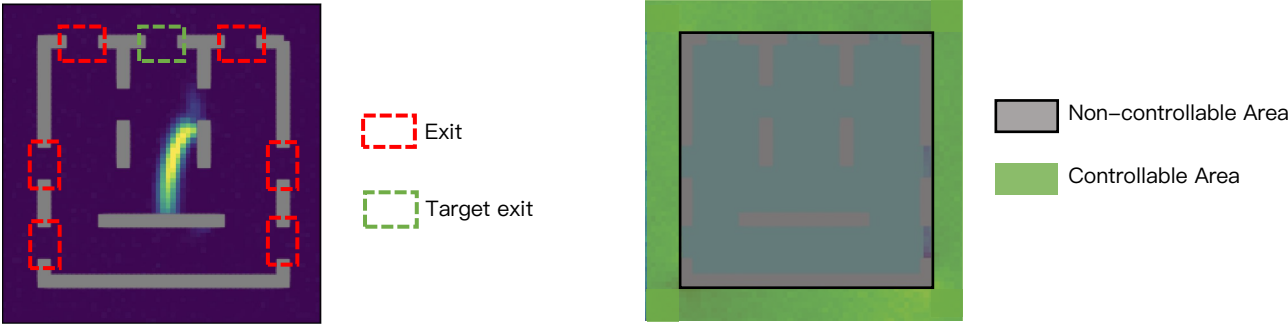
The propulsive performance and vortex dynamics of a two-dimensional model for the jellyfish-like propulsion with burst-and-coast strategy are investigated using a penalty-immersed boundary method. The simplified model comprises a pair of pitching flexible plates with their leading edges connected. The effects of two key parameters are considered, i.e., the duty cycle (DC , the ratio of the closing phase to the whole period) and the bending stiffness (K). Three different wake patterns, i.e., periodic symmetric, periodic asymmetric, and chaotic wakes, are identified in the DC - K plane. Numerical results indicate that a significant **fast-close-slow-open motion is more likely to achieve higher speed, efficiency, and stability** than a slow-close-fast-open motion, and proper higher bending stiffness is conducive to improving efficiency. A force decomposition based on the weighted integral of the second invariant of the velocity gradient tensor is performed to gain physics insight into the self-propulsive mechanism. It is found that the repulsive force induced by the strain-rate field between the body and the previous vortex pair is the main driving force of the jellyfish-like motion and that capturing the previous vortex pair during the closing phase can significantly enhance the strain rate as well as the thrust. This clarifies why the jellyfish can achieve thrust by pushing back vortex pairs. This study provides inspiration for the design and control of flexible jet propulsion devices.

“Numerical results indicate that a significant **fast-close-slow-open motion is more likely to achieve higher speed, efficiency, and stability**”

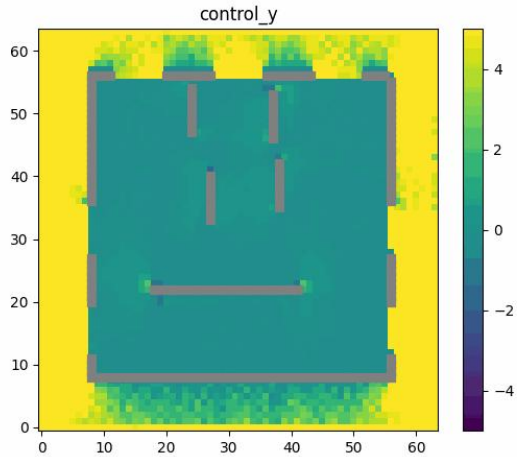
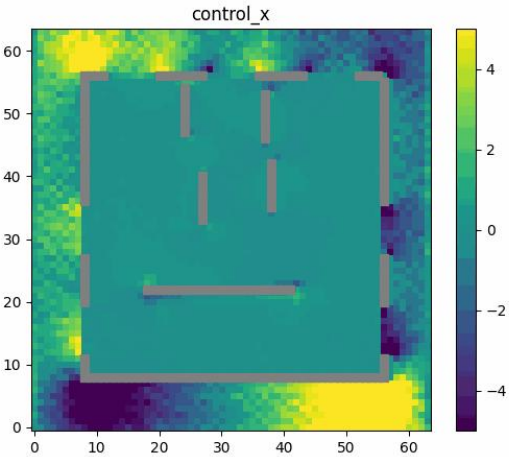
--Kang et al, *Physics of Fluids*, 2023

Experiment 3: Indirect Control of smoke in a fluid maze

Method	$\mathcal{J} \downarrow$
BC	0.3085
BPPO	0.3066
SAC (surrogate-solver)	0.3212
SAC(offline)	0.6503
DiffPhyCon-lite (ours)	0.2324
DiffPhyCon (ours)	0.2254



Smoke in a fluid



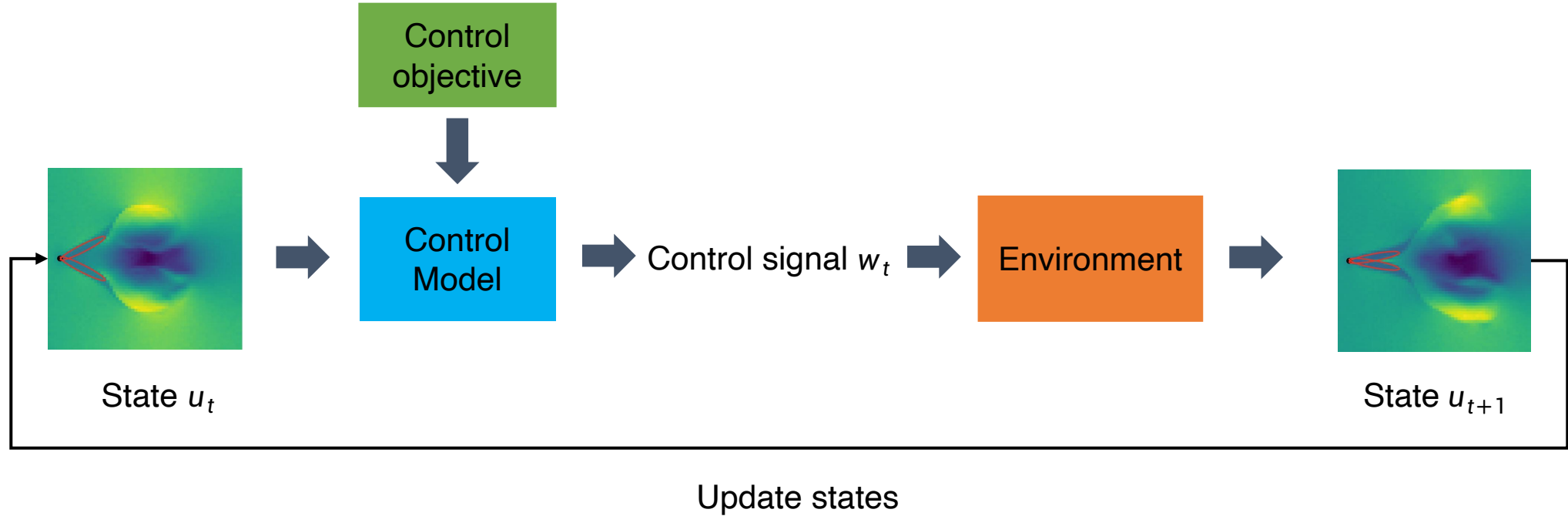
External control

Paper and code

Papers: [Compositional Generative Inverse Design](#) (ICLR 2024 spotlight)
[A Generative Approach to Control Complex Physical Systems](#) (NeurIPS 2024)

Code: <https://github.com/AI4Science-WestlakeU/cindm>
<https://github.com/AI4Science-WestlakeU/diffphycon>

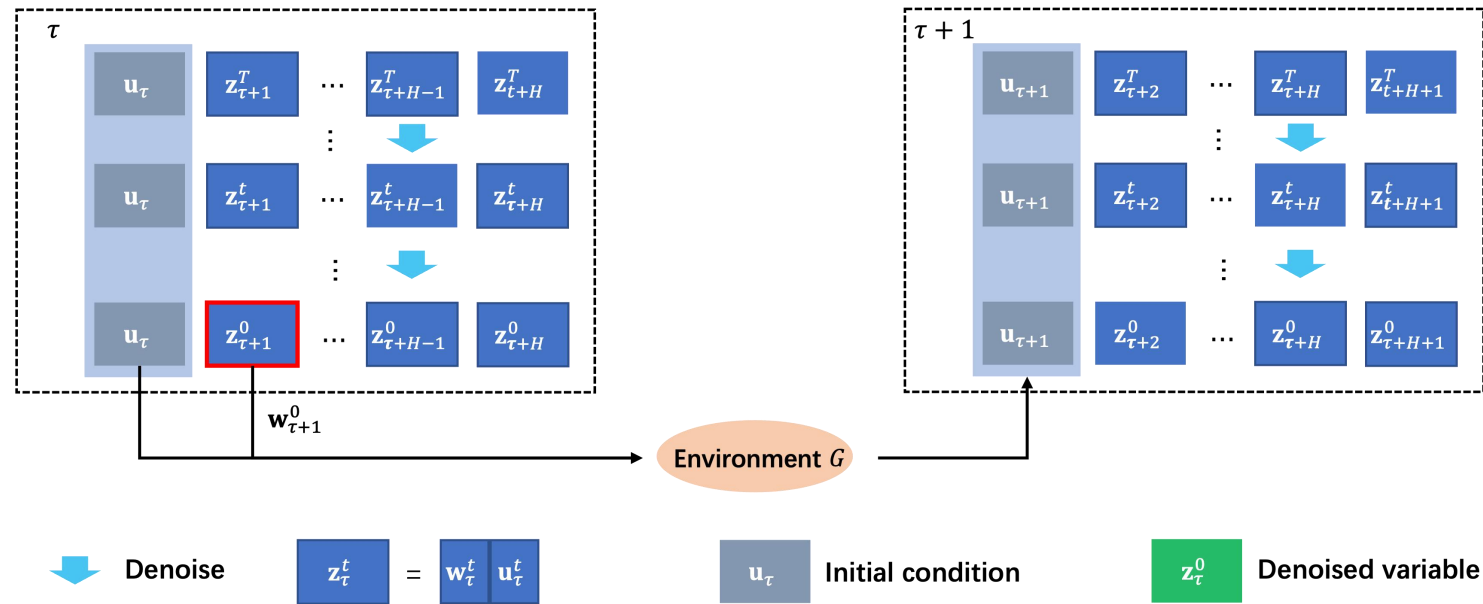
Closed-loop Control



- Closed-loop control is an **inherent constraint** for effective control
- Limitations of previous DiffPhyCon
 - Not closed-loop
 - Memory expensive for long trajectory

How to extend DiffPhyCon to a closed-loop form?

A Naïve Approach to Extend DiffPhyCon to Closed-loop

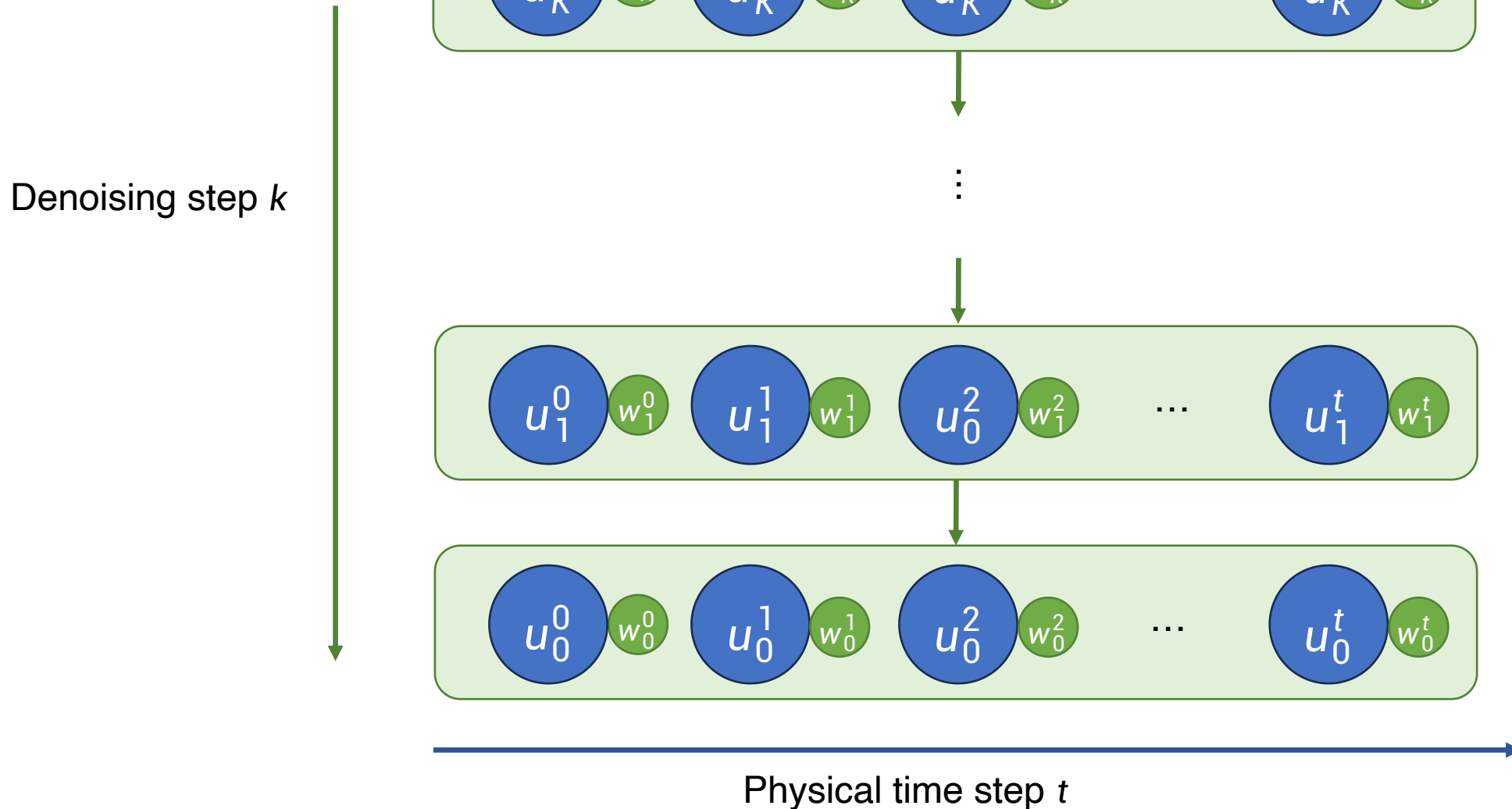


- How to determine when to replan from scratch?
 - Small time gap: high computational cost; inconsistent trajectory
 - Large time gap: violate closed-loop requirement
 - Adaptive gap [7]: extra computation; extra hyperparameters; violate closed-loop requirement

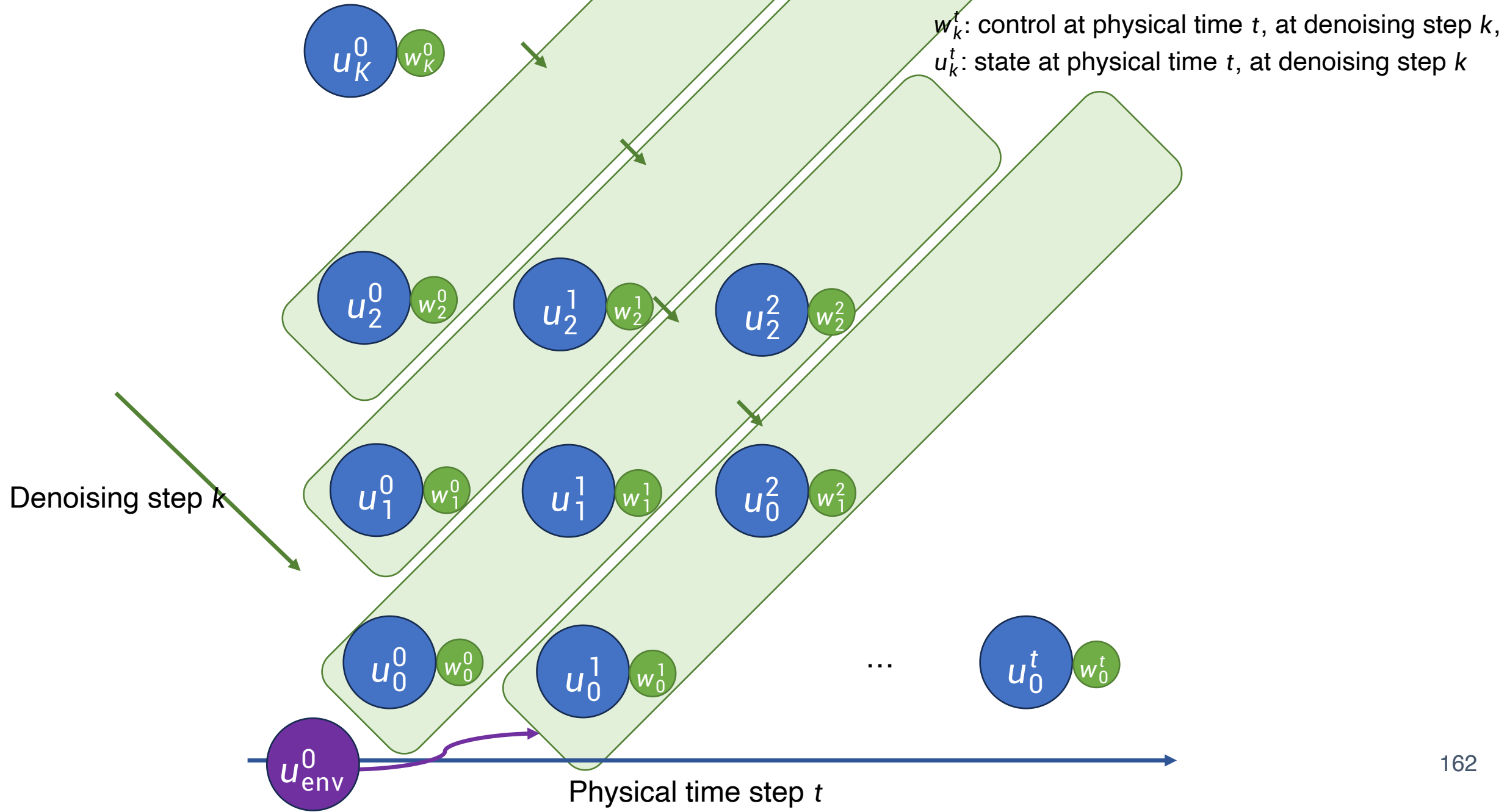
The main obstacle is the **synchronous denoising schedule** for different time steps

Recall: DiffPhyCon

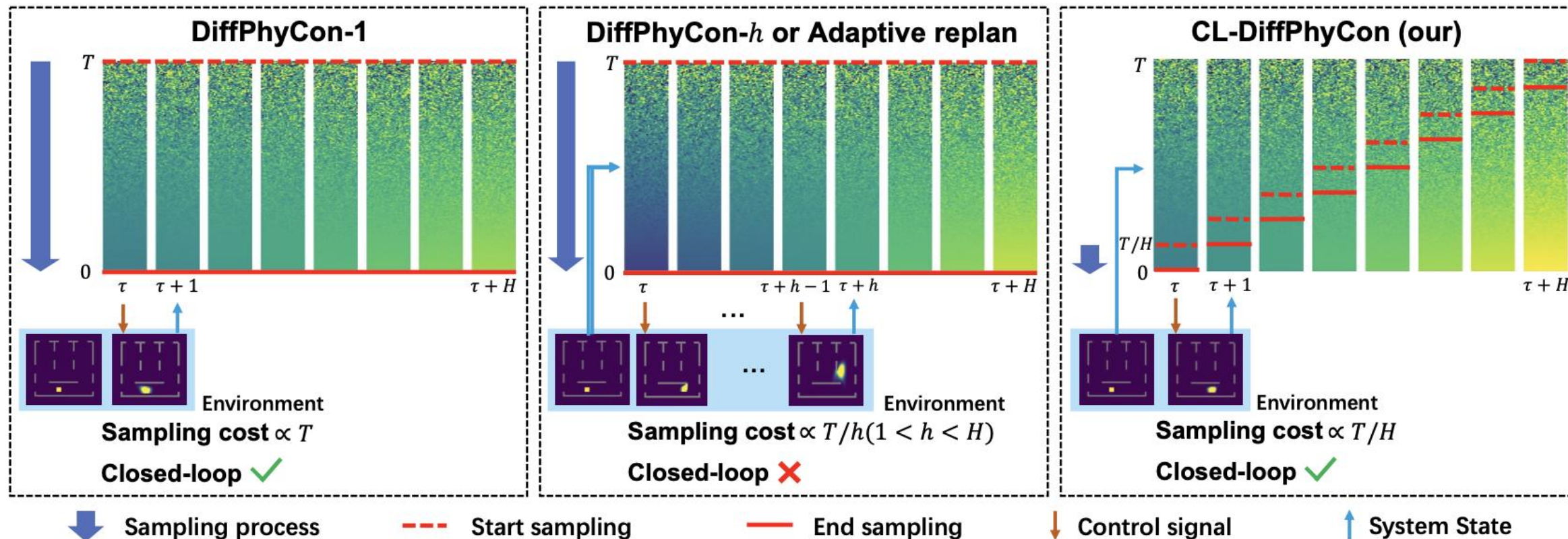
w_k^t : control at physical time t , at denoising step k ,
 u_k^t : state at physical time t , at denoising step k



Our contribution:

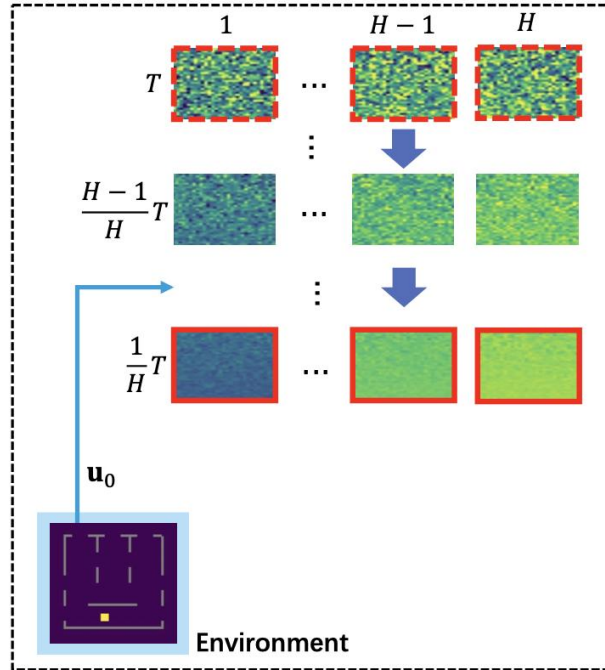


How to decompose the probability for a dynamical system's trajectory?

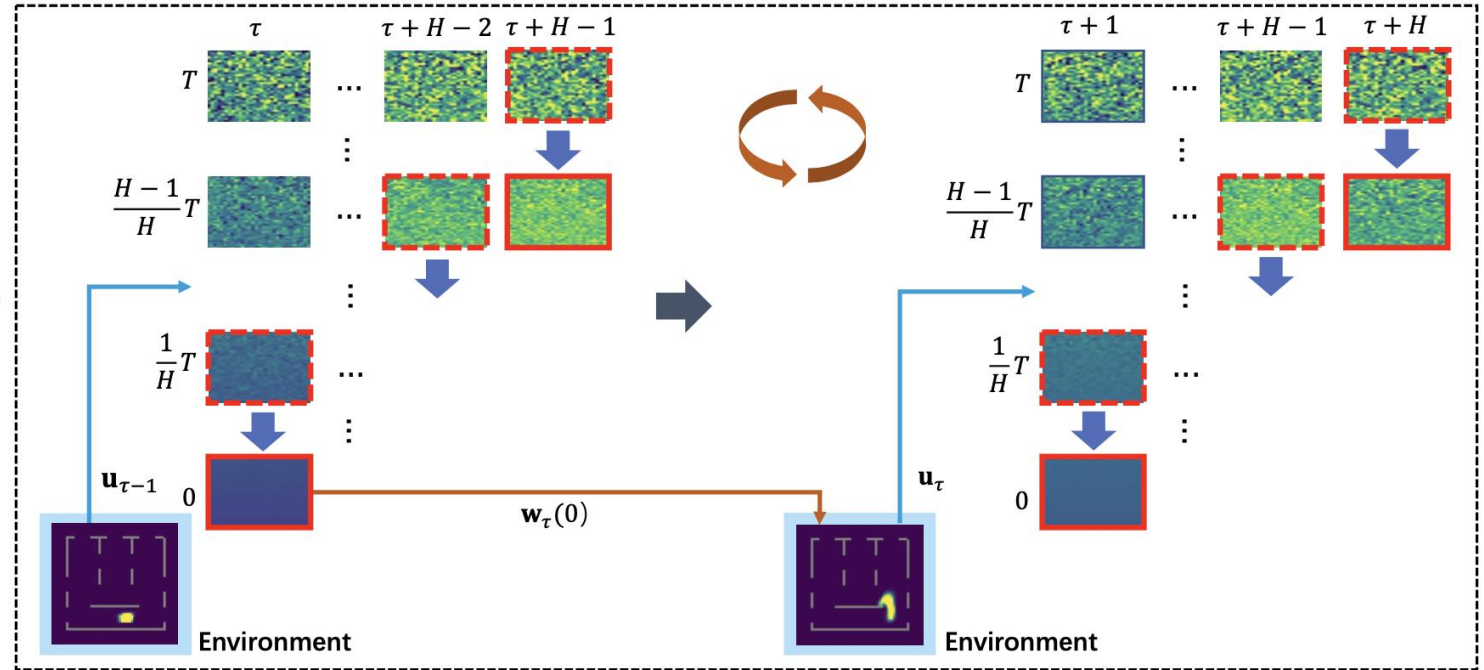


Our Approach

Initialization



Closed-loop Control ($\tau \geq 1$)



- We propose an **asynchronous denoising schedule** for different time steps, with several benefits:
 - Closed-loop control: receive real-time feedback from the environment as a condition for generation of the next control signal
 - Efficient: denoising steps between two successive time steps is reduced to $S = T/H$

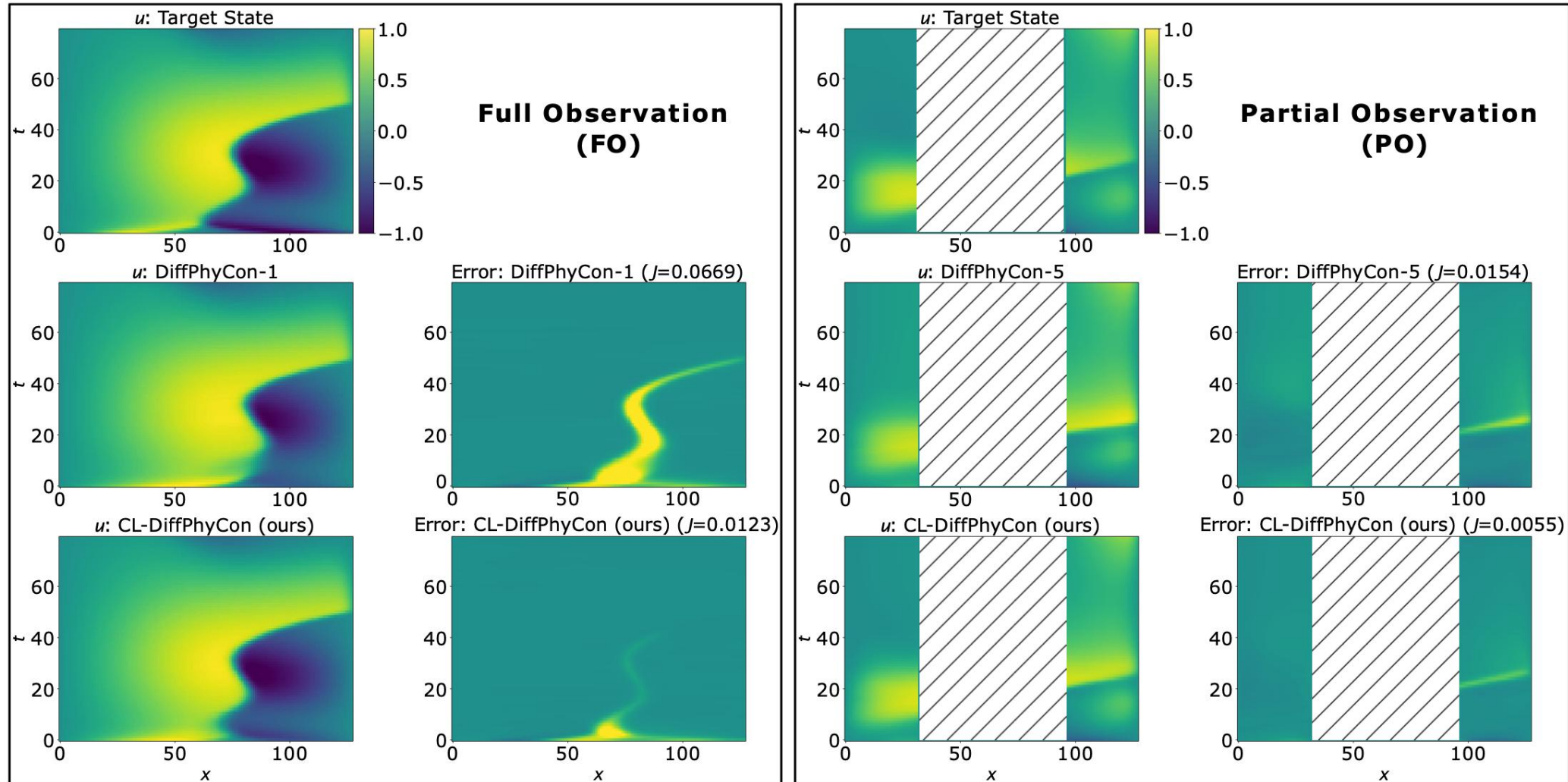
Experiment 1: 1D Burgers' Equation Control

- Online control steps: 128
- Model horizon: $H=15$

	FO		PO	
	$\mathcal{J} \downarrow$	time (s) $\downarrow$	$\mathcal{J} \downarrow$	time (s) $\downarrow$
BC	0.4708	0.7236	0.2558	0.7150
BPPO	0.4686	0.7524	0.2033	0.7342
PID	0.3250	<u>0.7281</u>	0.2212	<u>0.7236</u>
DiffPhyCon-1	0.0210	37.1969	0.0196	39.9720
DiffPhyCon-5	0.0252	8.1562	0.0184	8.4481
DiffPhyCon-15	0.0361	3.7378	0.0192	3.8935
RDM	0.0296	6.6077	0.0196	9.8153
CL-DiffPhyCon (ours)	0.0096	3.4919	0.0090	3.7516
CL-DiffPhyCon (DDIM, ours)	<u>0.0112</u>	0.7585	<u>0.0104</u>	0.7628

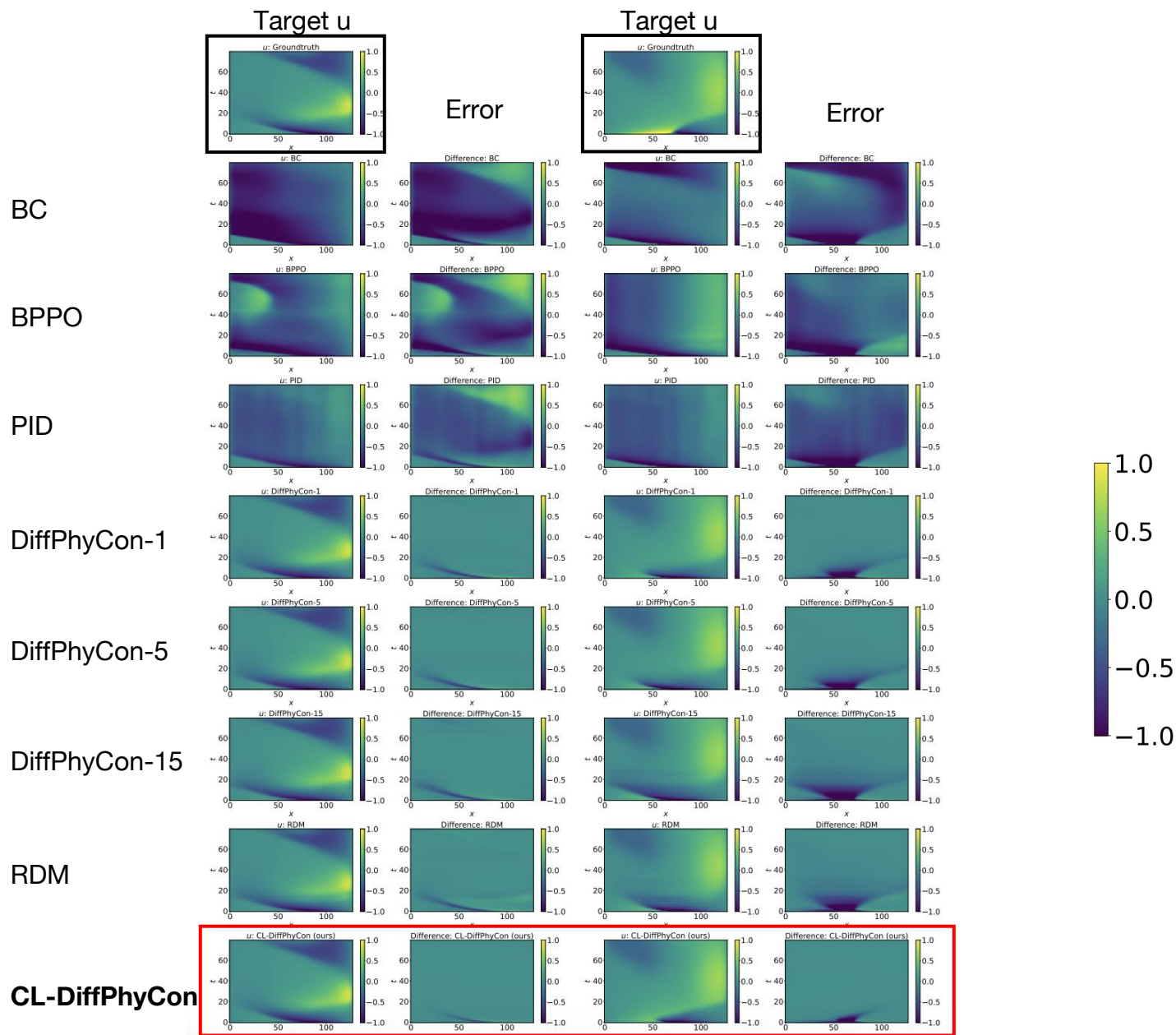
Our method achieves better control performance with less time

Experiment 1: 1D Burgers' Equation Control



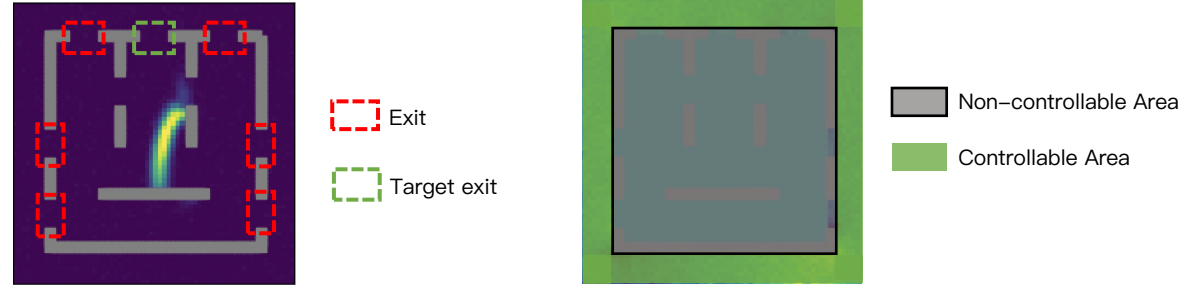
Experiment 1: 1D Burgers' Equation Control

- Online control steps: 128
- Model horizon: $H=15$



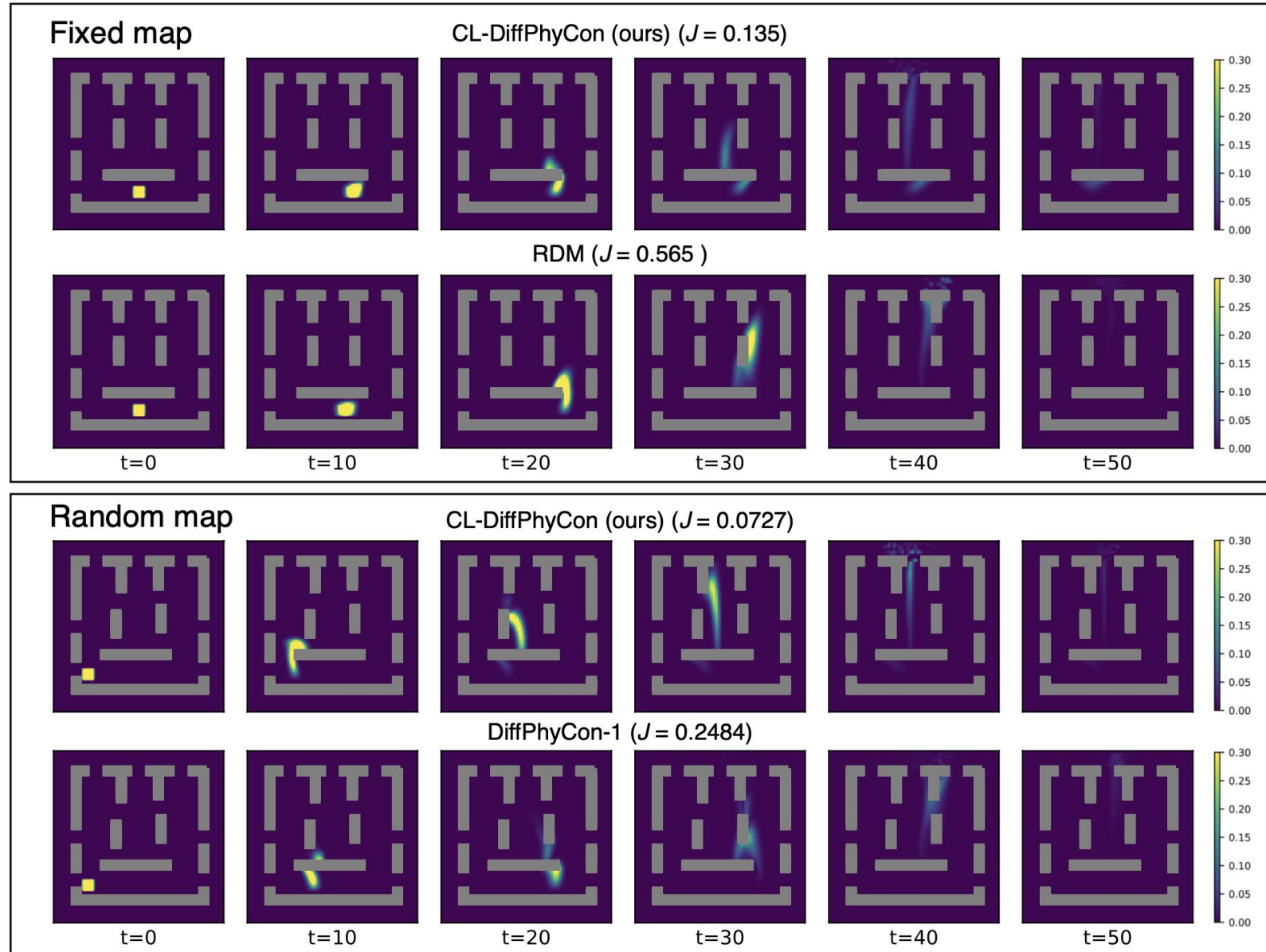
Experiment 2: Indirect Control of Smoke in a Fluid Maze

- Online control steps: 64
- Model horizon: $H=10$



	Fixed map (FM)		Random map (RM)	
	$\mathcal{J} \downarrow$	time (s) $\downarrow$	$\mathcal{J} \downarrow$	time (s) $\downarrow$
BC	0.6722	4.61	0.7046	4.62
BPPO	0.6343	<u>4.62</u>	0.6524	<u>4.66</u>
DiffPhyCon-1	0.5454	<u>1677</u>	<u>0.3754</u>	<u>1692</u>
DiffPhyCon-5	0.5051	345.85	0.5458	397.95
DiffPhyCon-14	0.5823	135.27	0.5621	154.28
RDM	<u>0.4074</u>	229.2	0.4356	344.4
CL-DiffPhyCon (ours)	0.3371	140.83	0.3485	140.54
CL-DiffPhyCon (DDIM, ours)	0.4100	20.75	0.4254	22.04

Experiment 2: Indirect Control of Smoke in a Fluid Maze



Paper: [Closed-loop diffusion control of complex physical systems](#) [1]

[1] Wei, Long, et al. "Closed-loop diffusion control of complex physical systems." *arXiv preprint arXiv:2408.03124* (2024).

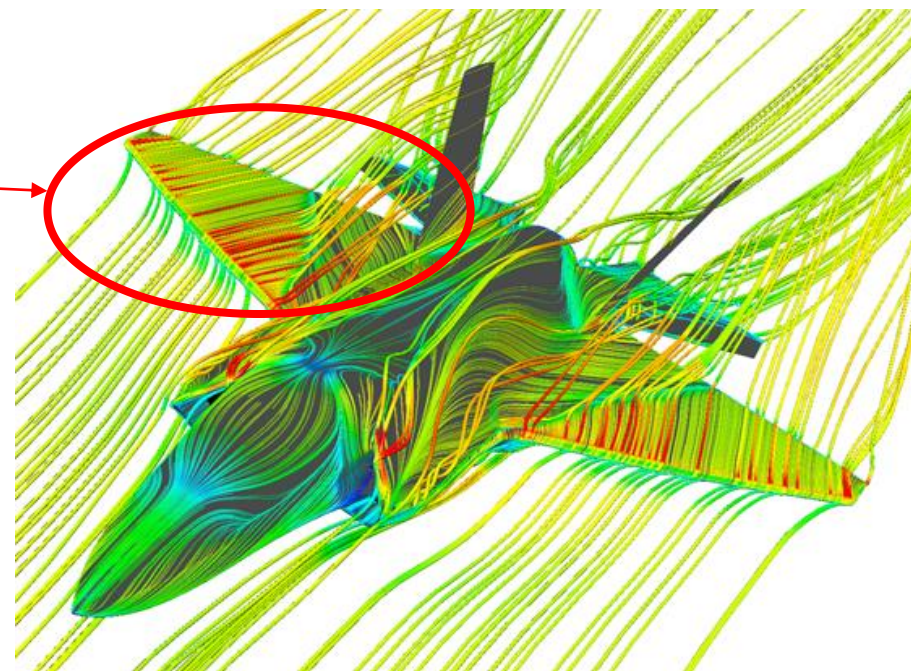
Compositional inverse design (组合逆向设计) : definition

Given objective $J(U(\gamma), \gamma)$, find design parameters γ that minimize J , where the **parameters γ and/or the state U are more complex** than in training.

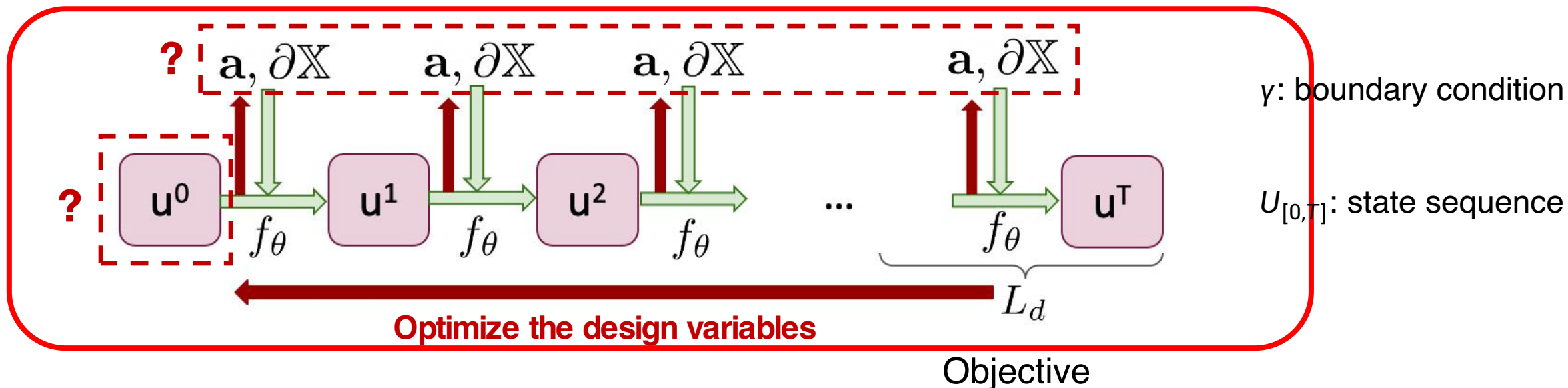
For example:

Training: we only see how the fluid interacts with each part of the airplane

Test: design the whole airplane shape



我们方法：将仿真和设计融为同一任务，联合生成

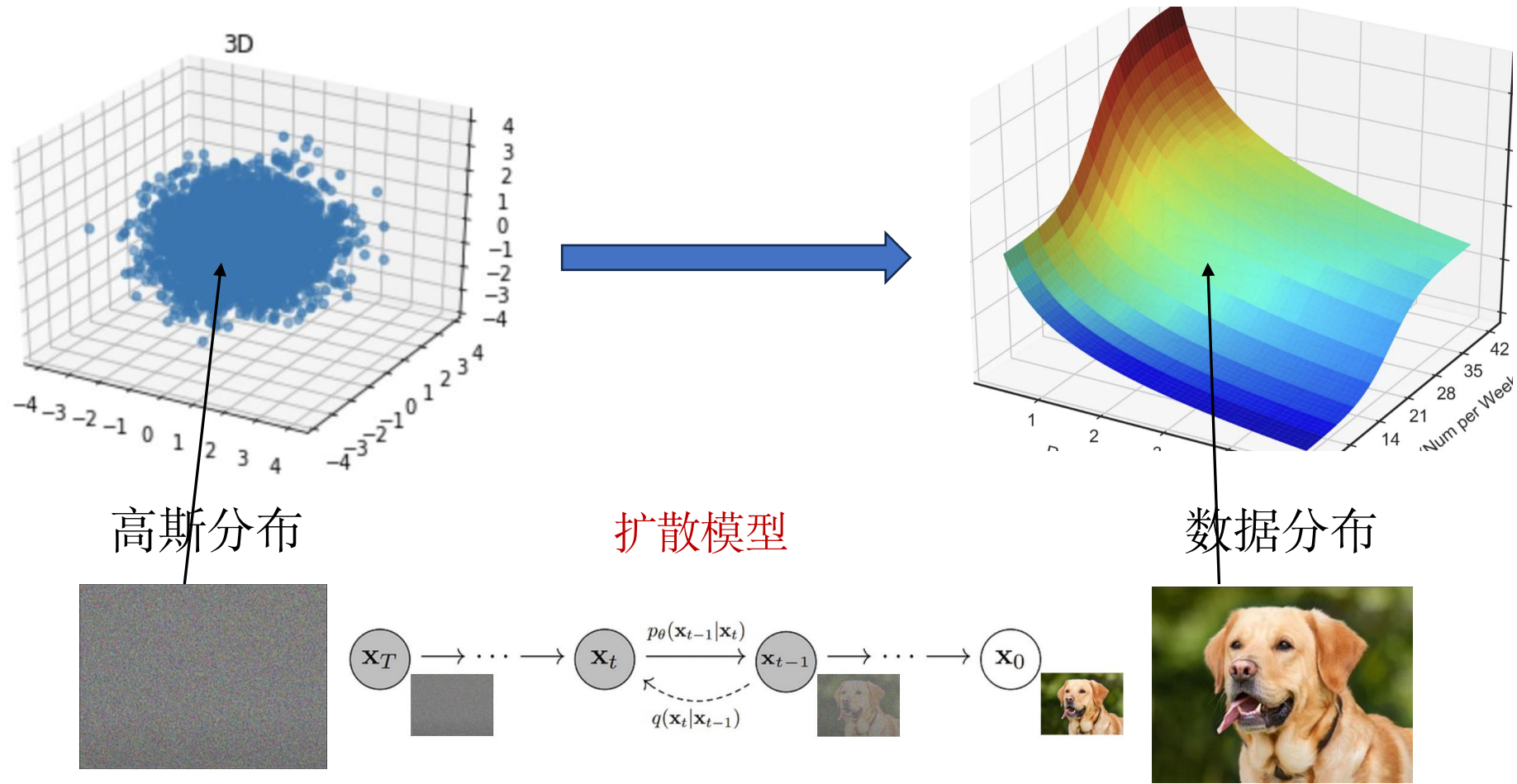


1. 训练时学习能量函数 $E_{\theta}(U_{[0,T]}, \gamma)$ 后，在设计时可同时生成系统的状态轨迹 $U_{[0,T]}$ 和设计参数 γ
2. 设计时，将能量函数 E_{θ} 进行相加，同时加上设计目标 J :

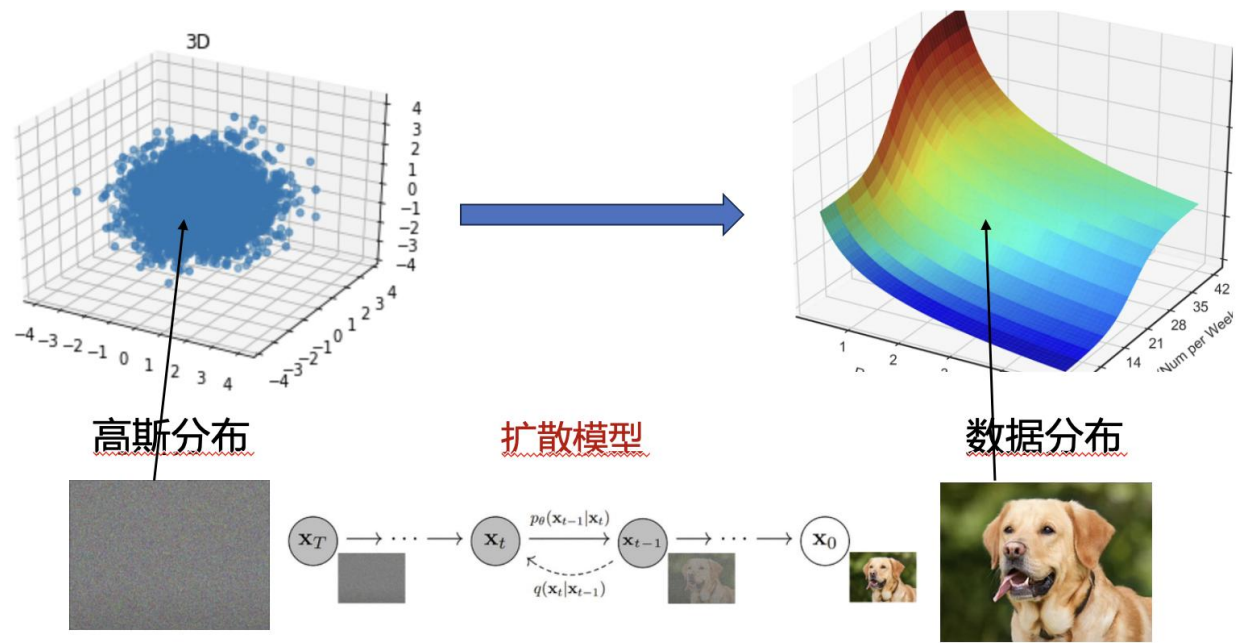
$$\hat{\gamma} = \operatorname{argmin}_{\gamma, U_{[0,T]}} \left(\sum_{k=1}^M E_{\theta}(U_{[0,T]}, \gamma_k) + \lambda \cdot J(U_{[0,T]}, \gamma) \right)$$

扩散生成模型

扩散模型：学习一个从高斯分布到数据分布的映射过程



扩散模型：本质在学习能量函数 $E_{\theta}(x)$



能量高，概率低

$$P(x) \propto e^{-E_{\theta}(x)}$$

x: 样本

能量低，概率高



DDPM: denoising diffusion probabilistic models [1]

训练:

Algorithm 1 Training

```
1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$ 
6: until converged
```

输入加噪 t 步的样本，预测噪声

推理（采样）：

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 
```

一步一步去噪

$\mathbf{x}_0$: 训练数据
 $\mathbf{x}_t$: 加了 t 步高斯噪声的数据
 ϵ_{θ} : 需要学习的去噪网络

局限性：未考虑对于目标函数的优化，只能采样出类似训练数据的分布

[1] Ho, Jonathan, Ajay Jain, and Pieter Abbeel. "Denoising diffusion probabilistic models." *Advances in neural information processing systems* 33 (2020): 6840-6851.

Our method: intuition

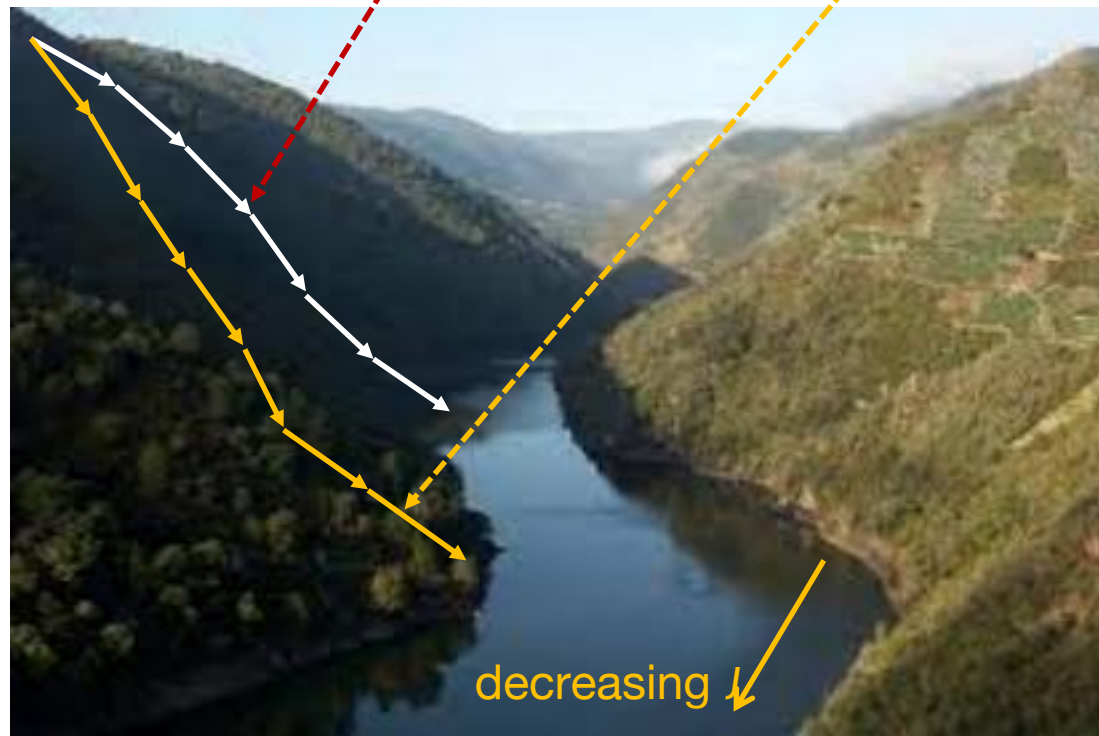
[1] Wu, Tailin, et al. "Compositional Generative Inverse Design." *NeurIPS 2023 AI for Science Workshop*. 2023.

$$\hat{\gamma} = \arg \min_{\gamma, U_{[0,T]}} [E_{\theta}(U_{[0,T]}, \gamma) + \lambda \cdot \mathcal{J}(U_{[0,T]}, \gamma)]$$

训练时候只训练 E_{θ}

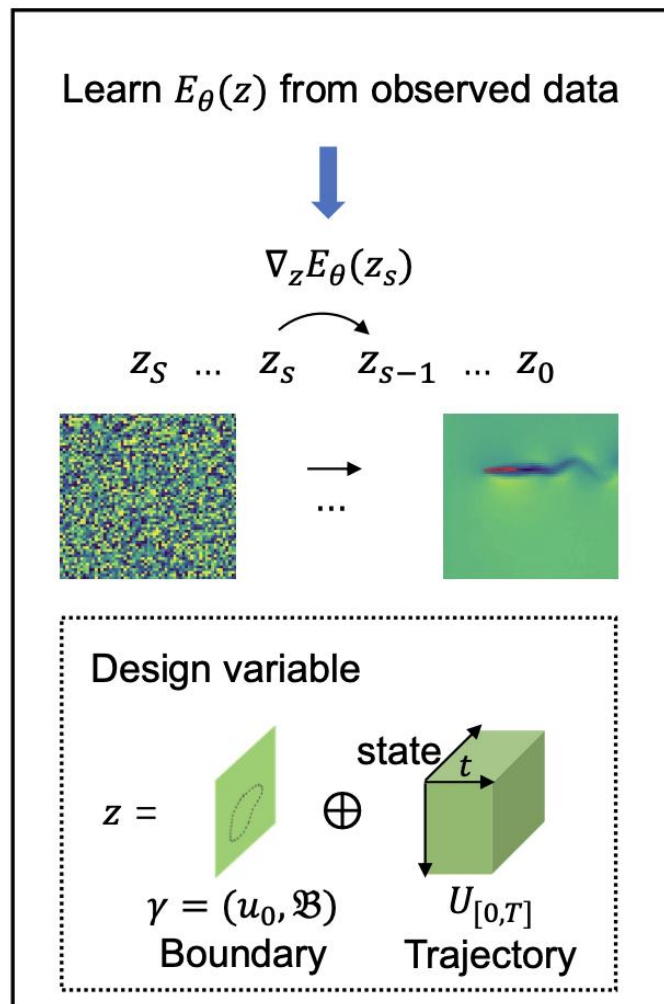
推理时候加上目标 J

$U_{[0,T]}$: state sequence (状态序列)
 γ : control variable (控制序列)
and boundary condition (边界条件)



Our method: architecture

Train



Design objective

$$\left. \begin{array}{l} \lambda \mathcal{J}(z) \\ + \\ E_\theta(z) \end{array} \right\} \rightarrow$$

Energy function

Compositional Design

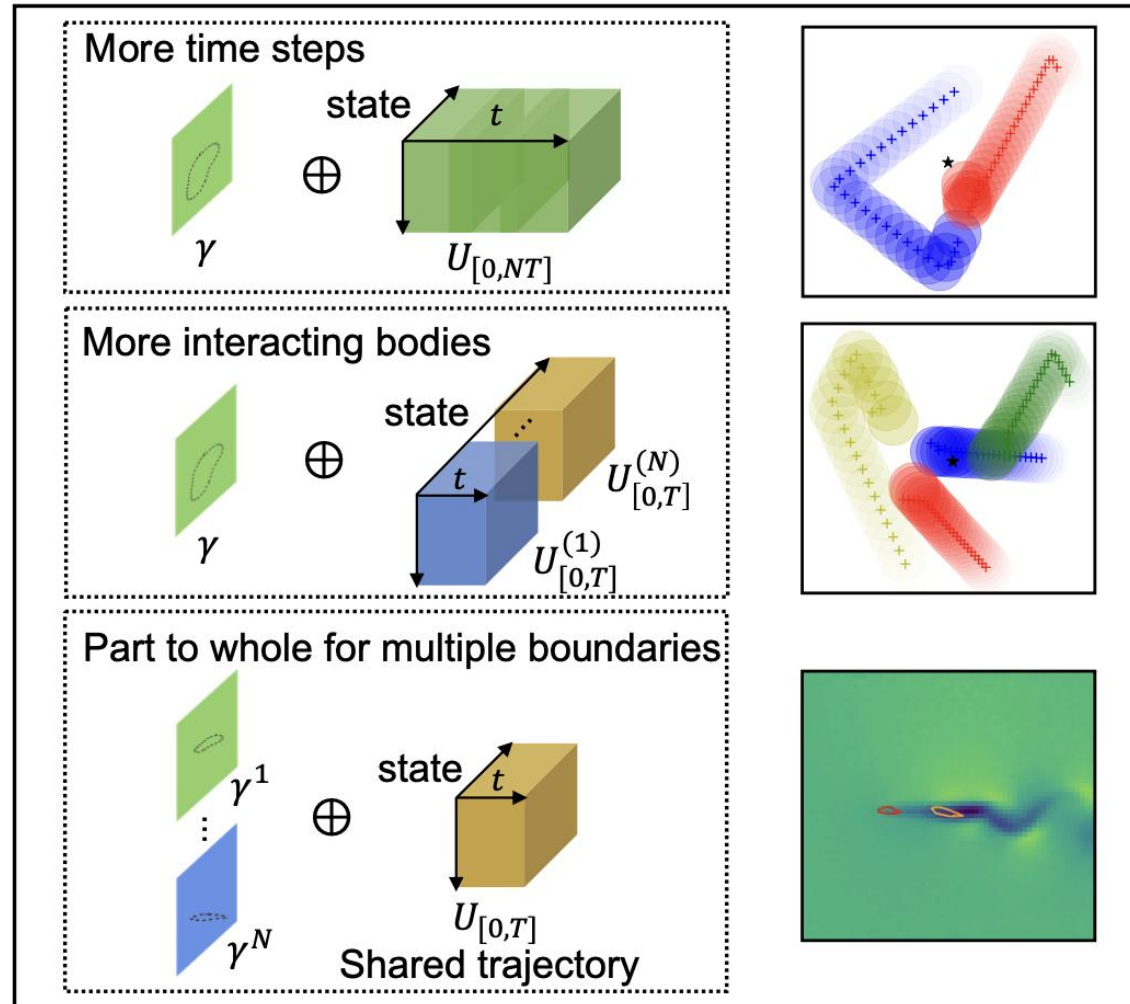
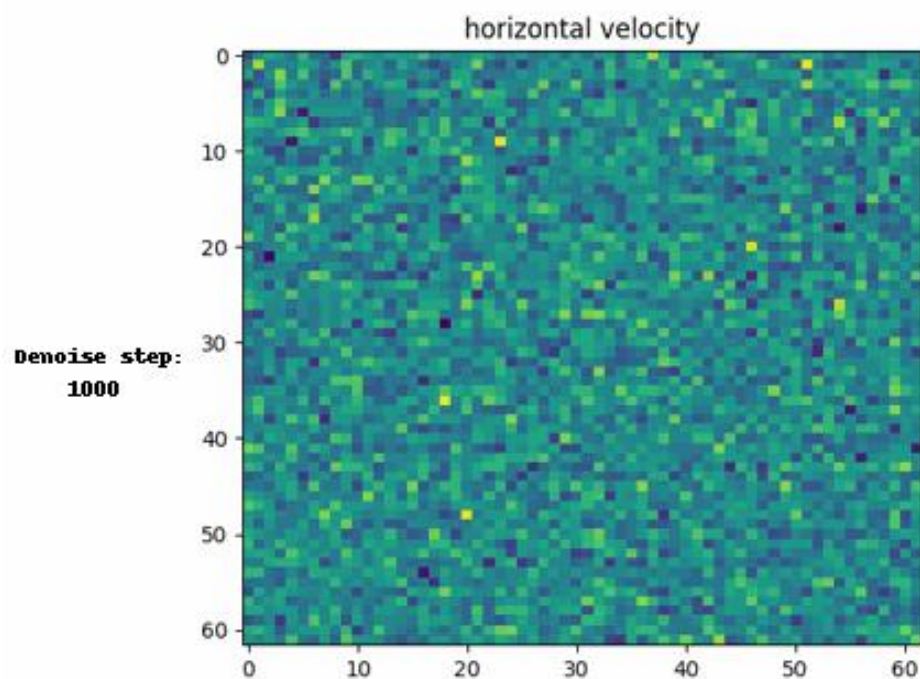


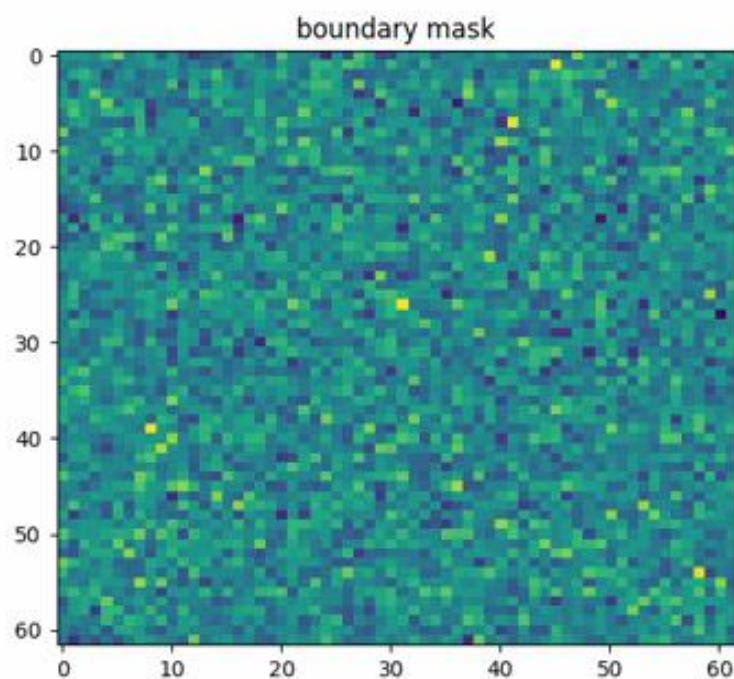
Fig.1 of CinDM. By composing generative models specified over subsets of inputs, we present an approach that design materials significantly more complex than those seen at training.

Learning process

推理生成过程： 从高斯分布到设计出来的速度场轨迹和机翼边界设计



流体速度场

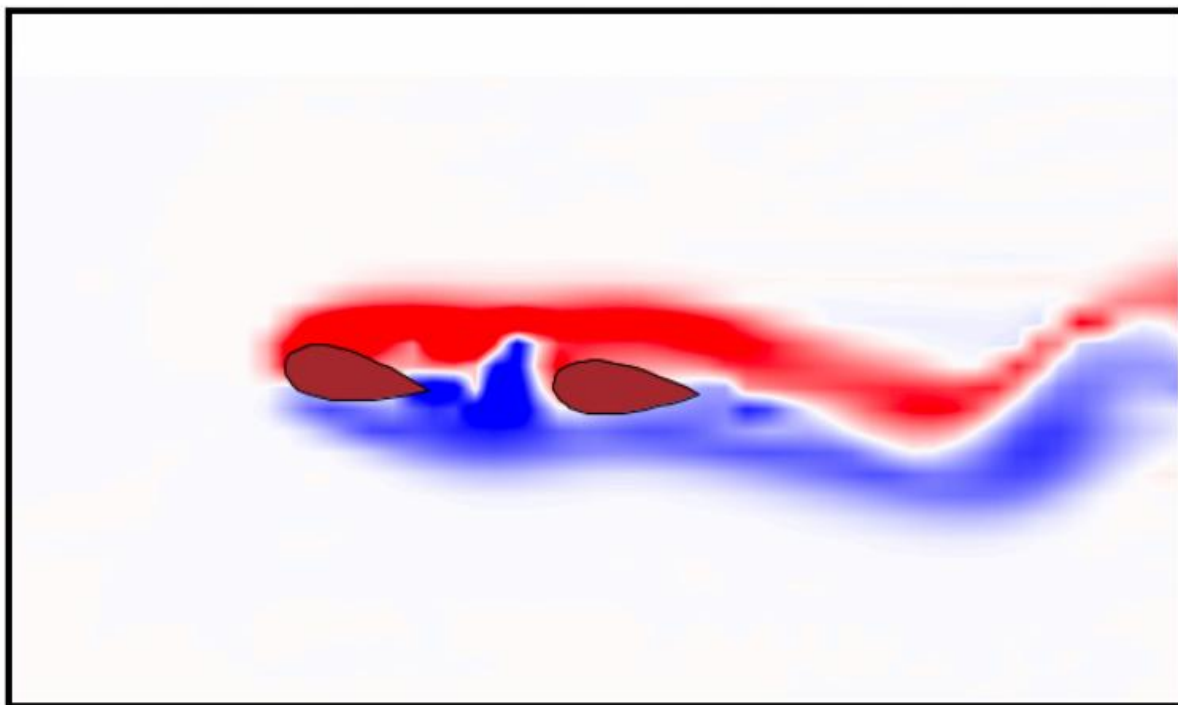


机翼边界设计

Results 1: inverse design

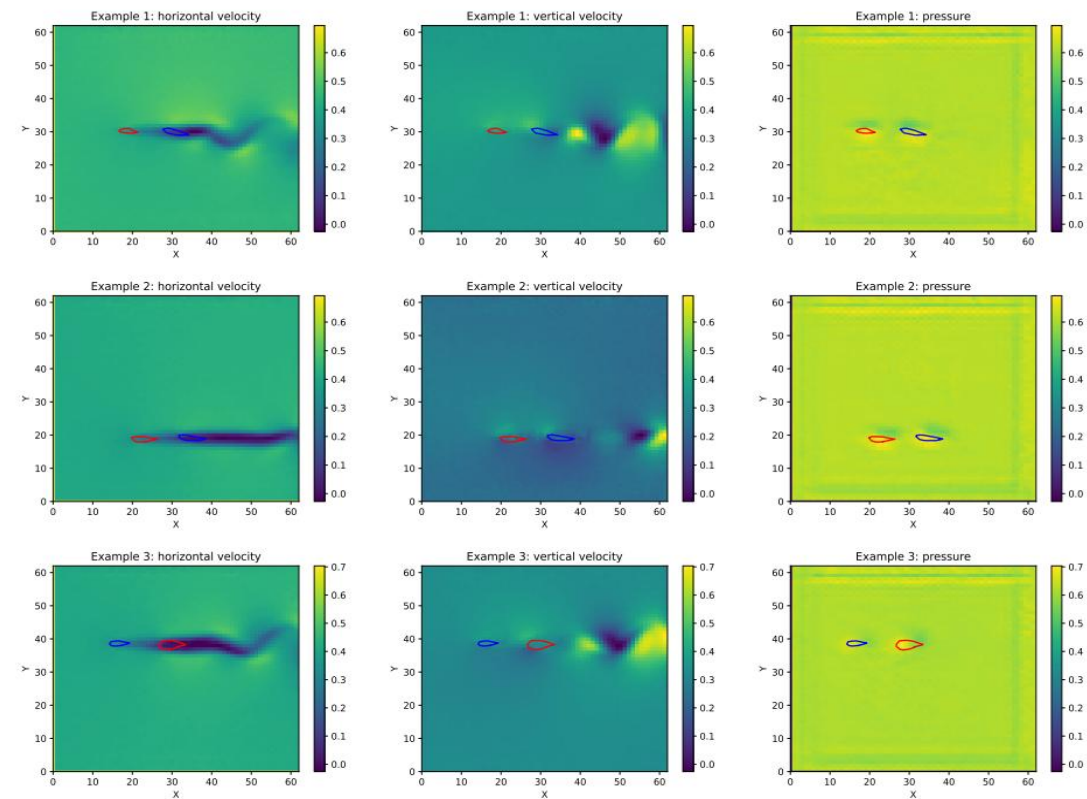
训练时：只训练**单个机翼**与气流的作用

推理时：考虑**多个机翼**，优化总的升阻比 $(= \frac{\text{升力}}{\text{阻力}})$



Example of Lily-Pad simulation.

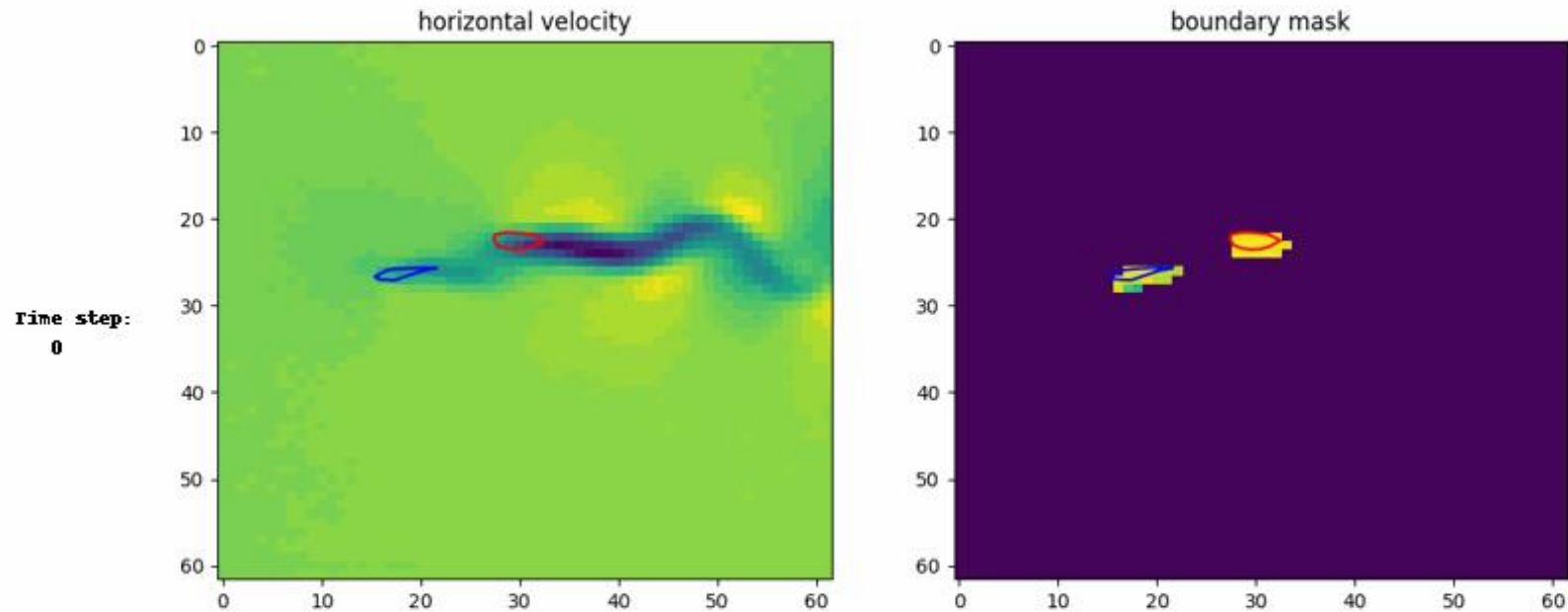
$$\hat{\gamma} = \arg \min_{\gamma, U_{[0,T]}} [E_{\theta}(U_{[0,T]}, \gamma) + \lambda \cdot \mathcal{J}(U_{[0,T]}, \gamma)]$$



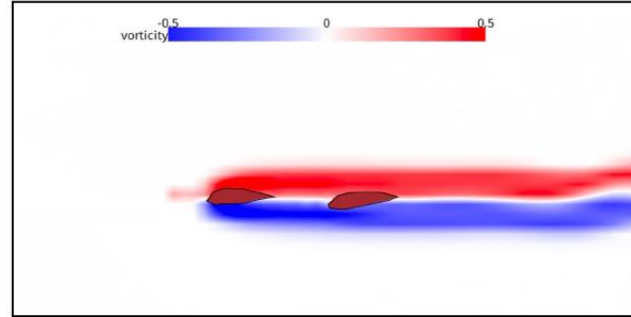
Compositional design results of our method in 2D airfoil generation. Each row represents an example. We show the heatmap of velocity in horizontal and vertical direction and pressure in the initial time step, ⁷⁹inside which we plot the generated airfoil boundaries.

Results 1: inverse design

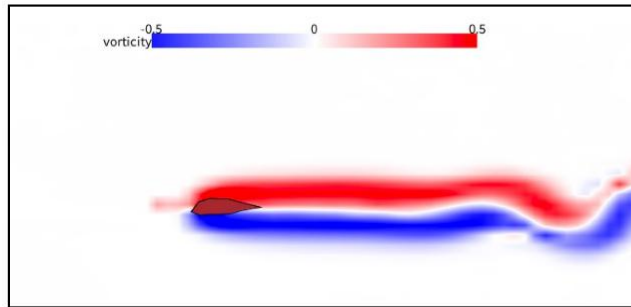
生成的速度场轨迹和机翼边界极其队列设计，以优化升阻比：



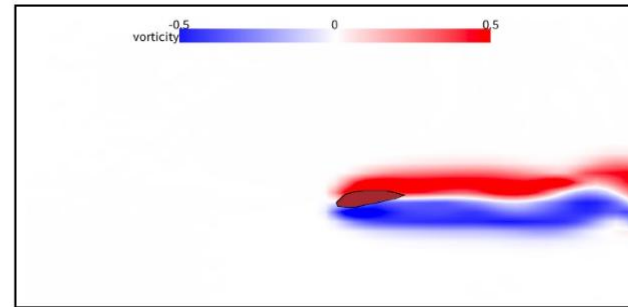
Results 1: inverse design



(a) Formation flying of airfoils A and B



(b) Single flying of airfoil A



(c) Single flying of airfoil B

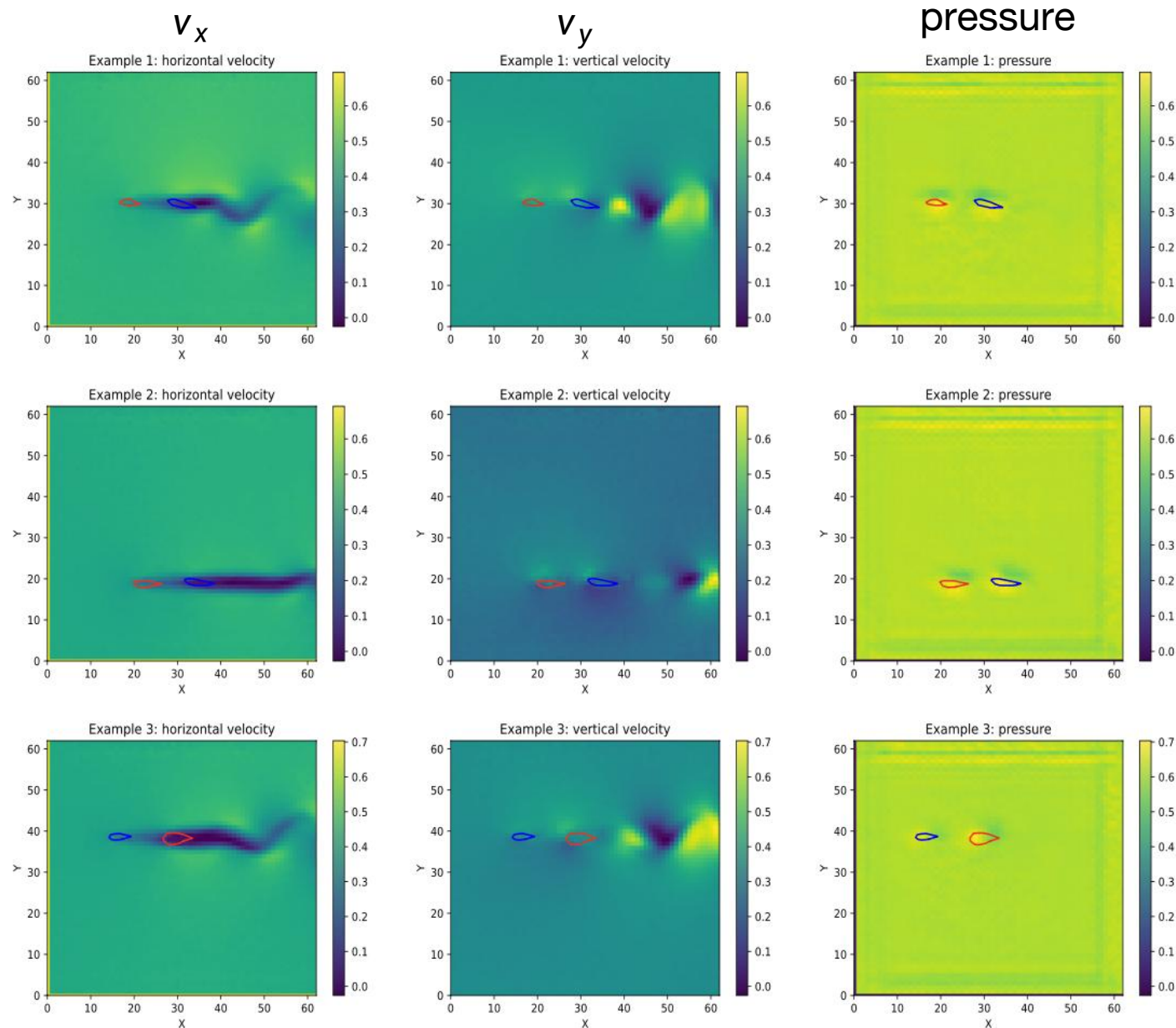
Our model discovers **formation flying** (编队飞行)

- Reducing the drag by 53.6%
- increasing the lift-to-drag ratio by 66.1%

Results 1: inverse design

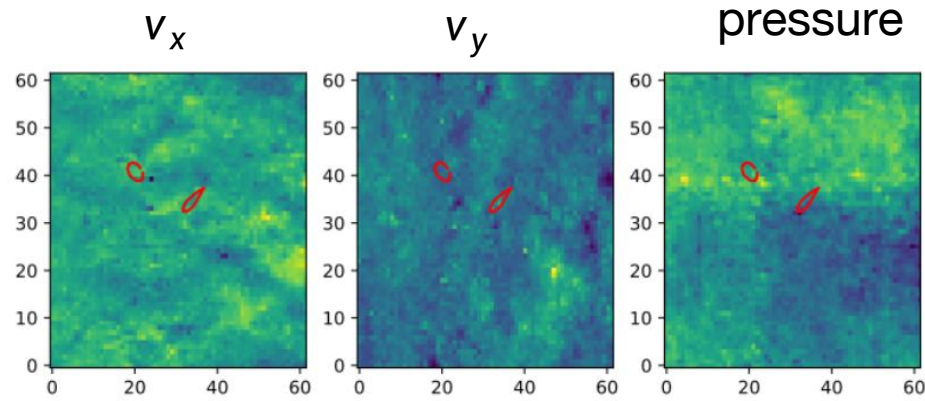
Method	1 airfoil		2 airfoils	
	design obj ↓	lift-to-drag ratio ↑	design obj ↓	lift-to-drag ratio ↑
CEM, FNO	0.0932	1.4005	0.3890	1.0914
CEM, LE-PDE	0.0794	1.4340	0.1691	1.0568
Backprop, FNO	0.0281	1.3300	0.1837	0.9722
Backprop, LE-PDE	0.1072	1.3203	0.0891	0.9866
CinDM (ours)	0.0797	2.1770	0.1986	1.4216

Results 1: inverse design by **our model**

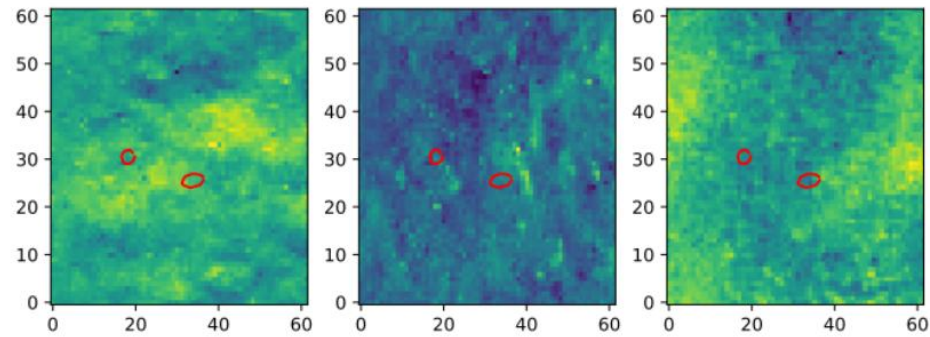


Results 1: inverse design by **baseline neural models**

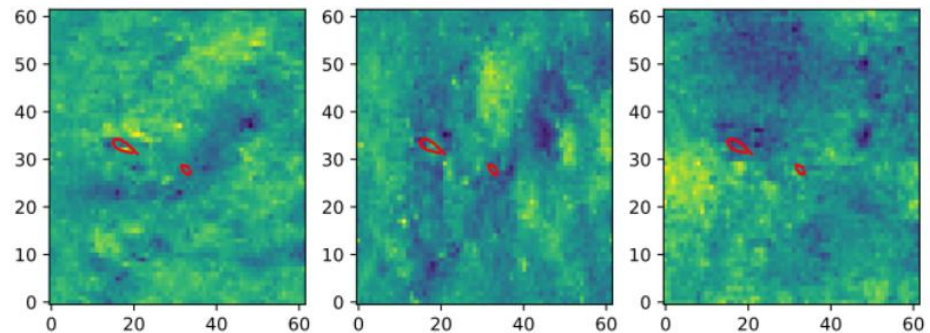
Adversarial modes by
baseline neural model
(FNO with CEM):



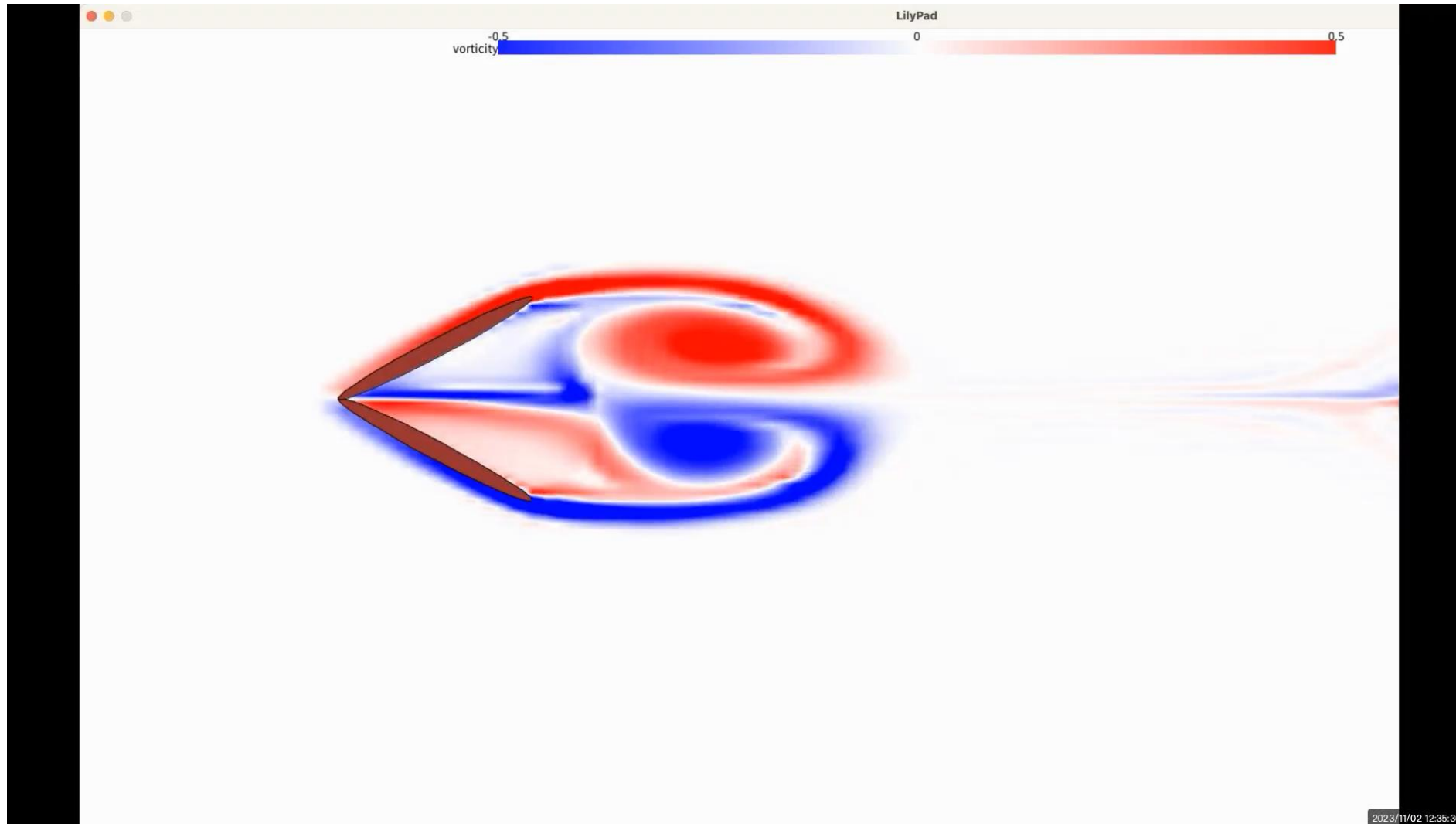
(a)



(b)

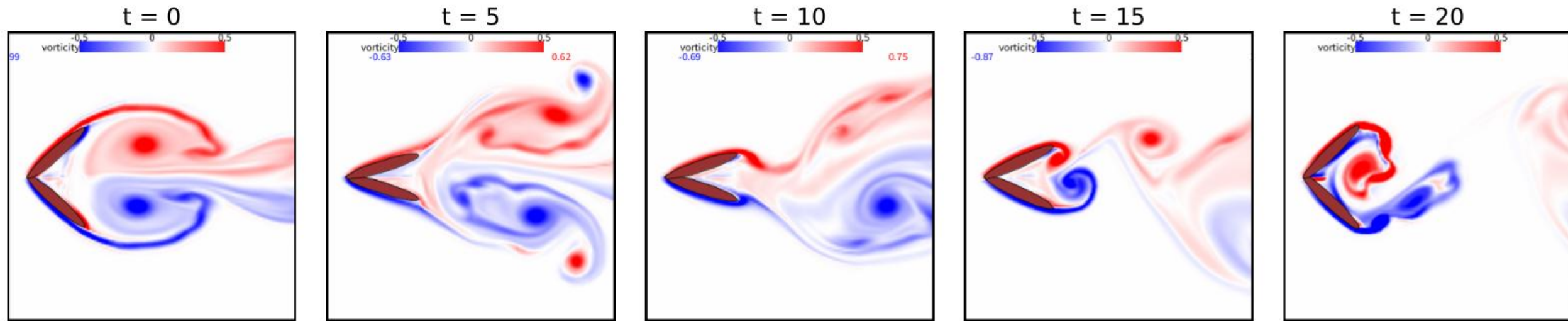


Experiment 3: Controlling a jellyfish



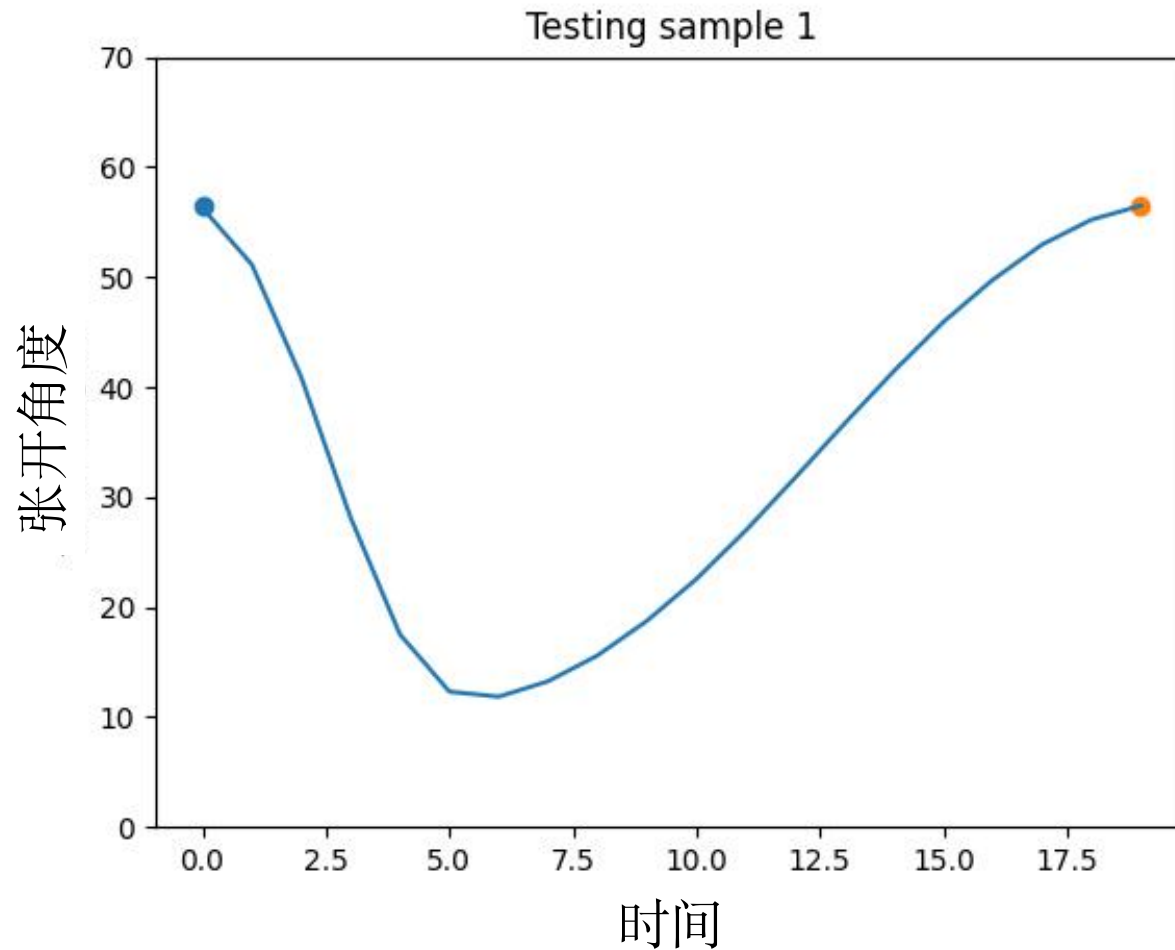
Experiment 3: Controlling a jellyfish

我们模型生成的控制轨迹



Experiment 3: Controlling a jellyfish

Discovered $\theta(t)$ by our diffusion model



Paper

Paper 1: Compositional Generative Inverse Design

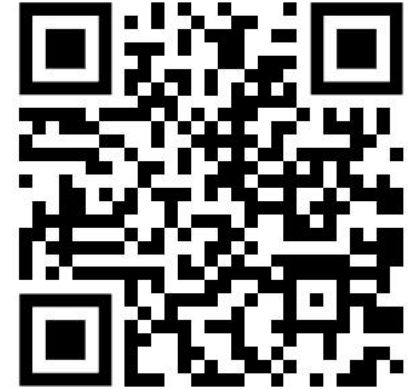
ICLR 2024 **spotlight**

<https://openreview.net/forum?id=wmX0CqFSd7>

Paper 2: Generative PDE control

NeurIPS 2024

<https://openreview.net/pdf?id=vaKnCahjdj>



总结

- **AI用于科学仿真**

两个重要架构的应用：图神经网络和神经算子

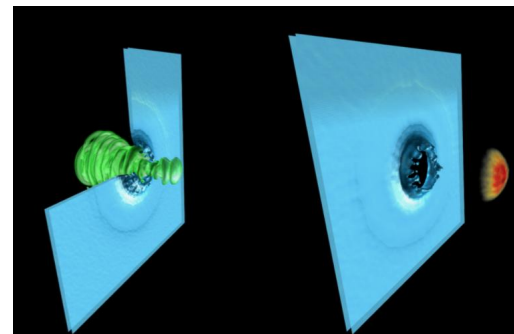
挑战1: **多分辨率**

方法：用两个图神经网络分别学习系统演化和优化网格

$$L = (1 - \beta) \cdot \text{Error} + \beta \cdot \text{Computation}$$

挑战2: **复杂边界条件**

方法：图神经网络学习边界和内部源对解的影响，Transformer编码边界



- **AI用于控制和逆向设计**

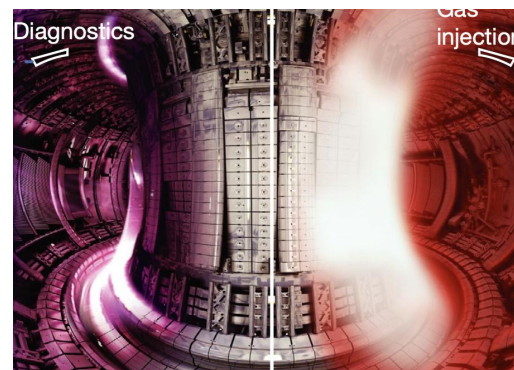
挑战：非线性、高维、强耦合的系统

方法：(1) 代理模型+反向传播

(2) 强化学习

(3) **扩散模型**同时学习物理以及优化设计/控制目标

$$\hat{\gamma} = \arg \min_{\gamma, U_{[0,T]}} [E_{\theta}(U_{[0,T]}, \gamma) + \lambda \cdot \mathcal{J}(U_{[0,T]}, \gamma)]$$



有用资源

[1] Zhang, Xuan, et al. "Artificial intelligence for science in quantum, atomistic, and continuum systems." *arXiv preprint arXiv:2307.08423* (2023).

- AI for Science综述： Review paper "[Artificial intelligence for science in quantum, atomistic, and continuum systems](#)."
- [Scientific Discovery in the Age of Artificial Intelligence](#), *Nature* 2023
- [集智AI + Science读书会](#)
 - [第一期](#)： AI + Science: motivation, advances and open problems [[slides](#)]
 - [第三期](#)： 利用AI代理模型和扩散模型辅助科学设计 [[代理模型slides](#)][[扩散模型slides](#)]
 - [第八期](#)： 数据驱动的物理仿真模拟： 神经算子与图神经网络 [[神经算子slides](#)][[图神经网络slides](#)]
- 《计算物理》： 人工智能加速科学仿真、设计、控制和发现 --吴泰霖